

Arquitectura conceptual

La arquitectura de Meta4Objects y sus componentes



© 2013 Meta Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: 20 de enero de 2002.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

1 Introducción a la arquitectura de componentes de Meta4

■ Acerca de este manual	13
-----------------------------------	----

2 Funcionalidad y uso de Meta4Objects

■ Acerca de este capítulo	15
■ Meta4Objects: concepto y definición	16
Elementos	17
Carga de Meta4Objects	19
■ Estructura de un Meta4Object	22
Meta4Objects mononodo y multinodo	23
Estructura del Meta4Object mononodo	24
Estructura del Meta4Object multinodo	24
Conectores	25
Tipos de conector y elementos de conexión	27
■ Funcionalidad de los Meta4Objects	30
Interfaz datos-aplicación	30
Control de integridad y subsistema de bloqueos	31
Percepción unificada de los datos	31
Definición de la lógica de los procesos	32
Visualización de datos	33
Formulario	34
Tabla	35
Seguridad	35
■ Presentaciones	37
■ Trabajar con Meta4Objects	38
Diseñador del modelo de datos	38
Diseñador de Meta4Objects	39
Editor de sentencias	40
Diseñador de presentaciones	41
Diseñador de eventos	42
Diseñador de informes	43

Diseñadores de seguridad	44
■ Conectividad	46
Cliente DCOM	46
Servidor de objetos	47

3 Estructuras de nodo: conceptos básicos

Acerca de este capítulo.	49
■ Concepto y funcionalidad	50
■ Creación de instancias y ejecución de estructuras de nodo.	52
■ Componentes de una estructura de nodo	55
Sentencia de recuperación de datos	55
Estructuras de nodo y lenguaje SQL	57
Instrucciones de procesamiento	58
El lenguaje LN4	58
LN4 y la interfaz API-SQL	59
■ Elementos de las estructuras de nodo	60
Funcionalidad de los elementos	60
Campos	61
Propiedades	61
Conceptos	61
Métodos	62
Ámbito de los elementos	63
Tipos de ámbito	64
Ámbito a nivel de registro	64
Alcance a nivel de bloque	64
Alcance a nivel de nodo	66
Visibilidad y ámbito de los elementos	66
Referencias a elementos con ámbito a nivel de registro	66
Referencias a elementos con ámbito a nivel de bloque	67
Referencias a elementos con ámbito a nivel de nodo	67
■ Filtros de datos	69
Estructura de un filtro	70
Operadores de comparación	72
Funciones predefinidas	73
Filtros con varias condiciones	74
Operador And	75
Operador Or	76
Operador NOT	77
Operador Exists	77

Orden de evaluación de condiciones	78
Cambio de la prioridad de los operadores	80
Ordenación de los registros	81
■ Agrupaciones de registros	84
Funciones de totalización	85
Diseño de agrupaciones	86
Agrupaciones por más de una columna	90
Ordenación de registros en agrupaciones	91
■ Eliminación de registros duplicados	92
■ Disparadores	94

4 Estructuras de nodo: conceptos avanzados

■ Acerca de este capítulo	95
■ Estructuras de nodo diseñadas sobre más de una tabla	96
Normalización	96
Enlaces	97
Ejecución de un enlace	99
Enlaces y estructuras de nodo	102
Diseño de estructuras de nodo con Enlaces	103
■ Tipos adicionales de enlaces	111
Valor nulo	111
Claves externas nulas	112
Valores no referenciados	113
Enlaces externos	114
Unión izquierda	114
Unión derecha	116
Enlaces externos en la tecnología Meta4	118
■ Métodos definidos por defecto en estructuras de nodo	121

5 Conectores

Acerca de este capítulo	131
■ Estructura jerárquica de un Meta4Object	132
Relaciones entre nodos y conectores	132
■ Funcionalidad de los conectores	134
■ Tipos de conectores y organización en memoria de los datos	135
Tipos de conectores	136
Registro-bloque	136
Bloque-bloque	137

Libre	137
■ Asociación de elementos tipo campo	138
Desplazamiento entre los registros de un nodo	141
Desplazamiento entre los registros de un nodo multibloque	141
Desplazamiento entre registros de un nodo multibloque conectado con otro nodo mediante un conector registro-bloque	142
Desplazamiento entre registros de un nodo multibloque conectado con otro nodo mediante un conector bloque-bloque	144
■ Conexiones entre elementos con ejecución	145
Propagación de la ejecución de elementos	145
Orden de ejecución de los elementos	146
Propiedad Tipo de precedencia (Tp. de precedencia)	146
Orden de ejecución cuando se desencadena la ejecución de dos o más elementos	146
Tipo de conectores y propagación de ejecución	147
Propagación de ejecución y ámbito de los elementos	147
Sincronización de argumentos de elementos con ejecución	149
Sincronización de los parámetros de dos métodos	150
Paso de argumentos por valor y por referencia	150
■ Borrado en cascada de registros	152
Borrado en cascada	154
■ Disparadores	156
Tipos de disparadores	156
Disparadores de estructura de nodo	156
Disparadores de nodo	159
Disparador de conector	163
Estructura de un disparador	164
Contexto de ejecución	164
■ Filtros en el conector	168
Estructura de un filtro de conector	170
Filtros de fechas	171
Filtros de conectores y reutilización de estructuras de nodo	172

6 Administración de memoria

Acerca de este capítulo	173
■ Meta4Objects e instancias de Meta4Objects	174
■ Tipos de carga	176
Carga completa	176
Carga automática de bloque	176
Carga automática de rama	176

Carga según Nodo.	177
■ Parametrización de la carga.	178
Carga de Meta4Object desde el Diseñador de Meta4Objects.	178
Carga de Meta4Objects desde llamadas de nivel 2.	179
Carga de Meta4Object desde la Barra de tareas del desarrollador.	180
Carga de Meta4Object desde el Diseñador de presentaciones.	180
■ Reutilización de Meta4Objects y llamadas de nivel 2.	182
■ Carga de datos en la memoria en el entorno C/S.	183
Parametrización de una carga en llamadas de nivel 2.	183
Parametrización de un Meta4Object con cachés.	188
Manejo de instancias de Meta4Object en Meta4Object Engine.	189

7 Listas y filtros

■ Acerca de este capítulo.	191
■ Listas: definición y funcionalidad.	192
Listas y grupos de remonte.	192
Grupos de remonte.	196
Filtros de carga de registros en nodos raíz.	197
■ Creación de listas.	199
Desde opción de menú Herramientas de desarrollo Desarrollar con el development framework.	199
Desde las herramientas de desarrollo tradicional:	199

8 Reglas

Introducción.	201
■ Métodos y conceptos.	203
■ Reglas.	205
■ Elementos, reglas y normas.	207
Reglas con periodos de validez solapados.	210
■ Metarreglas.	212
■ Argumentos.	213
Propiedades de los argumentos.	215
Tipo de argumento.	216
Posición.	218
Tipo básico.	218
Argumentos fijos y variables.	219
■ Ejecución de reglas.	220
Ejecución desde controles de presentación.	221

Ejecución desde una regla LN4	223
Sintaxis NombreElemento (arg1, ... argn)	224
Ejecución con tramos	224
Sintaxis NombreElemento(arg1, ..., argn)..IdRegla	225
Ejecución de elementos en disparadores y eventos	225
Resumen	226
■ Dependencias entre elementos	229
Dependencias de valor y de ejecución	229
Tipos de dependencias	230
Dependencias de nivel 2	232
Uso de alias de Meta4Object en reglas LN4	234
■ Dimensiones: ejecución condicional de reglas	236
Periodo de validez de una condición	237
Dimensiones y metarreglas	238
■ Creación de metarreglas	243
■ Actualización de metarreglas	244
Creación de una nueva norma	244
Solapamiento con una norma de la metarregla	244
Solapamiento con dos o más normas de la metarregla	248
Creación de nuevas reglas sin dimensiones	255
Creación de reglas con dimensiones	259
■ Apéndice: ejemplos de actualización de metarreglas	262
Ejemplo 1	262
Ejemplo 2	266
La regla no tiene dimensiones	267
La nueva regla cuenta con dimensiones	268
■ Ventajas de trabajar con etiquetas	270
■ Conclusiones	271

9 Elementos tramados

■ Introducción	273
■ Introducción a tramos	274
■ Parametrización de tramos	276
Definición de las normas	276
Parametrización del elemento	277
División en tramos y elemento de tramos	277
Comportamiento en tramos	279
Codificación en LN4	279
■ Activación de tramos	280

Modo de tramos	280
Fechas de trabajo	280
Fechas de ejecución	282
■ Creación de tramos	283
Ejecución de un concepto	283
Lineal	284
No lineal	284
Sin tramos	285
Asignación de elementos	285
Funciones LN4 de nómina	286
Unflatten	286
Flatten	287
JoinSlices	287
TransferApplyValue	288
YTDSearchFiltered	288
Atributos método de LN4	288
SysAddNewSlice	288
SysCreateSlice	288
SysSplitSlice	289
■ Lectura de tramos.	289
Valor base	290
Valor final	290
Unidad	291
Incidencia.	292
Excepciones.	293
Redondeo	293
Ajuste a 30 días	295
■ Compactación de tramos	296
Valor base	297
Valor final	298
Unidad	298
Incidencia.	299
Redondeo	299
■ Funciones LN4 para el manejo de tramos	300
■ Trazabilidad de tramos	302

10 Arquitectura cliente/servidor en las aplicaciones Meta4

■ Acerca de este capítulo	303
■ Arquitectura cliente/servidor: fundamentos básicos	304

Introducción: arquitecturas de 3 capas lógicas y físicas	304
Rendimiento y escalabilidad	305
Nivel de almacenamiento de datos	306
Nivel de aplicación	306
Nivel de presentación	308
Implementación física de la arquitectura c/s	308
Servidores de bases de datos	310
Independencia	310
Bases de datos distribuidas	311
Conexiones lógicas	311
Servidores de aplicaciones	312
Funcionalidad del servidor de aplicaciones	312
Componentes del servidor de aplicaciones	313
Servidores de presentación: puestos cliente	314
Interfaz del puesto cliente	315
Cliente ligero	316
Tipos de ejecución en arquitectura C/S	317
OLTP o modo transaccional	317
Delta o modo transacción optimizado	317
Proxy o modo conversacional	318
Ejecución programada: Planificador de tareas	318
■ Procesamiento distribuido en C/S	319
Procesamiento distribuido de Meta4Objects	319
Meta4Object en servidor y en cliente	319
Modos de serialización o transmisión de datos	320
■ Meta4Objects en C/S: metodología de desarrollo	321
Parametrización de Meta4Objects C/S	321
Parametrización del comportamiento del Meta4Object	322
Parametrización de los elementos: Propiedad ID Tipo C/S	324
Posibles usos de los elementos en C/S	327
Parametrización de la transmisión de datos para elementos de tipo Ambas partes (sync.)	330
Tipo de ejecución de OLTP	330
Limitaciones	330
Modos avanzados de transferencia o serialización	331
Registro	332
Registro y sus hijos	332
Bloque	333
Bloque y sus hijos	333
Documento	334
Persist	334

Ninguno	335
Tipo de ejecución Delta	335
Tipo de ejecución de Proxy	336
Limitaciones	336
Resumen	336
Conexión de los métodos en entornos C/S	337
Visualización de los métodos	337
Optimización del flujo de ejecución: idas y vueltas	338
Instancias de Meta4Objects en entornos C/S	339
■ Operatividad en modo C/S	341
Configuración del cliente Meta4 en C/S	341
Timeouts	341
Funcionamiento de las cachés	342
Tipos de caché	342
Caché de metadatos	342
Caché del diccionario	343
Caché de datos	343
Caché de datos: comportamiento de un Meta4Object cacheable	343
Refresco de las cachés	344
En el servidor de aplicaciones	344
En el cliente	344
En la estación de desarrollo	345
Parámetros de configuración de la caché de datos y metadatos	345
Parametrización de la caché	345
Configuración de caché en parte cliente y servidor a través de registro	347
Configuración de caché en parte servidor (startup.obl)	349
Supervisión de cachés desde la herramienta de administración del servidor	350
Borrado de cachés del servidor	351
Comportamiento de cachés en C/S	352
Sincronía/asincronía	352
Consideraciones especiales en el diseño con cachés	352
■ Trazas cliente/servidor	354
Requerimientos de alto nivel	354

11 Apéndice: Introducción al APISQL

■ ¿Que es APISQL?	357
■ Uso del APISQL:	357
Tipos de acceso a datos a través de API SQL	357
Empleando un concepto invocado desde LN4	357

Empleando el elemento SysSentence	358
■ Sintáxis de APISQL	359
Valores posibles de m4type	360
■ Tipos de sentencias:	361
■ Palabras clave de APISQL	362
■ Funciones APISQL	363

Introducción a la arquitectura de componentes de Meta4

1

Acerca de este manual

La tecnología Meta4 se basa dentro de su nivel de aplicación en el concepto de Meta4Object.

Un Meta4Object es un elemento que encapsula datos e instrucciones de procesamiento. Al definir la funcionalidad de la aplicación a través de Meta4Objects, la tecnología Meta4 incorpora a sus soluciones la utilización intensiva de la orientación a objetos y aprovecha las ventajas que ofrece esta tecnología:

- Reducción del tiempo y costes de desarrollo de nuevas funcionalidades: el responsable de diseño podrá reutilizar componentes definidos con anterioridad.
- Mayor facilidad en el mantenimiento de las aplicaciones.
- Seguridad: únicamente se obtendrá acceso a la información que encapsula un elemento a través de los métodos definidos en la interfaz.
- Adecuación de las estructuras de datos a los modelos de negocio de las organizaciones. La orientación a objetos permite modelizar la realidad con los mismos conceptos que utilizan los usuarios de la tecnología.
- Distribución de la lógica de las aplicaciones: los distintos elementos que implementan la funcionalidad de la aplicación se pueden distribuir entre distintos servidores. De esta forma, se garantiza la escalabilidad de las aplicaciones.

En los siguientes capítulos de este manual se explican los siguientes aspectos:

- Concepto de Meta4Object
- Estructuras de un Meta4Object
- Concepto de estructura de nodo
- Funcionalidad de los Meta4Objects, su estructura y tipos

-
- El diseño de estructuras de nodo que recuperen registros de dos o más tablas mediante un *Enlace*
 - Concepto de *Conector*
 - Gestión de memoria y diseño de Meta4Objects
 - Listas y filtros sobre Meta4Objects
 - Gestión y administración de Meta4Objects dentro de un entorno Cliente/Servidor

Funcionalidad y uso de Meta4Objects

2

Acerca de este capítulo

El nivel de aplicación de la tecnología Meta4 se basa en el concepto de Meta4Object.

Un Meta4Object es un elemento que encapsula datos e instrucciones de procesamiento. Al definir la funcionalidad de la aplicación a través de Meta4Objects, la tecnología Meta4 incorpora a sus soluciones la utilización de la orientación a objetos y aprovecha las ventajas que ofrece esta tecnología.

En los siguientes apartados se van a tratar los siguientes temas relativos al uso de Meta4Objects:

- Concepto y definición de Meta4Object y estructura de nodo.
- Estructura de un Meta4Object, diferenciando entre Meta4Objects mononodo y multinodo.
- Funcionalidad de los Meta4Objects
- Trabajo con Meta4Objects
- Conectividad y posibilidad de integración de la tecnología Meta4 mediante el uso de Meta4Objects.

Meta4Objects: concepto y definición

Un Meta4Object es un componente cuya estructura determina:

- Los datos que se van a extraer de la base de datos
- Las operaciones que se pueden realizar con esos datos

Un Meta4Object puede estar formado por uno o más nodos. Cada nodo se diseña sobre una estructura. La estructura de un nodo especifica:

- De qué tablas se quieren extraer datos
- De qué columnas de las tablas se quieren obtener los valores
- Qué condiciones deben cumplir los registros para que sus valores sean recuperados

De esta forma, a partir de una estructura de nodo se puede generar una sentencia SQL que se ejecute contra la base de datos y recupere un conjunto de registros.

A nivel de recuperación de datos, las estructuras de nodo ofrecen toda la capacidad expresiva del lenguaje SQL, siendo posible definir condiciones de selección de registros complejas, agrupaciones y totalizaciones, etc.

Una estructura de nodo también puede incluir código procesable escrito en el lenguaje de programación de desarrollo propio: LN4. Este código se escribe en un tipo especial de elementos, los de tipo método y concepto. Cada método y concepto puede interpretarse como programa ejecutable de forma independiente.

LN4 ofrece toda la capacidad expresiva de un lenguaje de programación, y permite tratar los valores recuperados por una estructura de nodo como si se tratase de variables o cursores de una base de datos.

Elementos

Una estructura de nodo está formada por un conjunto de elementos, que pueden recoger:

- Valores que se leen de la base de datos.
- Valores pedidos al usuario en tiempo de ejecución u obtenidos como resultado de un cálculo.
- Instrucciones de procesamiento. El término *elementos ejecutables* hace referencia a este tipo de elementos. Dentro de los elementos ejecutables se distingue entre conceptos y métodos.
 - Los conceptos recogen el valor que se obtiene tras su ejecución. Tras ejecutar un concepto, se puede leer su valor sin necesidad de volver a ejecutar el cálculo.
 - Los métodos ejecutan las instrucciones de procesamiento, pero no recogen el valor que se obtenga como resultado. Esto no significa que este valor no pueda quedar recogido en otro elemento distinto.Tanto los métodos como los conceptos devuelven un valor; la diferencia consiste en que los conceptos lo recogen y los métodos no.
Sin embargo, el valor que devuelve un método se puede asignar a otro elemento distinto.



Los elementos de una estructura de nodo siempre recogen valores escalares. Es decir, un único valor de tipo entero, cadena de caracteres, fecha, etc. No pueden recoger valores compuestos por más de un valor: listas, matrices, etc.

EJEMPLO

El método B contiene una fórmula que devuelve un valor. Si se hace una llamada al método desde la regla LN4 de un elemento de tipo campo o propiedad utilizando la sintaxis

`Elemento = B ( )`

el elemento *Elemento* recogerá el valor que devuelve B.

Los elementos se pueden clasificar en varios tipos, según la función que cumplan. La siguiente tabla recoge los distintos tipos de elementos.

Tipo de elemento	Descripción
Campos	Se corresponden con campos de la base de datos. Su valor se obtiene del campo de la base de datos al que están asociados. Cuando se asigna valor a un elemento de tipo campo y se ejecuta la orden de grabar, el valor propuesto se graba en el campo de la base de datos al que está asociado el elemento de tipo campo.
Propiedades	Recogen valores constantes, que no dependen de ningún otro valor. El valor de este tipo de elementos puede ser una constante predeterminada en la definición de la estructura de nodo o un valor que se solicita al usuario en tiempo de ejecución.
Conceptos	Recogen código procesable escrito en lenguaje LN4, C++ o llamadas a métodos de librerías de enlace dinámico (DLL) externas. Cuando se ejecuta el código de un concepto, el valor que se obtiene se asigna al concepto. Los conceptos pueden grabar su valor en una columna de la base de datos lógica.
Métodos	Recogen código procesable escrito en lenguaje LN4, C++ o llamadas a métodos de DLL externas. El resultado obtenido al procesar este código no se asigna al elemento. Sin embargo, desde la fórmula de un método se pueden asignar valores a otros elementos de tipo campo o concepto.



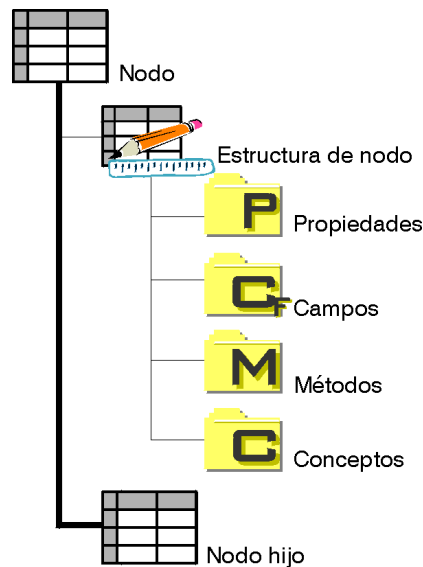
Un Meta4Object encapsula datos y procesamiento: información y todas las operaciones necesarias para gestionarla. Un Meta4Object puede incluir o no estos dos tipos de elementos. Por ejemplo, es posible definir Meta4Objects que carezcan de instrucciones de procesamiento y Meta4Objects que únicamente contengan instrucciones de procesamiento.

EJEMPLO

Se va a definir un Meta4Object que extraiga los datos de la tabla Empleados, correspondientes a los empleados adscritos al área de Marketing, y que realice un procesamiento simple con esos datos; por ejemplo, calcular la media de sus edades.

A lo largo de este manual se utilizan los siguientes iconos para representar las estructuras de nodo y los distintos tipos de elemento:

Figura 1. Estructura del Meta4Object



Carga de Meta4Objects

Los usuarios de la aplicación podrán trabajar con los Meta4Objects solicitando su carga en memoria o *ejecución*. Un usuario de la aplicación es cualquier persona y aplicación que interactúe con la aplicación.



La carga de datos en memoria puede distribuirse entre el servidor de aplicaciones y el puesto cliente. La herramienta de diseño de Meta4Objects permite parametrizar la carga de datos en memoria para obtener las máximas ventajas de la arquitectura cliente/servidor.

Al cargar en memoria un Meta4Object, el servidor de aplicaciones realiza las operaciones de lectura sobre la base de datos indicada en la definición del Meta4Object. Los datos obtenidos tras estas operaciones de lectura se copian en la memoria principal con un formato estructurado.

Este formato organiza los datos en tablas, de forma similar a como lo hacen las tablas de la base de datos. La diferencia consiste en que los datos de un Meta4Object sólo existen en la memoria principal; no tienen persistencia física. Cada fila se corresponderá con un registro y cada columna con un campo.

Los datos que recupera un Meta4Object reflejan el estado en el que se encuentra la base de datos. Si se agregan nuevos registros o se borran

registros de una tabla, el Meta4Object definido sobre esa tabla reflejará estos cambios cuando se vuelva a cargar en memoria.



Un Meta4Object tiene una naturaleza dinámica, y presenta unos datos u otros, según el estado en que se encuentre la base de datos al solicitar la carga del Meta4Object.

Desde la interfaz del puesto cliente, y mediante reglas LN4, el usuario puede recorrer los registros disponibles en memoria, pasar a un registro específico, visualizar el valor de los distintos campos y ejecutar las operaciones definidas en los métodos del Meta4Object.

Las operaciones que se pueden realizar sobre un Meta4Object no se limitan a la lectura de datos. La tecnología Meta4 permite al usuario insertar, modificar y eliminar registros del conjunto de registros recuperados en memoria tras la carga de un Meta4Object.

Estas operaciones se realizan en la memoria principal y no surten efecto en la base de datos hasta que el usuario lo indica de manera explícita mediante una llamada a la orden de grabar.

Este último punto es muy importante: los Meta4Objects constituyen la única interfaz entre la información de la base de datos y el nivel de aplicación y presentación de la aplicación. Las operaciones de lectura y escritura nunca se hacen directamente sobre las tablas de la base de datos: se realizan siempre mediante Meta4Objects. La estructura real de las tablas de la base de datos será transparente para el usuario final de la aplicación.

Tablas, estructuras de nodo y Meta4Objects

La tecnología Meta4 ofrece varios niveles de abstracción: tablas, tablas físicas, estructuras de nodo y Meta4Objects.

Las tablas determinan las estructuras en las que se va a almacenar la información de la base de datos. Realmente, una tabla no contiene ningún dato: es un recubrimiento de la tabla física. Los datos propiamente dichos se almacenan en tablas físicas. Una tabla se puede corresponder con dos o más tablas físicas.

Este primer nivel de abstracción permite liberar al programador de las limitaciones que presentan los distintos sistemas gestores de bases de datos en la asignación de nombres a tablas y columnas (número máximo de caracteres permitidos), tratamiento de claves primarias, valores nulos, etc.

La base de datos lógica será visible para los programadores de nuevas funcionalidades, que podrán desarrollar modelos de datos transferibles a instalaciones que utilicen distintos sistemas gestores de bases de datos como depósitos de información.

Tablas, estructuras de nodo y Meta4Objects(*Continuación*)

Las estructuras de nodo se diseñan sobre las tablas de la base de datos lógica. Una estructura de nodo puede recuperar datos de una única tabla o de más de una. La estructura de nodo determina la tabla o tablas de las que obtener datos, las columnas cuyos valores se quieren obtener y las condiciones que deben cumplir los registros de la tabla para que sus valores sean recuperados. Además, en la definición de una estructura de nodo se pueden proponer alias para nombrar los distintos campos cuyos valores se extraen de la base de datos.

Las estructuras de nodo sólo son visibles para los programadores de nueva funcionalidad.

Los Meta4Objects constituyen la interfaz real entre los niveles anteriores y los usuarios finales de la aplicación. Los Meta4Objects se diseñan a partir de estructuras de nodo. Se dice que el Meta4Object crea instancias de estructuras de nodo. Un usuario nunca podrá trabajar con una estructura de nodo si no se ha creado una instancia de ésta en un Meta4Object.

Si un Meta4Object crea una instancia de una única estructura de nodo, el Meta4Object es mononodo. Si crea una instancia de dos o más estructuras de nodo, el Meta4Object es multinodo. Cada nodo crea una instancia de una estructura de nodo y los nodos se relacionan entre sí mediante conectores.

Los conectores cumplen distintas funciones: determinar la estructura del Meta4Object, permitir la propagación de valores o la ejecución de métodos, y determinar los datos que se cargan en memoria de forma dinámica a partir de los que se hayan cargado en otros nodos del Meta4Object. Para obtener una explicación más detallada de este concepto, consulte el capítulo *Conectores*.

Estructura de un Meta4Object

Un Meta4Object puede extraer datos de una única tabla, o de más de una, sin que exista ningún tipo de restricción en este aspecto.

Si un Meta4Object recupera datos de más de una tabla, es aconsejable distribuir la carga de datos en bloques de memoria diferentes. De esta forma, no será necesario recuperar todos los datos en una sola operación de lectura y la carga de registros podrá hacerse de forma progresiva.

EJEMPLO

Se quiere diseñar un Meta4Object que recupere datos de las tablas Persona y Periodo_RH. La tabla Persona contiene información genérica de todas las personas relacionadas con la organización: nombre, fecha y lugar de nacimiento, tratamiento, etc. La tabla Periodo_RH contiene los periodos de tiempo durante los cuales una persona ha prestado sus servicios profesionales a la empresa: fases de alta, contratos como autónomo, etc.

El responsable de diseño podrá optar entre:

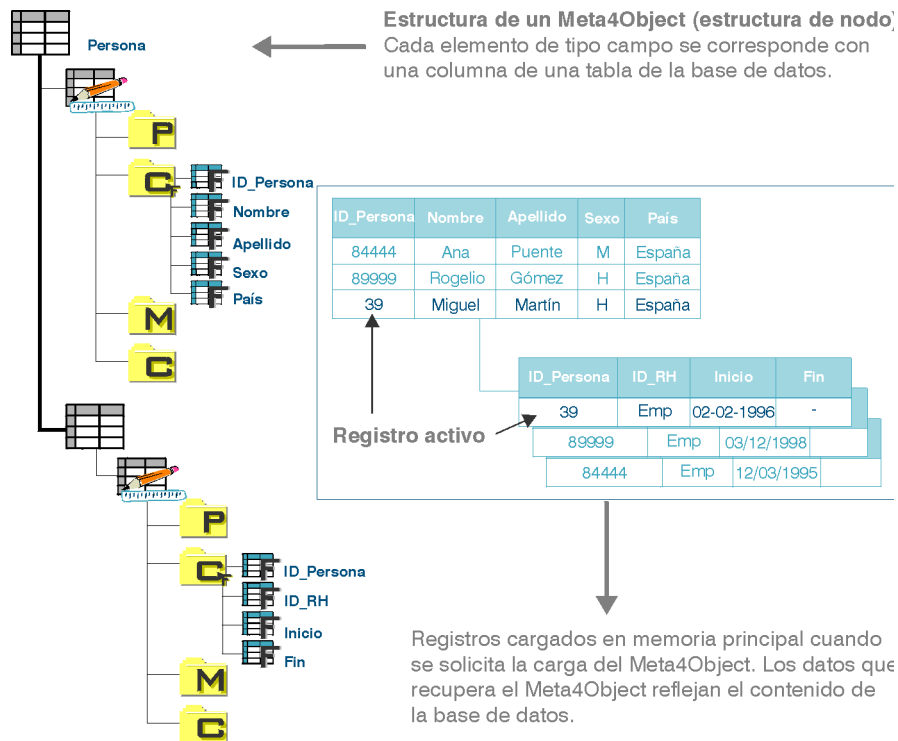
- Crear un Meta4Object que recupere de una sola vez datos de las dos tablas
- Crear un Meta4Object que permita la recuperación progresiva de los datos

En el segundo caso, se definirá una estructura de nodo para cada tabla y, posteriormente, se establecerá una correspondencia entre los registros recuperados por cada estructura de nodo. Esta correspondencia permitirá saber qué registros de Periodo_RH corresponden a los distintos registros de la tabla Persona.

Un Meta4Object puede estar formado por varias estructuras de nodo diferentes, que se pueden cargar en memoria de forma selectiva. La definición de cada estructura de nodo contiene:

- La sentencia que extraerá datos de una o varias tablas de la base de datos
- Las instrucciones necesarias para procesar los datos recuperados por la sentencia anterior

Figura 2. Estructura y datos recuperados por un Meta4Object multinodo



Si un Meta4Object está formado por una sola estructura de nodo, se trata de un Meta4Object mononodo. Si está formado por más de uno, es multinodo.

Meta4Objects mononodo y multinodo

Un Meta4Object mononodo es aquel que crea una instancia de una sola estructura de nodo.



La estructura de un Meta4Object determina la forma en que se va a distribuir en memoria la información recuperada de la base de datos.

El multinodo crea instancias de dos o más estructuras de nodo. Como en el caso anterior, cada una de estas estructuras de nodo puede recuperar datos de una o más tablas.

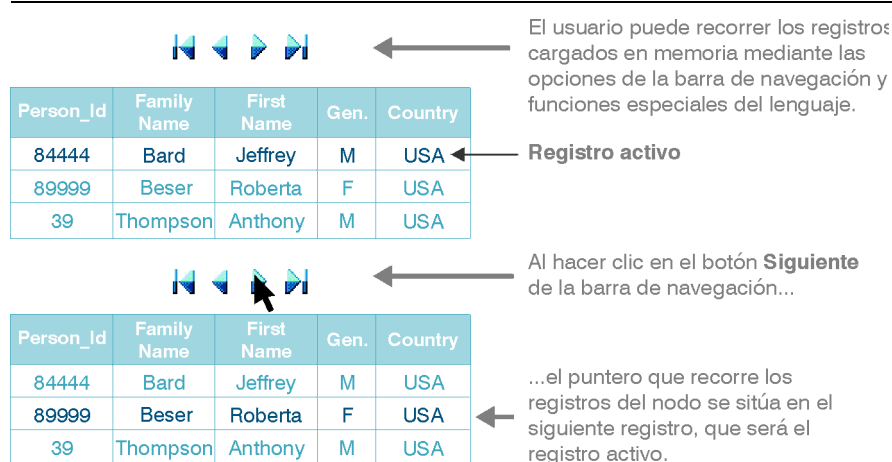
Los datos recuperados por un Meta4Object multinodo se distribuirán en distintos bloques de memoria. Estos bloques pueden estar relacionados, de forma que los registros que se carguen en un bloque dependerán de los que se hayan cargado en el bloque con el que se relaciona.

Estructura del Meta4Object mononodo

Un Meta4Object mononodo se define sobre una sola estructura de nodo, que recupera datos de una o más tablas.

Al cargar en memoria un Meta4Object mononodo, el sistema crea una tabla en la memoria principal con todos los datos extraídos de la base de datos. El usuario puede trabajar sobre esta tabla agregando nuevos registros, borrando y modificando registros, ejecutando métodos, etc.

Figura 3. Desplazamiento por los registros cargados en memoria



Los datos recuperados como resultado de la ejecución de un Meta4Object mononodo se distribuyen en un solo bloque.

Estructura del Meta4Object multinodo

El Meta4Object multinodo se diseña a partir de dos o más estructuras de nodo relacionadas. Cada una de las instancias de estructuras de nodo que conforman el Meta4Object forma parte de un nodo.

El concepto de nodo permite diferenciar entre una estructura de nodo propiamente dicha y las ocasiones en las que se utiliza esa estructura de nodo en la definición de un Meta4Object. Estableciendo un paralelismo con los lenguajes de programación orientados a objetos, una estructura de nodo equivaldría a una clase, y un nodo, a las instancias de clase.

Una estructura de nodo puede utilizarse en más de una ocasión en la definición de un Meta4Object multinodo. En este caso, el Meta4Object crea instancias en más de una ocasión de la misma estructura de nodo.

El hecho de utilizar la misma estructura en distintos nodos no implica que éstos contengan los mismos registros. Dependiendo de la posición que ocupen en el Meta4Object y de los nodos con los que estén relacionados, los datos de cada uno de ellos serán diferentes.



Un Meta4Object se compone de nodos y conectores.

En la estructura de un Meta4Object multinodo se distinguen los siguientes conceptos:

- *Nodo raíz*

Se trata del que se va a cargar en primer lugar. Todos los registros del nodo raíz ocupan un solo bloque de memoria.

- *Conectores entre nodos*

Los nodos de un Meta4Object multinodo se relacionan mediante conectores. Éstos determinan los registros que se van a cargar en cada uno de los nodos, según los registros cargados en el nodo con el que están conectados. También permiten conectar métodos para que se ejecuten automáticamente.

- *Nodo principal y subordinado*

Cuando dos nodos se conectan mediante un conector, uno de ellos será el principal y el otro, el subordinado. El principal es el que se carga en primer lugar en memoria.

Conectores

Los conectores relacionan los pares de nodos de un mismo Meta4Object. Gráficamente, los conectores se representan mediante una línea que une dos nodos, y cumplen las siguientes funciones:

- Determinar la relación entre los registros de los nodos que se conectan. Esta relación hace que, cuando se activa un registro específico del nodo principal, se activen los registros del nodo subordinado que cumplen una condición indicada en la definición del conector.



Estas relaciones permiten superar las limitaciones impuestas por el proceso de normalización del modelo de datos y ofrecer una visión unificada de todos los registros de la base de datos que hacen referencia a una misma entidad.

- Determinar cómo se van a organizar en la memoria principal los datos que recuperan los nodos que se conectan.
- Propagar la ejecución de métodos y conceptos.
- Propagar valores entre elementos de tipo propiedad, de forma que cuando se asigne un valor a un elemento, se asigne automáticamente a otro elemento de otro nodo.

Respecto a este segundo punto, durante la fase de diseño de un Meta4Object se puede indicar cómo se deben organizar en la memoria principal los datos que recupera un Meta4Object. El sistema permite distribuir la carga de datos entre dos o más equipos.

Cuando se conectan dos nodos de un Meta4Object, uno de ellos es el nodo principal de la conexión y el otro el subordinado. El principal es el primero que se va a cargar en memoria.

Para seleccionar el nodo que se debe cargar antes, se debe considerar la función que va a cumplir el conector. Se distinguen tres casos:

- *El conector determina la relación lógica entre los registros del nodo principal y los del nodo subordinado.*

Se suele producir al conectar nodos que crean instancias de estructuras de nodo que recuperan datos de dos tablas entre las que existe una asociación definida en el diccionario de datos. Las asociaciones entre tablas se explican en el manual *Metodología de diseño de modelos de datos*.

En toda asociación entre tablas, se distingue entre tabla principal y tabla subordinada. Subordinada es la que puede contener más de un registro que haga referencia a uno de la otra tabla.

EJEMPLO

La tabla Recurso humano muestra información sobre todas las personas y organizaciones que han mantenido una relación con la empresa. Los datos de cada persona u organización se recogen en un registro independiente.

La tabla RH en Periodo refleja las fases de alta de todos los empleados. La información de cada una de estas fases ocupa un registro independiente.

Entre las tablas Recurso humano y RH en Periodo existe una asociación, ya que RH en Periodo contiene un campo que hace referencia a los valores que permiten identificar a cada persona en la tabla Recurso humano. RH en Periodo contiene una clave externa definida sobre la tabla Recurso Humano.

La tabla RH en Periodo puede contener varios registros que hagan referencia a un mismo registro de la tabla Recurso Humano. Recurso humano es la tabla principal de la asociación y RH en Periodo, la tabla subordinada.

Al diseñar un Meta4Object sobre tablas de este tipo, el nodo principal corresponderá al nodo que recupera datos de la tabla principal, y el subordinado, al que los recupera de la tabla subordinada.

EJEMPLO

Se quiere crear un Meta4Object que recoja, para cada empleado, todas sus fases de alta.

El Meta4Object contendrá dos nodos conectados: uno recogerá datos de la tabla Recurso humano y el otro recuperará datos de la tabla RH en Periodo.

Según la relación que se establece entre las dos tablas, el nodo definido sobre Recurso humano será el principal, y el definido sobre RH en Periodo será el subordinado.

- *El conector propaga la ejecución de métodos o conceptos.*

Si el conector se va a crear para propagar la ejecución de métodos de dos nodos diferentes, y no va a cumplir ninguna otra funcionalidad, se recomienda, como norma de estilo, que el nodo principal sea el que contenga el elemento cuya ejecución va a desencadenar la del segundo método.

Siempre que se conectan dos elementos con ejecución, es posible indicar cuál se debe ejecutar antes y cuál después, independientemente de que formen parte del nodo principal o del subordinado de la conexión.

- *El conector propaga los valores de elementos de tipo propiedad o campo.*

El nodo principal será el que contiene el elemento cuyo valor se quiere propagar. El subordinado será el que contiene el elemento que va a heredar el valor.

EJEMPLO

Se crea un Meta4Object con dos nodos. Uno de ellos recoge una serie de valores solicitados al usuario en tiempo de ejecución: nombre de usuario y rol.

El otro nodo se encarga de leer de la base de datos los permisos sobre Meta4Objects asignados al usuario.

Para que el segundo nodo sepa cuál es el nombre de usuario y el rol propuestos en el primer nodo, se crea un conector que permita propagar estos valores en cuanto el usuario los proponga.

El nodo principal de la conexión recoge los valores indicados por el usuario. El nodo subordinado utiliza estos valores para obtener los permisos que le corresponden.

Tipos de conector y elementos de conexión

Es necesario distinguir entre el tipo del conector y los elementos que se conectan.

- *Tipo de conector*

El tipo de un conector determina los bloques de memoria que se van a reservar para el nodo subordinado de la conexión. Los registros que recupera este nodo se distribuyen entre los bloques de memoria reservados.

Los tipos de conectores disponibles son Registro-bloque, Bloque-bloque y Libre. Se utilizará un tipo de conector u otro según el tipo y alcance de los elementos que se conecten. El alcance de los elementos se explica en el capítulo dedicado a las estructuras de nodo.

■ *Elementos que se conectan*

Los conectores están formados por pares de elementos conectados. Cada uno de ellos conectará un elemento del nodo principal con uno del hijo.

Las conexiones entre pares de elementos pueden utilizarse para:

- Fijar el conjunto de registros activos en el bloque de memoria reservado para el nodo subordinado, según registro activo en el nodo principal.
- Desencadenar la ejecución de elementos con ejecución de nodos diferentes (métodos y conceptos).

EJEMPLO

Se va a diseñar un Meta4Object multinodo que extraiga de la base de datos información de las tablas RH en Periodo y Discapacidad RH. Se quieren recuperar todas las fases de alta activas, así como los periodos de discapacidad que se hayan producido en cada una de ellas. El Meta4Object tendrá dos nodos, uno para cada tabla, relacionados mediante un conector.

Para optimizar la distribución de recursos en memoria, los dos nodos se diseñan sobre dos estructuras de nodo diferentes.

El nodo principal de la conexión recoge las fases de alta activas. El subordinado incluye los absentismos.

El conector es de tipo Registro-bloque. De esta forma, todos los registros del nodo subordinado (Absentismos) correspondientes a un empleado y una fase de alta determinados, se sitúan en un mismo bloque de memoria. Los registros correspondientes a fases de alta diferentes ocupan bloques de memoria distintos.



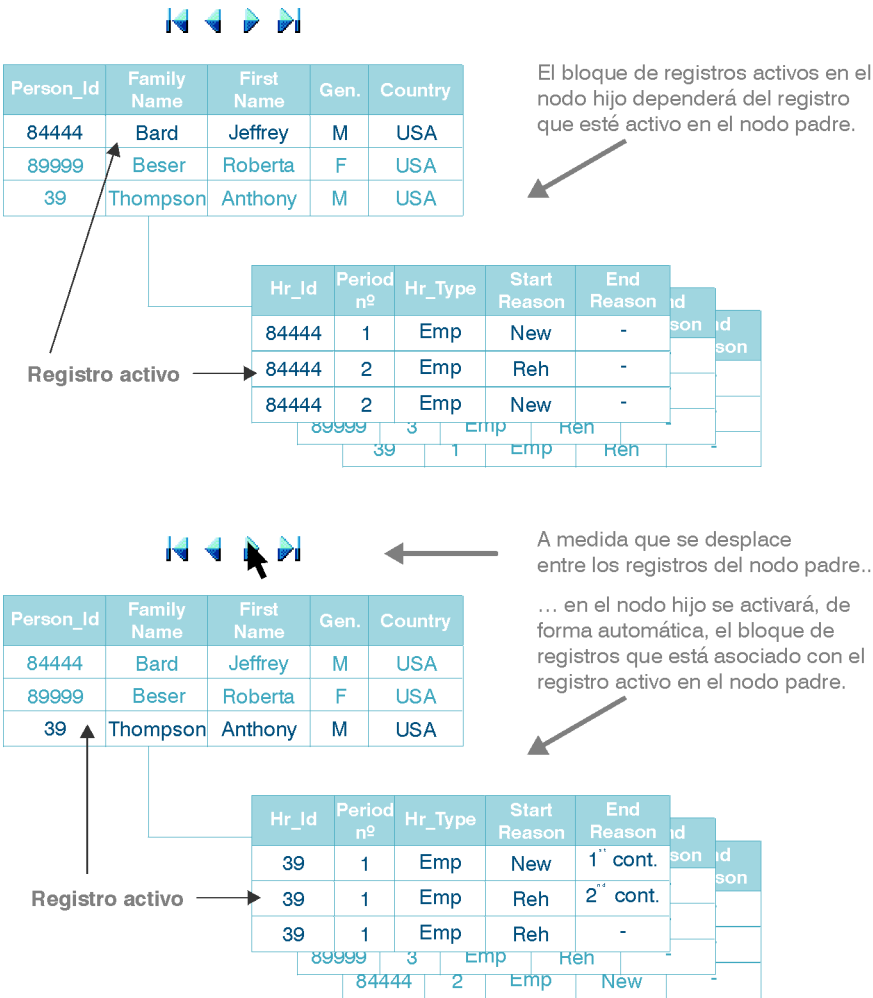
Para poder relacionar estos dos elementos mediante un conector Registro-bloque, los elementos deben tener alcance a nivel de registro. De no ser así, se producirá un error en tiempo de ejecución.

Para que el sistema sepa qué bloque de registros del nodo subordinado se corresponde con cada fase de alta, se conecta el elemento ID_RH de los dos nodos. ID_RH identifica de forma inequívoca a cada empleado.

Si el puntero que recorre el nodo principal está situado en el registro correspondiente a la fase de alta del empleado 01 con fecha de inicio 01-ENE-1999, en el nodo subordinado se activarán los registros correspondientes a los absentismos registrados para ese empleado y fecha de alta.

Si se sitúa el puntero en el siguiente registro, correspondiente al empleado 02 con fecha de inicio 13-JUN-1999, en el nodo subordinado se activarán los registros correspondientes a los absentismos registrados para esta fase de alta.

Figura 4. Desplazamiento en Meta4Objects multinodo



Funcionalidad de los Meta4Objects

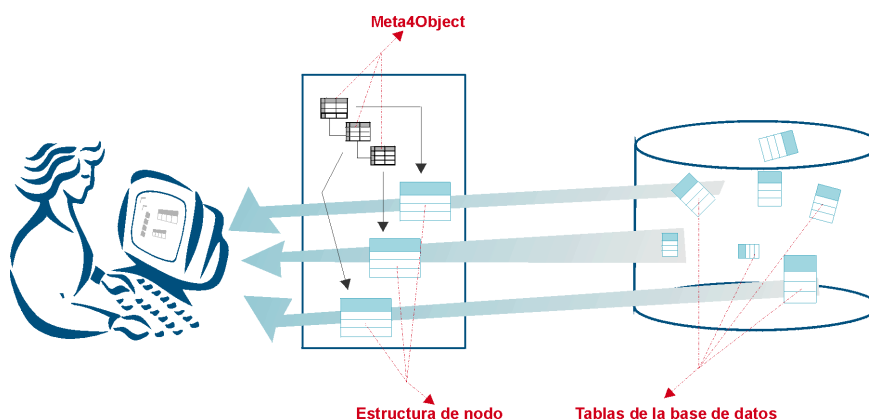
Los Meta4Objects cumplen las siguientes funciones:

- Actúan como interfaz entre la base de datos y la aplicación.
- Ofrecen una percepción unificada de los datos correspondientes a una misma entidad.
- Definen la lógica de la aplicación y el procesamiento que debe hacerse con los datos mediante un lenguaje de programación propio: LN4.
- Ofrecen múltiples visualizaciones de los datos que facilitan el análisis de la información en los procesos de toma de decisiones.
- Implementan mecanismos de seguridad que determinan qué datos van a ser accesibles para los distintos usuarios de la aplicación y qué instrucciones de procesamiento se van a poder ejecutar.

Interfaz datos-aplicación

Los Meta4Objects recuperan información de la base de datos de forma estructurada, y constituyen la única interfaz válida entre la información de la base de datos y los procesos que admite la aplicación.

Figura 5. Los Meta4Objects actúan como interfaz con la base de datos



Siempre que se quiera leer, agregar, actualizar o borrar registros de la base de datos, debe hacerse a través de un Meta4Object.

Como se ha indicado anteriormente, al cargar en memoria un Meta4Object, el sistema crea una imagen de las tablas sobre las que se ha definido.

El usuario puede trabajar con el Meta4Object como si se tratara de una tabla convencional, creando registros, actualizando valores, etc. Todos estos

cambios permanecerán en la memoria principal hasta que decidan grabarse los cambios. De la misma forma, en cualquier momento se podrán actualizar los registros cargados en memoria, para que muestren el contenido real de las tablas de la base de datos.

Para sincronizar los cambios realizados en los registros de un Meta4Object con el contenido de los campos de la base de datos, la aplicación reconoce la equivalencia entre los registros y las columnas de las tablas, así como la información que se ha cargado en la memoria principal.

Control de integridad y subsistema de bloqueos

Durante una sesión de trabajo, es posible que haya varios usuarios trabajando con un mismo Meta4Object. En este caso, el servidor de aplicaciones genera una copia en memoria diferente para cada usuario. Estas copias en memoria se denominan *instancias*.

Dos usuarios pueden cargar en memoria el mismo Meta4Object, sin que el sistema imponga restricción alguna. Cada uno puede realizar cambios en la memoria principal, aunque se estén haciendo cambios simultáneamente sobre los mismos datos.

El usuario que graba en primer lugar es el que valida los cambios que ha realizado. Cuando los demás usuarios que estaban trabajando con otras instancias del mismo Meta4Object intenten grabar sus cambios, y éstos afecten a los datos modificados por el primer usuario, el sistema les avisará de que otro usuario ha modificado los datos de ese Meta4Object, y no tendrán efecto los cambios.

Percepción unificada de los datos

La aplicación utiliza una base de datos relacional normalizada. La normalización de un modelo de datos permite minimizar la redundancia de datos sin que se produzcan pérdidas de información.

Como resultado de la normalización, la información correspondiente a una misma entidad del mundo real se distribuirá en un número variable de tablas.

EJEMPLO

Los datos correspondientes a un proceso de selección se encuentran en tablas diferentes:

- Una tabla contiene las características generales del proceso: fecha de inicio, de fin, código identificativo del supervisor, número de candidatos entrevistados, número de plazas ofertadas, etc.
 - Una segunda tabla registra las plazas ofertadas en el proceso.
 - Otra tabla recoge los candidatos que se han entrevistado, etc.
-

Para poder disponer de toda la información relativa a una misma entidad (por ejemplo, un proceso de selección), es posible diseñar un Meta4Object que la recupere de todas las tablas entre las que está distribuida.

Al cargar el Meta4Object, se recuperan en la memoria principal todos los datos correspondientes a una misma entidad, independientemente de la tabla en la que estén registrados.

Así, los Meta4Objects hacen transparente a los usuarios la estructura real de las tablas del modelo de datos y ofrecen visiones unificadas de la información.

Definición de la lógica de los procesos

Los Meta4Objects encapsulan datos e instrucciones de procesamiento. Las instrucciones de procesamiento se codifican en los elementos de tipo métodos y conceptos mediante el lenguaje LN4. Además de código LN4, los métodos pueden contener llamadas a código C++ o a métodos de librerías de enlace dinámico.

LN4 ofrece un amplio margen de posibilidades al usuario avanzado. No sólo se pueden realizar operaciones de lectura, escritura y navegación entre registros y bloques de memoria de los Meta4Objects; también se puede solicitar la carga en memoria de otros Meta4Objects, leer y escribir sus registros, ejecutar sus métodos, etc.

En resumen, LN4 se caracteriza por una gran capacidad expresiva. Cualquier funcionalidad se puede desarrollar fácilmente mediante las posibilidades de cálculo y acceso a datos que ofrece este lenguaje.

Al definir un proceso, se pueden utilizar distintos Meta4Objects, cada uno con sus propios datos e instrucciones de procesamiento.

EJEMPLO

Un proceso de selección se compone de distintas etapas: abrir el proceso, registrar las plazas que se quieren cubrir, buscar candidatos en la base de datos, registrar las notificaciones enviadas a los candidatos que se vayan a entrevistar, contabilizar los costes y cerrar el proceso de selección cuando se hayan cubierto las plazas.

En la modelización de este proceso, se diseñan distintos Meta4Objects que permiten registrar toda la información necesaria en las tablas de la base de datos.

El Meta4Object que implementa el cierre del proceso contiene instrucciones para ejecutar determinadas operaciones, como totalizar los costes del proceso de selección y grabar el resultado en una tabla de la base de datos. El proceso de totalización debe leer los costes registrados en una tabla diferente (a través del Meta4Object al que está asociado), realizar los cálculos y grabar el resultado en una segunda tabla a la que se tiene acceso a través de un Meta4Object distinto.

De esta forma, es necesario que el Meta4Object que implementa el cierre de los procesos de selección tenga acceso a datos e instrucciones de procesamiento de otros Meta4Objects.

La posibilidad de establecer llamadas entre Meta4Objects permite reutilizar su definición, y evita la duplicación de componentes que sean comunes a distintos módulos funcionales.

Visualización de datos

Los datos recuperados por un Meta4Object se pueden ver desde distintas perspectivas: presentaciones tabulares, formularios convencionales y de edición rápida, etc. Esta capacidad de la aplicación facilita el análisis e interpretación de los datos.

La capacidad de mostrar la información recuperada por un Meta4Object mediante distintos modos de visualización la implementa el software cliente de la aplicación.

Las opciones de visualización disponibles dependen del tipo de tabla de la que el Meta4Object extrae los datos. Hay dos modos de visualización: formulario y tabla. Las opciones de inserción, borrado y actualización de registros están disponibles con ambos modos.

Formulario

Es el modo de visualización por defecto. Está disponible para todos los Meta4Objects, cualquiera que sea el tipo de la tabla de la que extraen datos.

Presenta un registro por pantalla. El usuario se desplaza por los registros disponibles en memoria mediante la barra de navegación de la interfaz cliente o mediante las funciones de navegación del lenguaje LN4.

La siguiente ventana muestra un ejemplo de este modo de visualización:

Figura 6. Visualización como formulario

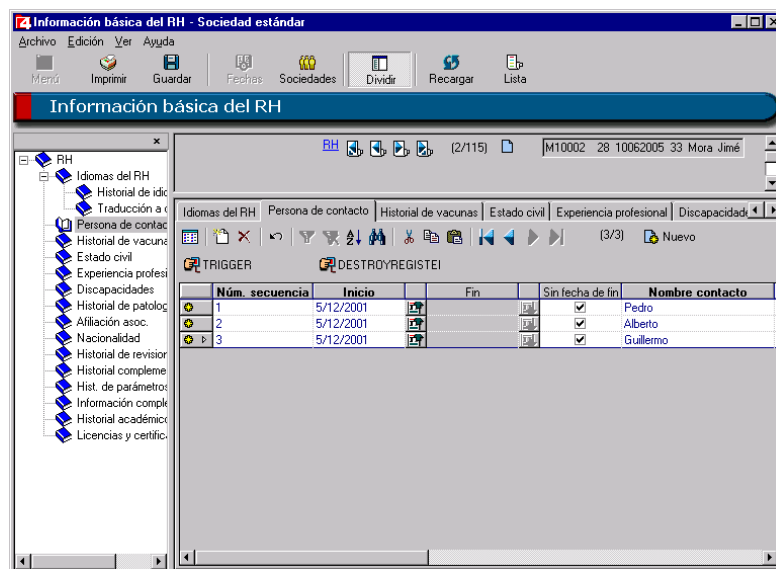
The screenshot shows a software window titled "Información básica del RH - Sociedad estándar". The window has a menu bar with "Archivo", "Edición", "Ver", and "Ayuda". Below the menu bar is a toolbar with icons for "Menú", "Imprimir", "Guardar", "Fechas", "Sociedades", "Dividir", "Recargar", and "Lista". The main area of the window is divided into two panes. The left pane is a tree view showing a hierarchy of data fields under the "RH" node, including "Idiomas del RH", "Historial de id", "Traducción a", "Persona de contac", "Historial de vacune", "Estado civil", "Experiencia profesi", "Discapacidades", "Historial de patologi", "Afiliación asoc.", "Nacionalidad", "Historial de revisio", "Historial compleme", "Hist. de parámetros", "Información compl", "Historial académic", and "Licencias y certific". The right pane displays a form for a specific record. The form has a title bar "Información básica del RH" and a toolbar with icons for "SSP_VALIDAR_NIF" and "Formatear el nº de SS". The form contains several input fields: "ID HR" (M10001), "Comentario" (A), "Num Provincia SS" (28), "Numero SS" (10062004), "Digito SS" (32), "Nombre" (Antonio), "Apellidos" (Toledo Nieto), and "2º Apellido" (Nieto). At the bottom of the form, there are five tabs: "Idiomas del RH", "Persona de contacto", "Historial de vacunas", "Estado civil", and "Experiencia profesional".

Tabla

Muestra una tabla con todos los registros cargados en un bloque de memoria. Cada columna de la tabla se corresponde con un campo del Meta4Object, y cada fila con un registro.

Este modo de visualización está disponible para todos los nodos de los Meta4Objects, cualquiera que sea el tipo de la tabla de la que extraen datos.

Figura 7. Visualización como tabla



Seguridad

Los Meta4Objects constituyen la interfaz entre el nivel de aplicación y la base de datos, e implementan la lógica de la aplicación mediante código ejecutable LN4.

Por lo tanto, es necesario establecer mecanismos de seguridad que impidan la lectura, inserción o actualización de registros a través de Meta4Objects a los usuarios no autorizados.



El sistema de seguridad de la aplicación permite definir las restricciones de seguridad a nivel de tabla, estructura de nodo y Meta4Object. Si se producen conflictos entre los permisos concedidos sobre estructuras de nodo y tablas, o entre estructuras de nodo y Meta4Objects, el sistema siempre aplicará los permisos más restrictivos.

La seguridad se define mediante máscaras. Una máscara es un conjunto de permisos concedidos sobre un Meta4Object y sobre sus componentes: estructuras de nodo, métodos ejecutables, etc.

Se puede definir un número ilimitado de máscaras para un mismo Meta4Object. De esta forma, distintos grupos de usuarios podrán trabajar sobre un mismo Meta4Object con máscaras diferentes.

EJEMPLO

Un Meta4Object tiene dos máscaras de seguridad diferentes.

La primera máscara permite leer todos los registros recuperados por el Meta4Object y ejecutar los métodos definidos.

La segunda máscara restringe el conjunto de registros visibles a aquellos registros que cumplan unas condiciones específicas y no deja ejecutar ningún método.

Los distintos grupos de usuarios utilizarán una máscara u otra para trabajar con la información del Meta4Object.

Presentaciones

Al ejecutar un Meta4Object, se cargan datos en la memoria principal. Sin embargo, estos datos (así como los métodos definidos en la estructura de nodo del Meta4Object) no se verán en pantalla si no van acompañados de una presentación.

Las presentaciones indican los elementos del Meta4Object que se deben mostrar en pantalla y el formato que deben tener: posición, formato, etc.

Todos los Meta4Objects que tengan que verse en pantalla deben contar con una presentación. Para facilitar la creación de presentaciones, la aplicación puede generar automáticamente presentaciones por defecto. El usuario avanzado puede editar las presentaciones por defecto y hacer todos los cambios que considere necesarios.

Las herramientas de desarrollo de la aplicación también permiten diseñar presentaciones que obtengan datos de distintos Meta4Objects y los presenten de forma unificada.

La estructura de la presentación se guarda en las tablas del repositorio de la aplicación, en forma de archivos de texto plano.

Cuando se ejecuta una opción de menú desde el panel de navegación de la aplicación, se solicita al servidor de aplicaciones la carga de una presentación. Este servidor transfiere al puesto cliente el archivo de texto con el formato de la presentación y carga en memoria los datos que recupera el Meta4Object.



El usuario puede solicitar progresivamente la presentación de los registros mediante las opciones de la barra de navegación.

A partir del contenido del archivo de presentación, la interfaz cliente de la aplicación dibuja la presentación en pantalla y muestra los datos a medida que el usuario los solicita.

Trabajar con Meta4Objects

La aplicación incluye un conjunto de herramientas de desarrollo que permiten al usuario avanzado:

- Editar y modificar la definición de Meta4Objects existentes
- Crear nuevos Meta4Objects para implementar funcionalidades adicionales a las de la versión estándar
- Crear estructuras de nodo reutilizables en la definición de más de un Meta4Object
- Optimizar el rendimiento del sistema y racionalizar la ejecución de procesos en arquitecturas cliente/servidor *multinivel*
- Automatizar la creación de Meta4Objects a partir de la definición de tablas
- Diseñar informes y representaciones gráficas con la información que recuperan los Meta4Objects
- Definir la seguridad de la aplicación a nivel de Meta4Object, además de conceder permisos y privilegios a usuarios o grupos de usuarios

En los siguientes apartados se explica la relación entre las herramientas de desarrollo de tecnología Meta4 y los Meta4Objects.

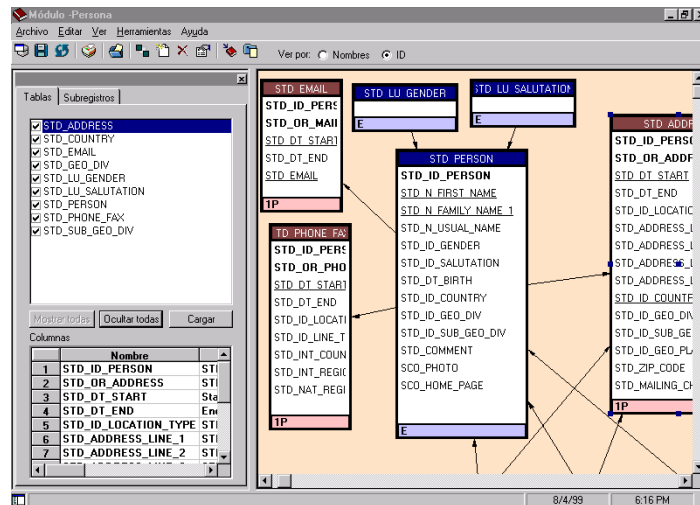
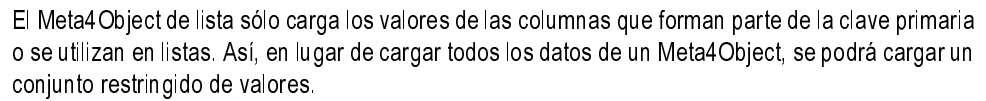
Diseñador del modelo de datos

Permite crear tablas y diseñar el modelo de datos que admitirá la lógica de la aplicación.

Además, con esta herramienta se pueden crear Meta4Objects mediante un asistente (*wizard*). Permite:

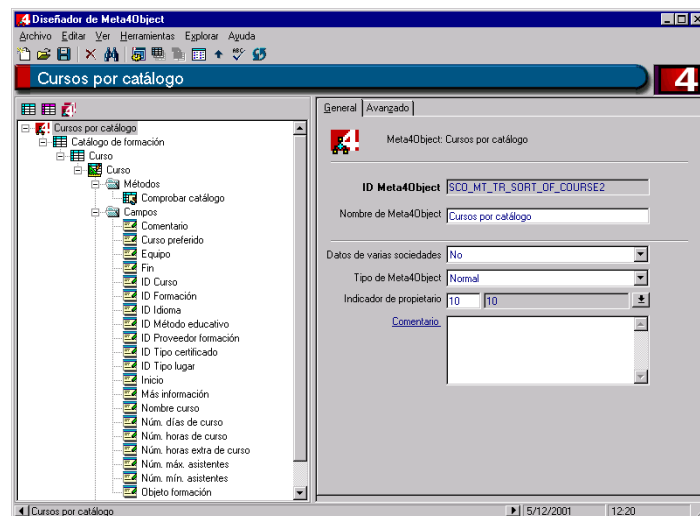
- Crear un Meta4Object mononodo para cada tabla que se diseña desde esta herramienta.
- Crear un Meta4Object multinodo: cada nodo toma como base una de las tablas incluidas en el mismo módulo. El Diseñador del modelo de datos genera una estructura de nodo para cada uno de los nodos.

En el caso de los Meta4Objects mononodos, el Diseñador de modelo de datos genera dos Meta4Objects: el de mantenimiento y el de lista. El de mantenimiento cuenta con un elemento por cada columna de la tabla. El de lista contiene un elemento por cada columna que forma parte de la clave primaria o se muestra en los listados de selección.



Diseñador de Meta4Objects

Permite diseñar Meta4Objects a partir de tablas y estructuras de nodo existentes.



Desde este diseñador, el usuario avanzado puede:

- Diseñar estructuras de nodo a partir de tablas existentes en la base de datos
- Reutilizar estructuras de nodo definidas con anterioridad por el mismo usuario, o por otros, de la aplicación



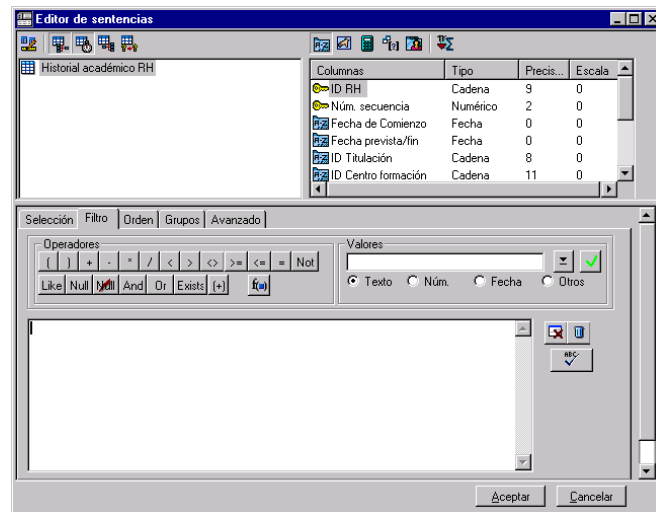
Las estructuras de nodo que se crean desde esta herramienta siempre están contextualizadas en un Meta4Object, en sus nodos. Es decir, la estructura de nodo se puede reutilizar en Meta4Objects creados posteriormente, pero esta herramienta no permite crear una estructura de nodo que sea independiente de un Meta4Object.

- Crear Meta4Objects formados por uno o varios nodos, y definir los conectores entre nodos con sus correspondientes propiedades
- Escribir el código LN4 de los elementos de tipo método o concepto de una estructura de nodo
- Parametrizar el comportamiento en entornos cliente/servidor del Meta4Object: carga de datos y ejecución distribuida, reutilización de instancias de Meta4Objects cargadas con anterioridad, etc.
- Diseñar máscaras de seguridad para el Meta4Object
- Generar una representación IDL acorde con las especificaciones DCOM, que permitan obtener acceso a los métodos del Meta4Object desde otras aplicaciones basadas en la orientación a objetos y acordes con estas especificaciones
- Diseñar gráficos e informes que tomen como base los datos recuperados por un Meta4Object

Editor de sentencias

Permite crear estructuras de nodo a partir de las tablas disponibles en la base de datos. Ofrece la capacidad expresiva del lenguaje SQL.

Figura 10. Editor de sentencias

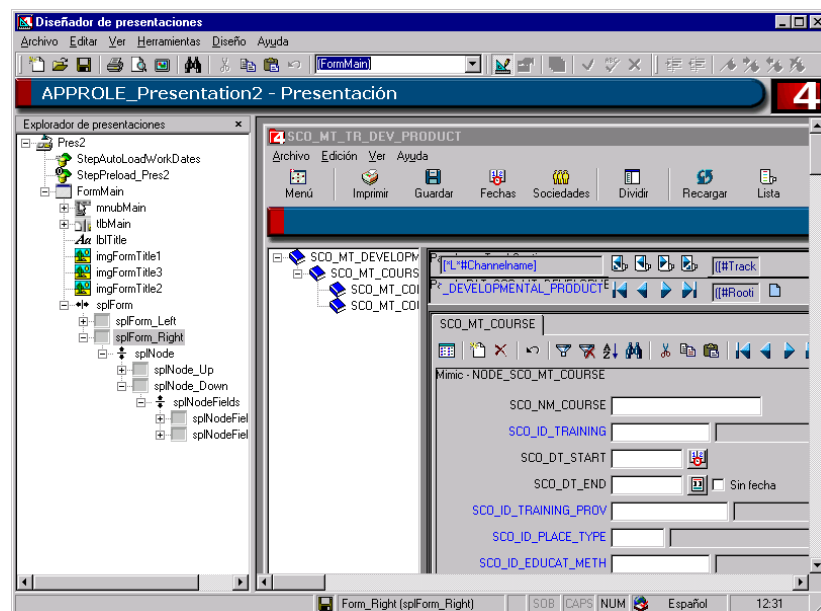


Diseñador de presentaciones

Permite crear presentaciones para los Meta4Objects diseñados desde el Diseñador del modelo de datos o desde el Diseñador de Meta4Objects.

Esta herramienta es capaz de generar presentaciones por defecto de forma automática y ofrece al usuario avanzado la posibilidad de crear presentaciones complejas.

Figura 11. Diseñador de presentaciones



El Diseñador de presentaciones funciona de forma similar a los diseñadores de ventanas de las herramientas de programación visual. El diseñador dispondrá de un conjunto de controles que podrá agregar a la presentación: texto estático, casillas y cuadros de texto editables, subformularios html, botones, etc.

Estos controles se podrán asociar a elementos de uno o varios Meta4Objects; por ejemplo, los métodos se pueden representar mediante botones, los campos mediante cuadros de texto, etc.

Para poder diseñar una presentación para un Meta4Object, es necesario que éste se haya creado con anterioridad desde el Diseñador de Meta4Objects o desde el Diseñador del modelo de datos.

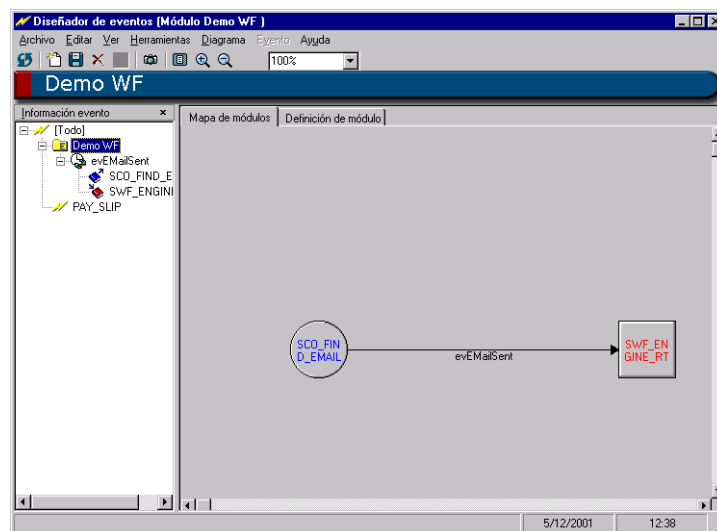
Diseñador de eventos

Esta herramienta automatiza la ejecución de métodos de distintos Meta4Objects.

Un evento está formado por un conjunto de métodos origen y métodos destino, de forma que, cuando se ejecute uno de los métodos declarados origen del evento, se ejecutarán uno tras otro todos los métodos destino.

El Diseñador de eventos utiliza métodos de Meta4Objects ya definidos.

Figura 12. Descripción gráfica de un evento



Diseñador de informes

Esta herramienta facilita la creación de informes e impresos que muestran la información registrada en la base de datos de la aplicación.

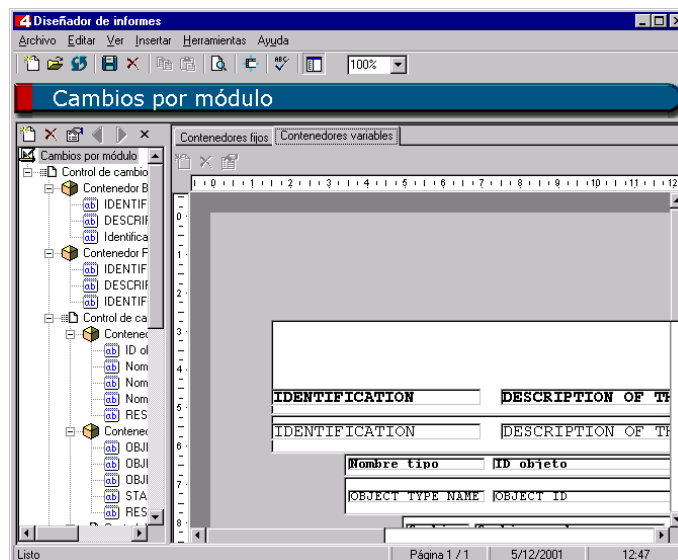


Los informes e impresos se diseñan siempre a partir de un Meta4Object que recupera toda la información que se va a mostrar en el informe.

La creación de un informe debe hacerse en dos fases:

1. Se crea un Meta4Object que extraiga la información que se quiere imprimir en el informe.
2. Se indica cómo se deben presentar estos datos en el informe: secciones, cabeceras, columnas, subsecciones, etc.

Figura 13. Diseñador de informes



El Meta4Object con los datos del informe se puede crear con la herramienta Consulta o desde el Diseñador de Meta4Objects. La presentación de los datos y su distribución en secciones se establece en el Diseñador de informes.

Diseñadores de seguridad

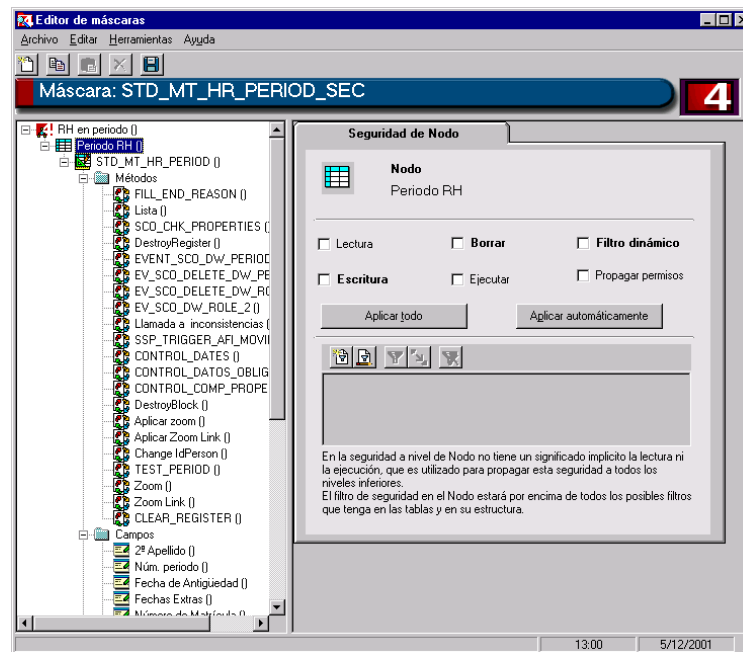
Estas herramientas, dirigidas a los gestores de la aplicación, permiten definir las restricciones de seguridad que se van a aplicar sobre los distintos elementos de la aplicación: tablas, estructuras de nodo, Meta4Objects, modelos de proceso, etc.

Hay tres herramientas para fijar la seguridad de la aplicación:

Módulos	Descripción
Gestor de usuarios de la aplicación	Facilita el mantenimiento de los usuarios de la aplicación: alta y baja de usuarios, indicación de si el usuario puede o no cambiar su contraseña, asignación de perfiles de seguridad por defecto, etc.
Gestor de roles de la aplicación	Permite asignar permisos para ejecutar tareas a los usuarios incluidos en un mismo rol de aplicación. Las tareas son acciones predefinidas ejecutables desde la interfaz de la aplicación. Por ejemplo: la ejecución de un informe, la grabación de un registro en un formulario, etc.
Gestor de MSR	Permite agrupar conjuntos de permisos concedidos sobre tablas y sobre Meta4Objects. Los permisos correspondientes a un modelo de sistema de roles (MSR) se asignan a roles de aplicación, de forma que todos los usuarios con el mismo rol disponen de los permisos que se hayan indicado en el MSR asociado a dicho rol.
Gestor de MSR de desarrollo	Permite visualizar o asignar permisos de seguridad sobre proyectos. Muestra las propiedades identificativas de todos los MSR de desarrollo definidos en el sistema.

Los permisos sobre tablas y Meta4Objects se definen en el módulo MSR de aplicación. Se pueden conceder permisos de lectura, escritura y modificación sobre los elementos de tipo campo y propiedad, así como de ejecución sobre los elementos de tipo método.

Figura 14. Diseñador de MSR: editor de máscaras



Conectividad

Las aplicaciones desarrolladas utilizando la tecnología Meta4 pueden actuar como cliente y como servidor de aplicaciones DCOM. Esta funcionalidad permite:

- Tener acceso a datos gestionados por otras aplicaciones desde el código LN4 de los elementos de tipo método y concepto. En este caso, se dice que la aplicación actúa como cliente de Meta4Objects.
- Tener acceso a datos gestionados por las aplicaciones desarrolladas por Meta4 desde otras aplicaciones. En este caso, se dice que la aplicación actúa como servidor de Meta4Objects.

EJEMPLO

La integración permite recoger datos recuperados de la base de datos a través de un Meta4Object para, posteriormente, cargarlos en un libro de Excel, generar circulares en Word y enviarlas por correo electrónico a través de Microsoft Exchange o Lotus Notes.

Cliente DCOM

La aplicación puede ejecutar llamadas a aplicaciones servidor DCOM (por ejemplo Microsoft Excel, Microsoft Word, Microsoft Project o Lotus Notes).



Las librerías de enlace dinámico OLEEXEC.DLL se encargan de gestionar la comunicación entre clientes y servidores de Meta4Objects.

En el caso de DCOM, las aplicaciones servidor presentan una interfaz pública (conjunto de propiedades y métodos) a la que se puede tener acceso desde un cliente DCOM (en nuestro caso, desde la aplicación en reglas LN4).

Para ello se ha creado un Meta4Object que asigna sus propiedades y métodos públicos para cada aplicación servidor DCOM.

Si desea utilizar la interfaz de una aplicación servidor DCOM, lo hará mediante referencias a los métodos y propiedades de los Meta4Objects que se han diseñado para actuar como interfaz con estas aplicaciones, denominados *Meta4Objects DCOM Automation*.

La versión estándar de la aplicación ofrece interfaces prediseñadas para interactuar con Microsoft Word, Excel, MAPI (Messaging Application Program Interface, desarrollado por Microsoft), Project 98 y Lotus Notes.

Aparte de esto, los desarrolladores pueden diseñar nuevos Meta4Object DCOM Automation que integren otras aplicaciones servidor DCOM, de forma que se pueda tener acceso a ellas desde el código LN4.

Servidor de objetos

La tecnología Meta4 también actúa como servidor de objetos DCOM. Ofrece una interfaz exterior de los Meta4Objects configurados para ello.

Para configurar un Meta4Object, con objeto de que sea mostrado a aplicaciones clientes DCOM, siga estos pasos:

1. Cree el Meta4Object y active las propiedades **Pública** y **Visible** de los elementos que se quieran mostrar al exterior.
2. Generar el IDL desde el Diseñador de Meta4Objects. El IDL es un archivo de texto en el que se describe la estructura de un objeto.

Existe un tipo de archivo IDL únicamente para DCOM. Por defecto, los IDL que genera el programa se guardan en el directorio *Unidad local:\TEMP* del puesto cliente, en un archivo que tiene como nombre el código identificativo del Meta4Object y como extensión .IDL.

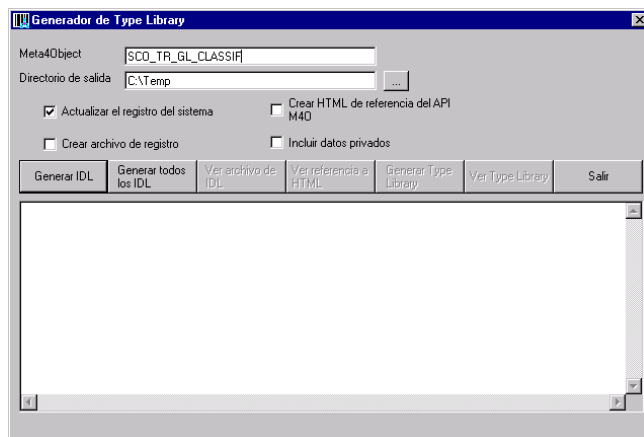
El generador de IDL también genera documentación HTML con la relación de propiedades y métodos que expone el Meta4Object a las aplicaciones cliente.

3. Para generar un elemento al que se pueda tener acceso desde un cliente DCOM, hay que compilar el código IDL para crear un archivo compilado TLB (Type Library, librería de tipos). Esto se puede hacer desde la aplicación siempre que se tenga instalado Microsoft Visual C++. El módulo de Visual C++ que compila IDL es el programa MIDL.

Además de generar el TLB, el sistema genera un archivo .REG que permite registrar el Meta4Object en el registro del cliente Windows. Siempre que se modifique un Meta4Object habrá que generar su IDL, volver a compilarlo y registrarlo. Los archivos TLB y .REG se guardan en el directorio *Unidad local:\TEMP* y tienen como nombre el código identificativo del Meta4Object y como extensión, respectivamente, .TLB y .REG.

La aplicación incluye un conjunto de Meta4Objects para los que ya se ha generado su IDL y que han sido compilados. Estos Meta4Objects reciben el nombre de *M4APIS*.

Figura 15. Generador de IDL



Los M4APIS no son exactamente los Meta4Objects, sino las interfaces que permiten el acceso a dichos Meta4Objects desde aplicaciones externas.

Estructuras de nodo: conceptos básicos

3

Acerca de este capítulo

En este capítulo se explica la composición y funcionalidad de las estructuras de nodo. Las estructuras de nodo son elementos reutilizables en distintos Meta4Objects que definen:

- La información que se quiere extraer de la base de datos
- Las instrucciones de procesamiento que se ejecutan con esos datos

Los nodos de un Meta4Object se definen a partir de una estructura. Se puede decir que el nodo es una instancia de la estructura de nodo o qué instancia a la estructura de nodo.

En los siguientes apartados, se suministra información sobre:

- El concepto y la funcionalidad de las estructuras de nodo
- La creación de instancias de estructuras de nodo en nodos de Meta4Objects
- La estructura de un nodo, con distinción entre:
 - La sentencia de recuperación de datos
 - Las instrucciones de procesamiento codificadas en métodos y conceptos
- La ejecución automatizada de elementos de tipo método y concepto mediante disparadores (*triggers*)

Concepto y funcionalidad

Las estructuras de nodo actúan de interfaz entre la base de datos y el nivel de aplicación de la tecnología Meta4.



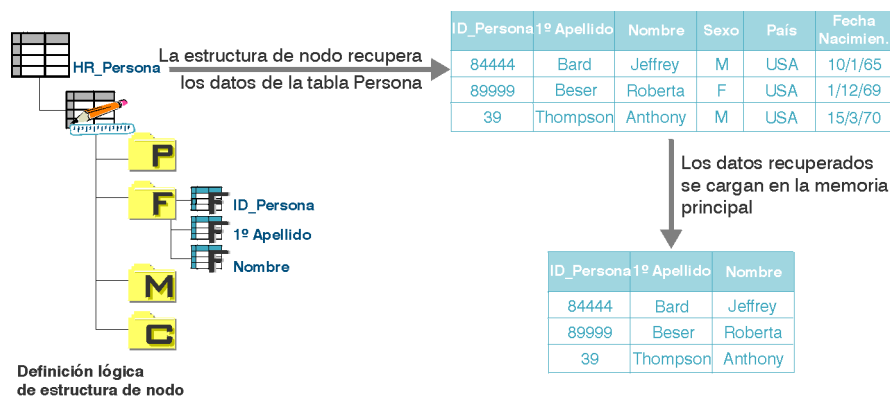
En el diseño de una estructura de nodo se puede indicar si las columnas que se recuperan de la base de datos tienen que cargarse en la memoria principal del puesto de trabajo cliente o en la del servidor de aplicaciones. La arquitectura cliente/servidor de la tecnología Meta4 se explica en el capítulo *Arquitectura cliente/servidor*.

Una estructura de nodo equivale a una sentencia SQL que se lanza contra el sistema gestor de bases de datos y recupera un conjunto de datos en memoria principal.

La estructura de nodo también puede incluir elementos que contienen código ejecutable escrito en lenguaje LN4, C++ o llamadas a métodos de archivos de enlace dinámico.

Cada elemento inicia un pequeño programa en el momento en que se ejecuta. El código de un elemento puede hacer llamadas a otros elementos.

Figura 16. Estructura de nodo: recuperación de datos desde la base de datos hasta la memoria principal



La utilización de las estructuras de nodo como interfaz entre la base de datos y el nivel de aplicación ofrece las siguientes ventajas:

- Aísla el nivel lógico y el procesamiento de datos de su situación física. Una estructura de nodo puede recuperar datos de distintas tablas que pertenecen a bases de datos diferentes.

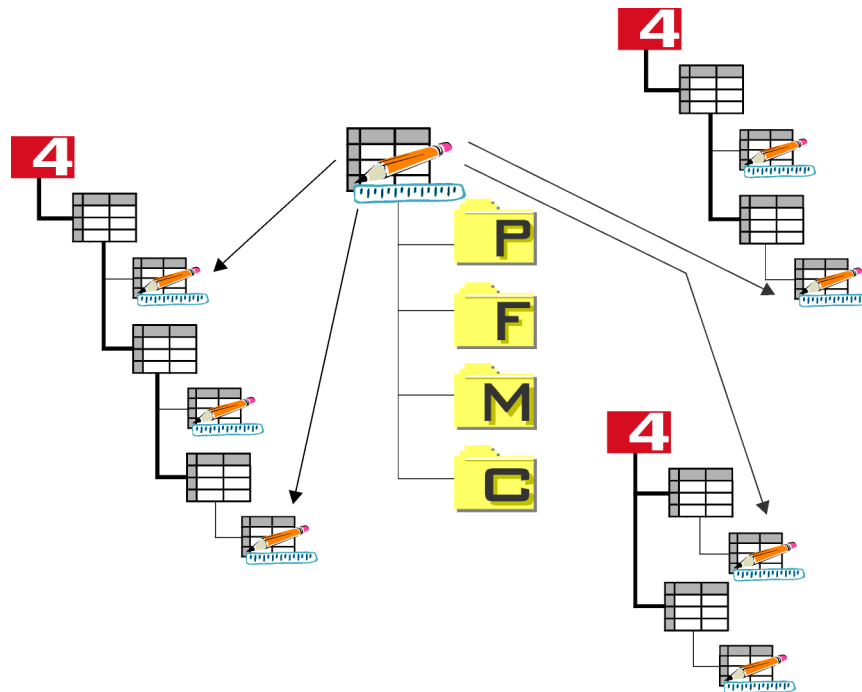
-
- Ejecuta con mayor rapidez los procesos, como cálculo de nóminas, beneficios, etc. Todos los datos necesarios para procesar los cálculos se gestionan en memoria principal, mediante estructuras de datos optimizadas.
 - Minimiza los costes de actualización y de desarrollo de nuevas funcionalidades. Puede utilizar la estructura de nodo en varios procesos.

Creación de instancias y ejecución de estructuras de nodo

Un Meta4Object está formado por uno o varios nodos. Cada nodo crea una instancia de una estructura de nodo. Es decir, el nodo recupera los datos y tiene los métodos y conceptos indicados en la definición de la estructura de nodo.

Las estructuras de nodo son reutilizables: se puede utilizar una estructura de nodo en varios nodos del mismo Meta4Object o de distintos Meta4Objects.

Figura 17. Reutilización de una estructura de nodo



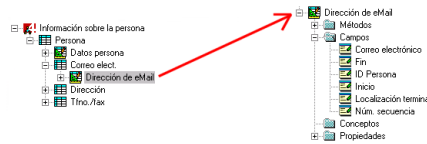
Cuando se solicita la carga en memoria de un nodo de un Meta4Object, se está solicitando la ejecución de la estructura de nodo sobre la que se ha definido.

Las estructuras de nodo se ejecutan siempre que:

- Se hace doble clic por primera vez en el nodo de un Meta4Object desde el panel de navegación de la interfaz cliente de la aplicación, y está activada el **Modo de carga**.
- Se hace una llamada a un elemento de un nodo desde una regla LN4, o desde cualquier aplicación externa que pueda tener acceso a las aplicaciones desarrolladas por Meta4, tales como Meta4 PeopleNet o Meta4 KnowNet.

- A continuación, el usuario solicita la carga de los registros del nodo **Correo elect.**; para ello, hace clic en el icono del panel de navegación. Este nodo se ha definido a partir de la estructura de nodo STD_MT_EMAIL.

2



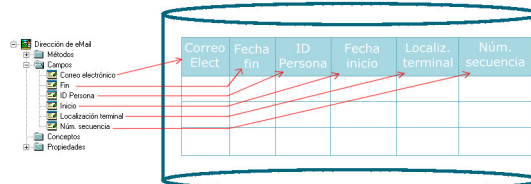
El nodo Correo Elect. instancia a la estructura de nodo Dirección de eMail



La ejecución de una estructura de nodo implica la lectura de datos de la base de datos y su carga en forma de estructura optimizada en la memoria principal.

- El sistema ejecuta la sentencia asociada a la estructura de nodo contra la base de datos.

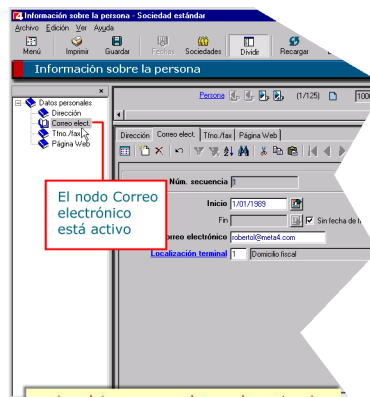
3



Se ejecuta la estructura de nodo Dirección de eMail y se lanza una sentencia SQL contra el SGBD. La estructura de nodo recupera los datos de la tabla con el mismo nombre

- Los datos se cargan en la memoria principal y se visualizan en pantalla. El usuario puede recorrer los registros del nodo mediante las opciones de la barra de navegación.

4



Los datos recuperados por la sentencia se cargan en memoria y el usuario puede trabajar con ellos

Componentes de una estructura de nodo

Una estructura de nodo se compone de:

- La sentencia que permite extraer registros de una o varias tablas de la base de datos cuando se solicita la ejecución de la estructura de nodo.
- Las instrucciones que procesan los registros extraídos por la sentencia anterior.



Una estructura de nodo puede incluir estos dos componentes o sólo uno de ellos. Así, hay estructuras de nodo que sólo recuperan datos y no incluyen ninguna instrucción de procesamiento, y estructuras de nodo que sólo contienen instrucciones de procesamiento.

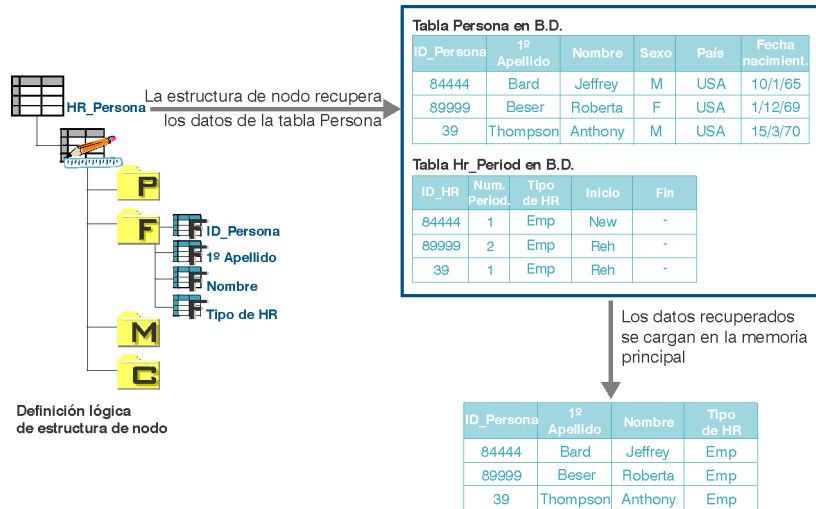
Sentencia de recuperación de datos

Cuando se ejecuta una estructura de nodo, se recupera el conjunto de registros que cumplen las condiciones indicadas en su sentencia. Si la definición de la estructura incluye un filtro de datos, únicamente se recuperarán los registros que cumplan las condiciones indicadas en él.

Para indicar qué datos se tienen que recuperar de los campos de la base de datos, la definición de la estructura de nodo debe incluir la siguiente información:

- La tabla o tablas de la base de datos de la que se quieren extraer los registros. El término *tabla base* hace referencia a esta tabla. Si la estructura de nodo recupera datos de más de una tabla, la tabla base será la primera que haya seleccionado el diseñador.
- Las columnas de las tablas anteriores cuyo valor se quiere recuperar. Estas columnas se denominan *elementos de tipo columna*.
- Las condiciones que deben cumplir los registros para poder recuperarlos. El término *filtro* de la estructura de nodo hace referencia a este conjunto de condiciones.
- Los criterios que se utilizan para ordenar los registros. Los registros recuperados por una estructura de nodo se pueden ordenar de forma ascendente o descendente, a partir de los valores de una o varias columnas.
- Las condiciones de agrupación y totalización que se van a aplicar a los registros recuperados.

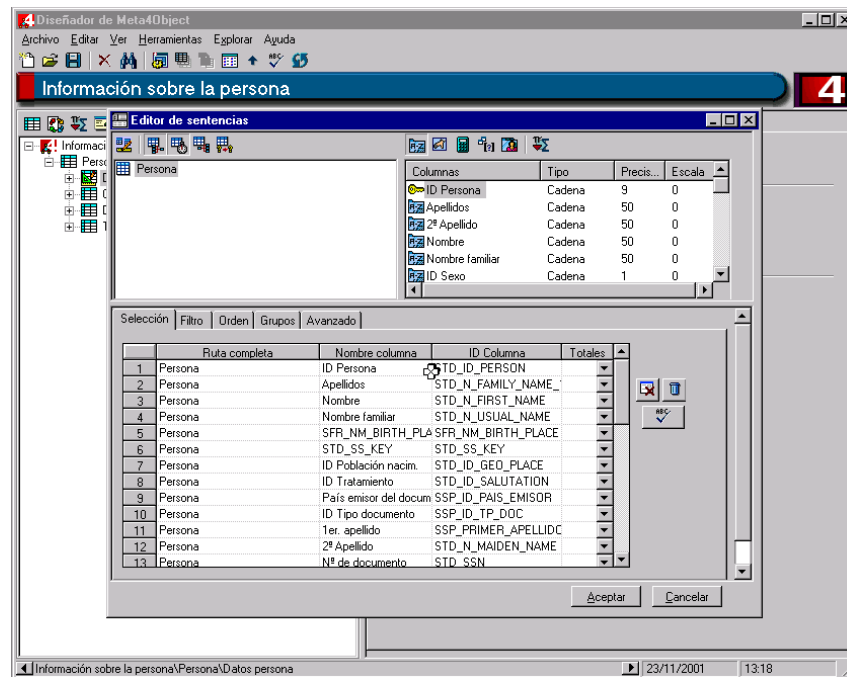
Figura 18. Estructura de nodo: recuperación de datos de varias tablas y unificación de resultados en una sola tabla



La definición de la sentencia de la estructura de nodo se puede hacer desde el Editor de sentencias de la aplicación.

Es posible obtener acceso a esta utilidad desde el Diseñador de Meta4Objects.

Figura 19. Editor de sentencias



La creación y el diseño de estructuras de nodo desde estas herramientas se explica en las ayudas en línea.

Estructuras de nodo y lenguaje SQL

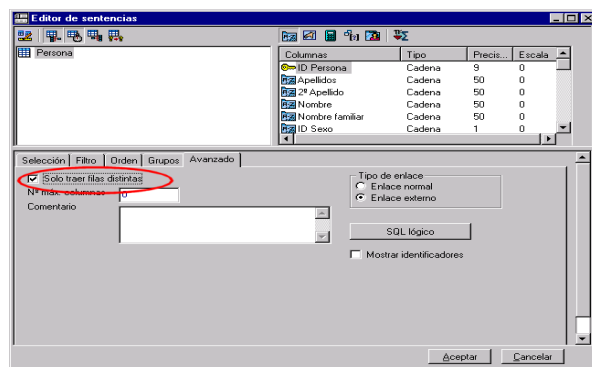
Las estructuras de nodo actúan de interfaz entre las tablas de la base de datos y el nivel de la aplicación de la tecnología Meta4.

Es posible establecer una equivalencia entre la composición de una estructura de nodo y la de una consulta en lenguaje SQL. Como en la de estructura de nodo, en la definición de una sentencia SQL se debe indicar:

1. El nombre de la tabla de la que desea extraer datos (cláusula **FROM**).
2. Las columnas de la tabla que desea recuperar (cláusula **SELECT**).
3. Las condiciones que deben cumplir los registros para ser recuperados (cláusula **WHERE**).
4. Los criterios que va a aplicar para ordenar los registros que recupera la sentencia (cláusula **ORDER BY**).
5. Los criterios que desea utilizar para agrupar registros que compartan valores comunes en una o en varias columnas, y las totalizaciones que se aplican sobre los valores de estos grupos de registros (cláusula **GROUP BY**).
6. Las condiciones que deben cumplir los valores totalizados tras una agrupación para ser recuperados (cláusula **HAVING**).

Cuando ejecute una estructura de nodo obtendrá un conjunto de registros que permanecerán en la memoria principal. Al igual que sucede con el lenguaje SQL, del conjunto de registros recuperados, pueden eliminarse los registros duplicados. En SQL, para eliminar estos registros debe agregar la palabra reservada **DISTINCT** detrás de la palabra reservada SELECT. En el Editor de sentencias, se incluye la casilla de verificación **Sólo traer filas distintas** con esta misma funcionalidad.

Figura 20. Editor de sentencias: opción Sólo traer filas distintas



Aunque se puede establecer una similitud entre una estructura de nodo y una sentencia Select de SQL, estos dos conceptos no son equivalentes. Las

estructuras de nodo no sólo recuperan datos de la base de datos, sino que también encapsulan datos y procesamiento en una única columna; es decir, las estructuras de nodo implementan la lógica de la aplicación.

Instrucciones de procesamiento

Las estructuras de nodo encapsulan datos y procesamiento. La definición de una estructura de nodo puede incluir, además de la sentencia de recuperación de datos, las instrucciones que permiten procesar los registros recuperados e implementan la lógica de la aplicación.

Las instrucciones de procesamiento se indican en un tipo especial de elementos: los de tipo método y los de tipo concepto. Cada método o concepto tiene asociado un programa que se ejecuta cuando el usuario lo solicita desde la interfaz cliente o desde el código de las reglas de otros elementos.

Para escribir el código se dispone de estas alternativas:

- El lenguaje LN4, desarrollado por Meta4
- El lenguaje C++
- También puede llamar a métodos de cualquier biblioteca de enlace dinámico (archivos .DLL).



El usuario dispone de un entorno de desarrollo integrado desde el que podrá modelar las estructuras de datos y definir reglas de procesamiento de datos.

Estas posibilidades amplían notablemente las capacidades de procesamiento de la aplicación. Al separar el tratamiento de los datos de los mecanismos de procesamiento que ofrecen los sistemas gestores de bases de datos (lenguajes propietarios, SQL integrado, etc.), se obtiene un alto grado de independencia respecto al sistema gestor de bases de datos en el que reside la información gestionada por la aplicación. Esta independencia facilita el desarrollo de nuevas funcionalidades y de código transferible.

El lenguaje LN4

El lenguaje LN4 permite al usuario-configurador procesar los registros recuperados por las estructuras de nodo de forma similar a como se haría con un cursor de bases de datos.

Entre las funciones de LN4 destacan las que permiten:

- Hacer referencia a un registro específico del conjunto de registros recuperados por una estructura de nodo.
- Hacer referencia a los elementos de la misma estructura de nodo, o de estructuras diferentes, utilizándolos como si se tratara de variables.
- Moverse en cualquier dirección entre los registros recuperados por una estructura de nodo.
- Leer y escribir el valor de elementos de la misma estructura de nodo, o de otras estructuras de nodo utilizadas en el mismo Meta4Object o en otro.

El código LN4 de un método o concepto se ejecutará:

- Cuando el usuario lo indique de manera explícita mediante una acción determinada sobre el control de la presentación del Meta4Object correspondiente al método o concepto.
- Cuando el método o concepto se ejecute como resultado de una secuencia de acciones encadenadas: disparadores, o disparadores y eventos.
- Cuando se invoque desde otro método o concepto.

EJEMPLO

El Meta4Object Evaluado está diseñado sobre la estructura de nodo SCO_MT_H_EVALUATE. La estructura de nodo incluye un elemento de tipo método, que se presenta al usuario como un botón en el momento en que se carga el nodo en memoria.

Para ejecutar el método, el usuario sólo tiene que hacer clic en el botón que lo representa. Asimismo, se puede llamar desde la regla LN4 de otro método o concepto, o mediante un disparador.

LN4 y la interfaz API-SQL

LN4 permite también obtener acceso a bases de datos SQL externas a la aplicación, mediante la interfaz API-SQL, perteneciente a la plataforma tecnológica de Meta4 y disponible con todas las versiones de nuestros productos.

La interfaz API-SQL amplía notablemente la capacidad de integrar la tecnología de Meta4 con el resto de sistemas de información de su organización.

Elementos de las estructuras de nodo

Una estructura de nodo está compuesta por un conjunto de elementos. Un elemento puede recoger:

- Datos procedentes de la base de datos.
- Datos propuestos por el usuario en tiempo de ejecución.
- Datos calculados por el programa.
- Instrucciones de procesamiento escritas en lenguaje LN4, C++ o llamadas a métodos de una DLL.

Funcionalidad de los elementos

Los elementos se clasifican en cuatro grupos según su funcionalidad:

Tipo de elemento	Descripción
Campos	Se corresponden con columnas de la base de datos. Obtienen su valor de la columna de la base de datos al que están asociados y, cuando se asigna valor a un elemento de tipo campo, el valor se graba en la columna de la base de datos al que está asociado el elemento.
Propiedades	Recogen valores que no proceden de la base de datos. Por ejemplo, valores solicitados al usuario en tiempo de ejecución o los resultados obtenidos al procesar un cálculo.
Conceptos	Recogen código procesable escrito en lenguaje LN4 o C++. Cuando se ejecuta este código, el valor que se obtiene se asigna al concepto. Los conceptos pueden grabar su valor en columnas de la base de datos; para ello, sólo hay que asociar el elemento de la estructura de nodo a una columna de la base de datos.
Métodos	Recogen código procesable escrito en lenguaje LN4 o C++, o llamadas a métodos de DLL externa. El resultado obtenido al procesar este código no se asigna al elemento. Desde la regla de un método se puede asignar valores a otros conceptos de tipo columna o propiedad y ejecutar elementos de tipo método o concepto.

A continuación, se explica con mayor detalle el comportamiento de los distintos tipos de elementos.

Campos

Los elementos de tipo campo están asociados a columnas de tablas de la base de datos.

Mediante los elementos de tipo campo, se puede:

- Recuperar valores de la base de datos para mostrarlos en pantalla, generar informes o procesarlos en cálculos complejos.
- Escribir y actualizar los valores de las columnas de la base de datos.

El valor que recoge un elemento de este tipo en un momento dado se corresponde con el valor incluido en la columna de la base de datos al que está asociado en el registro activo.

Una columna no lleva asociadas instrucciones de procesamiento. Sin embargo, puede procesar los valores de los elementos de tipo campo en las reglas LN4 de elementos de tipo método o concepto. El lenguaje LN4 permite leer, escribir y actualizar el valor de los elementos de tipo campo y trabajar con ellos como si fueran variables.

Propiedades

Estos elementos recogen valores que no se leen de la base de datos. Tampoco llevan asociadas instrucciones de procesamiento.

Se utilizan para recoger valores solicitados al usuario en tiempo de ejecución, u obtenidos durante la ejecución de la regla LN4 de un elemento de tipo método o concepto.

Aunque estos elementos no lean su valor de ninguna columna de la base de datos, sí podrán escribir el mismo en una columna de dicha base de datos.

Conceptos

Estos elementos obtienen su valor mediante un cálculo o regla, que se compone de instrucciones procesables escritas en lenguaje LN4, llamadas a métodos de DLL externas o C++. El valor que se obtiene al procesar estas instrucciones se asigna al elemento.

Si el elemento está asociado a una columna de una tabla de la base de datos, el resultado del cálculo se grabará en la base de datos en cuanto se dé al programa la orden de grabar.

EJEMPLO

El elemento de tipo concepto *Calcula_Edad* está asociado a una regla. Ésta muestra el siguiente código LN4:

```
dFechaNacimiento=FechaNacimiento
dFechaActual=Today()
dEdad=Years(dFechaActual-dFechaNacimiento)
Return(dEdad)
```

La regla asigna a las variables *dFechaNacimiento* y *dFechaActual* el valor del elemento *FechaNacimiento* y el de la fecha de hoy, obtenida con una llamada a la función *Today()*.

A continuación, resta las dos fechas para obtener el tiempo transcurrido entre las dos y aplica la función *Years()* para convertir el valor en años.

Finalmente, la función *Return()* contiene, entre paréntesis, el nombre de la variable cuyo valor se va a devolver como resultado del cálculo. Este valor se asignará al concepto *Calcula_Edad*.

A partir del momento en el que se calcula la regla de un concepto, cuando se solicite su valor durante la misma sesión de trabajo, no será necesario ejecutar el cálculo.

Métodos

Estos elementos contienen instrucciones procesables escritas en LN4, C++ o llamadas a métodos de bibliotecas de enlace dinámico (archivos .DLL).

A diferencia de los conceptos, el método no recoge el valor obtenido como resultado de la ejecución de su regla. Desde la regla LN4 de un método, lee y escribe los valores de otros elementos, y ejecuta otros métodos y conceptos del mismo *Meta4Object* o de uno diferente.

EJEMPLO

Desde LN4, puede asignar el valor de un elemento a una variable. La asignación tiene la siguiente sintaxis:

```
SVariable = NombreElemento()
```

En el primer caso, si los elementos a los que se asigna un valor están asociados a columnas de la base de datos en modo escritura, los valores que se les asignen se escribirán en la base de datos en cuanto se dé la orden de grabar.

Elementos y columnas de la base de datos

Los elementos de una estructura de nodo pueden estar asociados a columnas de la base de datos para lectura o escritura. En el primer caso, el elemento recogerá directamente los valores de esa columna en la base de datos, cuando se cargue en memoria principal la estructura de nodo. Los elementos de tipo campo, propiedad o concepto pueden tener asociado una columna de lectura.

Si el valor que recoge un elemento, cualquiera que sea su tipo (campo, concepto o propiedad), se graba en una columna de una tabla de la base de datos, el elemento de la estructura de nodo deberá tener asociado esta columna en calidad de *columna de escritura*.

La correspondencia entre un elemento de una estructura de nodo y una columna de la base de datos (para lectura o escritura) se indica en el momento de definir la estructura de nodo.

Ámbito de los elementos

Los elementos también se pueden clasificar según su ámbito.

El ámbito de un elemento indica si su valor (en el caso de campos, conceptos o propiedades) o su código (en el caso de métodos y conceptos) se aplica:

- A un único registro recuperado por la estructura de nodo.
- A todos los registros que se encuentran en un mismo bloque de memoria.
- A todos los registros recuperados por una estructura de nodo, independientemente de que se encuentren en bloques de memoria diferentes.

La siguiente tabla recoge los tres tipos de ámbito disponibles:

Ámbito	Descripción
Registro	El valor del elemento puede ser diferente en cada uno de los registros de la estructura de nodo. Varía si se cambia el registro activo.
Bloque	El valor del elemento es el mismo para todos los registros incluidos en un mismo bloque de memoria.
Nodo	El valor del elemento es el mismo para todos los registros de un mismo nodo, aunque se encuentren en bloques diferentes.

Tipos de ámbito

A continuación se explican con mayor detalle los distintos tipos de ámbitos disponibles.

Ámbito a nivel de registro

El valor del elemento puede cambiar para cada uno de los registros del nodo.

El ámbito a nivel de registro lo tienen:

- Los elementos de tipo campo, que obtienen su valor de una columna específica de la base de datos.
- Los elementos de tipo método o concepto, cuyas reglas toman como base los valores de elementos de tipo campo. Es decir, los métodos o conceptos que se tienen que ejecutar para cada uno de los registros recuperados por la estructura de nodo.

EJEMPLO

Los elementos de tipo campo tienen ámbito de tipo registro, ya que su valor puede ser distinto en cada registro.

Imagine una estructura de nodo definida a partir de una tabla de la base de datos que recoge información sobre los empleados.

Cada registro de la tabla corresponde a un empleado determinado; es decir, no hay dos registros con datos del mismo empleado.

Los elementos de la estructura de nodo que obtienen su valor de las columnas de la tabla de la base de datos son de tipo campo, con ámbito a nivel de registro.

En la estructura de nodo, se crea un método que calcula el número de días en alta del empleado, tomando como base del cálculo la fecha de alta del empleado y la fecha del sistema. El método no graba su valor en la base de datos.

Para definir este método, se indicará que:

- El elemento es de tipo método, ya que su valor no se quiere asignar al elemento.
- Su ámbito es a nivel de registro, ya que su valor puede cambiar para cada uno de los empleados, dependiendo de la fecha de alta.

La fecha de alta es un elemento de tipo campo y ámbito a nivel de registro, ya que puede tomar valores distintos en cada registro.

Alcance a nivel de bloque

Un elemento tiene ámbito de tipo bloque cuando su valor es idéntico para todos los registros incluidos en un mismo bloque.



Una de las características más destacables de la aplicación es la correspondencia entre la estructura de los bloques en memoria y el diseño lógico del modelo de datos. Así, se consigue que la gestión de memoria se acople a las operaciones que realiza el usuario conforme trabaja con la aplicación.

Un Meta4Object establece la forma en que los registros deben transferirse de la base de datos a la memoria principal del puesto de trabajo que solicita la carga del Meta4Object. De este modo, los registros de cada nodo se pueden distribuir en uno o varios bloques.

En el segundo caso, el número de bloques correspondientes a un nodo depende de cómo se ha conectado éste con el nodo que ocupa la posición inmediatamente superior en la jerarquía del Meta4Object.

La organización de la memoria física establece una jerarquía entre registros, bloques y nodos. Un bloque contiene varios registros.

De esta forma, puede interpretarse un bloque como una matriz u hoja de cálculo cuyas celdas se rellenan con los valores asignados a los elementos de cada uno de los registros incluidos.

Estos valores pueden extraerse de la base de datos u obtenerse al ejecutar un cálculo (en el caso de los conceptos).

Así, un nodo de múltiples bloques se puede considerar como un conjunto de hojas de cálculo relacionadas jerárquicamente. Si el nodo distribuye sus registros entre dos o más bloques de memoria, habrá una hoja de cálculo para cada bloque de memoria. Cada una de estas hojas se relaciona lógicamente con un registro de uno de los bloques de memoria del nodo superior.

La relación lógica entre los registros del bloque del nodo subordinado y los del bloque del nodo principal se establece mediante una condición que iguala o compara los valores de los elementos de uno y otro nodo.

El nodo que ocupa el primer lugar en la jerarquía de un Meta4Object distribuye sus registros en un solo bloque.

La distribución de registros en distintos bloques de memoria determina su visibilidad. En un momento dado, únicamente estarán visibles para la aplicación los registros del bloque de memoria activo.

La aplicación permite el desplazamiento entre los registros de un bloque de memoria mediante las opciones de la barra de navegación de la interfaz cliente, o desde reglas LN4, mediante unas funciones específicas: *Gonext()*, *Goprevious()*, etc.

Si los registros de un nodo se distribuyen entre dos o más bloques de memoria, cuando se alcanza el último registro de un bloque de memoria y se quiere

continuar recorriendo los registros del siguiente bloque, será necesario desplazarse al siguiente registro del bloque de memoria del nodo principal; es decir, al registro del nodo principal relacionado lógicamente con los registros del siguiente bloque de memoria del nodo subordinado.

Todos los elementos que forman parte del mismo bloque de memoria pertenecen también al mismo nodo.

Alcance a nivel de nodo

Los elementos con ámbito de tipo nodo tienen el mismo valor para todos los registros del mismo nodo, aunque los registros estén distribuidos en bloques diferentes.

Se pueden utilizar para recoger el resultado de cálculos realizados con todos los registros del nodo: totalizaciones, medias, valores máximos, mínimos, etc.

Visibilidad y ámbito de los elementos

El lenguaje LN4 permite hacer referencia en la regla de un método o concepto a otros elementos. Así, puede utilizar el valor de uno o varios elementos para calcular el valor de otro concepto o para ejecutar un método.

El ámbito de un elemento no determina su visibilidad. Se puede hacer referencia a un elemento desde una regla LN4, independientemente del ámbito que tenga, sin que se produzca ningún error.

Sin embargo, el valor del elemento al que se hace referencia varía en función del ámbito.

Referencias a elementos con ámbito a nivel de registro

Si se hace referencia a un elemento con ámbito a nivel de registro, la regla LN4 leerá el valor del elemento al que se hace referencia en el registro sobre el que se encuentra el puntero al llamar a la función.

Si se escribe únicamente el nombre del elemento, se utilizará su valor en el registro activo en el momento de hacer la referencia. Para recoger el valor en un registro diferente, es necesario desplazarse hasta él, o cualificar el nombre del elemento con el índice correspondiente a ese registro.



LN4 permite hacer referencia a los valores de los elementos con ámbito de tipo registro, en registros distintos al activo.

Esto sucede así cuando el registro al que se hace referencia se encuentra en el mismo bloque que el registro desde cuya regla se hace referencia.

Si los registros se encuentran en bloques distintos, será necesario desplazarse hasta el bloque que contiene el registro al que se quiere hacer referencia para poder recoger su valor.

Referencias a elementos con ámbito a nivel de bloque

LN4 siempre toma el valor correspondiente al elemento en el bloque de memoria activo.

Para hacer referencia a un elemento con ámbito de tipo bloque, es necesario activar ese bloque en memoria. Como el bloque activo depende del registro del nodo principal activo, para ver un nodo de tipo bloque hay que desplazarse por los registros del nodo principal, hasta alcanzar el correspondiente al bloque del elemento con el que se quiere trabajar.

Referencias a elementos con ámbito a nivel de nodo

En este caso, no se puede dar este problema, ya que siempre se recupera el mismo valor, independientemente de cuál sea el registro o el bloque de memoria en el que se encuentre el cursor.

Para hacer referencia a un elemento con ámbito a nivel de nodo desde un nodo distinto al que contiene el elemento, es necesario cualificar el nombre del elemento con el código identificativo del nodo.

Desde el código de reglas LN4, se puede hacer referencia a otros elementos del mismo Meta4Object o de otro diferente. La sintaxis empleada depende del ámbito del elemento al que se quiere hacer la llamada.

Se distinguen los siguientes casos, el elemento tiene ámbito a nivel de:

- Registro
- Bloque
- Nodo

Sintaxis LN4 y ámbito de los elementos

El elemento tiene ámbito a nivel de registro	Para utilizar el valor que recoge el elemento en el mismo registro en el que se va a ejecutar la regla, sólo hay que escribir el nombre del elemento. Para recoger el valor de este elemento en un registro distinto a aquél desde el que se va a ejecutar la regla, debe escribirse su nombre acompañado por el índice del registro del que se quiere obtener el valor. Este índice se escribe, entre corchetes, a continuación del nombre del elemento. Por ejemplo, <code>Nodo[10].Elemento</code> toma el valor que recoge el elemento <code>NombreElemento</code> en el registro que ocupa la undécima posición en el bloque. A continuación debe comprobarse si el elemento cuyo valor se quiere obtener está en el mismo nodo que el método para el cual está la regla o en un nodo diferente. Si está en un nodo diferente del mismo <code>Meta4Object</code>, se escribirá el nombre del elemento precedido por el nombre del nodo del que forma parte. El nombre del elemento se separa del nombre del nodo mediante un punto, sin espacios en blanco. Por ejemplo, para utilizar el valor del elemento <code>NombreElemento</code> del registro que ocupa la segunda posición en el nodo <code>Nodo_2</code>, hay que escribir <code>Nodo_2[1].NombreElemento</code>. Si el elemento está en un nodo de un <code>Meta4Object</code> diferente, delante del nombre del elemento irán los del <code>Meta4Object</code> y del nodo del que forman parte, en este orden. El nombre del <code>Meta4Object</code> se debe separar del nombre del nodo mediante el signo "!", y éste último del nombre del elemento mediante el carácter ".", sin espacios en blanco.
El elemento tiene ámbito a nivel de bloque	En este caso, el elemento al que se quiere hacer referencia contiene distintos valores para cada bloque. Si se quiere utilizar el valor que recoge este elemento en el bloque desde el que se va a ejecutar la regla, habrá que escribir el nombre del elemento. Si se quiere utilizar en un bloque distinto, se deberá activar el bloque que contiene el elemento al que se hace referencia, recoger su valor en una variable y volver a activar el bloque desde el cual se ha ejecutado la regla. Para activar un bloque, en el nodo principal del nodo que contiene el bloque que se desea activar, sitúese en el registro asociado a dicho bloque. Si el elemento al que se quiere hacer referencia se encuentra en un nodo o en un <code>Meta4Object</code> diferente, su nombre deberá escribirse precedido por el código identificativo del nodo que lo contiene, o por los códigos identificativos del <code>Meta4Object</code> y del nodo que los contiene.
El elemento tiene ámbito a nivel de nodo	En este último caso, el elemento recoge el mismo valor independientemente de cuál sea el registro o bloque activos. Por lo tanto, no es necesario situarse en un registro o bloque específico. Si el elemento al que se quiere hacer referencia forma parte de un nodo distinto a aquel que contiene el elemento desde el cual se va a hacer la llamada, se deberá escribir cualificado por el nombre del nodo en el que está incluido. Si el elemento al que se quiere hacer referencia forma parte de un nodo de otro <code>Meta4Object</code>, se escribirá el nombre, precedido por los del <code>Meta4Object</code>, seguido por el del nodo. El primero se separará del segundo mediante el signo "!", y éste último se separará del nombre del elemento mediante ".".

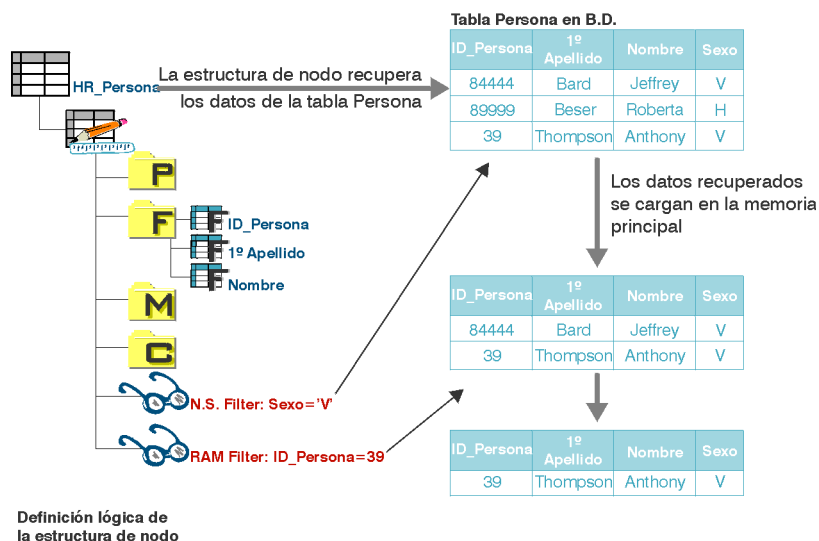
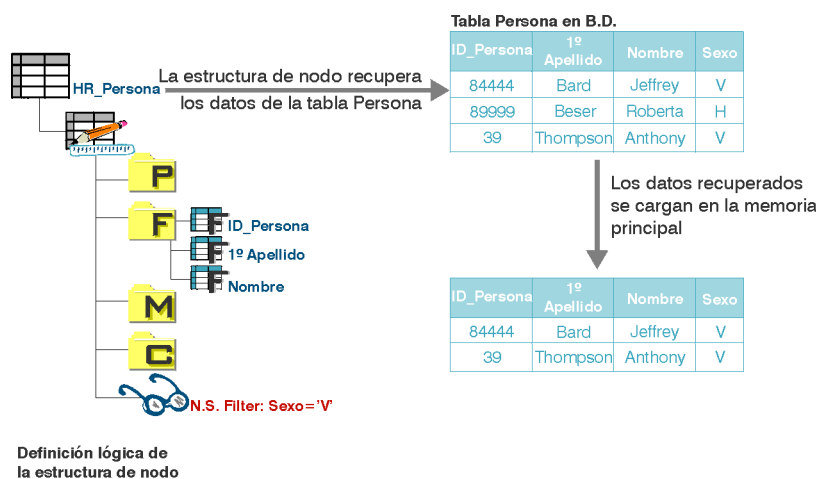
Filtros de datos

Las estructuras de nodo actúan como interfaz entre la base de datos y la aplicación. En la definición de éstas, se puede indicar:

- Las tablas de las que se quiere extraer los datos.
- Los campos cuyo valor se desea recuperar, cada uno de los cuales se corresponde con una columna de las tablas.

Las estructuras de nodo pueden hacer una recuperación selectiva de registros. Para ello, se debe agregar a la estructura de nodo un filtro con todas las condiciones que deben cumplir los registros que se quieren recuperar al ejecutar la estructura de nodo.

Figura 21. Filtros sobre estructuras de nodo



Filtros de estructura de nodo y filtros en RAM

Los registros se filtran en la base de datos, no en memoria principal. Es decir, la estructura de nodo sólo carga en RAM los registros que cumplen las condiciones indicadas en el filtro.

La aplicación también permite definir otro tipo de filtros, los filtros en RAM. Estos filtros recuperan un subconjunto de los registros que se han cargado en memoria principal al ejecutar la estructura de nodo. Los filtros en RAM se ejecutan sobre el conjunto de registros disponibles en memoria.

Estructura de un filtro

Un filtro puede estar formado por una o más condiciones. Cada condición se define sobre una de las columnas de la tabla, e indica qué valor debe recoger esa columna o en qué rango de valores debe encontrarse su valor, para que el registro sea recuperado.

Si el registro cumple la condición, se recuperará. Si, por el contrario, no la cumple, no se recuperará.

Las condiciones tienen la forma siguiente:

```
< Columna Operador  
[elementoTpPropiedad|elementoTpConcepto|Constante] >
```

Donde cada elemento cumple la función que se indica en la siguiente tabla:

Elemento	Descripción
Columna	Corresponde a la columna de una de las tablas de las que la estructura de nodo extrae registros. Si el valor de esta columna cumple la condición que se indica en el filtro, la estructura de nodo recuperará el registro.
Operador	Indica cómo se van a comparar los valores de la columna situado a la izquierda del operador, con el valor del elemento de tipo propiedad, concepto o valor constante de la columna situada a la derecha. Pueden utilizarse los operadores = (Igual que), <> (Distinto a), > (Mayor que), < (Menor que), >= (Mayor o igual que), <= (Menor o igual que) y LIKE.
Elemento	Corresponde a un elemento de tipo propiedad o concepto con ámbito a nivel de bloque, incluido en la definición de la estructura de nodo. Este elemento recoge el valor con el que se van a comparar los valores de la columna indicado en Columna, en cada uno de los registros de las tablas de las que la estructura de nodo extrae datos. Los elementos de tipo propiedad se suelen utilizar para recoger un valor solicitado al usuario durante el tiempo de ejecución del programa.

Elemento	Descripción
Constante	Recoge un valor constante. Con este valor se compara el de la columna de la tabla para evaluar la condición. El valor se debe indicar en el momento de diseñar la estructura de nodo. El tipo de dato del valor debe corresponder al de la columna.

EJEMPLO

Se diseña una estructura de nodo sobre la tabla SCO_MT_HR_ROLE -. Esta tabla recoge la adscripción de los empleados a las distintas unidades organizativas de la compañía.

ID RH	ID Rol	Fecha inicio	Fecha fin	Unidad organizativa
1	1	22-06-1998	Nulo	3- Gestión Calidad
2	1	13-01-1999	Nulo	2 - Marketing
3	1	07-07-1998	15-MAY-1999	2 - Marketing
3	2	16-05-1999	Nulo	3 - Gestión Calidad
4	1	13-02-1999	Nulo	5 - Comercial
5	1	05-04-1998	Nulo	5 - Comercial
6	1	10-09-1999	Nulo	6 - Soporte Técnico

En la estructura de nodo, se diseña un filtro sobre la columna **Unidad organizativa** para recuperar exclusivamente los registros correspondientes a la Unidad organizativa 3, 'GESTIÓN DE CALIDAD'.

Al ejecutarse la estructura de nodo, únicamente se recuperarán los registros de los empleados pertenecientes a la unidad organizativa de Gestión de Calidad.

ID RH	ID Role	Fecha inicio	Fecha fin	Unidad organizativa
1	1	22-10-1998	Nulo	3 - Gestión Calidad
3	2	16-09-1999	Nulo	3 - Gestión Calidad

Operadores de comparación

Las condiciones de un filtro comparan el valor que recoge una columna en los registros de una tabla, con:

- Un valor constante indicado en el diseño del filtro.
- El valor de un elemento de tipo propiedad o campo, y ámbito a nivel de bloque.

Para comparar el valor del elemento de tipo campo con otro valor, se utilizan los siguientes operadores:

Operador	Nombre operador	Descripción
=	Igual	El registro se recuperará si la columna indicada a la izquierda del operador contiene un valor igual al que se indica a la derecha. Este operador se puede utilizar con valores de tipo numérico, fecha o cadena de caracteres.
<>	Distinto a	El registro se recuperará si la columna indicada a la izquierda del operador contiene un valor distinto al que se indica a la derecha. Este operador se puede utilizar con valores de tipo numérico, fecha o cadena de caracteres.
>	Mayor que	El registro se recuperará si la columna indicada a la izquierda del operador contiene un valor mayor que el que se indica a la derecha. Este operador se puede utilizar con valores de tipo numérico, fecha o cadena de caracteres.
<	Menor que	El registro se recuperará si la columna indicada a la izquierda del operador contiene un valor menor que el que se indica a la derecha. Este operador se puede utilizar con valores de tipo numérico, fecha o cadena de caracteres.
>=	Mayor o igual que	El registro se recuperará si la columna indicada a la izquierda del operador contiene un valor mayor o igual que se indica a la derecha. Este operador se puede utilizar con valores de tipo numérico, fecha o cadena de caracteres.

Operador	Nombre operador	Descripción
<=	Menor o igual que	El registro se recuperará si la columna indicada a la izquierda del operador contiene un valor menor o igual que el que se indica a la derecha. Este operador se puede utilizar con valores de tipo numérico, fecha o cadena de caracteres.
LIKE	LIKE	Permite buscar registros que contengan una cadena de caracteres determinada dentro de una columna de texto. Por ejemplo, la condición UNIDAD_ORGANIZATIVA LIKE '%MA%' Recuperará los registros que en la columna UNIDAD_ORGANIZATIVA contengan la cadena de caracteres 'MA', por ejemplo 'Marketing', 'Mantenimiento', etc. El carácter % actúa como comodín, sustituyendo a cualquier cadena de caracteres. Este carácter especial puede escribirse al principio, al final o en cualquier posición de la cadena que se busca. El operador LIKE sólo se aplica a columnas de tipo texto.

Funciones predefinidas

En el filtro de una estructura de nodo se pueden aplicar funciones a los campos y valores sobre los que se define la condición.

El Editor de sentencias permite utilizar un conjunto de funciones predefinidas, denominadas **funciones lógicas**, ya que abarcan la implementación particular de cada sistema gestor de bases de datos.



El usuario puede utilizar el mismo conjunto de funciones con independencia del sistema gestor en el que residan los datos.

La base de datos lógica (BDL) mantiene la equivalencia entre las funciones lógicas que se utilizan en el Editor de sentencias y la forma en la que estas funciones están implementadas en los distintos sistemas gestores de bases de datos.

Cuando el usuario solicita la carga en memoria de un nodo asociado a una estructura de nodo en cuya definición se han utilizado funciones lógicas, la base de datos lógica traduce estas funciones a su correspondiente implementación en el sistema gestor de bases de datos (SGBD) donde residen los datos.

La implementación en el sistema gestor de bases de datos puede consistir en otra función, o en su defecto, en un procedimiento con el que se obtienen los resultados requeridos.

El usuario-configurador y el usuario final de la aplicación no necesita conocer cómo se implementan estas funciones en los distintos sistemas gestores de bases de datos. La base de datos lógica realiza automáticamente todas las traducciones necesarias.

El sistema incluye funciones para obtener la fecha de hoy: `HOY()`; convertir datos: `CONVERTIR A CADENA()`, `CONVERTIR A FECHA()`, `CONVERTIR A NÚMERO()`, etc.

EJEMPLO

El filtro

```
St_date <= HOY()
```

Devuelve los registros cuyo campo **St_date** recoge una fecha menor o igual a la del sistema.

Filtros con varias condiciones

En un filtro, se puede indicar más de una condición sin ningún tipo de restricción.

Cada condición está definida sobre un elemento de tipo columna de la tabla de la que se extraen datos.

Las condiciones se unen mediante los operadores lógicos AND, OR y NOT.



Existe un cuarto operador, EXISTS, que se explicará en otro apartado de este capítulo.

Operador And

Cuando se unen dos condiciones mediante el operador AND, el registro se recuperará si cumple ambas condiciones.

EJEMPLO

ID RH	ID Role	Fecha inicio	Fecha fin	Unidad organizativa
1	1	22-06-1998	Nulo	3 - Gestión Calidad
2	1	13-01-1999	Nulo	2 - Marketing
3	1	07-07-1998	15-MAY-1999	2 - Marketing
3	2	16-05-1999	Nulo	3 - Gestión Calidad
4	1	13-02-1999	Nulo	5 - Comercial
5	1	05-04-1998	Nulo	5 - Comercial
6	1	10-09-1999	Nulo	6 - Soporte Técnico

El filtro

```
Unidad organizativa='5 - Comercial' And Fecha inicio > CONVERTIR  
A FECHA('01-ENE-1999')
```

Recuperará los registros que:

- Recogen en la columna **Unidad organizativa** el valor *5 - Comercial*
- Recogen en la columna **Fecha inicio** una fecha posterior al *1 de enero de 1999*

Así, se obtendrá el registro siguiente:

ID RH	ID Role	Fecha inicio	Fecha fin	Unida organizativa
4	1	13-02-1999	Nulo	5 - Comercial

Operador Or

Cuando se unen dos condiciones mediante el operador OR, el registro se recuperará si cumple al menos una de las dos condiciones.

EJEMPLO

Se define el siguiente filtro sobre la tabla Roles por Unidad organizativa:

```
Unidad organizativa='5 - Comercial' Or Fecha inicio > CONVERTIR  
A FECHA('01-ENE-1999')
```

La tabla HT_HR_IN_ROLE_WU contiene los siguientes registros:

ID RH	ID Role	Fecha inicio	Fecha fin	Unidad organizativa
1	1	22-06-1998	Nulo	3- Gestión Calidad
2	1	13-01-1999	Nulo	2 - Marketing
3	1	07-07-1998	15-05-1999	2 - Marketing
3	2	16-05-1999	Nulo	3 - Gestión Calidad
4	1	13-02-1999	Nulo	5 - Comercial
5	1	05-04-1998	Nulo	5 - Comercial
6	1	10-09-1999	Nulo	6 - Soporte Técnico

Al ejecutar la estructura de nodo con el filtro, se cargan en memoria:

- Los registros que recogen en el campo **Unidad organizativa** el valor 5 - *Comercial*
- Los registros que recogen en el campo **Fecha inicio** una fecha posterior al 1 de enero de 1999

Es decir, los que se muestran en la siguiente tabla:.

ID RH	ID Role	Fecha inicio	Fecha fin	Unidad organizativa
2	1	13-01-1999	Nulo	2 - Marketing
3	2	16-05-1999	Nulo	3 - Gestión Calidad
4	1	13-02-1999	Nulo	5 - Comercial
5	1	05-04-1998	Nulo	5 - Comercial
6	1	10-09-1999	Nulo	6 - Soporte Técnico

Operador NOT

Cuando se unen dos condiciones mediante el operador NOT, el registro se recuperará si cumple la primera condición (es decir, la situada delante del operador NOT) y no cumple la segunda (situada detrás del operador).

EJEMPLO

Se define el siguiente filtro sobre la tabla HT_HR_IN_ROLE_WU:

```
ID_WORK_UNIT='5 - Comercial' NOT DT_START > CONVERTIR A  
FECHA('01-ENE-1999')
```

La tabla HT_HR_IN_ROLE_WU contiene los siguientes registros:

ID RH	ID Role	Fecha inicio	Fecha fin	Unidad organizativa
1	1	22-06-1998	Nulo	3 - Gestión Calidad
2	1	13-01-1999	Nulo	2 - Marketing
3	1	07-07-1998	15-MAY-1999	2 - Marketing
3	2	16-05-1999	Nulo	3 - Gestión Calidad
4	1	13-02-1999	Nulo	5 - Comercial
5	1	05-04-1998	Nulo	5 - Comercial
6	1	10-09-1999	Nulo	6 - Soporte Técnico

Al ejecutar la estructura de nodo con el filtro, se cargan en memoria los registros que:

- Recogen en la columna **Unidad organizativa** el valor *5 - Comercial*
- Tienen una fecha de inicio anterior al *1 de enero de 1999*

Es decir, los que se muestran en la siguiente tabla:

ID RH	ID Role	Fecha inicio	Fecha fin	Unidad organizativa
5	1	05-04-1998	Nulo	5 - Comercial

Operador Exists

Este operador permite definir condiciones de búsqueda sobre las tablas que mantienen una relación 1-N con la tabla base. Es decir, sobre aquellas tablas en las que, para cada entidad a la que hace referencia un registro de la tabla base, pueden existir uno, dos o más registros.

De esta forma, se pueden definir condiciones de selección del tipo:

- ¿Cuáles son los nombres de los empleados que han trabajado en la unidad organizativa de marketing durante el año pasado?
- ¿A qué unidades organizativas se destinó en algún mes anterior una cantidad superior a los 10.000.000 ptas. para pagar comisiones?

El operador EXISTS irá precedido por uno de los operadores lógicos AND, OR o NOT.



El operador Exists condiciona el cumplimiento del filtro al cumplimiento de una condición sobre otra tabla.

Orden de evaluación de condiciones

Un filtro puede estar formado por más de dos condiciones. Éstas se evalúan en el orden establecido por la prioridad de los operadores que conectan las condiciones, y no en el que se escriben las condiciones en el filtro.

El operador con mayor prioridad es el operador NOT, seguido de AND y OR.

EJEMPLO

La siguiente tabla recoge una proyección de los costes salariales retrospectivos, totalizados por concepto de nómina y agrupados por paga mensual y unidad organizativa.

Es decir, cada registro de la tabla recogerá cuánto se ha pagado por concepto de nómina, en una misma paga, a todos los empleados de una misma unidad organizativa.

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Marketing	25-07-1999	25-07-1999	20.700.000	50.000
Marketing	25-08-1999	25-08-1999	20.540.000	300.000
Logística	25-07-1999	25-07-1999	35.000.000	0
Logística	25-08-1999	25-08-1999	35.000.000	0
Fabricación	25-07-1999	25-07-1999	37.890.000	0
Fabricación	25-08-1999	25-08-1999	150.000.000	158.000
Fabricación	25-09-1999	25-09-1999	150.715.000	230.000

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Financiero	25-07-1999	25-07-1999	15.670.000	0
Financiero	25-08-1999	25-08-1999	15.780.000	0
Financiero	25-09-1999	25-09-1999	16.780.000	0

La siguiente sentencia:

Unidad organizativa = 'Marketing' OR **Unidad organizativa** = 'Fabricación' AND **Dietas** > 100.000

No se evalúa de izquierda a derecha de acuerdo con el orden en el que se han escrito las condiciones del filtro, sino en el orden que establece la prioridad de los operadores. Así:

- En primer lugar, se evalúa la condición **Unidad organizativa** = 'Fabricación' AND **Dietas** > 100.000.
- A continuación se evalúa la condición **Unidad organizativa** = 'Marketing', unida mediante el operador OR al conjunto de registros obtenido como resultado de la combinación de la condición anterior.

La sentencia recuperará:

- Los registros correspondientes a la unidad organizativa Fabricación cuya columna **Dietas** recoja un valor superior a 100.000
- Todos los registros correspondientes a la unidad organizativa Marketing

Es decir, los que recoge la siguiente tabla:

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Marketing	25-07-1999	25-07-1999	20.700.000	50.000
Marketing	25-08-1999	25-08-1999	20.540.000	300.000
Fabricación	25-08-1999	25-08-1999	150.000.000	158.000
Fabricación	25-09-1999	25-09-1999	150.715.000	230.000

Cambio de la prioridad de los operadores

Es posible cambiar el orden en el que se evaluarán las condiciones del filtro, para ello se utilizan paréntesis en los que se incluyen las condiciones que se van a evaluar en primer lugar.

EJEMPLO

Este ejemplo se basa en la tabla del ejemplo anterior.

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Marketing	25-07-1999	25-07-1999	20.700.000	50.000
Marketing	25-08-1999	25-08-1999	20.540.000	300.000
Logística	25-07-1999	25-07-1999	35.000.000	0
Logística	25-08-1999	25-08-1999	35.000.000	0
Fabricación	25-07-1999	25-07-1999	37.890.000	0
Fabricación	25-08-1999	25-08-1999	150.000.000	158.000
Fabricación	25-09-1999	25-09-1999	150.715.000	230.000
Financiero	25-07-1999	25-07-1999	15.670.000	0
Financiero	25-08-1999	25-08-1999	15.780.000	0
Financiero	25-09-1999	25-09-1999	16.780.000	0

Para evaluar las condiciones según el orden que ocupan en el filtro, se debe encerrar entre paréntesis las dos condiciones unidas mediante el operador OR, de la siguiente forma:

```
(Unidad organizativa = 'Marketing' OR Unidad organizativa =
'Fabricación') AND Dietas > 100.000
```

En este caso, el filtro recuperará:

- Todos los registros correspondientes a las unidades organizativas Marketing y Fabricación
- Los registros incluidos en el grupo anterior que muestran en la columna **Dietas** un valor superior a 100.000

Es decir, los registros que recoge la siguiente tabla:

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Marketing	25-08-1999	25-08-1999	20.540.000	300.000
Fabricación	25-08-1999	25-08-1999	150.000.000	158.000
Fabricación	25-09-1999	25-09-1999	150.715.000	230.000



En el diseño de filtros para estructuras de nodo que incluyen más de dos condiciones, es necesario considerar el orden en que se quieren evaluar dichas condiciones. De otra forma, se pueden obtener resultados no deseados.

El ejemplo anterior muestra la posibilidad de cambiar el orden de evaluación de las condiciones utilizado por defecto mediante el uso de paréntesis.

En primer lugar, se evalúan las condiciones escritas entre paréntesis y, a continuación, el resto de condiciones. En el caso de que se aniden más de un grupo de condiciones entre paréntesis, se evaluarán en primer lugar las condiciones entre los paréntesis más anidados; repitiendo el ciclo sucesivamente.

Ordenación de los registros

En la definición de una estructura de nodo, se puede indicar el orden en que se desea recuperar los registros.

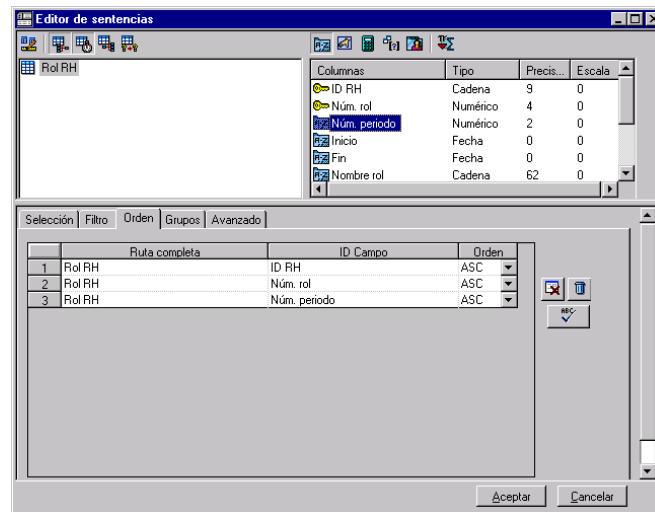


Sólo se pueden utilizar como criterios de ordenación los elementos de tipo campo. No se pueden utilizar los de tipo método o concepto.

Los registros se ordenan a partir de los valores que tienen en una o varias columnas. El orden puede ser ascendente o descendente.

El elemento o los elementos utilizados para ordenar los registros se deben haber seleccionado dentro del conjunto de elementos que debe recuperar la estructura de nodo.

Figura 22. Editor de sentencias: ordenación



En cada uno de los campos incluidos en la lista de criterios de ordenación, indique si se tiene que utilizar un orden ascendente o descendente.

Si indica más de una columna en la lista de criterios de ordenación, los registros que recupere la estructura de nodo se ordenarán de la siguiente forma:

- En primer lugar, como primer criterio de ordenación, los registros a partir de los valores de la columna seleccionado.
- Si hay más de un registro con el mismo valor en esa columna, estos registros se ordenarán a partir de los valores que recojan en la columna seleccionado como segundo criterio de ordenación, y así sucesivamente.

EJEMPLO

Se diseña una estructura de nodo sobre una tabla que recoge los costes salariales por concepto y paga mensual, totalizados por unidad organizativa.

El siguiente cuadro recoge el contenido de esta tabla en la base de datos.

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Marketing	25-07-1999	25-07-1999	20.700.000	50.000
Marketing	25-08-1999	25-08-1999	20.540.000	300.000
Logística	25-07-1999	25-07-1999	35.000.000	0

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Logística	25-08-1999	25-08-1999	35.000.000	0
Fabricación	25-07-1999	25-07-1999	37.890.000	0
Fabricación	25-08-1999	25-08-1999	150.000.000	158.000
Fabricación	25-09-1999	25-09-1999	150.715.000	230.000
Financiero	25-07-1999	25-07-1999	15.670.000	0
Financiero	25-08-1999	25-08-1999	15.780.000	0
Financiero	25-09-1999	25-09-1999	16.780.000	0

La estructura de nodo incluye uno de los siguientes filtros:

Unidad organizativa = 'Fabricación'

Unidad organizativa = 'Logística'

Y una cláusula de ordenación para que los registros recuperados se ordenen por **Unidad organizativa** (en orden ascendente) y **Salario base** (en orden descendente).

Unidad organizativa Asc

Salario base Desc

De esta forma, la estructura de nodo recupera los registros correspondientes a las unidades organizativas Fabricación y Logística, ordenados alfabéticamente por *Unidad organizativa*. Los registros que tienen el mismo valor en esta columna se ordenan a partir de los valores que recoge su columna **Salario base**, situándose en primer lugar los registros que tienen un valor mayor en esta columna.

Si hay registros con el mismo valor en las dos columnas seleccionados como criterios de ordenación, **Unidad organizativa** y **Salario base**, se ordenarán de forma aleatoria.

Los registros permanecen ordenados en memoria principal. La siguiente tabla recoge el conjunto de registros resultado:

Unidad organizativa	Fecha imputación	Fecha pago	Salario base	Dietas
Logística	25-07-1998	25-07-1998	35.000.000	0
Logística	25-08-1998	25-08-1998	35.000.000	0
Fabricación	25-09-1998	25-09-1998	150.715.000	230.000
Fabricación	25-08-1998	25-08-1998	150.000.000	158.000
Fabricación	25-07-1998	25-07-1998	37.890.000	0

Agrupaciones de registros

Una estructura de nodo puede recuperar valores que se hayan obtenido al aplicar agrupaciones y totalizaciones con los datos de las tablas de la base de datos.

Los registros se agrupan a partir de los valores que tienen en común en una o más columnas. Estas columnas son los criterios de ordenación de la sentencia de la estructura de nodo.

Al aplicar la agrupación, se obtienen grupos de registros con el mismo valor en las columnas seleccionadas como criterios de agrupación.

Sobre los registros incluidos en cada grupo, se puede aplicar funciones de totalización: medias, valores máximos y mínimos, sumatorios, etc. Una agrupación siempre irá acompañada por una o varias totalizaciones.

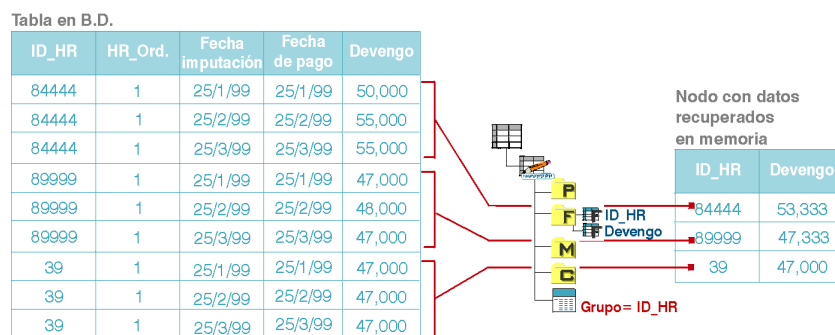


Se tiene que aplicar una función de totalización sobre todos los elementos de tipo campo de la estructura de nodo que no se han utilizado como criterios de agrupación. Es decir, si una columna no se ha utilizado como criterio de ordenación, tendrá que aplicarse una función de totalización sobre sus valores. De no hacerse así, se producirá un error.

Cuando se aplica una agrupación/totalización de registros, se obtiene como resultado un registro para cada grupo de registros que tienen el mismo valor en las columnas utilizadas como criterios de agrupación.

En la siguiente figura, para cada grupo de registros que tienen el mismo valor en los campos seleccionados como criterios de ordenación, se genera un único registro en la tabla resultado con los valores totalizados.

Figura 23. Agrupaciones



Las agrupaciones y totalizaciones son un eficaz mecanismo de análisis de la información. Sobre una misma tabla se pueden definir tantas estructuras de nodo como totalizaciones de los valores de la tabla que se necesiten.

EJEMPLO

Sobre la tabla que recoge los resultados retrospectivos del cálculo de nómina, se puede crear una estructura de nodo que totalice resultados para cada trimestre, cuatrimestre o año.

Agrupaciones en estructuras de nodo y lenguaje SQL

Las agrupaciones que se pueden diseñar al crear una estructura de nodo utilizan los mecanismos de agrupación y totalización disponibles en el lenguaje ANSI-SQL.

Realmente, cuando se ejecuta una estructura de nodo que contiene agrupaciones y totalizaciones, se diseña una sentencia SQL equivalente que se ejecutará contra el sistema gestor de bases de datos.

Este sistema se encarga de agrupar los registros de la tabla a partir de los valores que recogen en las columnas seleccionadas como criterios de agrupación, y ejecuta las totalizaciones indicadas para las demás columnas.

A continuación, el sistema gestor devuelve al servidor de aplicaciones (o al puesto cliente), un conjunto de registros ya agrupados y totalizados. Ni el servidor de aplicaciones ni el puesto de trabajo cliente tienen que realizar operación alguna para agrupar registros o totalizar valores.

Funciones de totalización

Las estructuras de nodo permiten aplicar las siguientes operaciones de totalización sobre los registros de cada grupo:

Función	Descripción
Count	Devuelve el número de registros incluido en el grupo. Es decir, devuelve el número de registros que tienen los mismos valores en las columnas seleccionados como criterios de agrupación. Esta función se aplica a cualquier columna, independientemente del tipo de dato que recoja.
Sum	Devuelve el valor obtenido al sumar los valores asignados a la columna sobre el que se aplica la totalización, en todos los registros de un mismo grupo. Esta función se aplica a las columnas que recogen datos de tipo numérico.
Avg	Devuelve el valor obtenido al calcular la media de los valores asignados a la columna sobre el que se aplica la totalización, en todos los registros de un mismo grupo. Esta función se aplica a las columnas que recogen datos de tipo numérico.

Función	Descripción
Max	Devuelve el valor más alto de todos los asignados a la columna sobre la que se aplica la totalización, en todos los registros del mismo grupo. Esta función se aplica a cualquier columna, independientemente del tipo de dato que recoja. Si se aplica a datos de tipo cadena de caracteres, devolverá la cadena del último lugar en orden alfabético descendente. Si se aplica a datos de tipo fecha, devolverá la fecha mayor.
Min	Devuelve el valor más bajo de todos los asignados a la columna sobre el que se aplica la totalización, entre los registros de un mismo grupo. Esta función se aplica a cualquier columna, independientemente del tipo de dato que recoja. Si se aplica a datos de tipo cadena de caracteres, devolverá la cadena del primer lugar en orden alfabético ascendente. Si se aplica a datos de tipo fecha, devolverá la fecha menor.

Diseño de agrupaciones

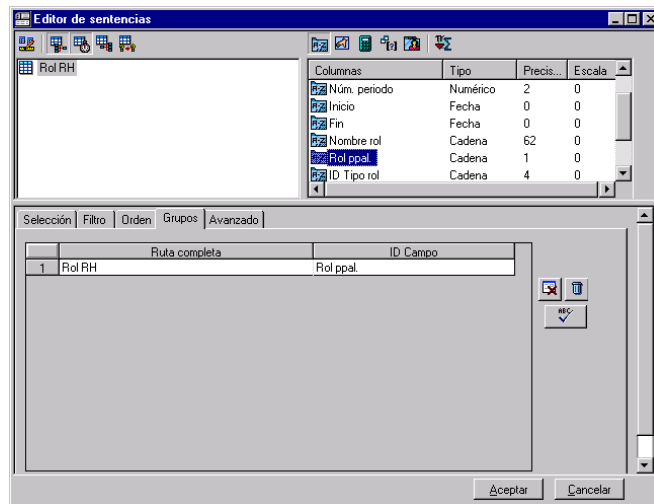
Al diseñar una estructura de nodo que va a recuperar registros agrupados y totalizados, se tiene que considerar:

- A partir de qué columnas se van a agrupar los registros, es decir, qué columnas van a utilizarse como criterios de agrupación.
- Sobre qué columnas se van a aplicar las funciones de totalización.

Todas las columnas cuyo valor sea recuperado por la estructura de nodo se deben incluir en uno de los dos grupos anteriores.

De esta forma, se tendrá que aplicar una función de totalización en todos los elementos de tipo campo que no se utilicen como criterios de agrupación de registros.

Figura 24. Editor de sentencias: Agrupaciones



Esta restricción no se aplica a los elementos de tipo método o concepto.

En el caso de que las reglas de los métodos o conceptos contengan referencias a los elementos de tipo campo de la estructura de nodo, las reglas utilizarán los valores totalizados.

En resumen:

- No se pueden aplicar funciones de totalización en las columnas utilizadas para agrupar registros
- En todas las columnas no utilizadas como criterios de agrupación de registros, se debe aplicar una función de totalización

EJEMPLO

La tabla PERIODO_PAGO recoge, para cada empleado y paga procesada, los valores asignados a cada concepto de nómina.

Se quiere diseñar una estructura de nodo sobre esta tabla. La estructura de nodo recuperará los valores totalizados de los conceptos de nómina *Devengos* y *Deducciones* para cada uno de los empleados a lo largo del primer trimestre del año 1999.

También se quiere averiguar la diferencia entre estos dos conceptos para cada empleado.

Para diseñar la estructura de nodo, siga estos pasos:

- Cree una estructura de nodo sobre la tabla PERIODO_PAGO que recupere los elementos de tipo campo **ID RH**, **Total devengos** y **Total deducciones**.
- Indique que se van a agrupar los registros a partir de los valores de la columna **ID RH**.

- Indique que se van a totalizar los valores de los elementos de tipo columna **Total devengos** y **Total deducciones**. En los dos casos se seleccionará *Sum* como función de totalización.
- Cree un filtro que recupere únicamente los registros correspondientes a las pagas del primer trimestre de 1999. Este filtro contendrá una condición como la siguiente:

```
Allocation_Date >= CONVERTIR A FECHA('1999-ENE-25') And
Allocation_Date <= CONVERTIR A FECHA('1999-MAR-25')
```
- Cree un elemento de tipo concepto con una regla LN4 que calcule la diferencia entre los conceptos totalizados **Total devengos** y **Total deducciones**.
- Este elemento tendrá ámbito a nivel de registro, ya que se quiere obtener la diferencia para cada empleado.

La siguiente tabla muestra una proyección de la tabla de Acumulado, en la que se recoge el valor asignado a todos los conceptos en cada una de las pagas procesadas, en la nómina de cada empleado.

ID RH	Fecha cuota	Fecha pago	Total devengos	Importe	Deducción		Retenciones
01	25-01-1999	25-01-1999	90.000	10.000	30.000		45.000
02	25-01-1999	25-01-1999	110.000	15.000	25.000		12.000
03	25-01-1999	25-01-1999	98.000	12.500	26.000		10.140
04	25-01-1999	25-01-1999	105.000	34.000	6.000		0
01	25-02-1999	25-02-1999	80.000	12.500	4.500.		0
02	25-02-1999	25-02-1999	100.000	16.700	3.200		0
03	25-02-1999	25-02-1999	150.715	8.700	2.000		23.000
04	25-02-1999	25-02-1999	90.670	4.500	7.500		0

La estructura de nodo recupera únicamente los valores de las columnas **ID RH**, **Total devengos** y **Retenciones**, para las pagas correspondientes al primer trimestre del año 1999; es decir:

ID RH	Total devengos	Retenciones
01	90.000	45.000
02	110.000	12.000
03	98.000	10.140
04	105.000	0
01	80.000	0
02	100.000	0

ID RH	Total devengos	Retenciones
03	150.715	23.000
04	90.670	0

A continuación, se agrupan los registros correspondientes a cada empleado y se totalizan los valores de los conceptos **Total devengos** y **Total deducciones**. Como resultado, se obtiene esta tabla:.

ID RH	Total devengos	Retenciones
01	170.000	45.000
02	210.000	12.000
03	248.715	33.140
04	195.670	0

Para cada empleado se han totalizado los valores correspondientes a los conceptos **Total devengos** y **Retenciones**, en los registros del primer trimestre del año 1998.

La información de la última tabla contiene los datos que devolverá la base de datos y que se cargarán en memoria al ejecutar la estructura de nodo.

Además de los elementos de tipo campo, en la definición de la estructura de nodo se ha incluido un elemento de tipo concepto y ámbito a nivel de registro. Este elemento restará, en cada registro recuperado por la estructura de nodo, los valores de los elementos **Total devengos** y **Retenciones**.

Al ejecutar el método en cada registro (por ejemplo, mediante un bucle que recorra todos los registros del nodo) se obtendrá una estructura de datos en memoria como la de la siguiente tabla:

ID RH	Total devengos	Retenciones	Concepto
01	170.000	45.000	125.000
02	210.000	12.000	198.000
03	248.715	33.140	215.575
04	195.670	0	195.670

Agrupaciones por más de una columna

Es posible agrupar los registros que tengan los mismos valores en más de una columna.

En este caso los registros se agruparán, en primer lugar, a partir de los valores que tienen en común en la columna seleccionada como primer criterio de agrupación. Dentro de cada uno de los grupos que se obtenga, se hará una división en subgrupos, cada uno de los cuales tendrá los registros que tengan el mismo valor en la columna seleccionada como segundo criterio de agrupación, y así sucesivamente.

EJEMPLO

La tabla PERIODO_INACTIVIDAD recoge los periodos de inactividad de los empleados.

Cada registro recoge:

- El código identificativo del empleado (**ID RH**).
- La fecha en que inició el periodo de actividad (**Fecha inicio**).
- La fecha en que finalizó el periodo de actividad (**Fecha fin**).
- El motivo que ha producido el periodo de inactividad (**Motivo baja**).
- Un ordinal que identifica los distintos periodos de inactividad de un mismo empleado (**Periodos**).

ID RH	Periodos	Fecha inicio	Fecha fin	Motivo baja
01	1	25-01-1998	27-01-1998	Enfermedad
01	2	15-06-1998	21-06-1998	Enfermedad
02	1	15-02-1998	19-03-1998	Accidente lab.
02	2	04-07-1998	05-07-1998	Enfermedad
02	3	17-08-1998	20-08-1998	Enfermedad
02	4	18-10-1998	30-10-1998	Accidente lab.
03	1	25-02-1998	27-02-1998	Permiso
03	2	05-05-1998	06-05-1998	Permiso

Se quiere diseñar una estructura de nodo sobre esta tabla que recupere el número de periodos de inactividad de cada empleado a lo largo del último año, producidos por una misma causa.

Para crear esta estructura de nodo, se eligen como criterios de agrupación los campos **ID RH** (código identificativo del empleado) y **Motivo baja** (motivo de la interrupción).

Para obtener el número de absentismos dentro de cada grupo, se aplica la función Count() al campo **ID RH**. Al ejecutar la estructura de nodo se recuperan los registros de la siguiente tabla:

ID RH	Motivo baja	Total
01	Enfermedad	2
02	Accidente lab.	2
02	Enfermedad	2
03	Permiso	2

Ordenación de registros en agrupaciones

La ordenación puede aplicarse a los registros obtenidos al aplicar una agrupación o totalización.

Se puede seleccionar como criterios de ordenación cualquiera de las columnas recuperadas por la estructura de nodo:

- Las columnas utilizados como criterios de agrupación de registros.
- Las columnas a los que se han aplicado las funciones de totalización.

Es posible utilizar dos o más columnas como criterios de ordenación. Si se aplican dos o más criterios sobre el resultado de una totalización, el sistema se comportará de la misma forma que en las estructuras de nodo sin totalización.

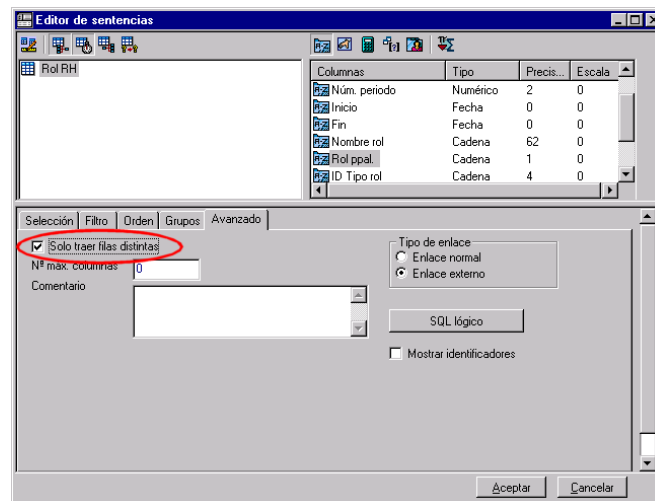
Eliminación de registros duplicados

La sentencia de recuperación de datos de una estructura de nodo puede recuperar los registros que tienen los mismos valores en todas sus columnas.

En la definición de la sentencia de la estructura de nodo, indique que se deben eliminar los registros duplicados de la lista de registros cargados en memoria principal, para que sólo aparezcan una única vez en la lista.

Para ello, en el Editor de sentencias active la opción **Solo traer filas distintas** de la pestaña **Avanzado**.

Figura 25. Editor de sentencias: opción Solo traer filas distintas



Esta opción equivale a la palabra reservada DISTINCT del lenguaje ANSI-SQL.

EJEMPLO

La tabla **Absentismos** contiene los siguientes registros:

ID RH	Periodos	Fecha inicio	Fecha fin	Motivo baja
01	1	25-01-1999	27-01-1999	Enfermedad
01	2	15-06-1999	21-06-1999	Enfermedad
02	1	15-02-1999	19-03-1999	Accidente lab.
02	2	04-07-1999	05-07-1999	Enfermedad
02	3	17-08-1999	20-08-1999	Enfermedad
02	4	18-10-1999	30-10-1999	Accidente lab.

ID RH	Periodos	Fecha inicio	Fecha fin	Motivo baja
03	1	25-02-1999	27-02-1999	Permiso
03	2	05-05-1999	06-05-1999	Permiso

Se quiere diseñar una estructura de nodo que enumere las razones que han motivado las interrupciones de cada empleado, es decir, las columnas **ID RH** y **Motivo baja**.

Al ejecutarse la estructura de nodo, se cargan en memoria los siguientes datos:

ID RH	Motivo baja
01	Enfermedad
01	Enfermedad
02	Accidente lab.
02	Enfermedad
02	Enfermedad
02	Accidente lab.
03	Permiso
03	Permiso

El conjunto de registros recuperados incluye datos redundantes. Para eliminarlos, active la opción *Solo traer filas distintas* cuando diseñe la estructura de nodo. En este caso, se recuperarán los siguientes registros:.

ID RH	Motivo baja
01	Enfermedad
02	Accidente lab.
02	Enfermedad
03	Permiso

Disparadores

La ejecución de los elementos de tipo método o concepto de una estructura de nodo se puede automatizar para que, cuando se lleve a cabo, se desencadene la ejecución de otros métodos o conceptos de la misma estructura de nodo.

Un disparador consiste en dos elementos de tipo método o concepto conectados, de forma que la ejecución de uno desencadena la del otro. El primero es el elemento origen de la llamada; el segundo, el elemento destino.

Los disparadores también permiten asegurar que dos elementos de tipo propiedad recogen el mismo valor.

Se puede definir un número variable de disparadores sobre una misma estructura de nodo. Un mismo elemento puede ser origen y destino de varios disparadores.

Los disparadores que se definen a nivel de estructura de nodo están activos en todos los nodos que crean la instancia de la estructura de nodo. Es decir, en todos los nodos que utilizan la estructura de nodo para extraer información de la base de datos. También se pueden diseñar disparadores que sólo estén activos en un determinado nodo.

La ejecución automática de métodos o conceptos no sólo se puede desencadenar mediante disparadores a nivel de estructura de nodo. También se puede utilizar en:

- Llamadas a métodos del mismo Meta4Object o de otros Meta4Objects en reglas LN4
- Unión de métodos de un mismo o diferentes nodos del mismo Meta4Object mediante conectores
- Declaración de eventos que automaticen la ejecución de métodos y conceptos de Meta4Objects diferentes

Estas posibilidades se explican en el capítulo Conectores de este mismo manual.

Estructuras de nodo: conceptos avanzados

4

Acerca de este capítulo

En el capítulo *Estructuras de nodo: conceptos básicos* se han descrito los conceptos básicos sobre nodos y estructuras de nodo: funcionalidad, estructura básica, tipos de elementos, filtros, ordenación de registros y agrupaciones.

En este capítulo se explica:

- El diseño de estructuras de nodo que recuperen registros de dos o más tablas mediante un *Enlace*
- El tratamiento de valores nulos en Enlaces, y el concepto de *Enlace externo*
- Los métodos definidos por defecto en las estructuras de nodo: métodos del sistema

Estructuras de nodo diseñadas sobre más de una tabla

Una estructura de nodo puede recuperar datos de una única tabla o de más de una tabla. En este último caso, la estructura de nodo recupera de forma unificada los registros de todas las tablas que se han especificado al diseñar la sentencia de la estructura de nodo.

Para los lectores que estén familiarizados con el lenguaje SQL, el efecto de crear una estructura de nodo sobre más de una tabla equivaldría a una sentencia Select sobre dos o más tablas unidas mediante un Enlace.

Normalización

La necesidad de trabajar con sentencias que recuperen datos de más de una tabla es consecuencia del proceso de normalización que se sigue en el diseño del modelo de datos.

Las aplicaciones basadas en la tecnología Meta4, Knownet o PeopleNet trabajan con un modelo relacional extendido. Una de las principales ventajas de los modelos relacionales es que permiten obtener un modelo de datos normalizado. Es decir, un modelo en el que se distribuye la información relacionada con una misma entidad u objeto en un número variable de registros y tablas.

El propósito de la normalización es reducir al mínimo la redundancia, sin que se pierda información.

EJEMPLO

La información relativa a los candidatos que se ofrecen para cubrir un puesto de trabajo se distribuye en varias tablas.

Por ejemplo, la tabla PERSONA recoge los datos personales de los candidatos; la tabla CANDIDATO RH contiene los códigos identificativos de todos los candidatos; SEGUIMIENTO CANDIDATO enumera las acciones de seguimiento que realiza la empresa durante el proceso de selección (entrevistas, pruebas, etc.) y ORIGEN CANDIDATO indica el origen de los candidatos (universidades, empresas de trabajo temporal, consultoras de selección, etc.).

De esta forma, para obtener una visión unificada de toda la información sobre un candidato, las fuentes de selección correspondientes y las acciones de seguimiento a las que se le ha sometido, debe recuperar información de estas tablas.



Cuando una o varias columnas de una tabla tienen que contener un valor que ya existe en un registro de otra tabla, se produce una relación de clave externa entre ambas tablas. Para obtener más información, consulte el manual *Metodología de diseño del modelo de datos*.

Para saber qué registros de las tablas SEGUIMIENTO CANDIDATO y ORIGEN CANDIDATO corresponden a cada uno de los candidatos indicados en la tabla CANDIDATO RH, SEGUIMIENTO CANDIDATO y ORIGEN CANDIDATO incluyen una columna que recoge los códigos identificativos de los candidatos a los que corresponde la información de cada registro.

Estos códigos deben haberse grabado con anterioridad en la tabla CANDIDATO RH. Es decir, en SEGUIMIENTO CANDIDATO y ORIGEN CANDIDATO no puede haber referencias a candidatos inexistentes.

Enlaces

Para resolver el problema que surge con motivo de la normalización, el lenguaje SQL ofrece la posibilidad de obtener una visión unificada de los registros distribuidos entre distintas tablas que recogen datos de una misma entidad.

Para obtener esta visión unificada de la información, el modelo relacional incluye una operación especial: el enlace entre tablas.

En la definición de un enlace se utilizan: las dos tablas de las que se quiere obtener una visión unificada y las columnas de estas tablas que se utilizan como criterios del enlace.

Para poder aplicar un enlace a un par de tablas, cada una de éstas debe tener al menos una columna que recoja el mismo tipo de datos. Así, no es posible definir un enlace entre una columna de una tabla con valores numéricos y una columna de otra tabla con valores de tipo fecha. No es necesario que las dos columnas tengan el mismo nombre.

Aparte de esta restricción, hay que tener en cuenta que los pares de columnas que se utilizan como criterios del enlace deben mantener entre sí una relación lógica. Aquí se distinguen dos casos:

- Las columnas que se utilizan para definir el enlace identifican a la misma entidad
- Las columnas que se utilizan para definir el enlace recogen información que quiere compararse y que no se refiere a una misma entidad

EJEMPLO

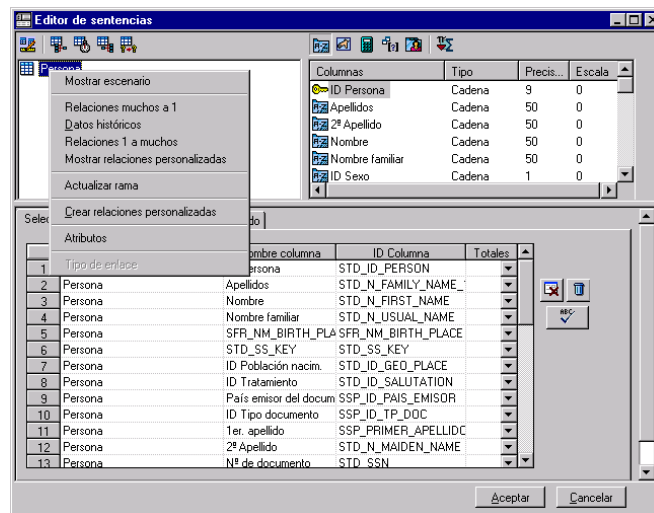
Un ejemplo del primer caso es el descrito anteriormente, acerca de la información sobre candidatos. En ese ejemplo, las tres tablas contienen una columna que recoge los códigos identificativos de los candidatos. Los registros de estas tablas que tengan el mismo valor en esta columna recogerán información sobre el mismo candidato.

Un ejemplo del segundo caso puede ser un enlace que se defina entre dos tablas que recogen el total facturado por los empleados de cada sucursal comercial durante un periodo de tiempo determinado, y el total que se ha pagado a cada uno de ellos en ese mismo periodo. Se quiere saber qué sucursales han facturado al menos el 75% del total que suponen sus costes salariales.

Para crear este enlace, se tienen que comparar los valores registrados en las columnas **Total_Facturado** y **Total_Abonado**.

El primer tipo de enlace es el más común. Al estar basado en el concepto de clave externa, la herramienta Editor de sentencias de la aplicación es capaz de generar automáticamente un listado con todas las tablas entre las que se puede establecer un enlace.

Figura 26. Relaciones entre tablas Editor de sentencias



La aplicación también permite diseñar enlaces adicionales a los que se generan tomando como base la definición de claves externas.

Ejecución de un enlace

Un enlace se define entre dos tablas. Por tanto, lo primero que hay que saber es de qué tablas queremos obtener información unificada. Las dos tablas tendrán que tener al menos un atributo que recoja el mismo tipo de dato.

EJEMPLO

Se quieren recuperar los datos de cada candidato, junto a las acciones de seguimiento que han realizado los técnicos de recursos humanos.

Para ello, se tiene que crear un enlace entre las tablas CANDIDATO RH y SEGUIMIENTO CANDIDATO.

ID candidato	Fecha inicio	Inicio seguimiento	Tipo candidato
01	22-07-1997	Solicitud externa	Externo
02	10-04-1998	Petición interna	Interno
03	03-03-1999	Presentación empleado	Externo
04	15-05-1998	Solicitud externa	Externo

ID candidato	Fecha inicio	OR seguimiento	OR selección	Tipo Seguimiento	Fecha Seguimiento
01	22-07-1997	1	1	Citación	01-08-1997
01	22-07-1997	2	1	Entrevista	06-08-1997
01	22-07-1997	3	1	Oferta inicial	12-08-1997
02	10-04-1998	1	3	Entrevista	13-04-1998
02	10-04-1998	2	3	Rechazo	16-04-1998
04	15-05-1998	1	10	Citación	19-05-1998
04	15-05-1998	2	10	Entrevista	25-05-1998
04	15-05-1998	3	10	Rechazo	26-05-1998

Las dos tablas contienen una columna que hace referencia a la misma entidad, **ID candidato**. Cada candidato se identifica de forma inequívoca mediante su **ID candidato**. La columna **ID candidato** se selecciona como criterio del enlace.

Al ejecutar un enlace entre dos tablas, el sistema gestor de bases de datos realiza las siguientes operaciones:

- Crea una tabla resultante que contiene los registros que se obtendrían al realizar el producto cartesiano de las dos tablas
- A continuación, elimina todos los registros de la tabla resultante que no tengan el mismo valor en las columnas indicados como criterios del enlace

EJEMPLO

Continuando con el ejemplo anterior, la ejecución de un enlace entre las tablas **CANDIDATO RH** y **SEGUIMIENTO CANDIDATO** realiza la siguiente operación:

- Obtiene el producto cartesiano de las dos tablas. El resultado de esta operación es una única tabla con tantas columnas como tienen las tablas CANDIDATO RH y SEGUIMIENTO CANDIDATO. Cada registro de esta tabla está formado por la yuxtaposición de los registros de CANDIDATO RH, con todos y cada uno de los registros de SEGUIMIENTO CANDIDATO, tal y como se muestra en la siguiente tabla:

ID candidato	Fecha inicio	Inicio seguimiento	Tipo candidatos	ID candidato	Fecha inicio	OR seguimiento	OR selección	Tipo Seguimiento	Fecha seguimiento
1	22-07-1997	Solicitud ext.	Ext.	1	22-07-1997	1	1	Citación	01-08-1997
1	22-07-1997	Solicitud ext.	Ext.	1	22-07-1997	2	1	Entrevista	06-08-1997
1	22-07-1997	Solicitud ext.	Ext.	1	22-07-1997	3	1	Oferta inicial	12-08-1997
1	22-07-1997	Solicitud ext.	Ext.	2	10-04-1998	1	3	Entrevista	13-04-1998
1	22-07-1997	Solicitud ext.	Ext.	2	10-04-1998	2	3	Rechazo	16-04-1998
1	22-07-1997	Solicitud ext.	Ext.	4	15-05-1998	1	10	Citación	19-05-1998
1	22-07-1997	Solicitud ext.	Ext.	4	15-05-1998	2	10	Entrevista	25-05-1998
1	22-07-1997	Solicitud ext.	Ext.	4	15-05-1998	3	10	Rechazo	26-05-1998
2	10-04-1998	Petición int.	Int.	1	22-07-1997	1	1	Citación	01-08-1997
2	10-04-1998	Petición int.	Int.	1	22-07-1997	2	1	Entrevista	06-08-1997
2	10-04-1998	Petición int.	Int.	1	22-07-1997	3	1	Oferta inicial	12-08-1997
2	10-04-1998	Petición int.	Int.	2	10-04-1998	1	3	Entrevista	13-04-1998
2	10-04-1998	Petición int.	Int.	2	10-04-1998	2	3	Rechazo	16-04-1998
2	10-04-1998	Petición int.	Int.	4	15-05-1998	1	10	Citación	19-05-1998

ID candidato	Fecha inicio	Inicio seguimiento	Tipo candidatos	ID candidato	Fecha inicio	OR seguimiento	OR selección	Tipo Seguimiento	Fecha seguimiento
2	10-04-1998	Petición int.	Int.	4	15-05-1998	2	10	Entrevista	25-05-1998
2	10-04-1998	Petición int.	Int.	4	15-05-1998	3	10	Rechazo	26-05-1998
3	03-03-1999	Presentación empleado	Ext.	1	22-07-1997	1	1	Citación	01-08-1997
3	03-03-1999	Presentación empleado	Ext.	1	22-07-1997	2	1	Entrevista	06-08-1997
03	03-03-1999	Presentación empleado	Ext.	1	22-07-1997	3	1	Oferta inicial	12-08-1997
03	03-03-1999	Presentación empleado	Ext.	2	10-04-1998	1	3	Entrevista	13-04-1998
03	03-03-1999	Presentación empleado	Ext.	2	10-04-1998	2	3	Rechazo	16-04-1998
03	03-03-1999	Presentación empleado	Ext.	4	15-05-1998	1	10	Citación	19-05-1998
03	03-03-1999	Presentación empleado	Ext.	4	15-05-1998	2	10	Entrevista	25-05-1998
03	03-03-1999	Presentación empleado	Ext.	4	15-05-1998	3	10	Rechazo	26-05-1998
04	15-05-1998	Solicitud ext.	Ext.	1	22-07-1997	1	1	Citación	01-08-1997
04	15-05-1999	Solicitud ext.	Ext.	1	22-07-1997	2	1	Entrevista	06-08-1997
04	15-05-1998	Solicitud ext.	Ext.	1	22-07-1997	3	1	Oferta inicial	12-08-1999
04	15-05-1998	Solicitud ext.	Ext.	2	10-04-1998	1	3	Entrevista	13-04-1998
04	15-05-1998	Solicitud ext.	Ext.	2	10-04-1998	2	3	Rechazo	16-04-1998
04	15-05-1998	Solicitud ext.	Ext.	4	15-05-1998	1	10	Citación	19-05-1998
04	15-05-1998	Solicitud ext.	Ext.	4	15-05-1998	2	10	Entrevista	25-05-1998
04	15-05-1998	Solicitud ext.	Ext.	4	15-05-1998	3	10	Rechazo	26-05-1998

A continuación, se eliminan de la tabla resultante los registros que no tengan el mismo valor en las columnas utilizadas como criterios del enlace. En este caso, las columnas **ID candidato**, que ocupan las columnas primera y quinta de la tabla

ID candidato	Fecha inicio	Inicio seguimiento	Tipo candidato	ID candidato	Fecha inicio	OR seguimiento	OR selección	Tipo Seguimiento	Fecha seguimiento
1	22-07-1996	Solicitud externa	Ext.	1	22-07-1996	1	1	Citación	01-08-1996
1	22-07-1996	Solicitud externa	Ext.	1	22-07-1996	2	1	Entrevista	06-08-1996
1	22-07-1996	Solicitud externa	Ext.	1	22-07-1996	3	1	Oferta inicial	12-08-1996
2	10-04-1997	Petición interna	Int.	2	10-04-1997	1	3	Entrevista	13-04-1997
2	10-04-1997	Petición interna	Int.	2	10-04-1997	2	3	Rechazo	16-04-1997
04	15-05-1997	Solicitud externa	Ext.	4	15-05-1997	1	10	Citación	19-05-1997
04	15-05-1997	Solicitud externa	Ext.	4	15-05-1997	2	10	Entrevista	25-05-1997
04	15-05-1997	Solicitud externa	Ext.	4	15-05-1997	3	10	Rechazo	26-05-1997

De esta forma, como resultado del enlace se obtiene una única tabla con la información unificada de las tablas CANDIDATO RH y SEGUIMIENTO CANDIDATO.

Enlaces y estructuras de nodo

La aplicación permite crear estructuras de nodo que recuperen los registros obtenidos al aplicar un enlace a dos o más tablas.

La estructura y funcionalidad de estos enlaces es igual a la de los Joins SQL descritos en el apartado anterior. Como criterios del enlace, se seleccionan columnas de las tablas que se unen mediante el enlace.

El enlace se define sobre tablas, y no sobre los registros recuperados en memoria principal por las estructuras de nodo. Por lo tanto, las operaciones requeridas por el enlace no se realizan en memoria principal del servidor de aplicaciones o del puesto cliente. Las operaciones de enlace entre tablas las realiza el sistema gestor de bases de datos.

Cuando se define un enlace entre dos tablas que tienen una relación de clave externa, no es necesario indicar qué columnas van a utilizarse como criterios del enlace. Éstos van a ser:

-
- La columna o columnas que conforman la clave primaria de la tabla padre de la relación
 - La columna o columnas que conforman la clave externa de la tabla hija de la relación



Estas condiciones del enlace que agrega el Editor de sentencias permanecen ocultas al usuario de la aplicación.

La herramienta Editor de sentencias incluye automáticamente la condición que iguala el valor de estas columnas dentro de las condiciones del filtro.

Cuando se crea una estructura de nodo que incluye un enlace, el responsable del diseño puede agregar condiciones adicionales, mediante las cuales se comparan o igualan los valores de otras columnas de las tablas que se relacionan mediante el enlace.

Estas condiciones se tienen que indicar de forma explícita. Se muestran en el Editor de sentencias siempre que se edite la definición de la estructura de nodo.

Por lo demás, las estructuras de nodo que se diseñan sobre dos tablas unidas mediante un enlace tienen la misma funcionalidad y comportamiento que diseñadas sobre una única tabla.

El responsable de diseño puede restringir el conjunto de columnas cuyo valor se quiere recuperar; asimismo, puede indicar criterios de ordenación, selección, agrupación y totalización de registros.

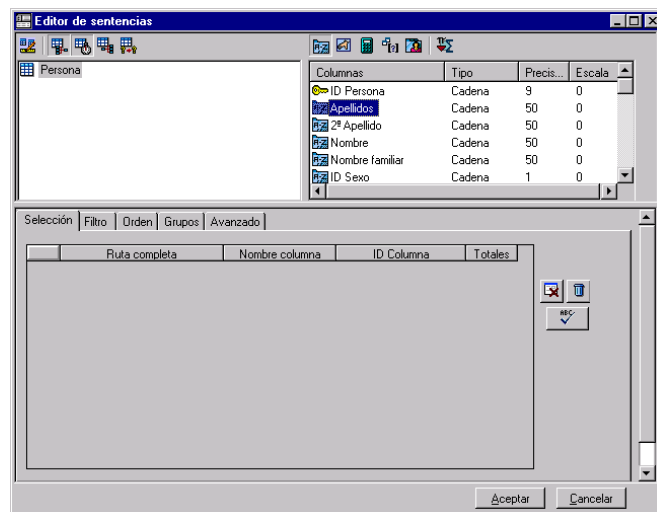
Diseño de estructuras de nodo con Enlaces

En este apartado, se explica la creación de una estructura de nodo que recupera datos de dos tablas unidas mediante un enlace.

Al iniciar el Editor de sentencias, se tiene que seleccionar una de las tablas de las que se quieren recuperar datos. Ésta es la tabla base.

El Editor de sentencias mostrará un aspecto similar al de la siguiente imagen:

Figura 27. Editor de sentencias: ventana inicial



El recuadro situado en la parte superior izquierda muestra la tabla que se ha seleccionado.

El recuadro de la parte superior derecha muestra las columnas de la tabla seleccionada en el recuadro anterior.

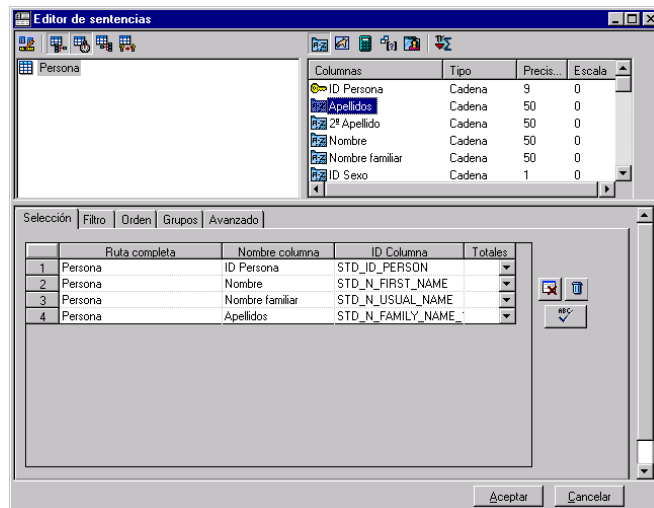
Seleccione las columnas que desee recuperar. Para cada columna que se seleccione, se agregará un elemento de tipo campo a la estructura de nodo.

EJEMPLO

En este ejemplo, se seleccionan las columnas **ID Persona**, **Nombre**, **Apellidos** y **Nombre familiar**.

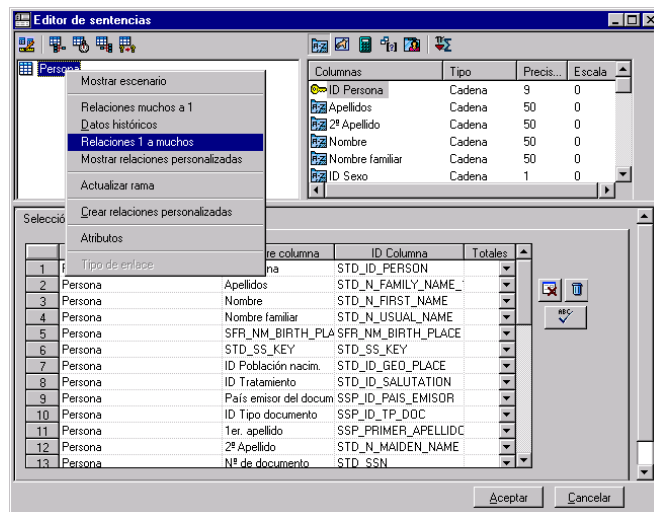
Para seleccionar las columnas, haga doble clic en cada una de ellas. A medida que se seleccionan, se van agregando a la rejilla situada en la parte inferior de la ventana.

Figura 28. Editor de sentencias con columnas seleccionadas



A continuación, se selecciona la segunda tabla de la que se quieren extraer datos. Para ello, haga clic con el botón derecho del ratón en la tabla base. Se despliega un menú emergente como el de la siguiente imagen:

Figura 29. Editor de sentencias: relaciones entre tablas



En esta ventana es posible seleccionar una de las tablas con las que se ha relacionado la tabla base al ser definida en el diccionario de datos desde el Diseñador del modelo de datos (opciones **Relaciones muchos a 1**, **Datos históricos**, **Relaciones 1 a muchos**), o definir una nueva relación mediante la opción **Crear relaciones personalizadas**.. Puede tener acceso a estas mismas opciones desde los botones que se encuentran en la parte superior de este panel.

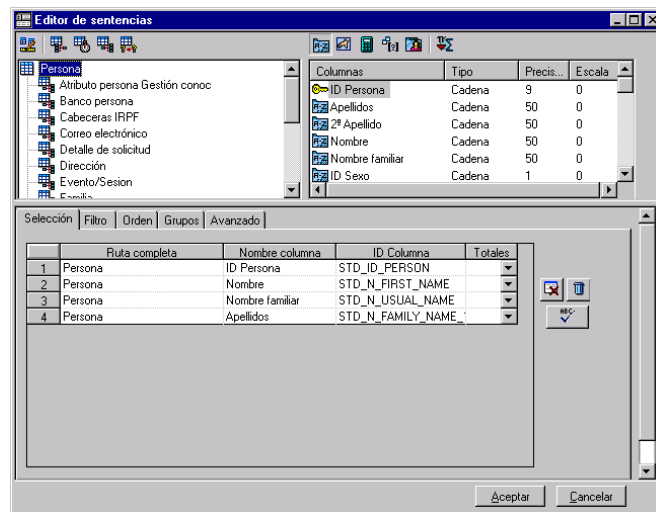
EJEMPLO

En el ejemplo, se selecciona la tabla **Dirección**, que contiene las direcciones de las personas registradas en la base de datos.

Como la relación entre las tablas **Persona** y **Dirección** está establecida en el modelo de datos estándar, se selecciona la opción de menú **Relaciones 1 a muchos**.

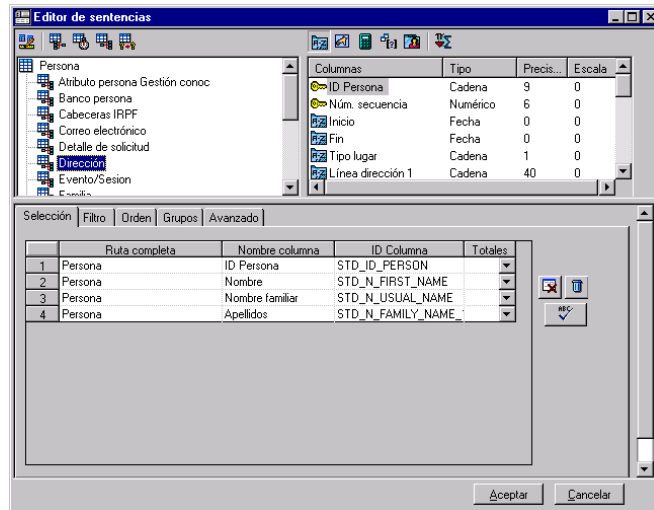
Seleccione la opción **Relaciones 1 a muchos** y, a continuación, haga doble clic en el icono que representa la tabla base. Se despliega una lista con todas las tablas que mantienen una relación en el diccionario de datos con la tabla base.

Figura 30. Lista de tablas relacionadas con la tabla base



En esta lista, haga clic en el icono correspondiente a la segunda tabla que se quiere unir en el enlace. El recuadro de la derecha muestra las columnas de esta segunda tabla.

Figura 31. Selección de la segunda tabla

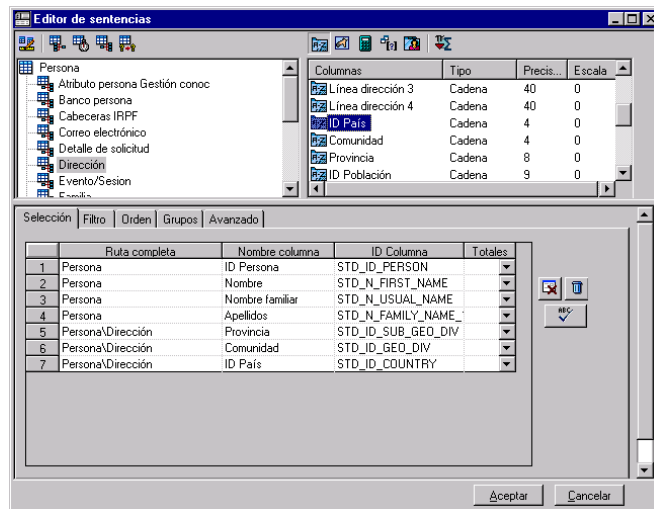


Seleccione las columnas de la segunda tabla cuyos valores quiere recuperar. Para ello, haga doble clic en dichos valores.

EJEMPLO

En el ejemplo, se seleccionan las columnas correspondientes a la dirección completa.

Figura 32. Selección de columnas de la segunda tabla



Si la estructura de nodo no va a incluir ningún filtro u ordenación de registros, su definición puede darse por finalizada.

En el caso de que se quiera agregar alguna condición de selección de los registros, se tendrá que diseñar su filtro. Como las tablas unidas mediante el enlace están relacionadas mediante una clave externa en el diccionario de datos, no es necesario igualar las columnas que constituyen el criterio del enlace (en el ejemplo, las columnas **ID_Persona**).



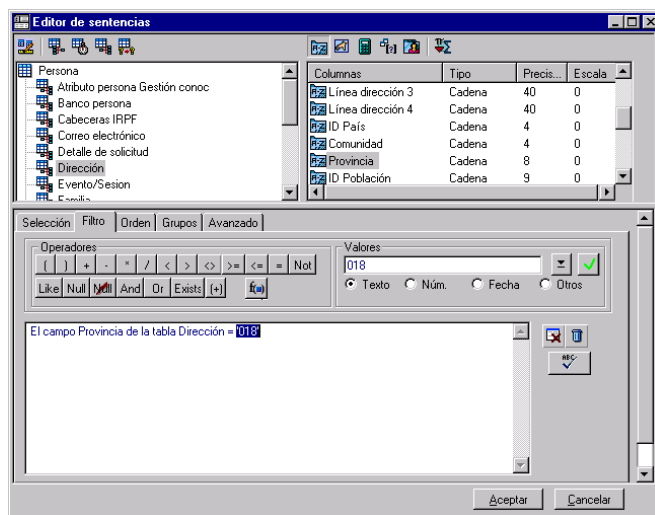
La condición del enlace sólo se agrega al filtro si se ha seleccionado una tabla mediante la opción **Nuevas Relaciones**.

EJEMPLO

En el ejemplo, sólo se va a recuperar la información correspondiente a los empleados cuyo domicilio se encuentra en un área geográfica determinada.

Por lo tanto, hay que agregar esta condición al filtro. No se debe agregar la condición que iguala los valores de las dos columnas que actúan como criterios del enlace. El Editor de sentencias genera internamente estas condiciones.

Figura 33. Editor de sentencias: opciones de filtro



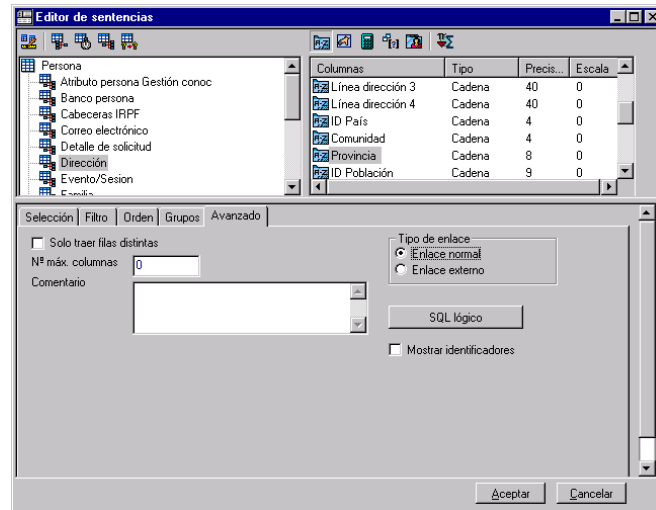
Para concluir, antes de grabar la definición de la estructura de nodo, es necesario indicar qué tipo de enlace se quiere utilizar: **Enlace normal** o **Enlace externo**.



Estos tipos de enlace se explican en el siguiente apartado de este capítulo.

En el Editor de sentencias, el tipo de enlace que se va a aplicar se indica mediante los botones de opción disponibles en la pestaña **Avanzado** (recuadro **Tipo de enlace**).

Figura 34. Tipos de enlace



Por defecto, siempre está activa la opción **Enlace normal**. Si quiere aplicar un enlace de otro tipo, active la opción **Enlace externo**.

EJEMPLO

Las tablas Persona y Direcciones tienen los registros que se indican a continuación:

ID Persona	Nombre	1er Apellido	2ª Apellido
01	Juan	Varela	Sanz
02	Ricardo	Pérez	Pérez
03	Francisco	Carrasco	González
04	María	Díez	Fenández
05	Susana	Cobo	Velando
06	Luis	Fernández	Herrera

ID_Persona	Dirección	Ciudad	Código País
01	Castellana 308	Madrid	SPA
01	Gran vía 104	Barcelona	SPA
02	Baeza 67	Sevilla	SPA
04	Villarroya 42	Valencia	SPA

La ejecución de la estructura de nodo recuperará en memoria los siguientes datos:

ID Persona	Nombre	1er Apellido	2ª Apellido	Dirección	Ciudad	Código País
01	Juan	Varela	Sanz	Castellana 308	Madrid	SPA
01	Juan	Varela	Sanz	Gran vía 104	Barcelona	SPA
02	Ricardo	Pérez	Pérez	Baeza 67	Sevilla	SPA
04	María	Díez	Fernández	Villarroya 7	Valencia	SPA

Tipos adicionales de enlaces

El enlace normal entre tablas tiene una limitación: únicamente se recupera información de la base de datos para aquellas entidades de las que hay información en las dos tablas que se unen mediante el enlace.

Cuando se ejecuta un enlace entre dos tablas, el sistema gestor de bases de datos devuelve los registros de las dos tablas siempre que, en la otra tabla, exista un registro con el mismo valor en las columnas seleccionadas como criterios del enlace.

La operación de enlace permite mostrar una visión unificada de los datos que corresponden a las mismas entidades y están distribuidos en distintas tablas.

Sin embargo, si una de las dos tablas contiene registros para los que no hay referencias en la otra tabla afectada por el enlace, el resultado no dará ninguna información sobre la existencia de esta información.

Esta situación se resuelve con los enlaces externos.

En este caso, el sistema no sólo devuelve información sobre las entidades para las que existen registros en las dos tablas, sino que también muestra los registros de aquellas entidades de las que sólo existen registros en una de las dos tablas unidas mediante el enlace.

Valor nulo

Los elementos de tipo campo de una estructura de nodo leen sus valores de las columnas de las tablas de la base de datos.

Las columnas de la base de datos pueden recoger valores nulos. El valor nulo se utiliza siempre que:

- Se desconoce el valor de una columna
- La columna corresponde a una propiedad que no es aplicable a la instancia para la que se rellena el registro

El valor nulo no debe confundirse con el valor 0 o con una cadena de caracteres vacía. Siempre que se quiera expresar una de las dos condiciones anteriores, debe utilizarse el valor nulo.

Claves externas nulas

Se establece una relación de clave externa entre dos tablas siempre que los valores que recoge una columna o grupo de columnas de una de ellas, aparezcan en uno de los registros de la otra tabla.

En este caso, la columna o grupo de columnas que recogen los valores presentes en la otra tabla constituyen una clave externa.

La tabla hija de la relación es la tabla que incluye la clave externa. La tabla padre de la relación es la que incluye la columna a cuyos valores hace referencia la clave externa.

Una clave externa puede ser de dos tipos: nula o no nula.

La clave externa no nula se utiliza cuando la columna o columnas que conforman la clave externa de la tabla hija no pueden recoger valores nulos.

La clave externa nula se utiliza cuando la columna o columnas que conforman la clave externa pueden recoger valores nulos. En el caso de que la clave externa esté formada por más de una columna, todas las columnas de la clave externa tendrán que ser nulos o no nulos.

EJEMPLO

Por necesidades de parametrización, se agregan al modelo de datos de la aplicación una tabla que recoge los distintos proyectos en los que participa un empleado a lo largo del tiempo y el tipo de tareas que realiza.

ID_RH	ID_Proyectos	ID_Tareas	Fecha_Inicio	Fecha_Fin
01	1004	02-A	22-07-1999	30-09-1999
01	1004	02-R	01-10-1999	NULO
01	1015	02-A	28-09-1999	NULO
02	1015	02-F	10-06-1999	30-09-1999
02	1015	02-G	20-09-1999	NULO
03	1004	NULO	01-10-1999	NULO

Esta tabla está relacionada con la tabla Recurso Humano (de la que toma valores para identificar a los empleados) y con otras dos tablas que recogen, respectivamente, los códigos identificativos de cada proyecto y los de las tareas que se pueden realizar.

Un empleado puede estar adscrito a un proyecto sin necesidad de especificar el tipo de tarea que va a realizar. De esta forma, entre la tabla Histórico de proyectos y la tabla de tareas se establece una relación de tipo clave externa nula.

En la tabla de ejemplo, el empleado con código identificativo 3 se ha asignado al proyecto 1004, pero no se ha especificado el tipo de tarea que va a realizar. Cuando se conozca la tarea, se actualizará este registro.

Valores no referenciados

Un problema parecido al anterior sucede cuando la tabla padre de la relación contiene registros para los que no existen referencias en la tabla hija.

EJEMPLO

La aplicación contiene una tabla en la que se enumeran todos los proyectos en los que participa un empleado.

No todos los empleados deben estar adscritos a un proyecto. Así, en las tablas de ejemplo, los empleados con códigos identificativos 04 y 05 no están trabajando en un proyecto específico. La tabla Histórico de proyectos no contiene registros que hagan referencia a estos dos empleados. Por ello, la tabla de empleados contiene valores a los que no se hace referencia en la tabla hija.

ID_RH	ID_Proyectos	ID_Tareas	Fecha_Inicio	Fecha_Fin
01	1004	02-A	22-07-1999	30-09-1999
01	1004	02-R	01-10-1999	NULO
01	1015	02-A	28-09-1999	NULO
02	1015	02-F	10-06-1999	30-09-1999
02	1015	02-G	20-09-1999	NULO
03	1004	NULO	01-10-1999	NULO

ID_RH	Nombre	1er_Apellido
01	Roberto	González
02	Juan	Varela
03	Ricardo	Pérez
04	María	Díez
05	Susana	Cobo

Si se ejecuta un enlace entre estas dos tablas, únicamente se obtendrá información para los registros de la tabla Empleados que están asignados a algún proyecto.

Los registros no referenciados permanecerán ocultos y no aparecerá ninguna información de los empleados que no están asignados a un proyecto.

Enlaces externos

Para resolver los problemas que surgen como consecuencia de los enlaces definidos sobre tablas con claves externas nulas y los valores no referenciados, el lenguaje SQL incorpora otro tipo de enlace: los enlaces externos.

Este tipo de enlace recupera no sólo la información de las entidades para las que existen registros en las dos tablas unidas mediante el enlace, sino también de las siguientes:

- Entidades registradas en la tabla padre de la relación a las que no se hace referencia en ningún registro de la tabla hija
- Entidades registradas en la tabla hija que contienen una clave externa nula

Por lo tanto, a la hora de crear un enlace externo se puede distinguir entre unión izquierda y unión derecha. A continuación se comentan las peculiaridades de cada uno de ellos aunque en definitiva se tratan ambos dos enlaces externos.

Unión izquierda

Para definir una sentencia SQL con un unión izquierda entre dos tablas, es necesario que la cláusula WHERE contenga estas columnas escritas en el mismo orden que el contemplado en la cláusula FROM.

Como norma de estilo en la creación de sentencias SELECT, escriba en primer lugar la tabla padre de la relación (que contiene la clave primaria a la que se hace referencia) y, en segundo lugar, la tabla hija (es decir, la que contiene la clave externa definida sobre la anterior).

Para indicar que se quiere crear un enlace con unión izquierda, agregue el carácter (+) tras el nombre de la columna situado a la izquierda del signo igual en los criterios del enlace.

EJEMPLO

Para diseñar un enlace con unión izquierda entre las tablas Tareas e Hco_Proyectos, debe escribir la siguiente sentencia:

```
Select A.Nombre_Tareas, B.Nombre_Proyecto
From Tareas A, Hco_Proyectos B
Where A.ID_Tareas(+) = B.ID_Tareas
```

Las columnas **ID_Tareas** de las dos tablas que se utilizan como criterio del enlace, ocupan el mismo lugar que las tablas que se han recuperado en la cláusula From de la sentencia SQL.

Como resultado de un enlace con unión izquierda entre dos tablas, se obtiene una tabla en la que se incluyen:

- Los registros que se obtendrían al hacer un enlace normal
- Los registros de la tabla escrita en segundo lugar que carecen de referencias en la primera tabla (bien porque la tabla contiene una clave externa nula, bien porque la tabla contiene valores no referenciados en la tabla de la izquierda)

En este segundo grupo de registros, las columnas de la tabla resultante correspondientes a la tabla de la izquierda se rellenan con valores nulos.

EJEMPLO

Las tablas Tareas e Hco._Proyectos contienen los siguientes registros:

ID_Tareas	Nombre_Tareas
02-A	Recogida datos
02-B	Contabilización costes
02-R	Entrevistas candidatos
02-F	Envío circulares
02-G	Notificación traslados

ID_RH	ID_Proyectos	ID_Tareas	Fecha_Inicio	Fecha_Fin
01	1004	02-A	22-07-1999	30-09-1999
01	1004	02-R	01-10-1999	NULO
01	1015	02-A	28-09-1999	NULO
02	1015	02-F	10-06-1999	30-09-1999
02	1015	02-G	20-09-1999	NULO
03	1004	NULO	01-10-1999	NULO

Se ejecuta la siguiente sentencia SELECT con un enlace con unión izquierda:

```
Select A.ID_Tareas, A.Nombre_Tareas, B.ID_Tareas,  
B.ID_Proyectos, B.ID_RH
```

```

From Tareas A, Hco_Proyectos B
Where A.ID_Tareas(+) = B.ID_Tareas

```

La sentencia recuperará los registros que recoge la siguiente tabla:

ID_Tareas	Nombre_Tareas	ID_Tareas	ID_RH	ID_Proyectos
02-A	Recogida datos	02-A	01	1004
02-R	Entrevistas candidatos	02-R	01	1004
02-A	Recogida datos	02-A	01	1015
02-F	Envío circulares	02-F	02	1015
02-G	Notificación traslados	02-G	02	1015
NULO	NULO	NULO	03	1004

A los registros que se obtendrían al hacer un enlace normal (los cinco primeros) se ha agregado un sexto registro que contiene la información del de la tabla Hco_Proyectos con la clave externa nula.

Las columnas de la tabla Tareas de este registro se han rellenado con el valor NULO.

La sentencia no ha recuperado el registro correspondiente a la tarea 02B-Contabilización de costes.

Unión derecha

Como en el caso anterior, las dos columnas que se igualan en la cláusula WHERE deben escribirse en el mismo orden que las tablas de las que forman parte en la cláusula FROM.

En esta última cláusula se escribe, en primer lugar, la tabla padre de la relación (que contiene la clave primaria a la que se hace referencia) y, a continuación, la tabla hija (es decir, la que contiene la clave externa definida sobre la anterior).

Para indicar que se quiere crear un enlace con unión derecha, agregue el carácter (+) tras el nombre de la columna situada a la derecha del signo igual en las columnas que se igualan en la cláusula WHERE.

EJEMPLO

Para diseñar un enlace con unión derecha entre las tablas Tareas e Hco_Proyectos, escriba la siguiente sentencia:

```

SELECT A.Nombre_Tareas, B.N_Proyectos
FROM Tareas A, Hco_Proyectos B
WHERE A.ID_Tareas = B.ID_Tareas(+)

```

Las columnas **ID_Tareas** de las dos tablas que se utilizan como criterio del enlace, ocupan el mismo lugar que las tablas que se han recuperado en la cláusula FROM.

Como resultado de un enlace con unión derecha entre dos tablas, se obtiene una tabla resultante en la que se incluyen:

- Los registros que se obtienen al hacer un enlace normal
- Los registros de la tabla escrita en primer lugar que carecen de referencias en la segunda tabla (la tabla padre recoge valores no referenciados)

En este segundo grupo de registros, las columnas de la tabla resultante correspondientes a la tabla de la derecha se rellenan con valores nulos.

EJEMPLO

Las tablas Tareas e Hco_Proyectos contienen los siguientes registros:

ID_Tareas	Nombre_Tareas
02-A	Recogida datos
02-B	Contabilización costes
02-R	Entrevistas candidatos
02-F	Envío circulares
02-G	Notificación traslados

ID_RH	ID_Proyectos	ID_Tareas	Fecha_Inicio	Fecha_Fin
01	1004	02-A	22-07-1999	30-09-1999
01	1004	02-R	01-10-1999	NULO
01	1015	02-A	28-09-1999	NULO
02	1015	02-F	10-06-1999	30-09-1999
02	1015	02-G	20-09-1999	NULO
03	1004	NULO	01-10-1999	NULO

Se ejecuta la siguiente sentencia SELECT con un enlace con unión derecha:

```
SELECT A.ID_Tareas, A.Nombre_Tareas, B.ID_Tareas,
       B.ID_Proyectos, B.ID_RH
FROM Tareas A, Hco_Proyectos B
WHERE A.ID_Tareas = B.ID_Tareas(+)
```

La sentencia recuperará los registros que recoge la siguiente tabla:

ID_Tareas	Nombre_Tareas	ID_Tareas	ID_RH	ID_Proyectos
02-A	Recogida datos	02-A	01	1004
02-R	Entrevistas candidatos	02-R	01	1004
02-A	Recogida datos	02-A	01	1015
02-F	Envío circulares	02-F	02	1015
02-G	Notificación traslados	02-G	02	1015
02-B	Contabilización costes	NULO	NULO	NULO

Como se puede apreciar, a los registros que se obtendrían al hacer un enlace normal (cinco primeros registros), se ha agregado un sexto registro que contiene la información de la tarea 02B-Contabilización de costes.

Las columnas de este registro que se extraen de la tabla Hco. Proyectos se han rellenado con el valor NULO.

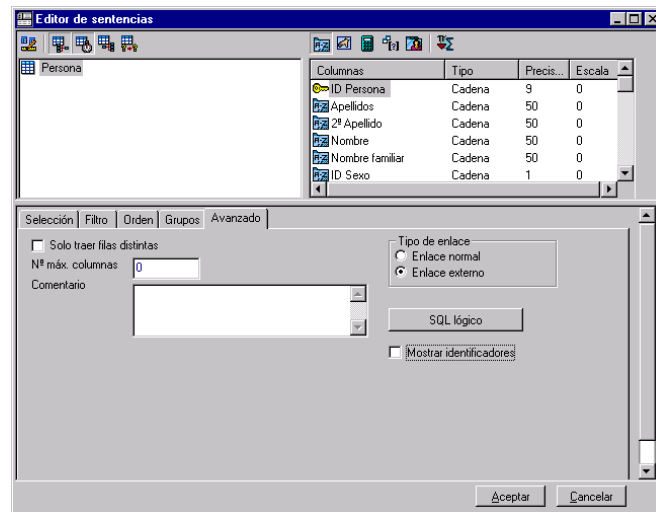
Por otra parte, el conjunto resultado no da ninguna información del registro de la tabla Hco_Proyectos, que recoge la clave externa nula.

Enlaces externos en la tecnología Meta4

El Editor de sentencias permite crear estructuras de nodo que extraigan datos de dos o más tablas unidas mediante un enlace externo.

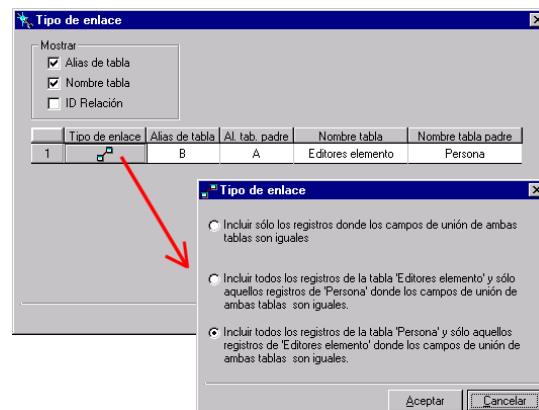
Para indicar qué tipo de enlace se quiere aplicar, active la opción correspondiente en el recuadro **Tipo de enlace** en la pestaña **Avanzado** del Editor de sentencias. Las dos opciones existentes son **Enlace normal**, que se encuentra seleccionada por defecto y **Enlace externo**.

Figura 35. Editor de sentencias: pestaña Avanzado



Para elegir el tipo de enlace, hay que situarse en la tabla y hacer clic en el botón derecho del ratón; aparece un menú contextual. Al hacer clic en la opción **Tipo de enlace**, se accede a una ventana desde la que puede seleccionarse el tipo de enlace entre las tablas.

Figura 36. Tipo de enlace



Al diseñar un enlace externo desde el Editor de sentencias, es necesario considerar el orden de los nombres de las tablas en la sentencia SQL generada por la herramienta.

La tabla padre de la relación siempre se escribe en primer lugar en la cláusula DE (FROM). La tabla hija se escribe en segundo lugar.

La columna de la tabla padre que se utiliza como criterio del enlace —cláusula DONDE (WHERE)— se escribe a la izquierda del signo igual.

La columna de la tabla hija que se utiliza como criterio del enlace —también en DONDE (WHERE)—, se escribe a la derecha del signo igual.

Este orden es importante, ya que desde el Editor de sentencias se pueden desplegar todas las tablas con las que está relacionada la tabla cuyas relaciones se quieren consultar.

Las tablas padre se despliegan al seleccionar la opción **Relaciones muchos a 1** y las tablas hijas al seleccionar la opción **Relaciones 1a muchos**.

En cualquier caso:

- Utilice el enlace con unión izquierda cuando desee obtener, además de los registros resultantes de un enlace, los de la tabla hija que contienen claves externas nulas
- Utilice el enlace con unión derecha cuando desee obtener, además de los registros resultantes de un enlace, los de la tabla padre para los que no existen referencias en la tabla hija

Métodos definidos por defecto en estructuras de nodo

Siempre que se ejecuta una estructura de nodo, el sistema agrega a su definición un conjunto de elementos de tipo método. Estos métodos se conocen como *métodos del sistema*.



Los métodos del sistema se agregan en tiempo de ejecución, y no forman parte de la estructura de nodo hasta que ésta no se ejecuta.

Estos métodos implementan funcionalidades básicas de la aplicación: inserción y borrado de registros, grabación de los cambios, desplazamiento entre registros, búsqueda de registros, etcétera.

Los elementos de la interfaz (botones de grabación, barra de navegación, etc.), implementan llamadas a estos métodos. También se pueden utilizar estos métodos desde reglas escritas en lenguaje LN4 del mismo Meta4Object o de un Meta4Object diferente.

Si se hace una llamada a uno de estos métodos desde la regla de un método o concepto del mismo nodo, sólo tendrá que escribir el nombre del método seguido de los argumentos que necesite. Escriba éstos entre paréntesis.

Si el método no requiere argumentos, escriba los paréntesis de apertura y cierre, sin indicar ningún valor dentro de ellos.

Si se hace una llamada a un método predefinido desde la regla de un método o concepto que está situado en otro nodo de mismo Meta4Object, la llamada al método debe ir precedida por el código identificativo de la estructura de nodo en la que está incluido o por el del alias indicado para esa estructura de nodo.

En la definición de un Meta4Object se puede utilizar más de una vez una estructura de nodo. Es decir, un Meta4Object puede incluir dos o más nodos que creen una instancia de la misma estructura de nodo.

Dependiendo de cómo se hayan conectado estas estructuras de nodo con su nodo padre en la estructura jerárquica del Meta4Object, es posible que cada nodo recupere datos diferentes, a pesar de que sean instancias de la misma estructura de nodo.

El lenguaje LN4 obliga a preceder el nombre de un elemento por el de la estructura de nodo del que forma parte. Ahora, si un mismo Meta4Object contiene más de un nodo que crea una instancia de la misma estructura de nodo, ¿cómo sabe el intérprete de LN4 qué método debe ejecutar?

Para evitar este problema, siempre que se necesite distinguir en una regla LN4 nodos que crean una instancia de la misma estructura de nodo, el programador puede definir un alias para cada uno de dichos nodos.

En el código LN4 de las reglas, se sustituirá el código identificativo de la estructura de nodo que precede al método por el del alias que se ha definido para el nodo con el que se quiere trabajar.

Sólo es necesario utilizar alias de estructuras de nodo cuando un mismo Meta4Object contenga dos o más nodos que creen una instancia de la misma estructura de nodo, y se tenga que hacer referencia a uno de estos nodos desde reglas LN4.

Los códigos identificativos de la estructura de nodo y del método se separan mediante un punto.

EJEMPLO

```
EN_Empleados.AddRegister( )
```

Este método agrega un nuevo registro en blanco al nodo que crea una instancia de la estructura de nodo EN_Empleados. El nodo que crea una instancia de EN_Empleados se encuentra en el mismo Meta4Object que el nodo que contiene el elemento desde cuya regla se hace la llamada.

Si se hace la llamada a un método del sistema de un nodo situado en un Meta4Object distinto al Meta4Object desde el que se hace la llamada, ésta deberá ir precedida de los códigos identificativos del Meta4Object y del nodo que contiene el método que se quiere ejecutar.

EJEMPLO

```
Empleados!EN_Direcciones.AddRegister( )
```

Con esta línea, se hace una llamada al método de sistema AddRegister() del nodo que crea una instancia de EN_Direcciones y que forma parte del Meta4Object Empleados.

La siguiente tabla recoge una descripción de todos los métodos predefinidos que agrega el sistema a las estructuras de nodo.

En la siguiente tabla, se indica que las llamadas a métodos de otros nodos desde la regla de un método o concepto deben ir cualificados por el código identificativo de la estructura de nodo que instancia al nodo que contiene el método que se quiere llamar.

Si se están utilizando alias para hacer referencia a las estructuras de nodo, será necesario cualificar el nombre del método con el alias que se ha definido para el nodo, en lugar de hacerlo con el código identificativo de la estructura de nodo que instancia.

Método	Descripción
AddRegister ()	Crea y agrega un registro vacío a continuación del último registro del bloque activo. Después, habrá que asignar valores a sus columnas mediante sentencias de asignación del tipo NombreElemento = Valor. La inserción del nuevo registro no se hace efectiva en la base de datos hasta que no se realiza una llamada a los métodos de grabación PersistTree() o Insert_Blkl (). Para agregar un registro en un nodo distinto al activo, es necesario cualificar la llamada al método con el nombre de la estructura que recibe la instancia del nodo al que se quiere agregar el registro. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el que se hace la llamada al método, cualifique el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo al que se quiere agregar el registro. AddRegister() no recibe ningún parámetro.
DeleteRegister()	Elimina el registro que se encuentra activo en el momento de llamar al método. El registro se marca como borrado; su eliminación de la base de datos no se hace efectiva hasta que no se realiza una llamada al método Delete_Blkl () o Persist_Tree(). Para marcar como borrado un registro que se encuentra en un nodo distinto al activo, cualifique la llamada al método con el nombre de la estructura que es instanciada por el nodo que incluye dicho registro. Antes debe haberse situado en ese registro mediante la función MoveTo(). Si este nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada al método, cualifique la llamada al método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo que contiene el registro. DeleteRegister() no recibe ningún parámetro.

Método	Descripción
InsertRegister()	Crea y agrega un registro vacío a continuación del que se encuentra activo. Después, es necesario asignar valores a sus columnas mediante sentencias de asignación del tipo NombreElemento = Valor. La inserción del nuevo registro no se hace efectiva en la base de datos hasta que no se realiza una llamada a los métodos de grabación PersistTree() o Insert_Blk(). Para agregar un registro en un nodo distinto al activo, es necesario cualificar la llamada al método con el nombre de la estructura que es instanciada por el nodo al que se quiere agregar el registro. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el que se hace la llamada, será necesario cualificar el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo al que se quiere agregar el registro. InsertRegister() no recibe ningún parámetro.
Delete_Blk()	Elimina de la base de datos física todos los registros que se hayan marcado como borrados en memoria principal. Los registros se pueden borrar mediante el método DeleteRegister(). La función Delete() únicamente elimina los registros que se hayan marcado como borrados en el nodo y bloque de memoria activos en el momento de llamar al método. Para eliminar de la base de datos los registros marcados como borrados en un nodo distinto al activo, cualifique la llamada al método con el nombre de la estructura instanciada por el nodo que contiene los registros que se quieren eliminar. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el que se hace la llamada, cualifique el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura instanciada por el nodo que contiene los registros que se van a borrar. El método Delete_Blk() debe estar incluido en una transacción. De cualquier otra forma, su ejecución no tendría efecto. Para incluir un método en una transacción, escríbalo entre las órdenes Begin_Transaction() y End_Transaction().
Delete_Prg()	Hace una llamada al método Delete_Blk(). Por lo tanto, su funcionalidad es equivalente a la de este método. La finalidad de este método es conectarse con el correspondiente del nodo padre y de los nodos hijos, para propagar la eliminación de registros que se hayan marcado como borrados en cualquier nodo del Meta4Object. Nunca se conectan los métodos Delete_Blk() de dos nodos.

Método	Descripción
Begin()	Activa el primer registro del bloque activo y sitúa el puntero que recorre el nodo en este registro. Para activar el primer registro de un nodo distinto al activo, cualifique la llamada al método con el nombre de la estructura instanciada por el nodo en cuyo primer registro se va a situar. Si el nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada al método, cualifique el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura instanciada por el nodo en cuyo primer registro se va a situar. El método Begin() no recibe parámetros.
End()	Sitúa el puntero en el último registro del bloque activo. Para situarse en el último registro de un nodo distinto al activo, cualifique la llamada al método con el nombre de la estructura instanciada por el nodo en cuyo último registro se va a situar. Si el nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada al método, cualifique el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura instanciada por el nodo en cuyo último registro se va a situar. El método End() no recibe parámetros.
Count()	Devuelve el número de registros que contiene el bloque activo. Para contar los registros de un nodo distinto al activo, cualifique la llamada al método con el nombre de la estructura instanciada por el nodo cuyos registros se quieren contar. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada al método, cualifique el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo cuyos registros se quieren contar. El método Count() no recibe parámetros.
GetCurrent()	Devuelve el índice correspondiente al registro que se encuentra activo. Los registros de un nodo se empiezan a contar a partir del número 0. Por ejemplo, si una estructura de nodo recupera 10 registros, al primer registro le corresponderá el índice 0, al segundo registro el índice 1 y así sucesivamente. Para obtener el índice de un registro situado en un nodo distinto al activo, cualifique la llamada al método con el nombre de la estructura que es instanciada por el nodo sobre el que se quiere aplicar el método. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada al método, cualifique el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo sobre el que se quiere aplicar el método. El método GetCurrent() no recibe parámetros.

Método	Descripción
IsEOF()	Devuelve el valor TRUE si el puntero que recorre el nodo ha alcanzado la variable lógica EOF. Si el puntero se encuentra en un registro del nodo activo, IsEOF() devolverá el valor FALSE. TRUE y FALSE son macros definidas en LN4 y equivalen a los valores 1 y 0. Para utilizar el método IsEOF() con otro nodo distinto al activo, cualifique la llamada al método con el nombre de la estructura que es instanciada por este segundo nodo. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada al método, cualifique el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo sobre el que se quiere aplicar el método. El método IsEOF() no recibe parámetros.
MoveToEOF()	Mueve el puntero que recorre el bloque hasta la variable lógica EOF(). No debe confundirse esta variable con el último registro del bloque. Para moverse a la variable lógica EOF de un nodo distinto a aquél desde el cual se hace la llamada al método, cualifique la llamada al método con el nombre de la estructura que es instanciada por el nodo en cuya variable lógica EOF va a situar el puntero. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada, cualifique la llamada al método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo en cuya variable lógica EOF va a situar el puntero. El método MoveToEOF() no recibe parámetros.
MoveTo()	Mueve el puntero al registro cuyo índice se pasa como parámetro al método. Si desea desplazarse a un registro determinado de un nodo distinto a aquél desde el cual se hace la llamada, cualifique la llamada al método con el nombre de la estructura que es instanciada por el nodo que contiene el registro en el desea situarse. Si este nodo se encuentra en un Meta4Object distinto a aquél desde el cual se hace la llamada, cualifique la llamada al método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo que contiene el registro en el que desea situarse. El método MoveTo() recibe como argumento el índice del registro sobre el que se quiere situar el puntero.
BeginTransaction()	Inicia una transacción lógica de base de datos. Las transacciones permiten agrupar operaciones de escritura (inserción y modificación) y borrado en la base de datos de forma que, si falla una operación dentro de la transacción, no se realizarán las demás operaciones incluidas en la transacción. Este método está asociado a EndTransaction(). BeginTransaction() no recibe argumentos.

Método	Descripción
EndTransaction()	Indica el final de las operaciones de escritura (inserción y modificación) y borrado en la base de datos que se agrupan en una transacción. Para iniciar una transacción, se utiliza el método BeginTransaction(). El método EndTransaction() recibe como argumento: ■ El valor 1, para ejecutar la transacción (commit) ■ El valor 0, para no ejecutar la transacción (rollback).
FindRegister()	Busca los registros de un nodo que cumplan una serie de condiciones. Las condiciones se pasan como argumentos al método. Para cada condición se pasan tres argumentos y se puede indicar un número ilimitado de condiciones. Los argumentos se pasan en este orden: ■ Nombre de la columna sobre la que se quiere realizar la búsqueda ■ Valor con el que se quiere comparar los valores de la columna indicada como primer argumento ■ Criterio que se va a utilizar para comparar los valores de la columna con el valor que se pasa como segundo argumento: Igual que, Distinto que, Mayor que, Menor que, Mayor o igual que, y Menor o igual que ■ Opcionalmente, el índice del registro a partir del cual se quiere realizar la búsqueda Si se pasa como argumento más de una condición, únicamente se recuperarán los registros que cumplan todas las condiciones. Es decir, las condiciones se unirán mediante el operador lógico AND.
Load_Blkl)	Carga en memoria principal todos los registros que recupera la sentencia asociada al bloque activo. Si se llama al método Load_Blkl) de un nodo monobloque, se cargarán todos los registros del nodo. Si se quieren cargar en memoria los registros de un bloque de un nodo distinto al activo, deberá cualificar la llamada al método con el nombre de la estructura que es instanciada por el nodo cuyos registros se quieren cargar. En el caso de que este nodo se encontrase en un Meta4Object distinto a aquél desde el cual se hace la llamada, sería necesario cualificar el nombre del método con el código identificativo del Meta4Object que contiene el nodo, y con el de la estructura que es instanciada por el nodo que contiene el bloque que se va a cargar en memoria. El método Load_Blkl) no recibe argumentos.
Load_Prgl)	Hace una llamada al método Load_Blkl). Por lo tanto, su funcionalidad es equivalente a la de este método. La finalidad de este método es conectarse con los Load_Prgl) del nodo padre y de los nodos hijos, para propagar la carga en memoria de los bloques del Meta4Object. Nunca se conectan los métodos Load_Blkl) de dos nodos.

Método	Descripción
PrepareJit()	Prepara el código LN4 que se le pasa como argumento. El código se pasa como argumento al método mediante una cadena de caracteres, encerrado entre comillas dobles. PrepareJit() compila el código y le asigna un valor numérico que se puede utilizar posteriormente para ejecutar el código resultado mediante el método ExecuteJit().
Persist_Tree()	Graba en la base de datos física los cambios realizados en los bloques del Meta4Object. El método Persist tree() realiza una transacción de base de datos. Internamente, este método llama a los métodos BeginTransaction(), Delete_Prg(), Update_Prg(), Insert_Prg() y EndTransaction(). Si se quieren grabar los cambios realizados en un Meta4Object distinto a aquél desde el que se hace la llamada, deberá cualificar la llamada al método con el código identificativo del Meta4Object cuyos registros desea grabar. El método Persist_Tree() no recibe argumentos.
RootLoad()	Carga en memoria principal los registros del nodo raíz del Meta4Object. Es decir, el primer nodo del Meta4Object, al cual se conecta el resto de nodo. Este método está conectado a Load_Prg(). Los registros del nodo raíz se cargan en un único bloque de memoria. Si desea cargar en memoria los registros del nodo raíz de un Meta4Object distinto a aquél desde el cual se hace la llamada, deberá cualificar la llamada al método con el nombre del Meta4Object cuyo nodo raíz desea cargar. El método Root_Load() no debe conectarse con el método Load_Prg() de otros nodos. Para desencadenar la carga de distintos nodos en memoria, se conectan los métodos Load_Prg() de los nodos.
ExecuteJit()	Ejecuta el código LN4 que el método PrepareJit() ha compilado con anterioridad. Recibirá como argumento el código identificativo numérico que haya devuelto la función PrepareJit().
ExecuteLN4Jit	Prepara y ejecuta el código LN4 que se le pasa como argumento. La ejecución de este método equivale a la conjunta de los métodos PrepareJit() y ExecuteJit(). La principal diferencia consiste en que ExecuteLN4Jit siempre prepara el código LN4 (aun en el caso de que se haya preparado con anterioridad). Frente a esto, PrepareJit() prepara el código una sola vez, por lo que no es necesario volver a prepararlo para posteriores ejecuciones.

Método	Descripción
Insert_Blk()	Inserta en la base de datos física los registros que se hayan agregado al bloque para el que se ejecuta la llamada. Si se quieren grabar los registros de un bloque situado en un nodo distinto al activo, deberá cualificar la llamada al método con el nombre de la estructura que es instanciada por el nodo cuyos registros se quieren grabar. En el caso de que este nodo se encontrase en un Meta4Object distinto a aquél desde el cual se hace la llamada, sería necesario cualificar el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo que contiene el bloque cuyos registros se van a grabar. El método Insert_Blk() no recibe argumentos.
Insert_Prg()	Este método hace una llamada al método Insert_Blk(). Por lo tanto, su funcionalidad es equivalente a la de este método. La finalidad de este método es conectarse con los métodos Insert_Prg() del nodo padre y de los nodos hijos; de este modo, se propaga la inserción en base de datos de todos los registros que se hayan agregado a cualquier nodo del Meta4Object. No se deben conectar los métodos Insert_Blk() de dos nodos distintos.
Update_Blk()	Actualiza en la base de datos física los registros que se hayan modificado en el bloque para el que se ejecuta la llamada. Si desea grabar los cambios realizados en los registros de un bloque situado en un nodo distinto al activo, deberá cualificar la llamada al método con el nombre de la estructura que es instanciada por el nodo cuyos registros desea actualizar. En el caso de que este nodo se encontrase en un Meta4Object distinto a aquél desde el cual se hace la llamada, sería necesario cualificar el nombre del método con el código identificativo del Meta4Object que contiene el nodo y con el de la estructura que es instanciada por el nodo que contiene el bloque cuyos registros se quieren actualizar. El método Update_Blk() no recibe argumentos.
Update_Prg()	Hace una llamada al método Update_Blk(). Por lo tanto, su funcionalidad es equivalente a la de este método. La finalidad de este método es conectarse con los métodos Update_Prg() del nodo padre y de los nodos hijos; de este modo, se propaga la inserción en base de datos de todos los registros que se hayan agregado a cualquier nodo del Meta4Object. No se deben conectar directamente los métodos Update_Blk() de dos nodos diferentes.

Acerca de este capítulo

Un Meta4Object está formado por uno o más nodos conectados. En el caso de que el Meta4Object cuente con dos o más nodos, éstos estarán conectados formando una estructura jerárquica.

Las relaciones entre los nodos de un Meta4Object se establecen mediante conectores. Éstos, además de definir la estructura de los Meta4Objects, cumplen otras finalidades:

- Determinar la relación lógica entre los registros de los nodos que se conectan
Esta relación hace que, cuando se active un registro específico del nodo principal, se activen los registros del nodo subordinado que cumplan las condiciones indicadas en la definición del conector.
- Determinar cómo se van a organizar en memoria principal los datos que recuperan los nodos que se conectan de acuerdo con una estructura optimizada
- Propagar la ejecución de métodos y conceptos
- Propagar valores entre elementos de tipo propiedad, de forma que cuando se asigne un valor a un elemento, se asigne automáticamente a otro elemento de otro nodo

En los siguientes apartados se explica el concepto de conector, sus distintos tipos, la conexión de elementos de tipo campo, de los nodos, la propagación de ejecución de elementos y la posibilidad de diseñar filtros en un conector.

Estructura jerárquica de un Meta4Object

Como ya se ha comentado a lo largo de este manual, un Meta4Object está formado por uno o varios nodos. Cada uno de los nodos toma como base una estructura de nodo que determina:

- Qué datos se van a recuperar de la base de datos
- Qué procesamiento se tiene que llevar a cabo con esos datos

En el caso de que el Meta4Object esté formado por dos o más nodos, los nodos se organizarán de acuerdo a una estructura jerárquica.

Relaciones entre nodos y conectores

Las relaciones entre los nodos de un Meta4Object se establecen a través de conectores.

Un conector incluye una serie de pares de elementos relacionados. El término *asociación* hace referencia a cada relación entre un par de elementos. Una asociación establece una condición de igualdad o una comparación entre los valores de un elemento del nodo padre, con uno de los elementos del nodo subordinado.



Los conectores tienen una existencia propia, independientemente de los pares de elementos asociados que incluyan. De esta forma, se podría tener un conector que no tuviese ninguna asociación de elementos, aunque, de ser así, el conector no tendría ninguna funcionalidad.

Un conector puede estar formado por una única asociación de elementos o por un número ilimitado de asociaciones. Cada asociación tendrá la forma:

`NodoPadre.elemento operador NodoHijo.elemento`

Donde:

- `NodoPadre.elemento` y `NodoHijo.elemento` se corresponden con los elementos que se conectan
- El elemento situado a la izquierda del operador será un elemento del nodo padre
- El elemento situado a su derecha será un elemento del nodo hijo
- Como operador, se podrá utilizar una de las siguientes opciones: *= igual*, *> mayor que*, *>= mayor o igual*, *< menor*, *<= menor o igual* ó *<> distinto*

Los elementos que se conectan pueden ser de cualquier tipo: métodos, conceptos, propiedades y campos, siempre que se conecten entre sí:

- Dos elementos que recojan un valor, es decir, campos, propiedades o conceptos
- Dos elementos con ejecución, es decir, métodos o conceptos

Requiere una especial atención la conexión de los métodos predefinidos, es decir, los métodos que están disponibles por defecto en todas las estructuras de nodo y que implementan las operaciones de grabación, inserción, eliminación de registros, etc. La conexión de este tipo de métodos se explica en un apartado posterior.

Conexiones válidas entre elementos

En la definición de un conector se pueden asociar entre sí:

- Un elemento de tipo método o concepto del nodo padre, con otro elemento de tipo método o concepto del nodo hijo. Con una conexión de este tipo, se consigue la ejecución del segundo elemento inmediatamente después de la ejecución del primero.
- Un elemento de tipo propiedad o campo del nodo padre, con otro elemento de tipo propiedad o campo del nodo hijo. La conexión de elementos que recogen un valor permite determinar qué registros se van a recuperar a medida que se carguen los distintos nodos del Meta4Object en memoria principal y el usuario se desplace entre los distintos registros del nodo padre de la conexión.
- Un elemento de tipo concepto con un elemento de tipo campo o propiedad. En este caso es importante recordar que, antes de ser ejecutada su regla, el concepto recoge el valor que se le haya asignado por defecto o el valor nulo. Por tanto, para que esta conexión sea efectiva, será necesario automatizar la ejecución del concepto en el momento en el que se carga el nodo en el que se ha definido.

Carecería de sentido conectar un elemento de tipo método con un elemento de tipo propiedad o campo, ya que la ejecución de los elementos de tipo método no devuelve ningún valor.

Funcionalidad de los conectores

Además de definir la estructura jerárquica de un Meta4Object, los conectores cumplen las siguientes funciones:

- Optimizar la distribución en memoria de los datos recuperados por las estructuras de nodo
- Relacionar los datos recuperados por las dos estructuras de nodo que se conectan, facilitando al usuario la percepción unificada de todos los datos relativos a una misma entidad que están distribuidos entre dos o más estructuras de nodo

Para los lectores que estén familiarizados con el lenguaje SQL, un conector entre dos nodos equivaldría a un *join* o a una cláusula *subselect*. Los nodos desempeñarían el papel de las tablas que se relacionan en el *join*, y cada asociación de elementos correspondería a cada una de las condiciones que se utilizan en la cláusula WHERE del *join*, para igualar las columnas de las dos tablas.

- Automatizar la ejecución encadenada de métodos y conceptos situados en distintas estructuras de nodo
- Realizar comprobaciones y validaciones antes de realizar una operación de escritura en la base de datos

Los conectores entre nodos permiten implementar funciones similares a las que ofrecen los disparadores (*triggers*) de los principales sistemas gestores de bases de datos.

Partiendo de estos puntos, se establece una distinción entre las conexiones entre elementos con ejecución (conceptos y métodos), y la conexión de elementos que recogen valores (campos, propiedades y conceptos).

Un conector entre dos nodos puede incluir, simultáneamente, conexiones entre pares de elementos con ejecución, y conexiones entre pares de elementos que recogen un valor.

Tipos de conectores y organización en memoria de los datos

La organización lógica de los datos recuperados en la carga de un Meta4Object se establece a partir del tipo de conector utilizado y de las asociaciones entre pares de elementos de los nodos.

Cuando se conectan dos nodos con un conector, es posible indicar cómo se van a distribuir en memoria principal los datos que recuperen los nodos del Meta4Object que se conectan.

Los registros de un nodo se distribuyen en bloques de memoria. Los registros de un nodo pueden ocupar un único bloque o más de un bloque. En el primer caso, se trata de un nodo monobloque. En el segundo caso, se trata de un nodo multibloque.

El nodo raíz de un Meta4Object siempre es monobloque. El resto de los nodos de un Meta4Object, es decir, los nodos que dependen del nodo raíz, reparten sus registros en uno o en varios bloques, según la forma en que se hayan conectado con los nodos de los que dependen.



Un nodo que ocupe el lugar de nodo raíz en un Meta4Object, únicamente podrá distribuir sus registros en un único bloque de memoria.

Por otra parte, un bloque podrá contener un único registro o un conjunto de dos o más registros. Desde la interfaz de la aplicación y desde las reglas LN4, es posible desplazarse entre los registros de un nodo o, si se trata de un nodo multibloque, del bloque que se encuentre activo.

La distribución de registros en distintos bloques se fija mediante los distintos tipos de conectores. Existen tres tipos de conectores: registro-bloque, bloque-bloque y libre. La selección de un tipo de conector o de otro determinará qué registros del nodo hijo van a estar visibles cuando se encuentre activo un registro específico del nodo padre.

Cuando se trabaja con un nodo multibloque, únicamente es posible acceder a los registros que se hayan cargado en el bloque que se encuentra activo en un momento dado. Para desplazarse por los registros que se han cargado en un bloque diferente, hay que activar este segundo bloque desplazándose, por el nodo padre, hasta el registro con el que está relacionado el segundo bloque.

Es importante recordar que un nodo sólo podrá distribuir sus registros en más de un bloque si está conectado con otro nodo con el que se haya establecido

una relación semántica mediante los pares de campos relacionados en el conector.



Para ampliar la información relativa a la distribución en memoria de los datos recuperados consulte el capítulo *Administración de Memoria* perteneciente a este mismo manual.

Tipos de conectores

La distribución en memoria de los datos recuperados por un par de estructuras de nodos se establece a partir del tipo de conector con el que se hayan conectado. Hay tres tipos de conectores: registro- bloque, bloque-bloque y libre.

Registro-bloque

Si se utiliza este tipo de conexión entre un par de nodos, para cada registro del nodo padre se crea un bloque de registros en el nodo hijo. En cada uno de estos bloques se cargan los registros del nodo hijo que estén relacionados con cada uno de los registros del nodo padre.

La relación entre los registros de los dos nodos se establece mediante la comparación de valores entre los elementos del conector.

Con un conector de este tipo, cuando el puntero que recorre los registros del nodo padre se sitúa en un registro determinado, se activará un nuevo bloque en el que se cargarán los registros del nodo hijo que estén relacionados lógicamente con el registro del nodo padre en el que se ha situado el puntero.

El tipo de conexión registro-bloque entre dos nodos se utiliza cuando las tablas sobre las que se han diseñado las estructuras de nodo asociadas a los dos nodos que se conectan están relacionadas mediante una relación de clave externa.

En este caso, se definen dos nodos, cada uno de ellos sobre una estructura de nodo. Estas estructuras de nodo tienen uno o más de un elemento de tipo campo en común. Si se relacionan los nodos definidos sobre estas dos tablas mediante una conexión de tipo registro-bloque y se igualan como elementos del conector los valores de los campos que tienen en común, se consigue que todos los registros del nodo hijo que tengan el mismo valor en los campos que conforman la clave externa se distribuyan en un mismo bloque de memoria.

Bloque-bloque

Si se utiliza este tipo de conexión entre un par de nodos, para cada bloque de memoria del nodo padre se reserva un bloque de memoria en el nodo hijo. De esta forma, hay el mismo número de bloques en el nodo padre y en el nodo hijo.

Tanto el bloque de memoria del nodo padre como el bloque de memoria del nodo hijo pueden tener un número variable de registros.

Al igual que sucedía con las conexiones registro-bloque, una conexión bloque-bloque no determina en sí misma ningún tipo de relación semántica entre los registros activos en los bloques correspondientes al nodo padre y los registros de los bloques correspondientes al nodo hijo.

Si se quiere establecer una relación que asocie los registros disponibles en cada bloque de memoria reservado para el nodo padre con un conjunto de registros disponibles en otro bloque de memoria del nodo hijo, deben conectarse elementos de los dos nodos que tengan un ámbito a nivel de bloque. Si se conectan elementos con ámbito a nivel de registro en una conexión entre nodos de tipo bloque-bloque, el sistema producirá un error.

Libre

La conexión de tipo libre entre un par de nodos no determina ninguna organización de datos ni ninguna reserva de espacio en memoria física.

Este tipo de conexión se utiliza cuando se quieren conectar únicamente elementos de tipo método de dos nodos. Si lo que se quiere es conectar registros de dos nodos, debe utilizarse una conexión de tipo registro-bloque o bloque-bloque.

Asociación de elementos tipo campo

La asociación de elementos de tipo campo de dos nodos conectados permite sincronizar los registros que recuperan cada uno de ellos.

De esta forma, cuando el puntero que recorre el nodo padre se sitúa en un registro, se activará el bloque de registros del nodo hijo que cumplan la condición indicada en la asociación.

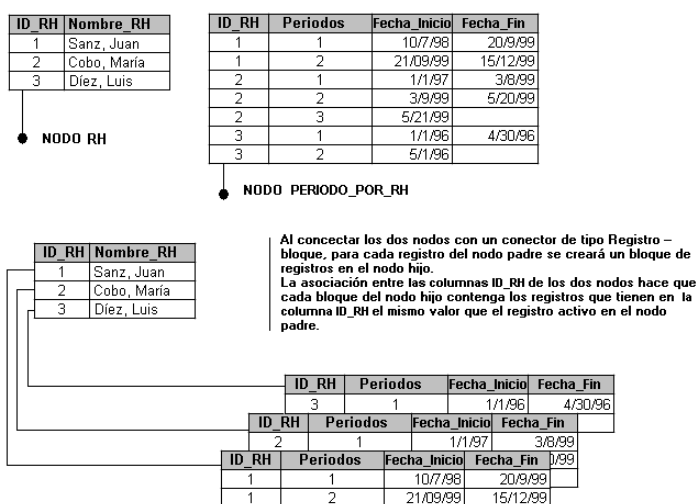
EJEMPLO

Un Meta4Object está formado por dos nodos unidos por un conector registro-bloque.

Se establece una asociación entre los elementos de tipo campo **ID_RH** de los dos nodos. La asociación iguala los valores de estos dos campos. Es del tipo:

NodoPadre.Id_Sociedad = NodoHijo.Id_Sociedad

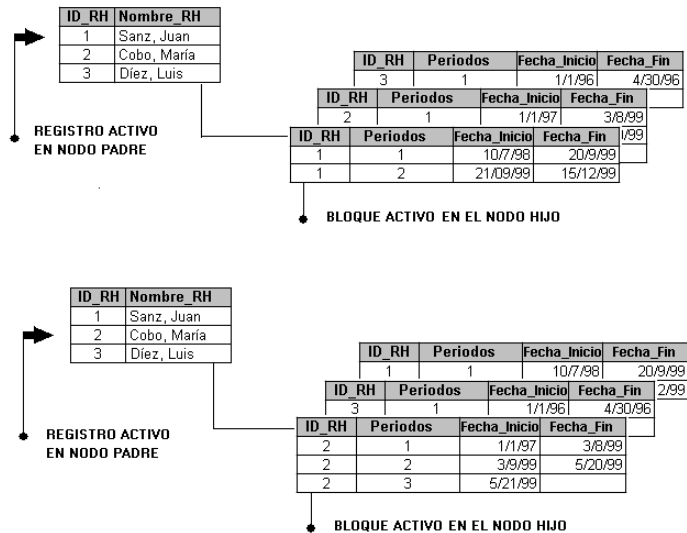
El siguiente gráfico muestra la organización de bloques de registros en el nodo hijo, y cómo los registros del nodo hijo con un mismo valor en el nodo hijo se agrupan en el mismo bloque:



En este mismo ejemplo, a medida que el cursor que recorre los registros del nodo padre se sitúa en un registro o en otro, se activa el bloque del nodo hijo que contiene los registros que tengan el mismo valor en la columna **ID_RH** que el registro activo en el nodo padre.

De esta forma, si el puntero está en el primer registro del nodo padre **ID_RH=1**, estará activo el bloque que aparece en primer plano en la imagen anterior.

Si el puntero del nodo padre se desplaza al segundo registro **ID_RH=2**, se activará el bloque con todos los registros de la columna que tengan **ID_RH** el valor 2 , y así sucesivamente.



La sincronización de registros determina qué registros del nodo hijo estarán visibles al situarse en cada uno de los registros del nodo padre. Al desplazarse por los registros del nodo padre se activa un conjunto de registros diferentes en el nodo hijo.

Mediante la sincronización de registros se establece un filtro dinámico entre los dos nodos conectados.

Los registros de dos nodos se sincronizan comparando los valores que se recogen en los campos que se hayan utilizado como elementos del conector. Los valores pueden compararse utilizando los operadores que aparecen en la siguiente tabla.

Operador	Nombre Operador	Descripción
=	Igual	Se cargarán en el nodo hijo los registros que tengan, en el elemento situado a la derecha del igual, el mismo valor que el elemento del nodo padre que se haya indicado a la izquierda del operador. Si se trata de un elemento con un ámbito de tipo bloque o registro, el valor que recoja dependerá, respectivamente, del bloque o del registro que se encuentre activo en cada momento.
<>	Distinto a	Se cargarán en el bloque de memoria correspondiente al nodo hijo los registros que tengan, a la derecha del signo igual, un valor distinto al del elemento del nodo padre que se haya indicado a la izquierda del operador. Si se trata de un elemento con un ámbito de tipo bloque o registro, el valor que recoja dependerá, respectivamente, del bloque o del registro que se encuentre activo en cada momento.
>	Mayor que	Se cargarán en el bloque de memoria correspondiente al nodo hijo los registros que tengan, en el elemento situado a la derecha del igual, un valor menor que el del elemento del nodo padre que se haya indicado a la izquierda del operador. Si se trata de un elemento con un ámbito de tipo bloque o registro, su valor cambiará dependiendo del bloque o del registro que se encuentre activo en cada momento.
<	Menor que	Se cargarán en el bloque de memoria correspondiente al nodo hijo los registros que tengan, en el elemento situado a la derecha del igual, un valor mayor que el del elemento del nodo padre que se haya indicado a la izquierda del operador. Si se trata de un elemento con un ámbito de tipo bloque o registro, su valor cambiará dependiendo del bloque o del registro que se encuentre activo en cada momento.
>=	Mayor o igual que	Se cargarán en el bloque de memoria correspondiente al nodo hijo los registros que tengan, en el elemento situado a la derecha del igual, un valor menor o igual que el del elemento del nodo padre que se haya indicado a la izquierda del operador. Si se trata de un elemento con un ámbito de tipo bloque o registro, su valor cambiará dependiendo del bloque o del registro que se encuentre activo en cada momento.
<=	Menor o igual que	Se cargarán en el bloque de memoria correspondiente al nodo hijo los registros que tengan, en el elemento situado a la derecha del igual, un valor mayor o igual que el del elemento del nodo padre que se haya indicado a la izquierda del operador. Si se trata de un elemento con un ámbito de tipo bloque o registro, su valor cambiará dependiendo del bloque o del registro que se encuentre activo en cada momento.

Desplazamiento entre los registros de un nodo

La conexión de elementos de tipo campo de los dos nodos permite obtener una presentación dinámica de los registros de los nodos. Como ya se ha indicado en un apartado anterior, los nodos de un Meta4Object se definen a partir de una estructura de nodo que determina qué información se quiere recuperar de una o de varias tablas de la base de datos. En la definición de la estructura de nodo se indican las condiciones que deben cumplir los registros para que sus valores sean recuperados.

Al cargar un Meta4Object en memoria principal, el sistema ejecuta las sentencias correspondientes a la estructuras de nodo sobre las que se han diseñado los nodos del Meta4Object y se recupera un conjunto de registros de la base de datos para cada uno de los nodos. La carga del Meta4Object puede hacerse en distintas etapas, según el modo de configuración de la carga de datos del Meta4Object.



Consulte el capítulo *Administración de memoria* de este mismo manual para obtener más información sobre la carga de datos del Meta4Object

El sistema permite realizar distintas operaciones sobre estos registros: añadir y eliminar registros, modificar los valores de sus campos, y desplazarlos entre ellos. Todas estas operaciones se harán en memoria principal.

Es posible desplazarse de un registro a otro desde la interfaz de la aplicación y mediante funciones del lenguaje LN4. De esta forma, un nodo implementa la funcionalidad de un cursor de base de datos y añade funciones adicionales, como la posibilidad de recorrer el conjunto de registros en orden ascendente o descendente, o la de situarse en un registro determinado del nodo sin necesidad de recorrer, uno tras otro, todos los registros que le preceden, etcétera.

Si todos los registros del nodo ocupan un mismo bloque de memoria, es posible desplazarse entre todos ellos sin ningún tipo de restricción. No sucede lo mismo si se trata de un nodo multibloque, es decir, de un nodo cuyos registros se han distribuido entre dos o más bloques de memoria. En el siguiente apartado se explica cómo desplazarse entre los registros de un nodo multibloque.

Desplazamiento entre los registros de un nodo multibloque

Los registros de un nodo pueden distribuirse en distintos bloques de memoria. La forma en la que se distribuyen los registros entre bloques de memoria

dependerá de cómo se haya conectado el nodo con sus nodos hijo o nodos padre en el Meta4Object.

Para que los registros de un nodo se distribuyan en más de un bloque, es necesario haber conectado este nodo con su nodo padre mediante un conector de tipo registro-bloque o bloque-bloque y haber conectado pares de elementos con un ámbito a nivel de registro en el primer caso, o a nivel de bloque en el segundo.

Se distinguen dos casos:

- Los dos nodos están conectados mediante un conector registro-bloque
- Los dos nodos están conectados mediante un conector bloque-bloque

No hay que olvidar que no se debe utilizar el tipo de conector Libre cuando se quieran conectar elementos de los nodos padre e hijo que recojan valor, es decir, elementos de tipo campo, propiedad o concepto.

Desplazamiento entre registros de un nodo multibloque conectado con otro nodo mediante un conector registro-bloque

Con una conexión registro-bloque, cuando el puntero que recorre los registros del nodo padre se sitúe en un registro determinado, el sistema activará un bloque de memoria reservado para el nodo hijo, en el que cargará aquellos registros que estén relacionados con el registro del nodo padre en el que está situado el puntero.

La relación entre los registros del nodo hijo y cada uno de los registros del nodo padre se establece mediante las asociaciones de elementos indicadas en la creación del conector. Cada asociación consiste en la comparación del valor de uno o varios elementos del nodo padre, con el de otro u otros elementos del nodo hijo.

En este contexto, al situarse en el nodo hijo, únicamente hay un bloque de memoria activo. El sistema permitirá el desplazamiento entre los registros del bloque activo desde la interfaz de la aplicación y mediante las funciones LN4. Sin embargo, no es posible moverse directamente entre los bloques de memoria reservados para el nodo hijo. Así, cuando se alcance el último registro del bloque activo, no será posible pasar al primer registro del bloque siguiente.

Para mover el puntero al primer registro del siguiente bloque, hay que moverse al siguiente registro del nodo padre. De esta forma, el sistema activa un nuevo bloque de memoria en el que carga los registros del nodo hijo que están asociados con el registro del nodo padre en el que acaba de situarse el puntero.

EJEMPLO

Se decide crear un Meta4Object que recupere datos de las tablas RH en periodo y Absentismos. El Meta4Object permitirá ver la información de todas las fases de alta de todos los empleados, y para cada una de ellas, todos los absentismos que se hayan registrado durante ese periodo.

- En primer lugar, se crean dos estructuras de nodo sobre las tablas RH en periodo y Absentismos. El nodo raíz del Meta4Object tomará como base la estructura de nodo diseñada sobre la tabla RH en periodo, y el segundo nodo tomará como base la estructura de nodo diseñada sobre la tabla Absentismos.
- Se establece una relación entre estos dos nodos, utilizando el tipo de conector registro-bloque. De esta forma, para cada registro del nodo raíz, es decir, para cada fase de alta de cada empleado, el sistema reservará un bloque de memoria para cargar los registros relacionados en el nodo hijo.

Para que el sistema sepa qué registros debe cargar en cada uno de los bloques de memoria reservados para los registros del nodo hijo, es necesario indicar cuál va a ser la relación lógica entre los elementos de los dos nodos. En este caso, se indicará que los elementos que se utilizan en las dos tablas para identificar a cada empleado y a cada fase de alta, ID_EMPLEADO e ID_PERIODO, deben recoger el mismo valor.

La relación que se establecerá entre los elementos que recogen el ID_EMPLEADO en las dos estructuras de nodo será la de igualdad; la relación que se establecerá entre los elementos que recogen el ID_PERIODO en las dos estructuras de nodo será también la de igualdad.

De esta forma, se indica al sistema que se quieren ver, únicamente, los registros de la tabla Absentismos correspondientes a la fase de alta que se encuentre activa en el nodo PERIODO_RH. Como cada registro del nodo padre se corresponde con una fase de alta distinta, la fase de alta activa será aquella sobre cuyo registro esté situado el cursor en cada momento.

De esta forma, cuando el puntero que recorre los registros del nodo padre se encuentre sobre un registro específico, el sistema recuperará, en uno de los bloques de memoria reservados para los registros del nodo hijo, los registros de la tabla Absentismos que tengan el mismo valor en las columnas ID_FASE_ALTA e ID_EMPLEADO que el registro que se encuentre activo en el nodo padre.

Por ejemplo, cuando esté activo en el nodo padre el registro correspondiente al PERIODO_RH 2 del empleado 01, el sistema cargará en un bloque aparte todos los registros de la tabla Absentismos correspondientes al empleado 01 durante su PERIODO_RH 2. De esta forma, se sincronizan los registros de los dos nodos, se optimiza la distribución de datos en memoria y se obtiene una visión unificada de datos distribuidos entre distintas tablas.

Es importante establecer las conexiones correctas entre los elementos de los dos nodos. En cualquier otro caso, la aplicación podría devolvernos resultados incorrectos o inconsistentes. Por ejemplo, si en el ejemplo anterior se hubiesen relacionado exclusivamente los elementos de tipo campos de las dos estructuras de nodo que recogen el Identificador del empleado, para cada

empleado se habrían generado registros en los que, para cada fase de alta, se listarían todos los absentismos registrados durante todas las fases de alta del empleado. Es decir, se encontrarían registros que recogerían absentismos registrados en una fecha no incluida en ese periodo de alta.

Desplazamiento entre registros de un nodo multibloque conectado con otro nodo mediante un conector bloque-bloque

El tipo de conector bloque-bloque es el segundo tipo de conector que se puede utilizar para unir dos nodos que recojan datos entre los que existe una relación semántica.

Con este tipo de conector, el sistema reservará un bloque de memoria para los registros del nodo hijo, por cada uno de los bloques de memoria reservados para el nodo padre. Es decir, habrá el mismo número de bloques de memoria reservados para el nodo padre y para el nodo hijo.

El tipo de conector bloque-bloque se utilizará cuando se conecten uno o varios elementos del nodo padre con un ámbito a nivel de bloque, con otro u otros elementos del nodo hijo que tengan un ámbito a nivel de bloque. Como ya se ha indicado, si se conectasen dos nodos mediante un conector bloque-bloque, y se conectase un par de elementos con un ámbito a nivel de registro, se produciría un error.

Cuando se conectan dos nodos mediante una conexión bloque-bloque, es posible desplazarse entre los registros del nodo padre y del nodo hijo de la forma habitual. Como en el caso anterior, no será posible desplazarse entre bloques de memoria distintos de una forma directa. Así, para moverse a un registro del nodo hijo que esté situado en un bloque distinto al bloque que se encuentre activo, será necesario activar su bloque correspondiente en el nodo padre. Para moverse a un bloque del nodo padre distinto del nodo activo, será necesario remontarse al nodo del cual depende este nodo, y situar el puntero en el registro que esté asociado con el bloque al que se quiere llegar, tal y como se ha explicado en el apartado anterior.

Conexiones entre elementos con ejecución

Los elementos de una estructura de nodo pueden ser de cuatro tipos: campos, propiedades, métodos y conceptos.

Los elementos de tipo método y concepto tienen asociadas reglas que contienen instrucciones de procesamiento escritas en lenguaje LN4, C++, o llamadas a métodos de bibliotecas de enlace dinámico.

De la misma forma que es posible conectar elementos que recogen un valor, también es posible conectar elementos de tipo método o concepto de nodos distintos de un mismo Meta4Object.

En este capítulo, sólo se describen las conexiones entre métodos situados en nodos que forman parte del mismo Meta4Object. Las conexiones entre métodos de Meta4Objects diferentes se explican en el manual *Flujos de trabajo y Eventos*.

La conexión de elementos de tipo método o concepto se utilizará cuando sea necesario:

- Propagar la ejecución de métodos o conceptos
- Sincronizar los argumentos que reciben dos elementos de tipo método de distintos nodos

En los siguientes apartados, se detallan los puntos anteriores. También se explica cómo implementar la funcionalidad de los disparadores de bases de datos mediante la conexión de métodos.

Propagación de la ejecución de elementos

Los conectores entre nodos permiten ejecutar métodos de forma encadenada. De esta manera, si dos métodos están conectados, en cuanto se ejecute uno de ellos se desencadenará la ejecución del segundo. Si el segundo método está conectado con un tercer método, se ejecutará a continuación este tercer elemento, y así sucesivamente.

En la conexión de dos elementos de tipo método o concepto, se distingue entre el elemento que desencadena la ejecución del segundo elemento, *elemento origen*, y el elemento cuya ejecución es desencadenada, *elemento destino*.

Orden de ejecución de los elementos

Propiedad Tipo de precedencia (Tp. de precedencia)

El hecho de que la ejecución de uno de los elementos desencadene la ejecución del otro no implica que el elemento origen vaya a ejecutarse antes que el elemento destino.

El orden en el que se ejecuten los elementos dependerá del valor que se asigne a la propiedad **Tp. de precedencia** en el momento en el que se define la asociación entre los dos elementos. El **Tp. de precedencia** puede tomar los valores *Después* o *Antes*.

Si se conecta un método o concepto del nodo padre con un método o concepto del nodo hijo y se asigna a la propiedad **Tp. de precedencia** el valor *Después*, el sistema ejecutará en primer lugar el elemento del nodo padre; inmediatamente después, ejecutará el elemento del nodo hijo.

Si se conecta un método o concepto del nodo padre con un método o concepto del nodo hijo y se asigna a la propiedad **Tp. de precedencia** el valor *Antes*, cuando el sistema vaya a ejecutar el elemento del nodo padre, detectará que tiene que ejecutar antes el elemento del nodo hijo, ejecutará este último y, a continuación, ejecutará el elemento del nodo padre.

En los dos casos, el elemento del nodo padre es el que desencadena la ejecución del elemento del nodo hijo. Sin embargo, el orden en el que se ejecuten los elementos dependerá de cómo se haya parametrizado la propiedad **Tp. de precedencia**.

Orden de ejecución cuando se desencadena la ejecución de dos o más elementos

Un método o concepto puede desencadenar la ejecución de dos o más elementos. Los elementos destino de la conexión pueden encontrarse en el mismo nodo o en nodos diferentes del mismo Meta4Object.

Cuando el elemento origen desencadena la ejecución de más de un elemento, el sistema tiene que saber en qué orden tiene que ejecutar los elementos destino de la conexión.

Para determinar el orden de ejecución de estos elementos, el sistema comprobará qué valor se ha asignado a la propiedad **Tp. de precedencia** en cada conexión. Si hay más de dos elementos destino con el mismo valor en esta propiedad, se ejecutarán los conceptos en el orden que se haya indicado en la propiedad **Orden**. Esta propiedad recogerá un valor numérico, de forma que las asociaciones con un valor más bajo se ejecutarán antes que las asociaciones con un valor más alto. El valor de la propiedad **Orden** se indicará

en el momento de definir las distintas asociaciones entre pares de elementos del conector.

Tipo de conectores y propagación de ejecución

En un apartado anterior se han explicado los distintos tipos de conectores que pueden utilizarse para relacionar dos nodos: registro-bloque, bloque-bloque y libre.

El tipo de conector determina cómo se van a distribuir los registros entre distintos bloques de memoria cuando se cargue el Meta4Object. Es una propiedad que se indica a nivel de conector y no a nivel de las asociaciones entre pares de elementos que conforman el conector.

Siempre que se conecten dos nodos con el propósito de sincronizar sus datos, deberá utilizarse el tipo de conector registro-bloque o bloque-bloque. De esta forma, el sistema será capaz de saber cómo debe distribuir los registros de cada nodo en bloques de memoria.

El tercer tipo de conector, libre, únicamente se utilizará cuando se conecten elementos de tipo método o concepto de los dos nodos.

Si el conector incluye, además de asociaciones entre elementos de tipo método, asociaciones entre elementos de tipo campo, propiedad o concepto, se tendrá que utilizar el tipo de conector registro-bloque o bloque-bloque. La selección de una u otra de estas dos opciones dependerá de la forma en la que se quieran distribuir los registros entre distintos bloques de memoria.

Propagación de ejecución y ámbito de los elementos

La forma en que se va a propagar la ejecución de métodos y conceptos conectados no sólo dependerá de los valores que se hayan asignado a las propiedades **Orden** y **Tp. de prec.** en cada asociación entre un par de elementos. Otro factor considerable es el ámbito de los elementos que se conectan. El ámbito de los elementos se explica en el capítulo *Estructuras de nodo: conceptos básicos* de este mismo manual.

El ámbito es una propiedad de los elementos que se utiliza para indicar si un elemento puede tomar valores distintos para cada uno de los registros que recupere la estructura de nodo, o si va a tomar el mismo valor para todos los registros incluidos en un mismo bloque de memoria, o para todos los registros de un mismo nodo.

El ámbito de un elemento puede ser de tres tipos: registro, bloque y nodo.

- Cuando el valor del elemento puede ser diferente en cada uno de los registros del mismo nodo, el elemento tendrá un ámbito a nivel de registro

- Cuando el valor del elemento tenga que ser el mismo para todos los registros incluidos en el mismo bloque de memoria, el elemento tendrá un ámbito a nivel de bloque
- Cuando el valor del elemento tenga que ser el mismo para todos los registros del nodo, independientemente del bloque de memoria en el que se carguen, el elemento tendrá un ámbito a nivel de nodo

Cuando se conectan dos elementos ejecutables, es decir, elementos de tipo método o concepto, tanto si se encuentran en el mismo nodo como si se encuentran en nodos diferentes, es importante saber cuál es su ámbito.

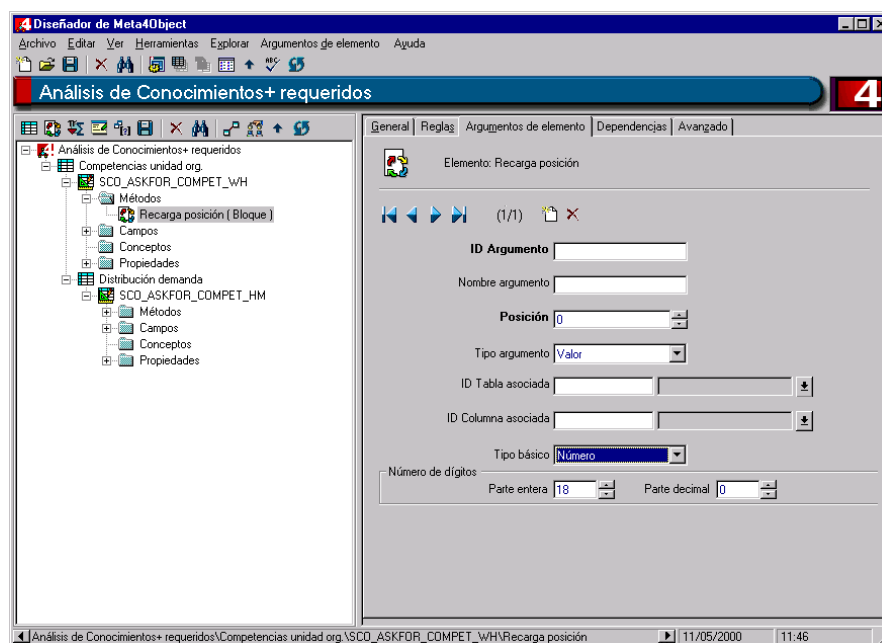
En los siguientes apartados se explica la propagación de la ejecución de elementos, según el tipo y ámbito de los elementos origen y destino, y el tipo de conector que se haya utilizado para conectar los nodos.

Sincronización de argumentos de elementos con ejecución

Las reglas de los elementos de tipo método o concepto pueden recibir argumentos; la regla tratará los argumentos como variables cuyo valor se conocerá en el momento en el que se solicite la ejecución de la regla.

La utilización de argumentos en la definición de métodos y conceptos permite trabajar con valores que no se conocen en el momento de crear la regla. De esta forma, se potencia la reutilización de código, y se pueden diseñar reglas que se ejecuten tomando como base valores que dependan del registro, bloque o nodo que se encuentre activo en el momento en el que se ejecuta la regla.

Figura 37. Definición de argumentos para reglas



Desde LN4, se puede recuperar el valor de cualquier elemento en una variable y ejecutar un método o concepto de cualquier otro Meta4Object pasándole como argumento el valor que se ha recogido en la variable.

Se pueden pasar como argumentos a un método o concepto:

- Un valor almacenado en una variable
- El valor que recoge un elemento del mismo Meta4Object o de un Meta4Object diferente
- La llamada a una función que, al ser ejecutada, devolverá un valor
- El valor que se ha pasado como argumento a otro elemento de tipo método o concepto utilizado en el mismo Meta4Object

Todas las posibilidades anteriores, a excepción de la última, están disponibles a través del lenguaje LN4; además de las posibilidades que ofrece este lenguaje, desde el Diseñador de Meta4Objects también es posible sincronizar los argumentos que reciben dos métodos de dos nodos distintos del mismo Meta4Object.

Sincronización de los parámetros de dos métodos

En el caso de que el método o concepto que recibe los argumentos tenga más de un argumento, es necesario sincronizar todos sus argumentos. De no hacerse así, se produciría un error.

No es necesario que los métodos que se sincronizan reciban el mismo número de argumentos; la única restricción es que el método que hereda los argumentos sincronice el valor de todos sus argumentos con los argumentos del método del cual los hereda. Tampoco es necesario que coincida el orden que ocupan los argumentos que se sincronizan.

EJEMPLO

El método A recibe dos argumentos,

A (arg₁, arg₂)

Se quieren sincronizar sus argumentos con los de otro método B, que recibe tres argumentos:

B (arg₁, arg₂, arg₃)

El argumento arg₃ del método B se sincronizará con el argumento arg₁ del método A, y el argumento arg₁ del método B se sincronizará con el argumento arg₂ del método A.

Es posible sincronizar dos o más argumentos de un método, con un único argumento de otro método; en este caso, los argumentos del primer método recibirán el mismo valor.

Paso de argumentos por valor y por referencia

Cuando se sincronicen los argumentos de dos elementos de tipo método, hay que tener en cuenta la forma en la que se han pasado los argumentos al elemento. Un argumento puede pasarse de dos formas: por valor y por referencia.

- Cuando se pasa un argumento por valor, el valor del argumento no puede modificarse desde el código de la regla que lo recibe como argumento
- Cuando se pasa un argumento por referencia, el valor del argumento puede modificarse desde el código de la regla que lo recibe como argumento

Por ejemplo, si la regla

```
CalculaTotal ( nTotalDevengado )
```

recibiese el argumento `nTotalDevengado` por valor y la regla contuviese una asignación del tipo

```
nTotalDevengado = nTotalDevengado * 5
```

el intérprete de reglas LN4 asignaría a la variable `nTotalDevengado` el valor obtenido al multiplicar por 5 el valor que ha recibido como argumento. Todas las referencias que apareciesen posteriormente a `nTotalDevengado` utilizarían este último valor. Sin embargo, en cuanto finalice la ejecución de la regla, `nTotalDevengado` mantendrá el valor que tuviese antes de la ejecución de la regla `CalculaTotal( )`. Es decir, como el argumento se ha pasado por valor, las modificaciones que se hagan sobre el argumento no tendrán ningún efecto fuera del entorno de la regla.

Si el argumento se hubiese pasado por referencia, todos los cambios que se hubiesen hecho sobre el argumento desde la regla permanecerían tras la ejecución de la regla. El valor de `nTotalDevengado` después de la ejecución de la regla no sería el mismo que era antes de su ejecución.

De esta forma, si se sincronizan los argumentos de dos elementos y el primero de ellos ha recibido el argumento por referencia, es posible que durante la ejecución de la regla del primer elemento se haya modificado su valor. De esta forma, aunque las reglas de los dos elementos reciban el mismo argumento, su valor no tiene por qué ser idéntico en los dos casos.

La forma en la que se pasa un argumento, por valor o por referencia, se indica en el momento de definir los argumentos del método, mediante la propiedad **Tipo argumento**. Esta propiedad recoge dos posibles valores: *Valor*, si el argumento se pasa por valor, o *Referencia*, si se pasa por referencia.

Borrado en cascada de registros

En las propiedades del conector entre un par de nodos pueden indicarse dos propiedades: **Tipo de conector**, que determina cómo se van a distribuir los registros de los nodos en distintos bloques de memoria y que se ha explicado en el apartado anterior, y **Eliminar hijos**.

La propiedad **Eliminar hijos** permite parametrizar los borrados en cascada.

Cuando se elimine un registro del nodo padre, se marcarán como borrados de forma automática los registros del nodo hijo con los que está relacionado.

En el momento de hacer efectivos los cambios mediante una llamada a la orden de grabación, se borrarán de la base de datos física los registros marcados como borrados en el nodo hijo y, a continuación, se borrará de la base de datos física el registro marcado como borrado en el nodo padre.

Esto es así porque, para poder borrar un registro de una tabla, es necesario borrar antes todos los registros del nodo hijo que le hagan referencia. De no hacerse así, los registros del nodo padre no podrían eliminarse.

EJEMPLO

Si el nodo hijo contiene tres registros que hacen referencia a un registro del nodo padre, éste último sólo se eliminará si se eliminan los tres registros del nodo hijo que le hacen referencia. Si se eliminase únicamente uno de estos tres registros, el registro del nodo padre no se borraría.

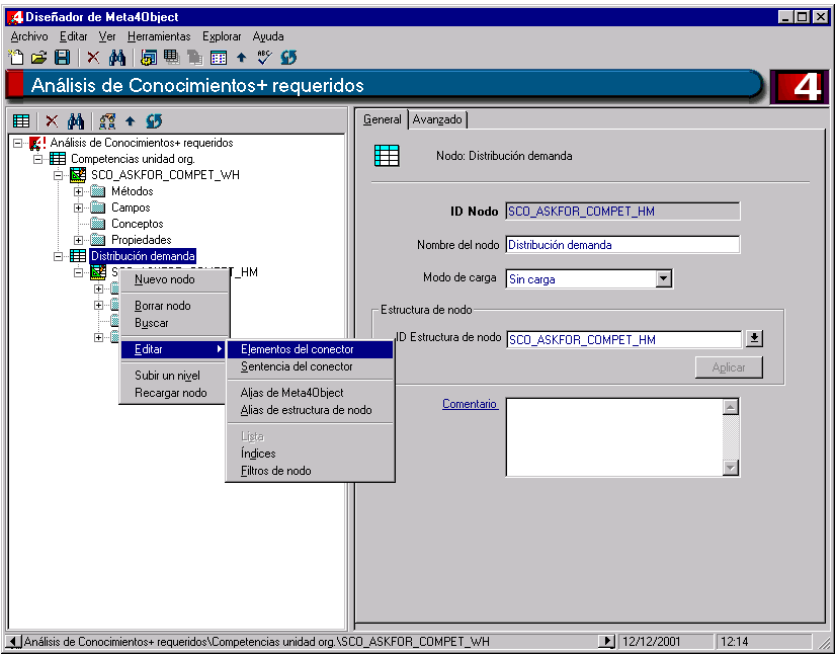
Para activar el borrado en cascada, basta con activar la casilla de verificación **Eliminar hijos**. Si no se activa esta casilla, el sistema no efectuará ninguna operación con los registros del nodo hijo cuando se borre un registro del nodo padre.

Esta propiedad se indica a nivel de conector cuando se crea el Meta4Object desde el Diseñador de Meta4Objects.

Para activarla, siga los siguientes pasos:

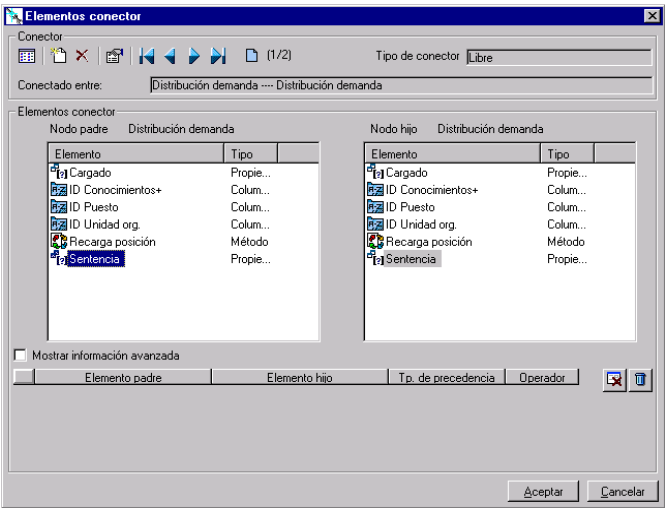
1. Desde el Diseñador de Meta4Objects, haga clic sobre el nodo que quiere conectar. Debe activar siempre el nodo hijo, situado en un nivel inferior de la jerarquía.
2. Haga clic en el botón derecho del ratón. Se desplegará un menú.
3. Seleccione la opción **Editar|Elementos del conector**.

Figura 38. Editar elementos del conector



El programa le presentará una ventana como la siguiente:

Figura 39. Elementos conector





- Haga clic en el botón **Nuevo**. Aparecerá la siguiente ventana:

Figura 40. Conector nuevo

- Mediante la lista desplegable **Eliminar hijos**, podrá indicar si quiere activar o no el borrado en cascada.

Borrado en cascada

La aplicación también permite implementar las operaciones de borrado en cascada basadas en claves externas, características de los principales sistemas gestores de bases de datos. Una operación de borrado en cascada implementa la siguiente función: siempre que se elimina un registro de una tabla a cuya clave primaria se le hace referencia en otra tabla mediante una relación de clave externa, se eliminan de la tabla que contiene la clave externa todos los registros que hagan referencia al registro que se ha eliminado en la tabla padre.

La aplicación permite implementar las operaciones de borrado en cascada desde la herramienta Diseñador de modelos de datos. Cuando se establece una relación de tipo clave externa entre dos tablas, puede activarse el borrado en cascada activando la casilla de validación **Borrar cascada**. A partir de ese momento, siempre que se elimine un registro en la tabla padre —cualquiera que sea el Meta4Object que implemente la interfaz con la tabla— se propagarán las operaciones de borrado en la tabla hija de la relación.

Figura 41. Diseñador de modelo de datos - creación de relaciones

Relaciones

2 de 2

ID: STD_EMAIL_1

Nombres:

Hija a padre:

Padre a hija:

Tabla hija:

ID: STD_EMAIL

Nombre: Correo electrónico

Tabla padre:

ID: STD_PERSON

Nombre: Persona

Temporalidad: Sin propiedades

☐ Hijo necesario ☐ Validación obligatoria común

Acciones automáticas:

☐ Desplazar Inicio en cascada ☐ Cierre automático ?

☐ Desplazar Fin en cascada ☒ Borrar cascada

Columnas de relaciones (hija/padre):

Columna hija	Columna padre
STD_ID_PERSON	STD_ID_PERSON

Comentario:

Estado: Datos correctos

Aceptar Cancelar

También sería posible implementar la funcionalidad del borrado en cascada en el nivel de aplicación, es decir, en el diseño del Meta4Object. Bastaría con conectar el método predeterminado `Delete_Prg()` del nodo padre, con un método diseñado en el nodo hijo que realizase las siguientes operaciones:

1. Buscar los registros del nodo hijo que tienen, en los elementos de tipo campo utilizados en la definición del conector, los mismos valores que, en estos mismos, recojan los registros del nodo padre que se ha marcado como borrado
2. Marcar como borrados todos los registros que cumplan la condición anterior
3. Guardar y persistir los cambios en la base de datos

Si bien cualquiera de los dos métodos anteriores podría utilizarse para implementar los borrados en cascada, el primero de ellos resulta el más apropiado y es el que se debe emplear en la mayoría de ocasiones.

Disparadores

Las conexiones de tipo método o concepto permiten implementar la función de los disparadores (*triggers*) disponibles en los principales sistemas gestores de bases de datos.

Los disparadores permiten desencadenar la ejecución de una serie de instrucciones cuando se produce una operación de inserción, actualización o borrado en la base de datos.

Por ejemplo, podría diseñarse un disparador que, tras insertar o actualizar un conjunto de registros en memoria principal, comprobase si los valores asignados a uno de sus campos están comprendidos dentro de un rango de valores válidos.

Tipos de disparadores

La aplicación permite diseñar tres tipos de disparadores:

- Disparadores diseñados en la definición de una estructura de nodo
- Disparadores diseñados en la creación de un nodo
- Disparadores definidos sobre dos nodos conectados jerárquicamente en la definición de un Meta4Object

En los tres casos, si se quisiese desencadenar la ejecución de uno o más métodos cuando el usuario intenta grabar los cambios realizados sobre los registros del Meta4Object, el método que desencadena la ejecución del segundo método deberá ser el que implementa la grabación en base de datos de los cambios que se hayan realizado en memoria principal; es decir, el método `Persist_Tree()` o `Persist_Prg()`.

A continuación se explica la función de los distintos tipos de disparadores:

Disparadores de estructura de nodo

Un disparador definido a nivel de estructura de nodo se creará desde el Diseñador de Meta4Objects. En su definición, se conectan dos de los métodos o conceptos de la estructura de nodo, de forma que la ejecución de uno de ellos desencadena la ejecución del segundo.

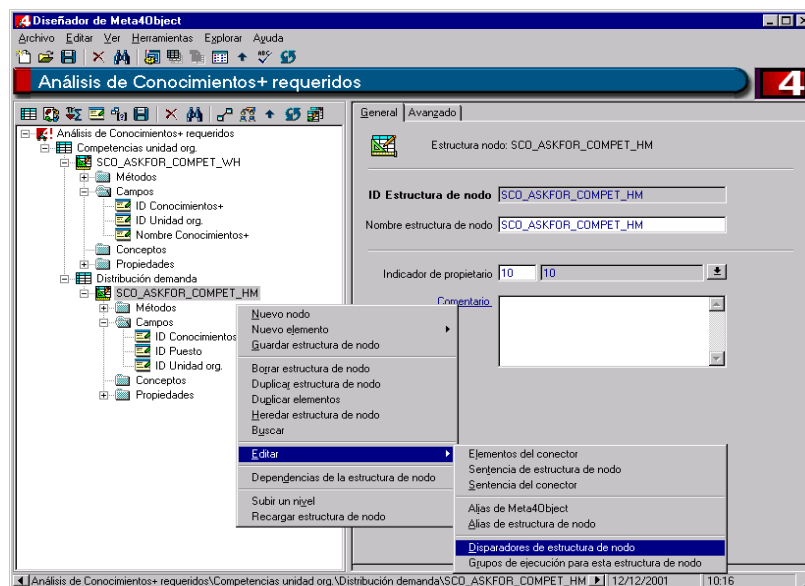
Si se define un disparador a nivel de estructura de nodo, el disparador se aplica en todas las instancias de esta estructura de nodo; es decir, en todos los nodos que se hayan definido tomando esa estructura de nodo como base.

El lector encontrará más información sobre este tipo de disparadores en el capítulo *Estructuras de nodo: conceptos avanzados*, de este mismo manual.

Para crear un disparador de estructura de nodo, siga los siguientes pasos:

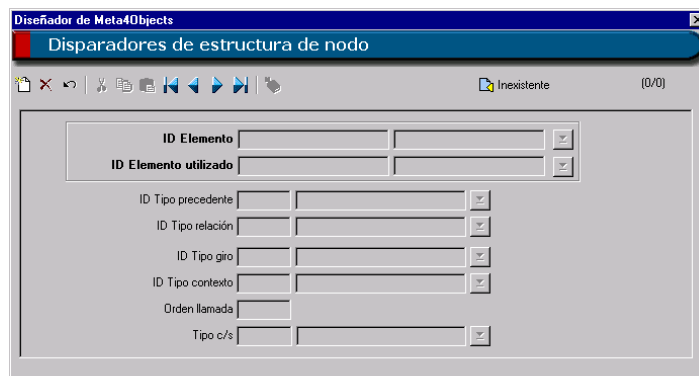
1. Desde el Diseñador de Meta4Objects, edite el Meta4Object que contiene la estructura de nodo para la que quiere crear el disparador.
2. Haga clic sobre el icono correspondiente a la estructura de nodo para la que quiere crear el disparador y pulse el botón derecho del ratón. Se desplegará un menú.
3. Seleccione la opción de menú **Editar|Disparadores de estructura de nodo**.

Figura 42. Editar disparadores de estructura de nodo



Aparecerá la siguiente ventana:

Figura 43. Elementos conectores de estructura de nodo



A continuación, rellene los campos con los datos que recoge la siguiente tabla:

Campo	Descripción
ID Elemento	Elemento destino del disparador. La ejecución de este elemento se desencadenará como resultado de la ejecución del elemento referenciado en la propiedad ID Elemento utilizado .
ID Elemento utilizado	Elemento origen del disparador. Cuando se ejecute este elemento se desencadenará la ejecución del elemento referenciado en la propiedad ID Elemento . El orden en el que se ejecutarán los dos elementos conectados mediante el disparador está determinado por el valor que se haya asignado a la propiedad ID Tipo precedente .
ID Tipo precedente	Determina el orden de ejecución de los dos elementos conectados. Puede tomar dos valores: <i>Después</i> y <i>Antes</i> . Si recoge el valor <i>Después</i> , en primer lugar se ejecutará el elemento origen (referenciado en ID Elemento utilizado) y a continuación el elemento destino (referenciado en ID Elemento). Si recoge el valor <i>Antes</i> , en primer lugar se ejecutará el elemento destino (referenciado en ID Elemento), y a continuación el elemento origen (referenciado en ID Elemento utilizado).
ID Tipo relación	Operador que se utilizará para comparar los valores de los elementos conectados. Al conectar dos elementos con ejecución (métodos o conceptos), no es necesario rellenar esta casilla.
ID Tipo giro	En el caso de que el elemento destino tenga un ámbito a nivel de registro, y el elemento origen tenga un ámbito a nivel de bloque o nodo, esta propiedad determina el orden en el que se ejecutarán las distintas instancias del método (habrá una instancia por cada registro de la estructura de nodo). Puede tomar dos valores: <i>Giro arriba</i> y <i>Giro abajo</i> . Si se asigna a esta propiedad el valor <i>Giro arriba</i> , las instancias del método se ejecutarán una tras otra, comenzando por el último registro recuperado por la estructura de nodo y finalizando con el primer registro. Si se asigna a esta propiedad el valor <i>Giro abajo</i> , las instancias del método se ejecutarán una tras otra, comenzando por el primer registro recuperado por la estructura de nodo, y finalizando con el último registro.
ID Tipo Contexto	En el caso de que el elemento destino tenga un ámbito a nivel de registro, esta propiedad determina para qué registros se va a ejecutar el método o concepto. Por ejemplo, sólo para los registros que hayan sido añadidos durante la sesión de trabajo, sólo para los modificados o borrados, o para todos ellos.
Orden llamada	En el caso de que el mismo elemento sea origen de dos o más disparadores (es decir, si desencadena la ejecución de dos o más métodos destino), y a la propiedad ID Tipo precedente de los disparadores se le haya asignado el mismo valor), esta propiedad determina el orden en el que se ejecutarán los elementos destino.
Tipo c/s	Determina el comportamiento del disparador en un entorno cliente/servidor. Por ejemplo, si se quiere conectar métodos en ejecución servidor, esta propiedad tendrá que asignarle el valor <i>Back</i> , si se quieren conectar dos métodos en ejecución cliente, esta propiedad recogerá el valor <i>Front</i> , etc. Encontrará más información en el capítulo <i>Arquitectura Cliente/Servidor</i> .

Tras completar estos valores, haga clic en el botón **Aceptar** y grabe los cambios en el Meta4Object.

Disparadores de nodo

Un disparador definido a nivel de nodo se creará desde el Diseñador de Meta4Objects. Se puede crear un disparador de nodo en el momento de crear un nodo o en cualquier momento posterior.

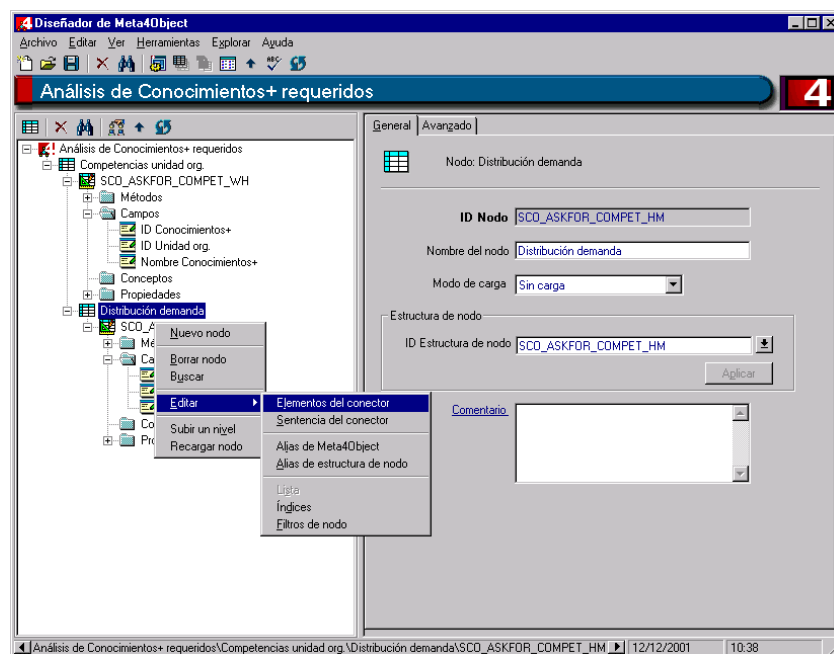
Cuando se crea un disparador de este tipo, se conectan dos de los métodos o conceptos de la estructura de nodo sobre la que se define el nodo, de forma que la ejecución de uno de ellos desencadenará la ejecución del segundo.

Si se define un disparador a nivel de nodo, el ámbito del disparador se verá restringido al nodo en el que se ha definido. Es decir, el disparador no funcionará en el resto de los nodos, a pesar de que estos nodos se hayan definido sobre la misma estructura de nodo.

Para definir un disparador a nivel de nodo, se deben seguir los siguientes pasos:

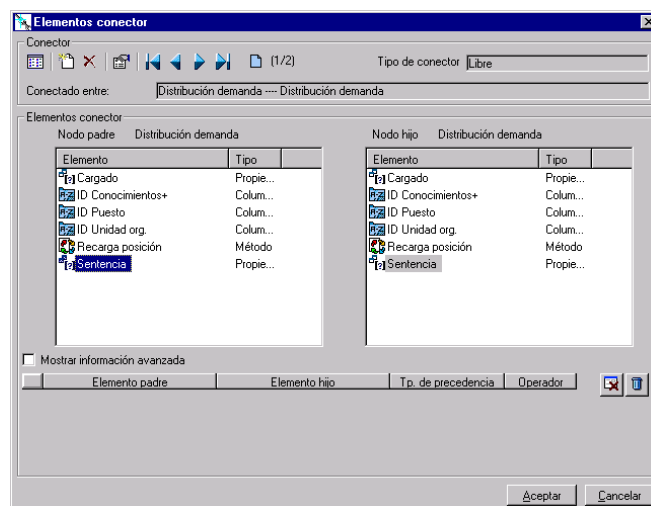
1. Desde el Diseñador de Meta4Objects, abra el Meta4Object que contiene el nodo para el que quiere diseñar el disparador, y haga clic sobre su icono correspondiente.
2. Haga clic en el botón derecho del ratón. Se desplegará un menú.
3. Seleccione la opción de menú **Editar|Elementos del conector**.

Figura 44. Opción de menú Editar elementos del conector



4. Se verá una ventana como la siguiente:

Figura 45. Ventana Elementos conector





5. Haga clic en el botón **Conector nuevo**. Se verá la siguiente ventana:

Figura 46. Ventana Conector nuevo

6. En esta ventana, podrá indicar las características del nuevo conector. Complete las casillas con la información que se indica en la siguiente tabla:

Propiedad	Descripción
Tipo de conector	Recoge el tipo de conector que se quiere utilizar. Puede ser de tipo registro-bloque, bloque-bloque y libre. En el caso de que se quieran conectar dos métodos del mismo nodo, se utilizará el conector Libre.
Orden de ejecución	Orden del conector. Este número determina el orden en el que se van a ejecutar los conectores, en el caso de que un mismo nodo esté conectado a más de un nodo.
Eliminar hijos	Indica si se quiere activar o no el borrado en cascada entre los dos nodos.
Propagar cancelación	Indica si se quiere propagar la cancelación entre los nodos.
Nodo padre	Recoge el nodo que se encontraba activo en el momento de crear el conector. Se corresponde con el nodo cuyos elementos se quieren conectar mediante un disparador.

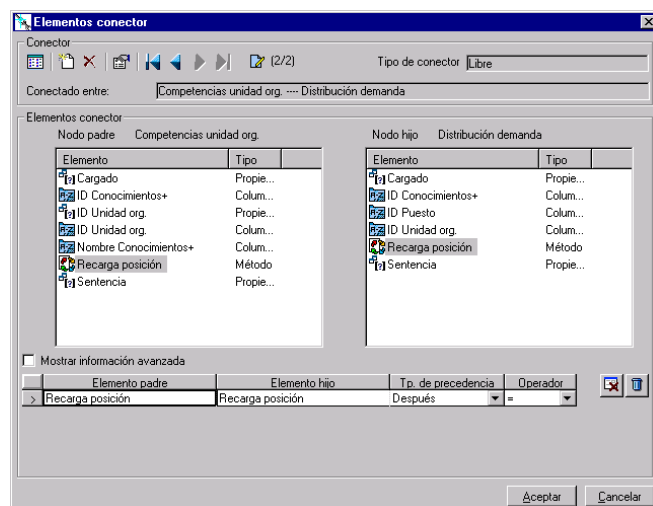
7. Tras completar estas propiedades, haga clic en el botón **Cerrar** para volver a la ventana anterior.

Por defecto, el programa le presentará la ventana con el nodo activo como nodo principal y como nodo secundario.

8. En el recuadro de la izquierda, haga clic sobre el elemento de tipo método o concepto que quiere utilizar como origen del disparador.

9. En el recuadro de la derecha, haga doble clic sobre el elemento que quiere utilizar como destino del disparador. Podrá apreciar que se ha añadido una nueva fila en la rejilla de la parte inferior de la ventana.

Figura 47. Elementos conector



Como elemento principal, figurará el elemento origen. Como elemento secundario, el destino.

10. Para parametrizar el comportamiento del disparador, rellene las propiedades de la rejilla de acuerdo a lo que se indica en la siguiente tabla:

Propiedad	Descripción
Tp. de precedencia	Determina el orden de ejecución de los dos elementos conectados. Puede tomar dos valores: <i>Antes</i> y <i>Después</i>. Si recoge el valor <i>Después</i>, en primer lugar se ejecutará el elemento origen (referenciado en ID Elemento utilizado y, a continuación, el elemento destino (referenciado en ID Elemento). Si recoge el valor <i>Antes</i>, en primer lugar se ejecutará el elemento destino (referenciado en ID Elemento) y a continuación, el elemento origen (referenciado en ID Elemento utilizado).
Sentido de pr.	En el caso de que el elemento destino tenga un ámbito a nivel de registro y el elemento origen tenga un ámbito a nivel de bloque o nodo, esta propiedad determina el orden en el que se ejecutarán las distintas instancias del método (habrá una instancia por cada registro de la estructura de nodo). Puede tomar dos valores: <i>Arriba</i> y <i>Abajo</i>. Si se asigna a esta propiedad el valor <i>Arriba</i>, las instancias del método se ejecutarán una tras otra, comenzando por el último registro recuperado por la estructura de nodo, y finalizando con el primer registro. Si se asigna a esta propiedad el valor <i>Abajo</i>, las instancias del método se ejecutarán una tras otra, comenzando por el primer registro recuperado por la estructura de nodo y finalizando con el último registro.

Propiedad	Descripción
Orden	En el caso de que el mismo elemento sea origen de dos o más disparadores (es decir, si desencadena la ejecución de dos o más métodos destino) y a la propiedad ID Tipo precedente de los disparadores se le haya asignado el mismo valor), esta propiedad determina el orden en el que se ejecutarán los elementos destino.
Cont. cond.	En el caso de que el elemento destino tenga un ámbito a nivel de registro, esta propiedad determina para qué registros se va a ejecutar el método o concepto. Por ejemplo, sólo para los registros que hayan sido añadidos durante la sesión de trabajo, sólo para los modificados o borrados, o para todos ellos.
Operador	Operador que se utilizará para comparar los valores de los elementos conectados. Al conectar dos elementos con ejecución (métodos o conceptos), no es necesario rellenar esta casilla.
Cliente/ Servidor	Determina el comportamiento del disparador en un entorno cliente/servidor. Por ejemplo, si se quieren conectar métodos en ejecución servidor, esta propiedad tendrá que recoger el valor <i>P. server</i> ; si se quieren conectar dos métodos en ejecución cliente, esta propiedad recogerá el valor <i>P. cliente</i> , etc. Encontrará más información sobre esta propiedad en el documento <i>Cliente/ Servidor</i> .
Args. Conect	Lista de argumentos del nodo padre que pueden utilizarse para filtrar los registros del nodo hijo



Para mostrar todas las propiedades, active la casilla de verificación **Mostrar información avanzada**

11. Desde esta ventana, puede añadir más conexiones entre elementos del mismo nodo. Para ello, repita los pasos anteriores para cada conector.

Disparador de conector

La aplicación permite crear un tercer tipo de disparador: los que conectan entre sí métodos o conceptos de dos nodos conectados en la definición de un Meta4Object.

En un disparador de este tipo, la ejecución del método situado en el nodo padre de la relación desencadenará la ejecución del método del nodo hijo con el que se ha conectado.

Este tipo de conectores se crea mediante una asociación de elementos del conector, desde el Diseñador de Meta4Objects. La creación de disparadores de conector es idéntica a la definición de un disparador de nodo.

Únicamente existe una diferencia: como nodo secundario, tendrá que seleccionar el nodo que contiene el elemento destino del disparador. Para ello:

1. Deberá escoger como nodo activo el nodo que contenga el elemento hijo del disparador.
2. En la ventana **Conector nuevo** podrá seleccionar como nodo principal el nodo que contiene el elemento padre del disparador.

Estructura de un disparador

La definición de un disparador, cualquiera que sea su tipo, comprende los siguientes componentes:

- El elemento de tipo método que desencadena la ejecución del segundo método (para hacer referencia a este método se utiliza el término *elemento origen*).
- El elemento de tipo método cuya ejecución se desencadena
Este método se puede utilizar para realizar comprobaciones sobre valores nuevos o actualizados, o para abrir nuevos registros en tablas relacionadas con la tabla en la que se hace la inserción de un nuevo registro. Este elemento se llama *elemento destino* del disparador.
- El indicador del momento de ejecución del método o concepto destino (el momento de ejecución se indica en la propiedad **Tp. de precedencia**).
- El rango de registros para los que se ejecutará el método destino.
Este rango sólo se indicará cuando el elemento destino tenga un ámbito a nivel de registro, y se indicará en la propiedad **Cont cond.**

A continuación, se explica cada una de estas propiedades:

Contexto de ejecución

La propiedad Contexto de ejecución, **Cont. cond.**, aplicable a nivel de asociación entre pares de elementos, permite especificar el rango de registros para los que se deben ejecutar las instrucciones indicadas en la regla del segundo método.

Cont. cond. puede tomar uno de los valores que recoge la siguiente tabla.

Valor	Descripción
<i>Todos los registros</i>	El método se ejecuta para todos los registros del nodo, independientemente de que estén distribuidos en bloques de memoria diferentes.
<i>Registros insertados</i>	El método se ejecuta para todos los registros que se hayan agregado al nodo durante la sesión de trabajo sin que se hayan grabado previamente en la base de datos.
<i>Registros actualizados</i>	El método se ejecuta para todos los registros del nodo que se hayan modificado durante la sesión de trabajo sin que se hayan grabado previamente en la base de datos.
<i>Registros insertados y actualizados</i>	El método se ejecuta para todos los registros del nodo que hayan sido agregados o modificados durante la sesión de trabajo.
<i>Registros borrados</i>	El método se ejecuta para todos los registros del nodo que se hayan borrado durante la sesión de trabajo sin que se hayan grabado previamente en la base de datos.
<i>Registros insertados y borrados</i>	El método se ejecuta para todos los registros del nodo que hayan sido agregados o borrados durante la sesión de trabajo.
<i>Registros actualizados y borrados</i>	El método se ejecuta para todos los registros del nodo que hayan sido modificados o borrados durante la sesión de trabajo.
<i>Registros actualizados, insertados y borrados</i>	El método se ejecuta para todos los registros del nodo que hayan sido modificados, agregados o borrados durante la sesión de trabajo.
<i>Registro actual</i>	El método se ejecuta únicamente para el registro activo en el momento de desencadenar el disparador.
<i>Registros actual e insertados</i>	El método se ejecuta para el registro activo y para los registros agregados durante la sesión de trabajo.
<i>Registros actual y actualizados</i>	El método se ejecuta para el registro activo y para los registros modificados durante la sesión de trabajo.
<i>Registros actual, actualizados e insertados</i>	El método se ejecuta para el registro activo y para los registros agregados y modificados durante la sesión de trabajo.
<i>Registros actual y borrados</i>	El método se ejecuta para el registro activo y para los registros borrados durante la sesión de trabajo.

Valor	Descripción
<i>Registros actual, actualizados y borrados</i>	El método se ejecuta para el registro activo y para los registros modificados y borrados durante la sesión de trabajo.
<i>Registros actual, actualizados, insertados y borrados</i>	El método se ejecuta para el registro activo y para los registros modificados, agregados y borrados durante la sesión de trabajo.

Los disparadores se utilizarán normalmente para conectar un método de inserción, actualización, borrado o grabación (todos ellos predeterminados en la definición de un nodo), con un método definido por el usuario-programador que implemente las instrucciones de procesamiento que se quieren desencadenar.

Normalmente, el método cuya ejecución se desencadena forma parte del mismo método desde el cual se desencadena la ejecución. De todas formas, no hay ninguna restricción en este sentido y el método destino podría estar situado en un nodo distinto, como en el caso que se está describiendo en este apartado.

Al diseñar un disparador, es importante considerar que los métodos de inserción y borrado de registros sobre los nodos de un Meta4Object se ejecutan en memoria principal, y que los cambios no se graban en la base de datos hasta que no se hace una llamada al método `Persist_Tree()`.

Por tanto, si se quisiera diseñar un disparador que realizase unas comprobaciones antes de añadir unos registros en la base de datos, se tendrían que seguir los siguientes pasos:

1. Crear un elemento de tipo método con una regla que implementase todas las comprobaciones necesarias. El elemento tendría un ámbito a nivel de registro.

La regla del elemento habrá sido codificada para abortar una transacción con la llamada al método de sistema `EndTransaction(Rollback)`, en el caso de que no se cumplan las condiciones necesarias.
2. Conectar los elementos del sistema `Insert_Bl()`, `Update_Bl()` y `Delete_Bl()`, esto es, el elemento que graba en la base de datos los cambios que se hayan realizado en memoria principal, con el método anterior.
3. Seleccionar en la propiedad **Cont. cond.** el valor *Registros insertados*, para que el método se ejecute únicamente para los registros que han sido insertados, y así minimizar el número de comprobaciones.

-
4. Seleccionar en la propiedad **Tp. de precedencia** el valor *Antes*, para que el método destino se ejecute antes que el método origen. De esta forma, en cuanto se haga una llamada al método `Insert_blk()`, `Update_blk()` y `Delete_blk()`, que actúa como origen del disparador, el sistema ejecutará el método de usuario que comprueba la validez de los valores propuestos para los nuevos registros. Si los valores indicados son correctos, el sistema ejecutará a continuación el método `Persist_Tree()`. Si los valores no son correctos, el código del método de usuario abortará el flujo de ejecución del disparador mediante una llamada al método de sistema `End_Transation(Rollback)`.

El método para diseñar disparadores que automaticen la ejecución de métodos situados en la misma estructura de nodo se explica en el capítulo Estructuras de nodo: conceptos avanzados de este mismo manual.

Filtros en el conector

Un conector de tipo registro-bloque o bloque-bloque actúa como un filtro dinámico que determina qué conjunto de registros estará cargado en uno de los bloques de memoria asignados al nodo hijo, a medida que se recorren los registros del nodo padre.

Cuando se asocian pares de elementos de dos nodos a través de un conector, se está definiendo un filtro que tendrá un efecto similar al de un *join* entre tablas de una base de datos.

El término *condiciones predeterminadas* hace referencia a las condiciones que establecen las asociaciones entre pares de elementos. Además de las condiciones predeterminadas, al crear un conector entre un par de nodos es posible añadir otras condiciones sobre los elementos del nodo hijo que recojan valor, es decir, los elementos de tipo campo, propiedad o concepto.



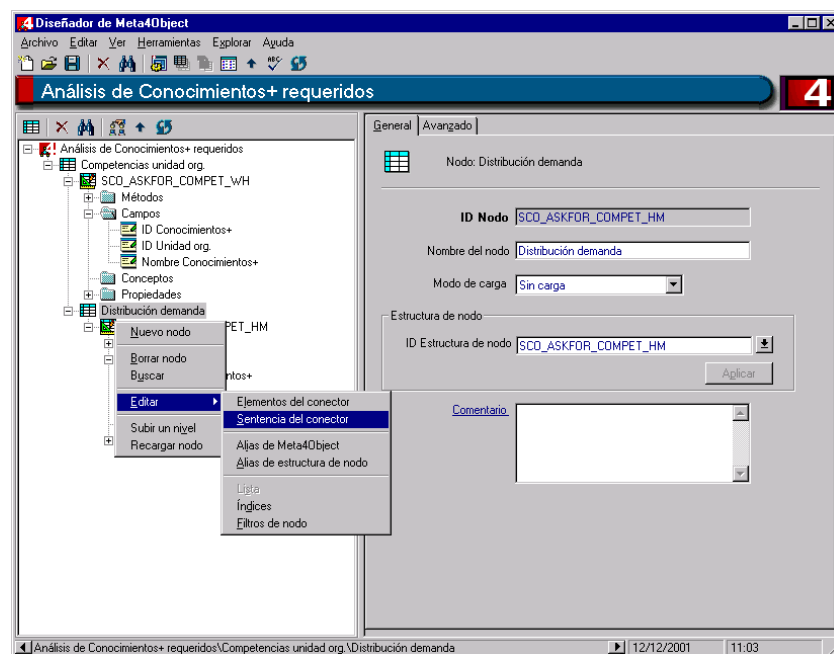
Cuando se defina una condición sobre un elemento de tipo concepto, es importante recordar que el concepto no recogerá el valor resultado de la ejecución de la regla a la que está asociado hasta que no se haya ejecutado esa regla de manera explícita. Hasta ese momento, el concepto recogerá el valor que tenga asignado por defecto; el valor por defecto de un concepto se indicará en el momento de su creación.

Las herramientas de diseño de la aplicación incluyen un módulo desde el que se pueden añadir nuevas condiciones a las condiciones predeterminadas del filtro: el Editor de sentencias.

Para acceder a esta herramienta y ver el filtro del conector, siga los siguientes pasos:

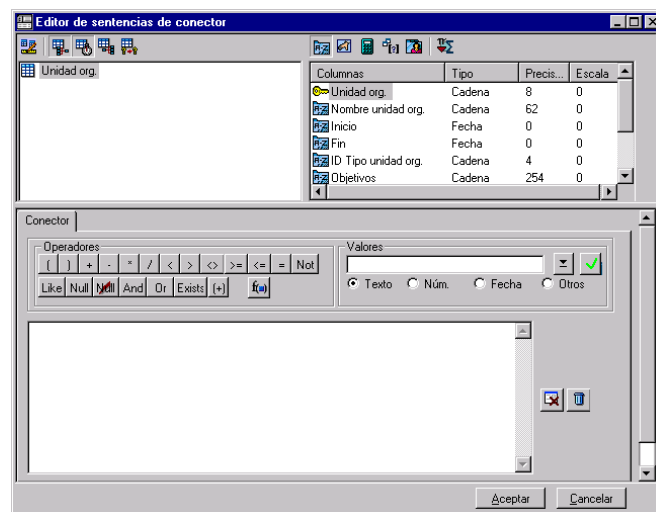
1. Desde el Diseñador de Meta4Objects, haga clic en el nodo cuya sentencia de conector quiere editar.
2. Haga clic en el botón derecho del ratón. Se desplegará un menú como el de la siguiente imagen.
3. Seleccione la opción de menú **Editar|Sentencia del conector**.

Figura 48. Editar instrucción del conector



El programa le presentará la siguiente ventana:

Figura 49. Editor de sentencias



La estructura de las condiciones que se pueden añadir al filtro de un conector es idéntica a la descrita para los filtros sobre estructuras de nodo en el capítulo *Estructuras de nodo: conceptos básicos* de este mismo manual.

Si bien es aconsejable consultar este capítulo para obtener una mayor información, en el siguiente apartado se explica brevemente la estructura de un filtro de conector.

Estructura de un filtro de conector

La estructura de un filtro de conector es idéntica a la de un filtro definido sobre una estructura de nodo. De esta forma, un filtro estará formado por una o varias condiciones. Cada condición consiste en la comparación del valor de un elemento del nodo hijo con un segundo valor escrito a la derecha del signo igual. En el caso de los filtros de conector, este segundo valor puede ser:

- Un valor constante, indicado en el momento de diseñar el filtro
- Un valor solicitado al usuario en tiempo de ejecución
- El valor de uno de los elementos del nodo padre con el que está conectado

En este último caso, cuando se compare el valor de uno de los elementos del nodo hijo con el valor de un elemento del nodo padre, se dice que el elemento del nodo padre se está pasando como parámetro a la condición del filtro. Es posible pasar como parámetros elementos de tipo campo, concepto y propiedad.



Para poder pasar como valor un concepto, es necesario que el concepto haya sido ejecutado con anterioridad o recoja un valor por defecto.

No se pueden pasar como parámetros los métodos, ya que no recogen ningún valor.

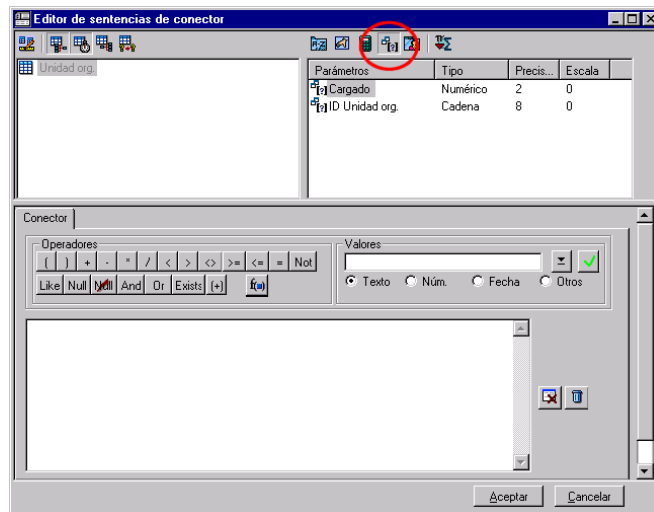
Para poder utilizar en el filtro un valor solicitado al usuario en tiempo de ejecución, el usuario-programador tiene que seguir los siguientes pasos:

1. Crear en el nodo padre un elemento de tipo propiedad; las propiedades **Ámbito** y **Visible** de este elemento deberán recoger, respectivamente, los valores *Nodo* y *Sí*.
2. En la sentencia del conector, indicar que el valor de uno de los elementos del nodo hijo debe compararse con el valor del elemento de tipo propiedad anterior.

Siempre que se crea un elemento de tipo propiedad en el nodo padre, para el que se han indicado los valores *Nodo* y *Sí* en las propiedades **Ámbito** y **Visible**, el sistema lo añadirá a la lista de argumentos del nodo padre que pueden utilizarse para filtrar los registros del nodo hijo.

De esta forma, para utilizar la propiedad como criterio de filtro en la condición del conector, bastará con seleccionarla de la lista de elementos que se muestran al hacer clic en el botón **Parámetros**.

Figura 50. Editor de sentencias - sentencia del conector



3. Cuando se abra el Meta4Object desde el software cliente de la aplicación y se solicite al sistema la carga en memoria de los registros correspondientes al nodo hijo, el sistema solicitará al usuario el valor para el elemento de tipo propiedad que se ha utilizado como filtro, antes de recuperar ningún dato.

Una vez el usuario haya indicado el valor para esta propiedad, el sistema cargará en memoria principal, únicamente, aquellos registros que cumplan la condición propuesta en el filtro del conector.

Respecto a los operadores de comparación que pueden utilizarse para comparar los valores de los elementos del nodo hijo, con los valores propuestos como condiciones del filtro, véase el apartado "Filtros de datos" del capítulo *Estructuras de nodo: conceptos básicos* de este mismo manual.

Filtros de fechas

Un tipo especial de filtros son los filtros de fechas. Estos filtros se definen sobre elementos del nodo hijo que recogen un valor de tipo fecha, con el fin de recuperar, únicamente, aquellos registros cuyos periodos de validez están comprendidos dentro de unas fechas de validez determinadas.

Generalmente, los filtros de fechas se diseñarán sobre estructuras de nodo que hayan sido creadas a partir de una tabla fechada: es decir, una tabla que contenga dos columnas que recojan, respectivamente, las fechas de inicio y de fin del periodo de validez de cada uno de sus registros. Por ejemplo, son tablas fechadas las tablas que recogen los absentismos de los empleados o su adscripción a un departamento, centro de trabajo, etcétera.

Los filtros sobre estructuras de nodo fechadas pueden definirse de dos formas:

- Comparando los valores de los elementos **Fecha_inicio** y **Fecha_fin** con un valor constante propuesto en el momento de diseñar el filtro de conector
- Comparando los valores de los elementos **Fecha_inicio** y **Fecha_fin** con los valores que recoja, en esas mismas columnas, el registro que se encuentre activo, o con un par de valores propuestos por el usuario en tiempo de ejecución

En este segundo caso, se dice que se ha diseñado un filtro dinámico entre los nodos padre e hijo.

Filtros de conectores y reutilización de estructuras de nodo

La definición de filtros en el momento de instanciar la estructura de nodo en un Meta4Object ofrece al usuario-programador la posibilidad de crear estructuras de nodo de carácter genérico, y por tanto, más fácilmente reutilizables.

Por ejemplo, si se necesita extraer de la tabla de acumulado, es decir, la tabla que recoge los resultados retrospectivos del cálculo de nómina, la totalización de las cantidades pagadas en concepto de dietas durante el último trimestre y durante el último semestre.

En este caso, cabrían dos posibilidades: diseñar dos estructuras de nodo, una que filtrase aquellos registros correspondientes al último trimestre del año, y otra que filtrase aquellos registros correspondientes al último semestre.

La posibilidad de definir filtros en el conector, en el momento de instanciar la estructura de nodo, permite crear una única estructura de nodo que podría utilizarse siempre que se necesitase recuperar información de las tablas de acumulado.



Cuando no se sepa con certeza si una estructura de nodo va a aplicar el mismo filtro todas las veces que se utilice, es aconsejable utilizar un filtro dinámico.

Es conveniente potenciar al máximo la reutilización de estructuras de nodo, ya que así se minimizan los costes necesarios para diseñar nuevas funcionalidades y se facilita el mantenimiento de la aplicación permitiendo realizar cambios globales.

Como regla metodológica, podría indicarse que sólo se debe definir un filtro en una estructura de nodo cuando esa estructura de nodo vaya a utilizarse siempre para filtrar esa misma información. Como este caso es impredecible, siempre será aconsejable aplicar las condiciones del filtro en las instanciaciones que se hagan de la estructura de nodo en distintos Meta4Objects.

Administración de memoria

6

Acerca de este capítulo

En el momento de diseñar un Meta4Object, es posible definir la política que debe utilizar el servidor de aplicaciones para cargar la información en memoria y gestionar las operaciones de lectura sobre la base de datos.

El Meta4Object Engine es un componente de la aplicación de Meta4Mind Set que se encarga de gestionar estas operaciones, así como la estructura de los datos en memoria.

En este capítulo, se explican los siguientes conceptos:

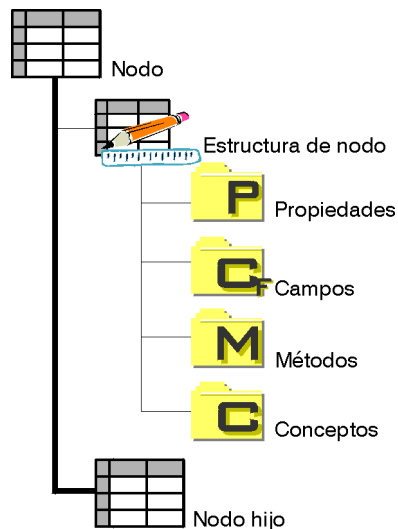
- Diferencia entre un Meta4Object y las instancias del mismo
- Carga en memoria de una instancia de un Meta4Object
- Posibilidad de compartir un Meta4Object en una sesión de la aplicación para reutilizar datos y recursos de hardware
- Distribución del procesamiento de un Meta4Object entre los puestos cliente y el servidor de aplicaciones

Meta4Objects e instancias de Meta4Objects

En el momento de definir un Meta4Object, el desarrollador indica qué información va a extraer de la base de datos y qué operaciones se deben realizar con esos datos, o con los recuperados por otros Meta4Objects.

La información que se quiere extraer se indica mediante elementos de tipo campo, los elementos auxiliares o de apoyo son de tipo propiedad y las instrucciones de procesamiento se reflejan en los elementos de tipo método o concepto.

Figura 51. Estructura de un Meta4Object



Toda la funcionalidad de la aplicación se implementa a través de Meta4Objects. De hecho, estos Meta4Objects son la única interfaz disponible entre la información registrada en la base de datos y el nivel de aplicación. Los procesos de cálculo o actualización del contenido de la base de datos siempre se deben hacer a través de un Meta4Object que actúa como interfaz.

En resumen, un Meta4Object equivale a una estructura en la que se indica:

- De qué tablas se quieren extraer datos
- De qué columnas de las tablas anteriores se quieren obtener los valores
- Qué condiciones deben cumplir los registros de las tablas para que sus datos sean recuperados
- Qué operaciones se quieren realizar con esos datos

Un Meta4Object únicamente especifica los puntos anteriores; no contiene en sí mismo ningún dato. Un Meta4Object es similar a una estructura de datos típica de los lenguajes de programación o a una clase de un entorno basado en la

orientación a objetos. El Meta4Object es una definición estática de la estructura guardada en el repositorio.

Cuando un usuario de la aplicación especifica que quiere trabajar con un determinado Meta4Object, bien desde la interfaz de la aplicación, bien desde el código de una regla LN4, el sistema crea una instancia del Meta4Object solicitado. Para llevar a cabo esto:

- El sistema reserva un espacio en la memoria principal en el que se recuperará la estructura del Meta4Object.
- Si se invoca la carga de la instancia, se carga en el espacio reservado la información de la base de datos que se ha indicado en la definición del Meta4Object; esta información consiste en los datos extraídos de tablas.

Los datos que se cargan en memoria dependen del estado en que se encuentra la base de datos cuando el usuario solicita la carga del Meta4Object.

De esta forma, la carga de un mismo Meta4Object puede recuperar datos diferentes dependiendo del momento en que se realice.

Tipos de carga

Para poder trabajar con datos es necesario realizar la carga de un Meta4Object. A continuación, se describen las diferentes maneras de realizar la carga, dependiendo de cómo se generan, parametrizan, etc.

Carga completa

Carga todos los bloques y todos los registros del Meta4Object. En este caso, para cargar los datos de un Meta4Object, el sistema llama a la función *Load()*. Esta carga hace que se invoque el método *RootLoad()* de todos los nodos raíz, que en caso de estar correctamente conectados al método *Load_prg()*, desencadena la carga de los hijos.

Carga automática de bloque

El sistema activa la carga del nodo a petición del usuario, esto es, la estructura del nodo se ejecuta cuando lo pide el usuario desde el árbol de navegación del Meta4Object o desde una regla LN4.

El sistema invoca al método *Load_blk()* del bloque al que se obtiene acceso. Si es desde una regla LN4, la carga se lleva a cabo cuando se accede a algún registro o a alguno de sus elementos o se ejecuta un *Count()* del bloque.

Carga automática de rama

Carga los registros del bloque activo y también desencadena la carga en memoria de todos los bloques de los nodos conectados con ese bloque.

La carga de datos en memoria se propaga hasta el siguiente nivel jerárquico.

El funcionamiento es similar a la carga automática de bloque, excepto que se invoca al método *Load_prg()*, que si se encuentra correctamente conectado, desencadena la carga de los bloques hijos.

Carga según Nodo

Depende del tipo de carga que tenga asignado el nodo. Esta carga puede tomar uno de los siguientes valores:

- 1: desactivada
- 2: de bloque
- 3: de rama

El funcionamiento de los tipos 2 (de bloque) y 3 (de rama) es similar a la carga automática de bloque y a la de rama, respectivamente.

En el caso del tipo 1 (desactivada), no se carga el bloque aunque se acceda a sus registros, y si se quiere cargar los datos, hay que hacerlo desde LN4 de forma normal, invocando la carga del bloque con los métodos *Load_blk()* o *Load_prg()*.

Parametrización de la carga

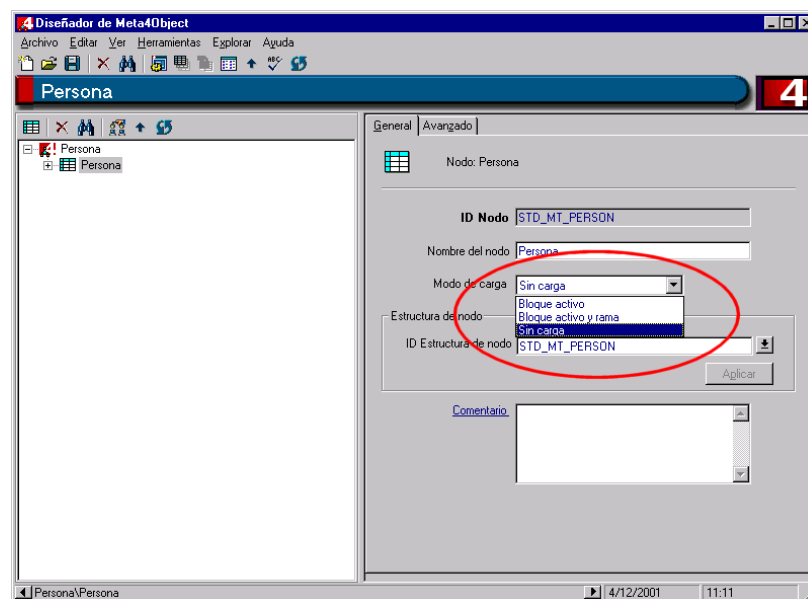
La aplicación permite indicar el tipo de carga del Meta4Object desde distintos lugares:

- Desde el Diseñador de Meta4Objects, a nivel de nodo y cuando se haga una llamada de nivel 2 a otro Meta4Object
- Desde la Barra de tareas del desarrollador
- Desde el Diseñador de presentaciones

Carga de Meta4Object desde el Diseñador de Meta4Objects

Desde el Diseñador de Meta4Objects se asigna el modo de carga a nivel de nodo, como se muestra a continuación:

Figura 52. Modo de carga



Los posibles modos de carga son los siguientes:

- Sin carga (Carga desactivada)
- Bloque activo
- Bloque activo y rama

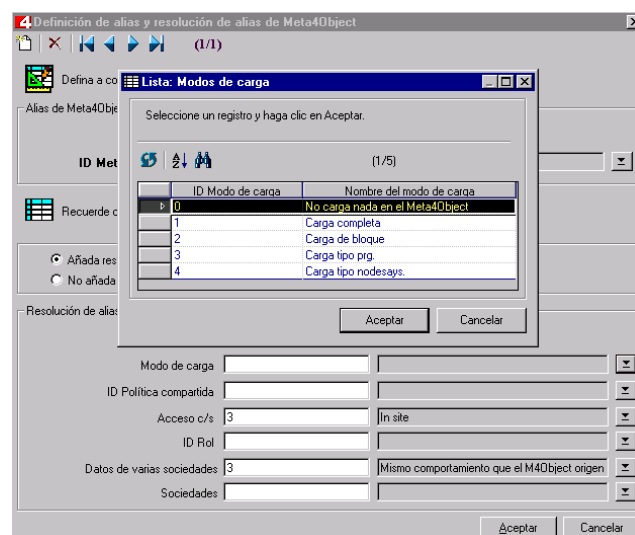
Estos valores sólo se tienen en cuenta cuando el modo de carga del Meta4Object es de tipo según Meta4Object.

Carga de Meta4Objects desde llamadas de nivel 2

Se produce una llamada de nivel 2 cuando desde la regla de un método o concepto de un Meta4Object se hace referencia a un elemento de otro Meta4Object diferente. El elemento origen es aquél cuya regla contiene la referencia al segundo elemento; el elemento destino es al que se hace referencia.

Cuando se haga una llamada de nivel 2 y sea necesario crear una nueva instancia de Meta4Object, se aplicará el modo de carga aquí definido. A continuación, se muestra una lista con los distintos modos de carga:

Figura 53. Modo de carga



Puede tomar uno de los siguientes modos de carga:

- **No carga nada en el Meta4Object:** carga en memoria la estructura del Meta4Object, sin ningún dato. Se utiliza cuando la llamada se hace a un elemento de tipo método cuya ejecución no requiere datos, o cuando se quiere utilizar el Meta4Object para crear nuevos registros.
- **Carga completa:** carga el Meta4Object en memoria con todos sus datos.
- **Carga de bloque:** carga los datos del bloque a medida que los necesita el código.
- **Carga tipo prg.:** carga los datos del bloque y se propaga la carga hasta el siguiente nivel jerárquico.
- **Carga tipo nodesays:** depende del tipo de carga definido en el nodo.

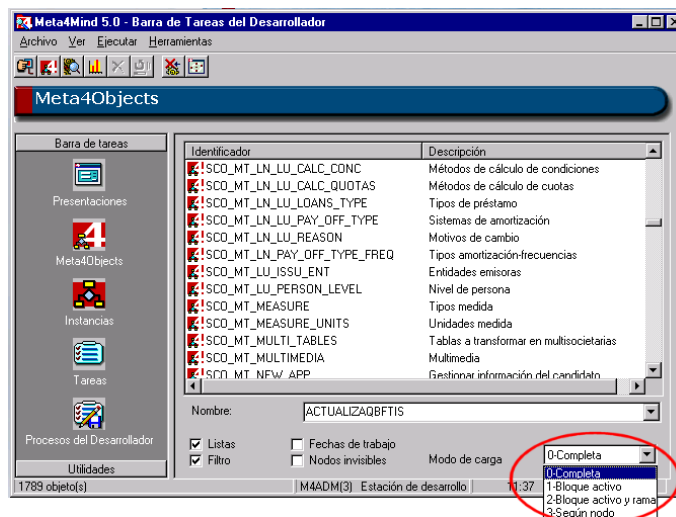
Al crear una llamada de nivel 2, el desarrollador puede parametrizar las opciones de compartición en memoria para indicar si se debe crear una nueva instancia o reutilizar una creada con anterioridad.

En cualquier caso, siempre que se tenga que hacer una llamada de nivel 2, es necesario definir un *alias* para el Meta4Object destino de la llamada.

Carga de Meta4Object desde la Barra de tareas del desarrollador

Desde la Barra de tareas del desarrollador, en la parte inferior de la pantalla aparece el cuadro de lista desplegable **Carga automática**, que contiene los tipos de carga que se muestran a continuación:

Figura 54. Modo de carga



Puede asignar uno de los siguientes tipos de carga automática:

- **Completa**
- **Bloque activo**
- **Bloque activo y rama**
- **Según nodo**

Carga de Meta4Object desde el Diseñador de presentaciones

Desde el Diseñador de presentaciones se puede definir el tipo de carga del Meta4Object que se realizará de forma automática al abrir la presentación.

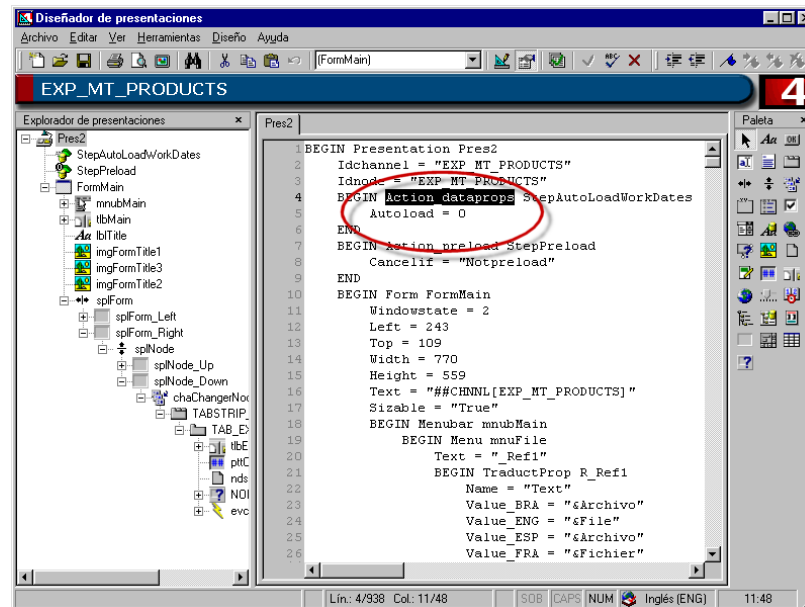
Para esto, abra una nueva presentación o una presentación ya creada desde la barra de herramientas, haga clic en el botón **Editor de código** para abrir el editor que contiene el código OBL del componente activo.



OBL es un lenguaje orientado a objetos desarrollado por Meta4, que se emplea en el diseño de presentaciones.

Ahora se permite realizar modificaciones del código mismo.

Figura 55. Modos de carga



El tipo de carga del Meta4Object se define en **Action_dataprops** y en concreto en la propiedad **Autoload**. Por defecto, el valor de **Autoload** es 0, aunque los valores que puede tomar son los siguientes:

- 0: desactivado
- 1: bloque activo
- 2: bloque activo y rama
- 3: carga según nodo

Para obtener más información, consulte la ayuda en línea correspondiente a la herramienta **Diseñador de presentaciones**.

Reutilización de Meta4Objects y llamadas de nivel 2

En un entorno de trabajo cliente/servidor, es posible que existan varias instancias de un mismo Meta4Object cargadas en la memoria principal de un servidor de aplicaciones.

El sistema permite compartir la misma instancia de un Meta4Object que se ha cargado en la memoria de un servidor de aplicaciones. En este caso, se está reutilizando la instancia del Meta4Object.

La reutilización de Meta4Objects minimiza los requisitos de memoria, aumenta la escalabilidad de la aplicación, y permite optimizar y racionalizar los recursos de hardware.

Para implementar la reutilización de Meta4Objects, se emplea un sistema de *alias*, que permite hacer referencia a instancias específicas de un Meta4Object desde el código LN4 mediante su *alias*.

Las llamadas de nivel 2 permiten parametrizar la política de reutilización que se seguirá al crear instancias del Meta4Object que contiene el elemento destino de la llamada.

En la resolución de *alias* se indica cuál va a ser la política de reutilización. Hay tres posibilidades:

- Crear una nueva instancia del Meta4Object
- Reutilizar la información generada por una instancia desde la misma solicitud de ejecución del Meta4Object del que parte la llamada de nivel 2
- Reutilizar la información obtenida con una instancia que se haya instanciado desde cualquier solicitud de ejecución de otro Meta4Object

Al hacer referencia a un Meta4Object desde una regla escrita en código LN4, el desarrollador debe indicar un *alias* para ese Meta4Object y comunicar al intérprete de reglas la información para la resolución del *alias*.

El modo de carga se debe seleccionar siempre por si finalmente es necesario crear una nueva instancia. Mediante estos modos de carga, el desarrollador puede indicar cómo se tiene que gestionar la carga de instancias de Meta4Objects, manteniendo un control real sobre los datos con los que está trabajando y gestionando de forma eficiente los recursos del sistema.

Carga de datos en la memoria en el entorno C/S

En el modelo cliente/servidor, el usuario solicita la ejecución de un proceso desde un puesto cliente.

El servidor de aplicaciones, tras aceptar la solicitud, inicia la ejecución del proceso requerido por el cliente. Durante dicha ejecución, el servidor de aplicaciones solicita toda la información que necesita al servidor de bases de datos. Como la aplicación sólo permite tener acceso a la base de datos a través de la interfaz que implementan los Meta4Objects, las instancias de los Meta4Objects que recuperan los datos y las instrucciones necesarias para ejecutar el proceso se cargan en la memoria principal del servidor de aplicaciones.



La utilización de un Meta4Object no implica su carga total en memoria. Es posible parametrizar la carga bajo demanda de los nodos de un Meta4Object. Se puede trabajar con un Meta4Object sin necesidad de que toda la información que recupera éste se encuentre disponible en la memoria principal del servidor de aplicaciones.

Las políticas de reutilización son las distintas alternativas disponibles. El servidor de aplicaciones no está capacitado para decidir cuál va a ser la política de reutilización; es el desarrollador el que la establece al crear las llamadas de nivel 2 entre Meta4Objects y las entradas de menú que permiten cargar en memoria los datos de un Meta4Object.

Parametrización de una carga en llamadas de nivel 2

Este apartado describe cómo se debe parametrizar una llamada de nivel 2 mediante el Diseñador de Meta4Objects.



Las llamadas de nivel 2 también se pueden parametrizar mediante el Diseñador de eventos. Para obtener más información sobre esta herramienta, consulte las ayudas en línea y el manual de *Eventos y Workflow*.

Desde las fórmulas LN4 es posible hacer referencia a elementos de otro Meta4Object. En este caso, se produce una dependencia o llamada de nivel 2. La sintaxis correcta para hacer este tipo de llamadas obliga a cualificar el nombre del elemento con un *alias* del Meta4Object y con el código identificativo del nodo en que está incluido el elemento al que se hace la llamada.

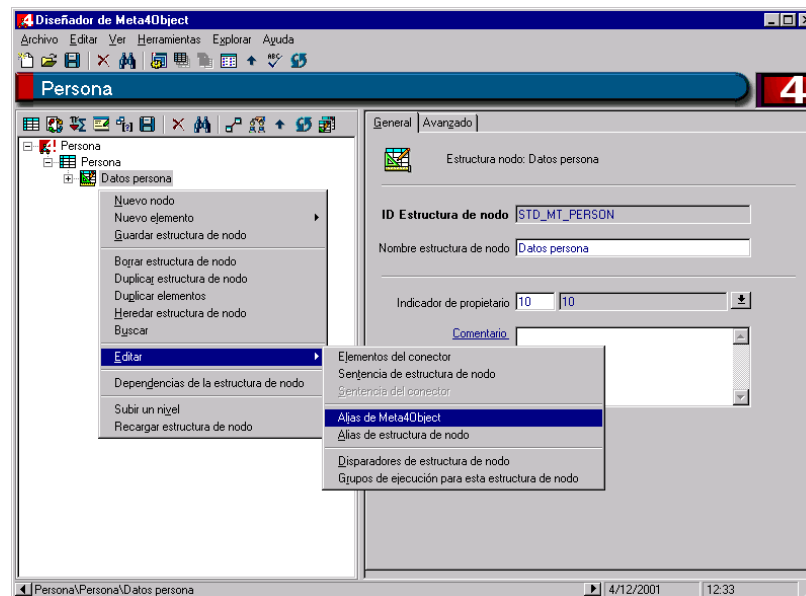
Si hay dos instancias activas del mismo Meta4Object en memoria, el intérprete LN4 necesitará saber a qué instancia se quiere hacer referencia desde la regla del elemento origen de la llamada.

Para resolver esta situación, el usuario que diseña la fórmula debe asignar un *alias* para hacer referencia a la instancia del Meta4Object con la que quiere trabajar. El *alias* consiste en un identificador textual que sustituirá al del Meta4Object en todas las referencias que se hagan a él en la regla.

Para declarar un *alias*, debe seguir estos pasos:

1. Desde el Diseñador de Meta4Objects, haga clic sobre el icono correspondiente a la estructura de nodo que contiene el elemento en cuya regla quiere utilizar el *alias* de Meta4Object.
2. Haga clic con el botón derecho del ratón. Aparece un menú emergente como el de la siguiente imagen:

Figura 56. Alias de Meta4Object



3. En este menú, seleccione la opción **Editar | Alias de Meta4Object**. Aparece el siguiente formulario:

Figura 57. Declaración de alias

4 Definición de alias y resolución de alias de Meta4Object

Defina a continuación un nuevo alias de Meta4Object.

Alias de Meta4Object

Alias

ID Meta4Object usado

Recuerde que sólo puede tener una instancia definida para un mismo alias de Meta4Object.

☐ Añada resolución de alias de Meta4Object

☒ No añada resolución de alias de Meta4Object

Resolución de alias de Meta4Object

Instancia

Modo de carga

ID Política compartida

Acceso c/s

ID Rol

Datos de varias sociedades

Sociedades

Aceptar Cancelar



4. Haga clic en el botón **Nuevo**.

5. En los campos **Alias** e **ID Meta4Object usado**, escriba, respectivamente el *alias* y el código identificativo del Meta4Object que contiene el elemento al que quiere hacer referencia.

6. Puede cerrar la ventana y grabar los cambios en el Meta4Object o definir la resolución del *alias*.



El *alias* sólo indica que se quiere utilizar una instancia específica de un tipo de Meta4Object, pero no recoge las características de esa instancia.

La resolución del *alias* recoge información que permite dar a conocer al sistema qué instancia tiene que utilizar cuando encuentre un *alias*.

La información sobre la instancia del Meta4Object correspondiente a cada *alias* se registra en el momento de definir la estructura del Meta4Object desde el que se hace la llamada.

En la resolución del *alias*, se indicará al sistema el Meta4Object al que se quiere hacer referencia en la llamada de nivel 2 que contiene la regla, y cómo se quiere tener acceso a él: creando una nueva instancia, reutilizando una instancia ya disponible, etc.

Esta información se indica en el momento de crear una instancia de la estructura de nodo que contiene el método o el concepto en un Meta4Object. Es decir, al crear un nodo que utilice la estructura de nodo desde el Diseñador de Meta4Objects.

Para declarar una resolución de *alias*, complete los siguientes datos en el

formulario Resolución de alias de Meta4Object:

Figura 58. Resolución de alias para Meta4Object

1. Haga clic en el botón de opción **Añada resolución de alias de Meta4Object**.
2. Rellene las casillas del formulario con la información que recoge la siguiente tabla:

Propiedad	Descripción
Instancia	Nombre propuesto para la instancia del Meta4Object que contiene el elemento al que se hace la llamada. El programa lo utiliza internamente para hacer referencia a una zona de memoria reservada para la nueva instancia del Meta4Object. También puede utilizarse en las funciones LN4 que utilizan llamadas de nivel 2 por indirección o dinámicas, aunque en este caso es aconsejable utilizar el <i>alias</i> del Meta4Object.
Modo de carga	Indica qué datos del Meta4Object se quieren recuperar. Puede tomar uno de los siguientes valores: ■ 0 - No carga nada en el Meta4Object ■ 1 - Carga completa ■ 2 - Carga de bloque ■ 3 - Carga tipo prg. ■ 4 - Carga tipo nodesays (según nodo)

Propiedad	Descripción
ID Política compartida	Mediante esta propiedad se indica si se quiere cargar una nueva instancia del Meta4Object o reutilizar una cargada con anterioridad. Puede tomar uno de los siguientes valores: ■ 1 - No compartido: se utiliza este valor cuando se quiere cargar una nueva instancia del Meta4Object. ■ 2 - Compartición local: se utiliza este valor cuando se quiere reutilizar una instancia del Meta4Object que se ha cargado desde una llamada de nivel 2 de otro elemento del mismo Meta4Object. ■ 3 - Compartición global: se emplea este valor cuando se quiera reutilizar una instancia del Meta4Object que se haya cargado con anterioridad desde cualquier elemento de un Meta4Object o desde la interfaz cliente.
Acceso c/s	Indica qué parte del Meta4Object se quiere cargar en memoria: la parte cliente (Meta4Object en cliente) o la parte servidor (Meta4Object en servidor). Se denomina Meta4Object en servidor a la parte del Meta4Object que se carga en el servidor de aplicaciones, y Meta4Object en cliente, a la que se carga en el puesto cliente. Esta propiedad debe tomar el siguiente valor: ■ 3 - In site: si la llamada de nivel 2 se hace desde un elemento que se ejecuta en el servidor de aplicaciones, se cargan los elementos del Meta4Object de la parte servidor. Si se hace desde un elemento que se ejecuta en el puesto cliente, se cargan los elementos del Meta4Object de la parte cliente. Para usuarios avanzados, esta propiedad también puede tomar los siguientes valores: ■ 1 - En cliente: se cargan los elementos del Meta4Object de la parte cliente. ■ 2 - En servidor: se cargan los elementos del Meta4Object de la parte servidor.
ID Rol	Se utiliza para asignar el perfil de seguridad del Meta4Object usado en caso de que se cree una nueva instancia del mismo o a la hora de buscar una instancia existente. A no ser que quiera cambiar de perfil de usuario para trabajar, se dejará vacío.
Datos de varias sociedades	Aparece cuando trabaje en un entorno multisociedad. Puede tomar uno de los siguientes valores: ■ 0 - Independiente de la sociedad: el Meta4Object no está sujeto a multisociedad. ■ 1 - No: los datos de cada sociedad pueden ser distintos, la información que maneja está filtrada por una única sociedad. ■ 2 - Sí: puede trabajar con datos de distintas sociedades al mismo tiempo. ■ 3 - Mismo comportamiento que el Meta4Object origen: hereda el comportamiento que tiene asignado el Meta4Object desde el que se realiza la llamada de nivel 2. ■ 4 - Cualquier comportamiento: no se especifica el comportamiento.
Sociedades	Aparece cuando se trabaja en un entorno multisociedad. Desde aquí puede seleccionar la sociedad con la que va a trabajar. Normalmente se deja vacío.

3. Cierre la ventana y grabe los cambios en el Meta4Object.

Parametrización de un Meta4Object con cachés

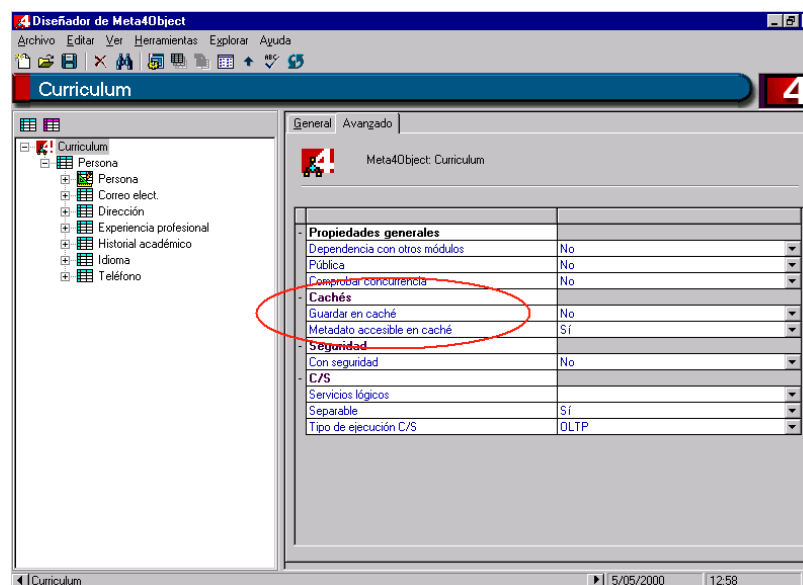
Las cachés son espacios de memoria temporales reservados para almacenar la estructura y el contenido de los Meta4Objects. Los Meta4Objects que gestiona la aplicación pueden cachearse y el comportamiento con cachés puede parametrizarse.

El Meta4Object cacheable se almacena en la memoria caché, que es un espacio de memoria RAM, de tal modo que cuando el sistema necesita leer datos sobre ese Meta4Object, lo hace directamente desde la caché y no desde la base de datos, lo que dinamiza las operaciones de lectura de datos y evita realizar operaciones de lectura en el SGBD.

Los Meta4Objects se almacenan temporalmente en la memoria caché del puesto cliente o del servidor de aplicaciones, dependiendo de dónde se invoque la carga del Meta4Object.

Se puede parametrizar el comportamiento con cachés de los Meta4Objects desde la pestaña de propiedades avanzadas que aparece en el Diseñador de Meta4Objects, tal y como se muestra a continuación:

Figura 59. Parametrización de la caché



Para obtener mayor información, consulte el capítulo *Arquitectura Cliente/servidor*.

Manejo de instancias de Meta4Object en Meta4Object Engine

La estructura cliente/servidor de la tecnología Meta4 para las aplicaciones como Meta4 PeopleNet o Meta4 KnowNet, y los mecanismos de administración de memoria utilizados por la aplicación pueden provocar que se solicite la ejecución de un mismo Meta4Object más de una vez. En este caso, el sistema mantiene espacios distintos en memoria principal para cada una de las instancias.

La arquitectura lógica de la tecnología Meta4 incluye un componente, el Meta4Object Engine, que se encarga de gestionar la creación de instancias de Meta4Objects y la administración de memoria interna.

Meta4Object Engine trabaja de la siguiente forma:

- Siempre que se inicia la aplicación, el Meta4Object Engine carga en memoria interna un Meta4Object que contiene información sobre la estructura del resto de Meta4Objects. La información necesaria para cargar este primer Meta4Object se registra en un archivo de inicialización; con esta información, el sistema puede cargar en memoria el resto de Meta4Objects definidos.
- A medida que los usuarios de la aplicación solicitan la ejecución de procesos al servidor, éste va creando instancias de los Meta4Objects necesarios.

Para saber qué instancias de Meta4Object se encuentran disponibles en cada momento, el sistema mantiene un registro de todas las instancias activas. Concretamente, para cada Meta4Object se ofrece información sobre el resto de Meta4Objects que han sido ejecutados desde sus elementos.

Las instancias que se han creado desde un elemento de un Meta4Object mediante llamadas de nivel 2, están disponibles durante todo el tiempo que permanecen activos los Meta4Objects desde los cuales se han creado las instancias. En el caso de que un Meta4Object esté utilizando esa instancia, ésta permanece activa hasta que se le deje de hacer referencia, independientemente de que el Meta4Object desde el que se creó la instancia permanezca o no activo.

Cuando se ejecuta una regla LN4 que necesita leer datos de otro Meta4Object, el Meta4Object Engine trabaja de la siguiente forma:

1. Si existe esa instancia disponible, porque ya ha sido utilizada desde alguna regla del Meta4Object, se trabajará con esa instancia.
2. Si una instancia no ha sido usada, se comprueba el tipo de política compartida que tiene:
 - Si es de tipo no compartido, se crea una nueva instancia.

- Si la compartición es local, se busca dentro de las instancias una que use el mismo Meta4Object, y si se encuentra, será ésta la que se utilice. En caso de no encontrar ninguna instancia, se creará una nueva.
 - Si la compartición es global, se busca una instancia que cumpla las condiciones, y si se encuentra, será ésta la que se utilice. En caso contrario, se crea una instancia nueva.
3. Si se creó una instancia nueva, se comprueba el modo de carga, para indicar qué datos del Meta4Object se quieren recuperar. Toma uno de los siguientes valores:
- No cargar
 - Carga completa
 - Carga automática de bloque
 - Carga automática de rama
 - Carga según nodo

En todos los casos anteriores, hay que tener en cuenta que el método empleado para crear instancias de Meta4Objects o reutilizar instancias disponibles no garantiza ninguna exclusividad sobre el control del Meta4Object. En cuanto se crea una instancia, cualquier otro Meta4Object puede trabajar con él y utilizar sus datos.

Listas y filtros

7

Acerca de este capítulo

La tecnología Meta4 ofrece distintos mecanismos para agilizar la carga de datos en memoria y optimizar la ejecución de los procesos de aplicación.

Entre estos recursos, se encuentra la posibilidad de restringir el número de registros recuperados por un nodo, los mecanismos de carga avanzados (modo Bloque activo, Bloque activo y rama, etc.) y las listas y los filtros predefinidos.

Las listas permiten cargar únicamente los datos de las columnas que conforman la clave primaria de una tabla. A partir de esta información se podrá identificar el registro o conjunto de registros con los que realmente se quiere trabajar durante la sesión de trabajo.

Los filtros permiten buscar aquellos registros de la lista que cumplen unas condiciones específicas.

En los siguientes apartados de este capítulo se explica:

- El concepto de las listas y su funcionalidad: grupos de remonte, validación y listas de selección de registros, etc.
- Creación de listas

Listas: definición y funcionalidad

Las listas son un tipo especial de Meta4Objects que implementan las siguientes funcionalidades:

- Listados de selección y validación de registros en grupos de remonte
- Restringen los datos que se van a recuperar cuando se carga un Meta4Object a las columnas que conforman la clave primaria de la tabla de la que extrae datos el nodo raíz del Meta4Object

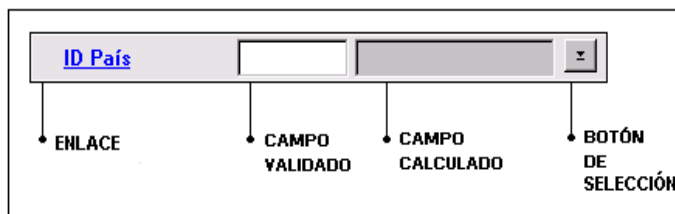
El principal propósito de las listas es reducir el volumen de datos y los recursos de memoria necesarios para trabajar con los Meta4Objects.

Listas y grupos de remonte

Uno de los objetos que se encuentran en las presentaciones de las aplicaciones Meta4 son los grupos de remonte.

Un grupo de remonte está formado por un enlace, un campo validado, un campo calculado y un botón de selección.

Figura 60. Ejemplo grupo de remonte.



EJEMPLO

En la imagen que se muestra a continuación, la carpeta activa contiene cuatro grupos de remonte para los elementos ID Titulación, ID Centro Formación, ID Especialidad y Tipo formación.

Figura 61. Meta4Object Historial del RH

Historial del RH - Multisociedad

Archivo Edición Ver Ayuda

Menú Imprimir Guardar Fechas Sociedades Dividir Recargar Actualizar

Historial del RH

Recurso humano

- Experiencia profesional
- Afiliación asociada
- Historial académico
- Licencias y certificados

Recurso humano (1/1) M10081

Experiencia profesional Afiliación asociada Historial académico Licencias y certificados

Inicio 01/09/1974

Fecha prevista/fin 01/06/1978

ID Titulación

ID Centro formación COL_2 COLEGIO SAGRADOS CORA

ID Especialidad

Tipo formación 0099 B.U.P.

Comentario

En las presentaciones, los grupos de remonte se aplican en la presentación de los elementos que recogen el valor de los campos que hayan sido definidos como claves externas en la fase de diseño del modelo de datos.

En el ejemplo anterior, los cuatro campos que tienen un grupo de remonte asociado son claves externas que validan su valor con las columnas que son clave primaria de otras tablas de la base de datos.

Uno de los conceptos más importantes en el diseño de bases de datos relacionales es la normalización. La normalización distribuye la información relativa a una misma entidad en un número variable de tablas, con el propósito de reducir la redundancia al mínimo imprescindible, sin que se produzcan pérdidas de información.

Por ejemplo, la información sobre los empleados se encuentra distribuida en distintas tablas: Persona recoge los datos personales de los empleados (nombre, fecha y lugar de nacimiento, sexo, etc.), RH en periodo contiene todas sus fases de alta, Direcciones, los domicilios, etc.

Para que el sistema sepa a qué entidad corresponden los registros de las tablas, cada entidad se identifica de manera unívoca mediante un atributo o conjunto de atributos que toman valores únicos: la clave primaria.

En el caso de los empleados, cada empleado tendrá un código identificativo, **ID_Persona**, que se utilizará en todas las tablas que contengan información sobre los empleados. Así, la tabla Direcciones contiene una columna **ID_Persona** en las que se indica a qué persona corresponde el domicilio que recoge cada registro.

Como la columna **ID_Persona** de la tabla Direcciones hace referencia a un valor que debe haber sido registrado con anterioridad en la tabla de personas, se dice que esta columna es una clave externa definida sobre la clave primaria de la tabla Persona.

Claves externas y presentaciones

En la tabla Direcciones, es decir, en la tabla que contiene la clave externa, únicamente se guarda el código identificativo de las personas y la información de sus domicilios.

Si se quiere ver el nombre de la persona, o cualquier otro dato, es necesario leerlo de la tabla Persona.

Cuando se diseña una presentación para un Meta4Object que incluya un elemento que se corresponda con una columna de la base de datos que ha sido definido como clave externa, es necesario ofrecer al usuario las siguientes posibilidades:

- Leer los valores significativos de la entidad a la que se está haciendo referencia, por ejemplo el nombre y apellidos de la persona referenciada mediante su código identificativo **ID_Persona**.
- Listar los valores de la tabla Persona para poder seleccionar a una persona mediante su nombre u otra información significativa.
- Validar los valores que se indican en **ID_Persona**, para ver si se corresponden con el código identificativo de una persona registrada con anterioridad en Persona.

La presentación debe permitir interactuar con los datos de la tabla que contiene la clave externa y con los de la tabla que contiene la clave primaria sobre la que se valida la clave externa.

Esta interacción se consigue mediante los grupos de remonte y mediante las listas definidas sobre las tablas que contienen las claves referenciadas.

Mediante los grupos de remonte, es posible:

- Proponer un valor para el elemento, escribiendo el nuevo valor en el espacio reservado al campo validado

El valor que se indique siempre deberá coincidir con uno de los valores registrados en la tabla que contiene la información de la entidad referenciada.

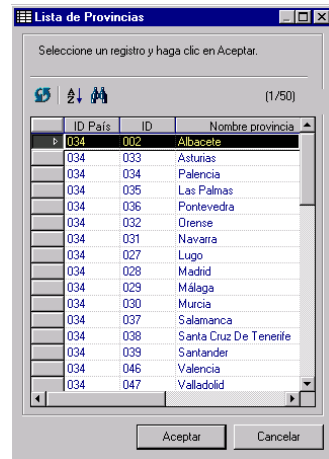
En el caso de que se proponga un valor que no exista en esa tabla, el sistema presentará un listado con todos los valores que se pueden seleccionar.



- Listar el conjunto de valores posibles para ese campo, mediante el botón de selección

Al hacer clic sobre este botón, el sistema mostrará en pantalla una relación con todos los registros disponibles en la tabla que contiene los valores que deben ser referenciados.

Figura 62. Lista de selección



Las columnas que se muestran en la lista se fijan en el momento de crear la tabla desde la herramienta *Diseñador de modelos de datos*, mediante la propiedad **Posición en lista**, disponible a nivel de columna.

- Si no se activa esta casilla de verificación, los valores de la columna no se mostrarán en la lista
- Si se activa esta casilla de verificación, la columna se mostrará en la lista y ocupará una posición igual al valor que se haya indicado

EJEMPLO

Si se selecciona la casilla de verificación **Posición en lista** y se le asigna el valor 1, ocupará la primera columna de la lista comenzando por la izquierda. Si se le asigna el valor 2 ocupará la segunda columna, y así sucesivamente.

- Añadir un nuevo registro en la tabla con la que se validan los datos introducidos en la columna (posteriormente se podrá utilizar este nuevo valor para rellenar el campo del grupo de remonte)

EJEMPLO

En la tabla que recoge las direcciones de los empleados, se quiere añadir una dirección en una ciudad que no existe en la tabla que recoge todas las ciudades.

Desde el enlace del grupo de remonte se podrá acceder al formulario asociado a la tabla **Ciudades** y crear un nuevo registro para la nueva ciudad.

A continuación, podrá utilizarse el nuevo valor para rellenar el campo que recoge la ciudad en el formulario **Direcciones**.

El enlace, situado a la izquierda del grupo de remonte y escrito en letra azul y subrayado, permite abrir en modo inserción el Meta4Object asociado a la tabla de validación. También permite editar el registro al que se hace referencia en el registro activo de la presentación que contiene el grupo de remonte.

Figura 63. Los enlaces se resaltan en azul

Grupos de remonte

Cuando se está añadiendo o modificando un registro a través de una presentación que contenga un grupo de remonte, es posible:

- Escribir un valor en el campo validado

Al escribir un valor en el campo validado (única sección editable del grupo de remonte), el programa comprobará si ese valor existe en la tabla con la que se valida el valor introducido en el grupo de remonte. Esto se denomina *proceso de validación de claves externas*.

En el caso de que el valor exista, en el campo calculado del grupo de remonte se mostrará el nombre descriptivo de la entidad referenciada por el código propuesto.

Si el valor propuesto no existe, se presentará un listado desde el cual se podrá seleccionar uno de los valores disponibles en la tabla de validación. Esta opción se denomina *proceso de listado*.

- Listar los valores que se pueden seleccionar

Ante la dificultad de conocer el valor que identifica a la entidad a la que se quiere hacer referencia en el grupo de remonte, el usuario podrá obtener una lista con todos los valores a los que puede hacer referencia en el grupo de remonte. Esta lista se obtendrá haciendo clic en el botón de selección.

- Crear un nuevo valor al que poder hacer referencia

En el caso de que no exista el valor al que se quiere hacer referencia en la tabla de validación, es posible crear un nuevo registro para este valor en la tabla de validación. Para ello, bastará con hacer clic en el enlace.

El enlace cargará la presentación correspondiente a la tabla de validación. Esta acción se denomina *proceso de remonte*.

Las tres funciones anteriores (validación, listados y remontes) se implementan mediante llamadas de nivel 2 a métodos de un Meta4Object de tipo lista que recupera los valores necesarios en cada uno de estos procesos.



Se produce una llamada de nivel 2 entre dos Meta4Objects cuando, desde la regla de un elemento de tipo método o concepto de uno de ellos se hace referencia a un elemento de otro Meta4Object, cualquiera que sea el tipo de este último: método, concepto, campo o propiedad.

La estructura de un Meta4Object de tipo lista y los métodos a los que se hacen las llamadas de nivel 2 desde los grupos de remonte se explican en un apartado posterior de este mismo capítulo.

Filtros de carga de registros en nodos raíz

Además de las funciones de validación y listado disponibles en los grupos de remonte, las listas implementan la posibilidad de reducir la cantidad de datos cargados en memoria al ejecutar un Meta4Object.

Todos los Meta4Objects que recuperan datos de la base de datos cuentan con un nodo raíz que instancia una estructura de nodo.



Las estructura de nodo tienen una sentencia SQL asociada que determina qué registros se deben recuperar cuando se solicita su ejecución.

La estructura de nodo determina qué datos se quieren recuperar de la base de datos y qué procesamiento se quiere hacer con ellos.

En el momento en el que se solicita la carga o ejecución de un Meta4Object, el sistema lanza la sentencia asociada a la estructura de nodo y recupera de la base de datos las columnas y los registros que se indican en esta sentencia.

El sistema cargará en memoria los valores de todas las columnas que se hayan indicado en la definición de la estructura de nodo instanciada por el nodo raíz del Meta4Object.

Dependiendo de cómo se haya configurado el Meta4Object en la fase de diseño, es posible que la carga de datos se propague hacia los niveles inferiores del Meta4Object (es decir, se podrán cargar los datos indicados en la definición de las estructura de nodos instanciados por los nodos hijos del nodo raíz).

En ocasiones, esto puede obligar a cargar un gran volumen de datos en memoria que no van a ser utilizados.

EJEMPLO

Si se quiere consultar la situación de un empleado en concreto, basta con recuperar los datos de ese empleado, sin que sea necesario recuperar los de todos los demás.

La utilización de listas permite cargar en memoria un volumen menor de datos. En lugar de cargar todas las columnas indicadas en la definición de la estructura de nodo instanciada por el nodo raíz, la lista sólo recuperará:

- Los valores de aquellas columnas que formen parte de la clave primaria de la tabla de la que extrae datos esta estructura de nodo
- Los valores de las columnas que se deban mostrar en las listas (es decir, los elementos que se corresponden con las columnas de la tabla que tengan activada la casilla de verificación **Posición en lista**)
-

Creación de listas

En Peoplenet existe una metodología para facilitar el desarrollo de componentes funcionales (development framework). El framework incorpora una serie de herramientas y plantillas que permiten resolver de forma común el desarrollo de las funcionalidades más usuales que se implementan en el producto: mantenimiento de datos, filtros, listas, formularios de información etcétera.

Para las listas, la plantilla consta de un Meta4Object y una presentación, y existen diferentes formas de acceder a las herramientas de creación y modificación de listas:

Desde opción de menú Herramientas de desarrollo | Desarrollar con el development framework

- Usar la opción de menú Crear lista y mantenimiento para crear una nueva lista
- Usar la opción de menú Modificar lista y mantenimiento para modificar una lista

Desde las herramientas de desarrollo tradicional:

- Diseñador de Meta4Objects:
 - Al crear un nuevo Meta4Object y seleccionar el tipo lista y mantenimiento
 - al abrir un Meta4Object de tipo lista y mantenimiento
- Diseñador de presentaciones:
 - al crear una nueva presentación y seleccionar el tipo Mantenimiento.
 - al abrir una presentación de tipo mantenimiento y utilizar la opción de menú Herramientas | Asistente del development framework.
- Diseñador del modelo de datos:
 - desde la opción de menú Herramientas | Asistente del development framework



Como puede acceder a las herramientas del framework desde varias opciones para crear o modificar Meta4Object, tenga en cuenta que en el caso de que lo que desee es hacer una modificación de un Meta4Object aparecerá una ventana de aviso que le advierte de la posibilidad de modificar dicho Meta4Object desde la herramienta del framework o desde el Diseñador de Meta4Objects. Es importante que recuerde que el impacto si usa el framework será que los cambios que efectúe sobre la estructura del Meta4Object se propagarán a la presentación asociada, es decir si elimina o añade algún elemento este se eliminará o se añadirá cuando la herramienta acceda al

diseñador de presentaciones. De manera inversa si usa el diseñador de Meta4Objects para la modificación los cambios que efectuó sobre el Meta4Object no se propagarán a la presentación asociada, así cuando abra el Meta4Object desde el diseñador de presentaciones no se verán reflejados los cambios realizados.

Para ampliar información sobre el development framework y herramientas utilice el manual del development framework y la ayuda en línea definida en cada una de las herramientas.

Introducción

La tecnología Meta4 implementa la lógica de la aplicación mediante Meta4Objects. Los Meta4Objects encapsulan información que extraen de la base de datos e instrucciones de procesamiento.

Las instrucciones de procesamiento se codifican en los elementos de tipo método y concepto mediante reglas escritas en el lenguaje LN4, llamadas a métodos de librerías de enlace dinámico (.dll) o a métodos desarrollados en lenguaje C++.

LN4 es un lenguaje de programación desarrollado por Meta4[®], que ofrece toda la capacidad expresiva de un lenguaje de programación de alto nivel.

Una regla es código LN4 que constituye una unidad de ejecución. Si bien las reglas pueden considerarse como programas independientes, desde una regla se puede solicitar la ejecución de las reglas de otros métodos y conceptos, formando así programas complejos.



El sistema de desarrollo propuesto por Meta4[®], basado en reglas y en el lenguaje LN4, ofrecen al desarrollador la modularidad y capacidad expresiva necesarias para poder desarrollar nuevas funcionalidades de forma rápida y sencilla, reutilizando el código creado para otros métodos y conceptos.

En los siguientes apartados se explica:

- La diferencia entre métodos y conceptos
- En qué consiste una regla y qué propiedades las describen: periodos de validez, código identificativo, etc.
- Métodos y conceptos con argumentos

- Diseño de reglas y lenguaje LN4
- Dependencias entre métodos y conceptos, y llamadas de nivel 2
- Ejecución de reglas con y sin argumentos
- El concepto de metarregla y la ejecución condicional de reglas mediante dimensiones

Métodos y conceptos

Un Meta4Object está formado por un único nodo, o por dos o más nodos conectados. Cada nodo de un Meta4Object crea una instancia de estructura de nodo que determina:

- Los datos que se quieren extraer de la base de datos
- Las instrucciones para procesar estos datos

Los datos de la base de datos se recuperan en los elementos de tipo campo. Las instrucciones de procesamiento se indican mediante reglas en los elementos de tipo método y concepto.

Una regla puede contener llamadas a métodos de DLL externas, a métodos C++ o una serie de instrucciones procesables escritas en lenguaje LN4. Las reglas implementan la lógica de la aplicación.

Tanto los métodos como los conceptos pueden utilizar reglas. Este tipo de elementos se denominan *elementos con ejecución*.

La diferencia entre los métodos y los conceptos consiste en que los conceptos recogen el valor obtenido como resultado de la ejecución, y los métodos no. Esto no significa que los métodos no devuelvan un valor. La ejecución de un método puede devolver un valor que puede ser asignado a otros elementos o a variables locales, pero este valor nunca será asignado al propio elemento, como sucede con los conceptos.

EJEMPLO

La siguiente regla suma el valor de dos elementos de tipo campo, lo divide entre dos y devuelve el resultado.

```
Resultado=nElementoPrimero + nElementoSegundo
Resultado=Resultado / 2
Return(Resultado)
```

Si se ejecuta esta regla para un elemento de tipo concepto, el valor devuelto por la función Return() se asignaría al concepto. A partir de este momento, siempre que se solicite el valor de este concepto se devolverá el valor devuelto por la variable Return() y no será necesario ejecutar de nuevo el código de la regla.

Si el concepto está asociado a un campo de escritura en la base de datos, cuando se llame a la orden de grabar, se grabará el valor calculado para el concepto en ese campo.

Si se ejecutase esta misma regla para un elemento de tipo método, el valor devuelto por `Return( )` no se asignaría al método, pero podría asignarse a otro elemento si la llamada al método se hubiese hecho de la siguiente forma:

```
IdElemento=ElementoMétodo( )
```

La regla de un método se ejecutará siempre que se llame al método y la fecha de ejecución seleccionada al iniciar la aplicación esté comprendida en el periodo de validez de la regla.

Reglas

Los elementos de tipo método y concepto pueden estar asociados a reglas que se ejecutan siempre que se hace una llamada al elemento.

Un *elemento con ejecución* puede tener un número variable de reglas. Cada regla cuenta con un código identificativo único, unos periodos de validez y un orden de ejecución.



El término *elementos con ejecución* hace referencia a los elementos de tipo método y concepto.

La siguiente tabla recoge las propiedades de las reglas:

Propiedad	Descripción
ID Regla	Código identificativo de la regla. Todas las reglas de un mismo elemento tendrán un código identificativo único, no podrá haber dos o más reglas del mismo método o concepto que tengan idéntico código identificativo. Sin embargo, sí podrá haber dos elementos con ejecución para los que haya una regla con idéntico código identificativo. Se utilizará como código identificativo una cadena alfanumérica con un máximo de 30 caracteres.
Orden ejecución	Orden de ejecución de la regla. Se utilizará cuando haya dos o más reglas para un mismo concepto con periodos de validez solapados en el proceso de creación de metarreglas. En este caso, el valor que se haya indicado en la propiedad Orden ejecución determinará el orden en que se deben ejecutar las reglas. Para un mismo elemento no puede haber reglas con periodos que se solapen e idéntico valor en la propiedad Orden ejecución. Este proceso se explica en un apartado posterior y lo gestiona la aplicación de forma automática.
Metarregla	Indicador de metarregla. Las metarreglas son reglas especiales que contienen llamadas a otras reglas. Cuando se solicita la ejecución de un elemento de tipo método o concepto, el sistema siempre ejecutará la metarregla que sea válida en el instante de tiempo en el que se solicita la ejecución.
Metarregla del sistema	Indica si la regla es una metarregla generada por el sistema. El sistema generará una metarregla automáticamente siempre que se crea la primera regla para un método o concepto y cuando se trabaja con dimensiones.
Inicio	Fecha de inicio del periodo de validez de la regla. El periodo de validez determina el periodo durante el cual se puede ejecutar la regla.

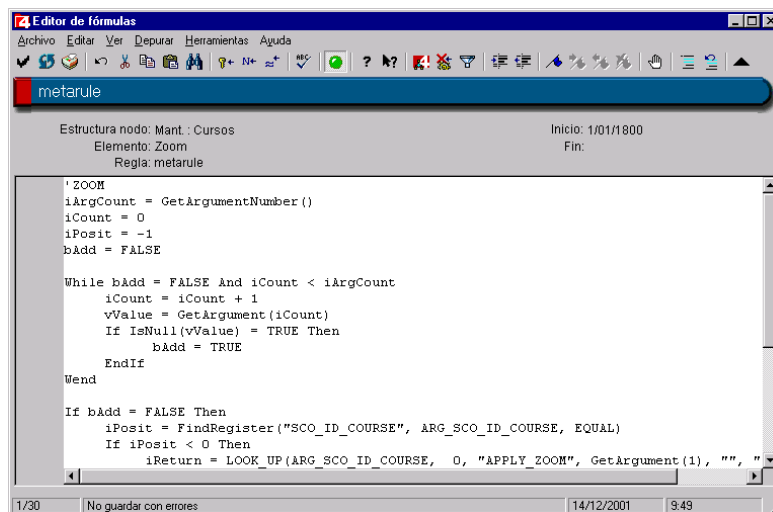
Propiedad	Descripción
Fin	Fecha de fin del periodo de validez de la regla.
Tipo de código	Tipo de código que va a ejecutar la regla. Una regla puede ejecutar código LN4, C++ o métodos de librerías de enlace dinámico (archivos .DLL)
ID Prioridad	Prioridad con la que se va a asignar valor al elemento como resultado de la ejecución de la regla. La prioridad puede ser un valor comprendido entre 0 y 3; siendo la prioridad 0 la máxima y 3 la mínima. Únicamente se utiliza en los elementos de tipo concepto.

Una regla puede tener distintos periodos de validez a lo largo del tiempo. Se denomina *norma* a cada uno de estos periodos. Los periodos de validez de las normas de una misma regla no se pueden solapar. Es decir, para una misma regla no podrá haber dos normas con periodos de validez solapados total o parcialmente.

Además de sus propiedades, la definición de una regla incluirá el código que debe procesar. Las distintas normas de una misma regla pueden ejecutar códigos diferentes.

La herramienta **Diseñador de Meta4Objects** incorpora un editor de código LN4 desde el que se pueden crear las reglas.

Figura 64. Diseñador de Meta4Objects: Editor de fórmulas



Elementos, reglas y normas

Un elemento de tipo método o concepto puede estar asociado a una o más reglas en el mismo instante de tiempo, o en instantes de tiempo diferentes. Cada regla tendrá uno o más periodos de validez, denominados normas.

Los periodos de validez de las normas de una regla pueden solaparse, con los periodos de validez de las normas de otras reglas del mismo elemento. Pero los periodos de validez de las normas de una misma regla no pueden solaparse.

Cuando se graba una nueva norma para una regla desde el Diseñador de Meta4Objects con un periodo de validez que se solapa con el de una norma ya existente para esa misma regla, el sistema hace los cambios necesarios para evitar que sus periodos de validez se solapen.



En los siguientes ejemplos, excepto en el último, se considera el comportamiento en el caso de que sólo exista una norma, previa a la creación de la nueva norma, con una única metarregla.

EJEMPLO

La interfaz del programa **Diseñador de Meta4Objects**, desde el que se crean las reglas de los elementos de tipo método y concepto, permite grabar normas para una misma regla con periodos de validez solapados.

De esta forma, si ya existe una norma con un periodo de validez que comprende del 1 de enero de 2000 al 31 de enero de ese mismo año, se podría grabar una nueva norma con fecha de inicio 15 de enero de 2000 y fecha de fin 28 de febrero de ese mismo año.

El programa no daría ningún error, pero haría lo siguiente:

- Grabaría una nueva norma con las fechas propuestas.
- Cambiaría la fecha de fin de la norma ya existente al 14 de enero de 2000.

Estos cambios se hacen de forma transparente para el usuario, aunque se reflejan de forma inmediata en la interfaz de la aplicación.

- Si el periodo de validez de la nueva norma está incluido totalmente en el periodo de validez de una norma ya existente:
 - Se corregirá la fecha de fin de la norma ya existente, cambiándola por la fecha inmediatamente anterior a la fecha de inicio de la nueva norma
 - Se creará una segunda norma con un periodo de validez indicado por el usuario

- Si el periodo de validez de la nueva norma está comprendido parcialmente dentro del periodo de validez de una norma ya existente y tiene una fecha de inicio anterior a la fecha de inicio de la norma con la que se solapa:
 - Se creará una nueva norma con el periodo de validez indicado
 - La fecha de inicio de la norma ya existente se cambiará por el día inmediatamente posterior a la fecha de fin de la nueva norma
- Si el periodo de validez de la nueva norma está comprendido parcialmente dentro del periodo de validez de una norma ya existente y tiene una fecha de fin posterior a la fecha de fin de la norma con la que se solapa:
 - Se creará una nueva norma con el periodo de validez indicado
 - La fecha de fin de la norma ya existente se cambiará por el día inmediatamente anterior a la fecha de inicio de la nueva norma
- Si el periodo de validez de la nueva norma incluye totalmente al periodo de validez de una norma existente:
 - Se creará una nueva norma con el periodo de validez indicado
 - Se eliminará la norma existente
- En el caso de que el periodo de validez de la nueva norma se solape con más de una norma, el sistema se comportará de la siguiente forma:
 - Graba la nueva norma con el periodo de validez que se haya indicado
 - Elimina las normas ya existentes cuyos periodos de validez estuviesen totalmente incluidos dentro del periodo de validez de la nueva norma
 - Si existe una norma con un periodo de validez que se solapa parcialmente con el de la nueva norma y con una fecha de inicio anterior a la de la nueva norma, el sistema actualizará la norma existente pasando la fecha de fin al día inmediatamente anterior a la fecha de inicio de la nueva norma
 - Si existe una norma con un periodo de validez que se solapa parcialmente con el de la nueva norma y con una fecha de fin posterior a la de la nueva norma, el sistema actualiza la norma existente pasando la fecha de inicio al día inmediatamente posterior a la fecha de fin de la nueva norma

EJEMPLO

Para un elemento A existen dos reglas: Regla1 y Regla2. Regla1 es la metarregla y Regla2 es la metarregla normal.

Para la Regla2 hay dos normas. La primera tiene un periodo de validez desde el 1 de enero al 30 de junio de 2000; la segunda, del 1 de agosto al 30 de septiembre de 2000.

Regla	Norma	Inicio	Fin
Regla2	1	01-01-2000	30-06-2000
Regla2	2	01-08-2000	30-09-2000

Se crea una nueva norma desde la interfaz de la aplicación con un periodo de validez comprendido entre el 15 de junio de 2000 y el 15 de septiembre.

En el momento en el que se grabe la nueva norma, se realizarán los siguientes cambios:

- La fecha de fin de la primera norma se cambiará por el 14 de junio de 2000.
- Se agregará una nueva norma con un periodo de validez comprendido entre el 15 de junio y el 15 de septiembre.
- La fecha de inicio de la segunda norma existente pasará al día inmediatamente posterior a la fecha de fin de la nueva norma, el 16 de septiembre de 2000, y mantendrá su fecha de fin.

Finalmente, se tendrán las siguientes normas para la regla:

Regla	Norma	Inicio	Fin
Regla2	1	01-01-2000	14-06-2000
Regla2	2	15-06-2000	15-09-2000
Regla2	3	16-09-2000	30-09-2000

Como se puede apreciar, los periodos de validez de las normas no se solapan realmente, a pesar de que la interfaz permita crear normas con periodos de validez solapados.

En un mismo momento podrá haber más de una regla válida para un mismo elemento. Pero para cada una de estas reglas sólo una de sus normas podrá ser válida en el mismo instante.

Reglas con periodos de validez solapados

Los periodos de validez de las distintas reglas de un mismo elemento se pueden solapar. Un mismo concepto o método puede tener más de una regla válida en el mismo momento. En este caso, el intérprete determinará qué regla tiene que ejecutar, según la sintaxis que se haya utilizado para hacer la llamada al método o al concepto, y de lo que se indique en la metarregla:

Si se ha llamado al método o concepto haciendo clic en el botón asociado en la presentación del Meta4Object, o se les ha llamado desde una fórmula LN4 escribiendo el nombre del concepto o del método seguido de sus argumentos entre paréntesis, el intérprete ejecutará aquella regla que se haya marcado como metarregla, siempre que su periodo de validez incluya a la fecha de aplicación.

La metarregla crea el programa de forma automática en el momento de definir la primera regla de un elemento. Podrá tener distintos periodos de validez o normas. Los periodos de validez de las distintas normas de una metarregla no se podrán solapar.

La metarregla determina qué reglas se deben ejecutar y en qué orden. Puede definir incluso ejecuciones condicionales de las reglas válidas durante el periodo de ejecución. La ejecución condicional se define mediante las dimensiones, que se explican más adelante.

La tecnología Meta4 incorpora un editor gráfico para diseñar las condiciones que se deben cumplir para que se pueda ejecutar una regla.

EJEMPLO

La fórmula utilizada para calcular un concepto de nómina cambia según el proyecto en el que haya trabajado el empleado durante el periodo que se paga.

Los proyectos se clasifican dependiendo de su complejidad en dos grupos diferentes. De esta forma, el concepto tendrá tres reglas válidas para un mismo instante. Para determinar cuál de estas reglas se tiene que ejecutar, el sistema creará una metarregla que establecerá qué regla se tiene que ejecutar en cada caso. La metarregla contendría un código similar al siguiente:

```
If UltimoProyecto = A Then
  Elemento( )..Regla1
End If
If UltimoProyecto = B Then
  Elemento( )..Regla2
End If
```

Si se ha llamado a una regla específica del método o del concepto, el intérprete ejecutará esa regla, independientemente de que haya otras reglas válidas durante ese periodo de tiempo, y de que la regla cuya ejecución se solicita se haya marcado como metarregla.

Desde LN4 se podrá llamar a una regla en concreto mediante la sintaxis:

```
NombreElemento( )..NombreRegla
```

Metarreglas

Las metarreglas son un tipo especial de reglas que genera el programa de forma automática, y que se ejecutan siempre que se hace una llamada al método o al concepto desde:

- La interfaz de la aplicación
- Código LN4, con la sintaxis nombre del elemento seguido de sus argumentos; es decir, sin especificar el código identificativo de una regla específica

Un elemento sólo puede tener una metarregla. A su vez, esta metarregla puede tener dos o más periodos de validez a los que se llaman normas. Las metarreglas se comportan igual que las reglas, por lo que los periodos de validez de sus normas no se pueden solapar.



En el caso de las metarreglas, las normas se generan de forma automática. El usuario no puede crear nuevas normas para una regla de tipo metarregla.

El código ejecutable en cada periodo de tiempo o norma puede cambiar. Es decir, las normas de una misma regla pueden ejecutar códigos diferentes.

Las metarreglas las genera el sistema de forma automática. Consisten en código LN4 que indica qué reglas del elemento se deben ejecutar y en qué orden.

En el caso de que se hayan diseñado dimensiones, es decir, condiciones para la ejecución de las reglas, la metarregla se encargará de evaluar las condiciones y decidir si las reglas afectadas por ellas se deben ejecutar o no.



En un apartado posterior se explica el concepto de *dimensión*.

Argumentos

Los elementos pueden recibir argumentos. Los argumentos actúan como variables cuyo valor se desconoce en el momento de crear la regla y que será determinado en tiempo de ejecución.

Por ejemplo, los argumentos se pueden utilizar cuando se quiera utilizar en el código de la regla un valor solicitado al usuario en tiempo de ejecución, el valor de un elemento o el de una variable a la que se le asigna valor en otro método.



Si se pasan como argumentos a un elemento con ejecución los nombres de otros elementos, éstos deberán escribirse precedidos por el código identificativo del nodo en el que están incluidos, salvo en el caso de que formen parte del mismo nodo que el método o concepto desde el cual se hace la llamada.

Se puede pasar como valor de un argumento:

- Un valor constante, por ejemplo una cadena de caracteres o un número
- El valor de otro elemento del mismo nodo o de otros nodos del mismo Meta4Object o de un Meta4Object diferente, siempre que el elemento recoja valor (sea de tipo campo, propiedad o concepto)
- Variables

Al definir un elemento que vaya a utilizar valores que se le pasen como argumentos, en primer lugar se tendrán que declarar cuales van a ser sus argumentos.

En el código de la regla se utilizará el nombre de estos argumentos como si se tratase de variables.

EJEMPLO

Se define el argumento FechaNacimiento para el concepto CalculaEdad.

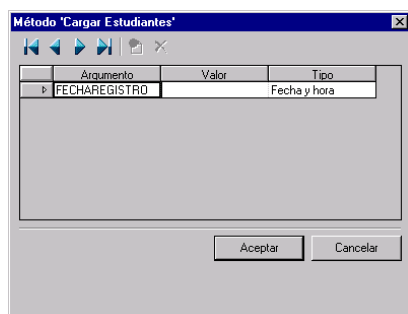
En la regla del concepto se utilizará el nombre del argumento como si se tratase de una variable.

```
iDias=Today( ) - FechaNacimiento  
Return(iDias)
```

Cuando se solicite la ejecución del elemento será necesario indicar qué valores se le van a pasar como argumentos. Esto podrá hacerse de dos formas:

- Si se solicita la ejecución del elemento desde la interfaz de la aplicación, al hacer clic en el botón que se le asocia en la presentación del Meta4Object, el sistema muestra una ventana en la que se pueden indicar los valores que se quieren pasar como argumentos.

Figura 65. Ventana de petición argumentos



- Si se solicita la ejecución del elemento desde el código LN4 de otro método o concepto, los valores que se pasen a los argumentos se escribirán entre paréntesis, detrás del nombre del elemento.

Si el elemento cuenta con más de un argumento, se deberá pasar valor a todos ellos. Los valores se separarán mediante comas <,>.

EJEMPLO

El método `CalculaEdad( )` cuenta con dos argumentos: `dFechaNacimiento` y `dFechaActual`.

La regla del método realiza la siguiente operación:

```
dDias=dFechaActual - dFechaNacimiento
Return(dDias)
```

Para solicitar la ejecución del método `CalculaEdad( )` desde la regla LN4 de otro elemento, se tendría que utilizar la siguiente sintaxis:

```
NombreElemento(arg1, ..., argn)
```

Si, por ejemplo, se quieren asignar como argumentos los valores 22-07-1973 para la fecha de nacimiento, y 01-04-2000 como fecha actual, la llamada al método se hará de la siguiente forma:

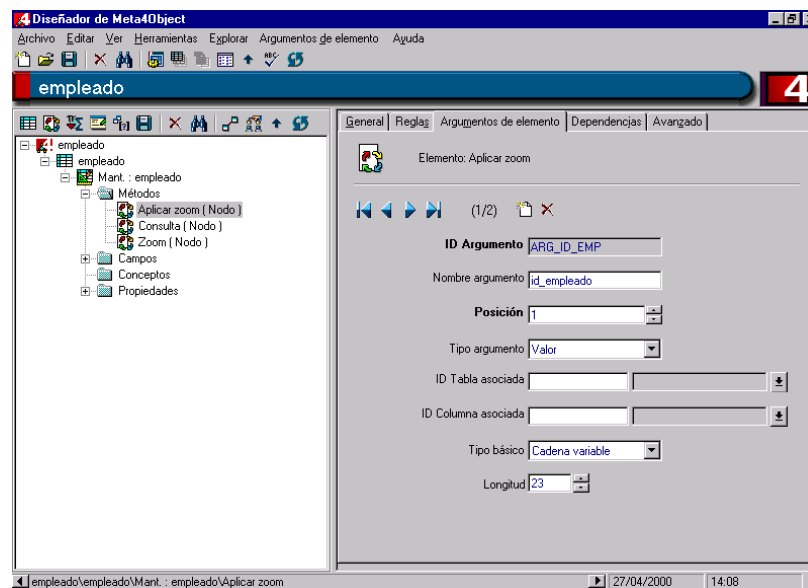
```
CalculaEdad({1973-07-22}, {2000-04-01})
```

Propiedades de los argumentos

Todos los argumentos cuentan con una serie de propiedades: tipo de dato, orden, tipo de argumento, etc.

El valor de estas propiedades se indica en el momento de diseñar la regla:

Figura 66. Definición de argumentos



La siguiente tabla describe cada una de las propiedades:

Propiedad	Descripción
ID Argumento	Código identificativo del argumento. Los argumentos de un mismo método o concepto deben tener un código identificativo único. Es decir, no puede haber dos argumentos de un mismo elemento que compartan el mismo código identificativo. Acepta un máximo de 30 caracteres.
Nombre argumento	Nombre descriptivo del argumento. Acepta un máximo de 50 caracteres.
Posición	Orden que ocupará el argumento en la lista de argumentos de entrada. Es importante saber el orden del argumento en el momento de hacer llamadas al elemento desde el código de reglas LN4, ya que el intérprete de reglas asignará los valores que se indiquen en la llamada a un parámetro o a otro, tomando como base el orden en el que se escriben.
ID Tabla asociada	Grupo de remonte para asociar una tabla al argumento.

Propiedad	Descripción
ID Columna asociada	Grupo de remonte para asociar una columna al argumento.
Tipo argumento	Tipo del argumento. Los argumentos pueden ser de dos tipos: Valor (Input) y Referencia (Reference) Si se pasa un argumento por Valor y se hace algún cambio en el valor del argumento en la regla del método o concepto que lo recibe como argumento, estos cambios no serán visibles una vez finalice la ejecución de la regla en la que se modifica su valor. El argumento conservará el valor que tuviese antes de ser pasado como argumento. Si se asigna el argumento por Referencia, se mantendrán los cambios que se hagan sobre el valor que se ha asignado inicialmente.
Tipo básico	Tipo de dato del argumento: cadena variable, fecha, número, etc.
Longitud	Longitud del tipo de dato del argumento.

En los siguientes apartados se explican algunas de las propiedades anteriores:

Tipo de argumento

Los argumentos pueden ser de dos tipos: *Valor* y *Referencia*.

Cuando el argumento toma el tipo Valor, la regla del elemento que lo recibe puede operar con el valor de ese argumento, pero no puede cambiarlo.

Si toma el tipo Referencia, la regla del elemento que lo recibe puede operar con el valor del argumento y modificarlo.

EJEMPLO

El método A contiene una llamada al método B. El método B recibe como argumento un valor numérico sobre el que realiza una serie de operaciones. En la definición del método B se ha utilizado el código identificativo «Arg 1» para nombrar a este argumento y para hacerle referencia en el código de las reglas.

El código de la regla A es el siguiente:

```
iValorTotal=10000
B(iValorTotal)
MessageBox("La regla A devuelve el valor "+iValorTotal)
```

El código de la regla B es el siguiente:

```
Arg_1=Arg_1 * 2  
MessageBox("La regla B devuelve el valor "+Arg_1)
```

Se describirá el comportamiento de los métodos en el caso de que el argumento «Arg 1» sea de tipo *Input* o *Reference*.

Si el argumento «Arg 1» es de tipo *Input*, es decir, se le asigna el tipo Valor, el programa se comportaría de la siguiente forma:

1. Comienza a ejecutar la regla del método A.
2. Asigna a la variable iValorTotal el valor 10.000.
3. Desde la regla de A, se solicita la ejecución del método B, pasándole como argumento el valor que recoge la variable iValorTotal, es decir, 10.000.
4. El intérprete de reglas comienza a ejecutar la regla del método B. Todas las referencias que aparezcan en esta regla al argumento «Arg 1» se sustituirán por el valor indicado en la llamada al método: 10.000.
5. La regla del método B multiplica por 2 el valor de «Arg 1» y asigna el resultado a la variable «Arg 1», es decir, a la misma variable que ha recogido el valor que se ha pasado como argumento.
6. A continuación se muestra un mensaje con el valor que recoge «Arg 1». El mensaje mostrará el texto "La regla B devuelve el valor 20.000".
7. Cuando finaliza la ejecución del método B, el flujo de ejecución continúa en el método A, en el punto en el que se quedó.
8. La última instrucción del método A muestra un mensaje en pantalla con el valor que recoge la variable iValorTotal. Como esta variable se ha pasado como valor a un argumento de tipo Input, su valor no se habrá visto modificado, y seguirá siendo el que tenía antes de que se ejecutase el método B: 10.000. El mensaje mostrará el texto: "La regla A devuelve el valor 10.000".
9. Sin embargo, si se hubiese pasado el valor de la variable iValorTotal a un argumento de tipo Reference, el cambio que se ha hecho en el valor del argumento en la regla del método B se hubiese reflejado en el resto de referencias que se hiciesen a la variable iValorTotal. Es decir, la última instrucción del método B devolvería el mensaje "La regla A devuelve el valor 20.000".

Es importante resaltar que el intérprete es capaz de reconocer que la variable iValorTotal se corresponde con el argumento «Arg 1».

Posición

Es el orden que ocupa el argumento en la lista de argumentos del método o del concepto.

En la definición de un método o concepto cuya regla vaya a aceptar argumentos, es necesario indicar un orden de entrada para cada uno de ellos.

Cuando se haga la llamada al método o concepto, se tendrá que indicar con qué valores se quieren sustituir las referencias que aparecen en el código LN4 a los argumentos.

Para que el intérprete de reglas pueda saber qué valor debe sustituir a cada argumento, los valores se deben indicar en el mismo orden indicado para los argumentos en el momento en el que se han definido.

EJEMPLO

El método `CalculaDiasAlta()` recibe dos argumentos, la fecha del sistema y la fecha de alta. Se llama a estos argumentos `FechaSistema` y `FechaAlta`.

En el momento de declarar los argumentos del elemento, se les ha asignado respectivamente el orden 1 y 2. De esta forma, la fecha del sistema se debe pasar en primer lugar y la de alta en segundo lugar.

Cuando se haga una llamada al elemento desde el código de una regla LN4, se tendrán que escribir los valores que se pasen a los argumentos en este mismo orden.

Por ejemplo, si el día de hoy es el 22-07-2001, y la fecha de alta del empleado es el 05-08-1996, la llamada a la ejecución del método se hará con la sintaxis:

```
CalculaDiasAlta ( { 2001-07-22 }, { 1996-08-06 } )
```

El intérprete de reglas sabe que la primera fecha debe sustituir a las referencias que se hayan hecho al argumento `FechaSistema`, y que la segunda fecha debe sustituir al argumento `FechaAlta`.

Tipo básico

Los argumentos van asociados a un tipo de dato, de forma que sólo aceptarán valores cuyo tipo de dato coincida con el que se les haya asignado en el momento de su definición.

No obstante, si se pasa como valor a un argumento un dato con un tipo de dato que no coincide con el indicado para el argumento, el intérprete de reglas tratará de convertir el valor al tipo de dato del argumento.

Si la conversión no puede hacerse correctamente, se producirá un error. Si la conversión es posible, continuará la ejecución de la regla.

Argumentos fijos y variables

Un elemento con ejecución puede recibir un número fijo de parámetros o un número variable.

En el primer caso, siempre que se haga una llamada al método deberán escribirse, entre paréntesis, un valor para cada uno de los parámetros que se hayan definido al crear la regla del elemento.

En el segundo caso, al llamar al elemento se podrán proponer argumentos adicionales a los estrictamente declarados en el momento de crear el método o el concepto. Todos los argumentos se escribirán entre paréntesis separados por comas.

Esto obliga a que, si se quiere pasar a un elemento un valor para un argumento situado en la una posición determinada de la lista de argumentos, se debe pasar obligatoriamente un valor para todos los anteriores. De no hacerse así, se produciría un error de compilación.

En los dos casos anteriores, siempre se deben pasar valores a los argumentos declarados en el momento de crear el elemento. La sintaxis del lenguaje LN4 no permite llamar a un método sin pasarle uno de sus argumentos obligatorios.

De esta forma, sería incorrecta una llamada del tipo:

```
Elemento (23, 12, ..., 34)
```

En su lugar, debería indicarse siempre un valor para el tercer argumento, por ejemplo:

```
Elemento (23, 12, 0, 34)
```

Los argumentos se definen a nivel de elemento, y no de regla, aunque es la regla la que recibe y utiliza los argumentos.

Ejecución de reglas

Las reglas de métodos y conceptos se pueden ejecutar de distinta forma:

- Desde la interfaz de la aplicación, mediante el control que esté asociado al elemento en la presentación del Meta4Object (generalmente un botón)
- Desde las reglas LN4 de otros métodos o conceptos
- Al desencadenar la ejecución automática de métodos mediante disparadores y eventos

En los tres casos se tendrá que distinguir si se hace una ejecución con o sin tramos. Las ejecuciones con tramos únicamente afectan a los elementos de tipo concepto (no a los de tipo método)



La tecnología Meta4 ofrece dos modos de ejecución: con tramos y sin tramos. La ejecución de los métodos no se ve afectada por el modo de ejecución seleccionado.

Los eventos sólo se utilizan con los métodos y un evento no puede automatizar la ejecución de conceptos.

Cuando se hace una ejecución sin tramos, el sistema devuelve un único valor para cada concepto.

En una ejecución con tramos, el usuario seleccionará un periodo de tiempo al que se llamará *periodo de ejecución*. Al procesar el cálculo, el sistema será capaz de interpretar los cambios que se hayan producido durante el periodo de tiempo seleccionado en:

- Las reglas de los conceptos
- Los valores de otros elementos utilizados por las reglas

Y generará valores diferentes para un mismo concepto en distintos periodos de tiempo. Estos periodos estarán comprendidos dentro del periodo de ejecución.

EJEMPLO

Si el cálculo de nómina mensual calcula el valor de un concepto, Precio_Total_Dietas, que recoge cuánto se debe abonar a cada empleado en concepto de dietas de desplazamiento, el precio de las dietas cambiará según el proyecto en el que haya trabajado el empleado.

Durante un mismo mes, un empleado ha trabajado en proyectos distintos. Por ejemplo, entre el día 1 y 20 estuvo en el proyecto A y entre el 21 y el 31 en el proyecto B.

La regla LN4 que calcula el valor del concepto Precio Total Dietas multiplica el precio del concepto Precio Dietas por el número de días trabajados en cada proyecto.

En una ejecución con tramos, el sistema podría generar dos tramos para este concepto: uno para el periodo comprendido entre el 1 y el 20 y otro para el comprendido entre los días 21 y 31. Cada tramo contendría un valor distinto para el concepto.

Los valores obtenidos para los distintos tramos de un mismo concepto se pueden totalizar de distinta forma: suma, media, valor máximo, mínimo, etc.

Ejecución desde controles de presentación

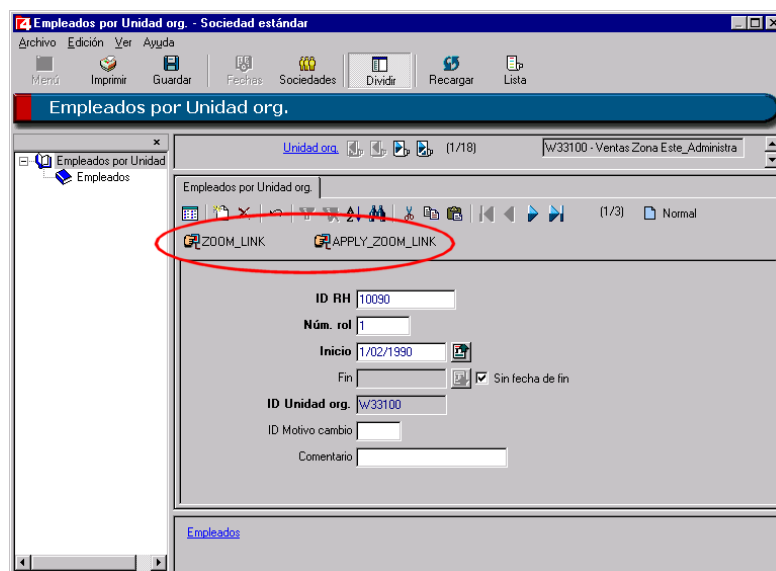
Cuando se crea un elemento de tipo método o concepto en una estructura de nodo, se podrá indicar si el elemento va a ser visible en la presentación de los Meta4Object que creen una instancia de la estructura de nodo.



Para indicar que un método debe ser visible, ha de establecer en *Sí* la propiedad **Visible** del elemento.

Si se indica que el método va a ser visible, cuando se cargue el Meta4Object en memoria, se mostrará al usuario un botón desde el que se podrá ejecutar el método. Para ello bastará con hacer clic en este botón.

Figura 67. Ejecución del elemento



Al solicitar la ejecución del elemento desde la interfaz, el sistema ejecutará la regla del método o del concepto que:

- Se haya marcado como metarregla
- Tenga un periodo de validez que comprenda la fecha de aplicación seleccionada por el usuario al comenzar la sesión de trabajo

EJEMPLO

Durante una ejecución con tramos, si en el momento de tener acceso a la aplicación, se selecciona una ejecución con tramos y se ejecuta un concepto desde el botón que tiene asociado en la presentación del Meta4Object, el sistema generará tramos para el concepto siempre que se cumpla una de las condiciones necesarias para ello; por ejemplo, la existencia de:

- Dos o más reglas cuyos periodos de validez estén comprendidos en el periodo de ejecución seleccionado
- Dependencias con otro elemento cuyo valor haya cambiado durante el periodo de ejecución

EJEMPLO

Con las fechas de filtro de datos, de reglas y fecha de aplicación, al iniciar una sesión de trabajo es posible indicar una serie de fechas que van a determinar los datos y reglas con las que se va trabajar durante la sesión.

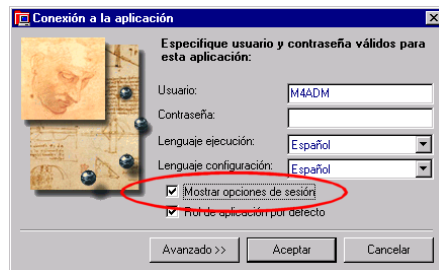
Es posible seleccionar las siguientes fechas:

- Fechas de filtro de datos: determinan un periodo de tiempo, de forma que, al cargar los datos de tablas fechadas a través de un nodo de un Meta4Object, únicamente se recuperarán aquellos registros cuyo periodo de validez comprenda total o parcialmente el comprendido entre las fechas de filtro de datos. Para que esto sea así, es necesario configurar el nodo con la propiedad **Filtro automático** como *Ambos*.
- Fechas de reglas: determinan un periodo de tiempo, de forma que cuando se ejecuten elementos de tipo concepto en una ejecución con tramos, sólo se ejecutarán las reglas cuyos periodos de validez estén comprendidos, total o parcialmente, en este periodo.
- Fecha de aplicación: se utiliza en la ejecución de elementos de tipo método y concepto, y en ejecuciones con y sin tramos.

Si se hace una ejecución sin tramos, cuando se ejecute un elemento, sólo se ejecutará la metarregla cuyo periodo de validez comprenda la fecha de aplicación. Como se ha indicado, esta metarregla puede ejecutar más de una regla, siempre que exista una norma para estas reglas que comprenda la fecha de aplicación.

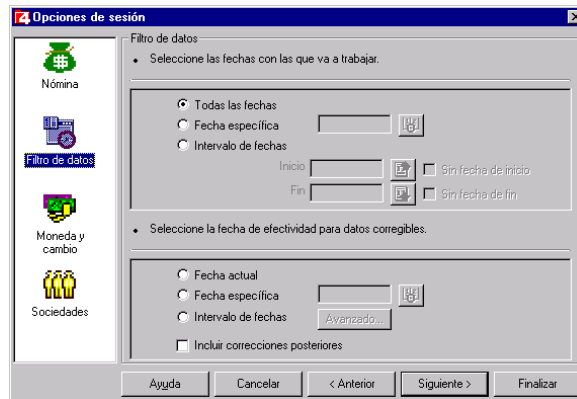
Para seleccionar las fechas de trabajo desde la ventana de conexión a la aplicación, debe activar la casilla **Mostrar opciones de sesión** una vez que haya indicado el nombre de usuario, la contraseña y el idioma de trabajo.

Figura 68. Ventana de conexión



Al hacer clic en el botón **Aceptar**, se abre la siguiente ventana:

Figura 69. Opciones de sesión



Haga clic en el icono denominado **Filtro de datos**; a la derecha se le ofrece la opción de seleccionar las fechas con las que va a trabajar y en la parte inmediatamente inferior, las fechas de filtro para los datos.

Si se hace una ejecución con tramos, cuando se ejecute un elemento de tipo método, sólo se ejecutará su metarregla si existe una norma cuyo periodo de validez comprenda la fecha de aplicación. En una ejecución con tramos, la fecha de aplicación no se utiliza para los elementos de tipo concepto.

Ejecución desde una regla LN4

Es posible ejecutar métodos y conceptos desde las reglas de otros elementos. LN4 permite utilizar dos sintaxis:

- El nombre del elemento seguido de sus parámetros escritos entre paréntesis.

```
NombreElemento (arg1, arg2, ... argn);
```

- El nombre del elemento seguido de sus argumentos y, a continuación, el código identificativo de la regla. El nombre del elemento se separará del código identificativo de la regla mediante dos puntos:

`NombreElemento(arg1, arg2, ..., argn)..IdRegla`

Sintaxis NombreElemento (arg₁, ... arg_n)

En el primer caso, el sistema ejecutará la regla que se haya marcado como metarregla, siempre que una de sus normas tenga un periodo de validez que comprenda la fecha de aplicación.

Ejecución con tramos

Si se ha seleccionado este tipo de ejecución, y se solicita la ejecución de un concepto desde el código LN4 de una regla con la sintaxis:

`NombreElemento (Arg1, Arg2, ..., Argn)`

El sistema ejecutará la metarregla del elemento, siempre que exista una norma con un periodo de validez que comprenda la fecha de ejecución.

La metarregla llamará a la ejecución del resto de reglas del elemento. Estas reglas sólo se ejecutarán si tienen al menos una norma cuyo periodo de validez:

- Esté comprendido, total o parcialmente, en el periodo de ejecución seleccionado
- Comprenda totalmente el periodo de ejecución

Para cada regla que se ejecute, se generarán distintos tramos y valores; al menos un valor para cada regla.

La ejecución con tramos no afecta a los elementos de tipo método. La ejecución de los métodos se hará de la misma forma con o sin tramos.

La fecha o el periodo de ejecución lo indica el usuario al comenzar una sesión de trabajo. Equivale a las fechas de filtro o fechas de reglas.

Durante una sesión de trabajo se pueden cambiar las fechas de reglas y la fecha de aplicación, tanto desde la interfaz de la aplicación como desde el código LN4, mediante las funciones `SetStartDate()`, `SetEndDate()` y `SetRunDate()`.

`SetStartDate()` y `SetEndDate()` fijan el periodo de ejecución y `SetRunDate()`, la fecha de aplicación.

Sintaxis NombreElemento(arg1, ..., argn)..IdRegla

Si se utiliza esta sintaxis en una ejecución sin tramos, el sistema ejecutará la regla que se indica en la llamada, siempre que exista una norma para esa regla con un periodo de validez que comprenda la fecha de aplicación.

En una ejecución con tramos, si la llamada se hace a un elemento de tipo concepto, se ejecutará la regla siempre que exista al menos una norma para esa regla que tenga un periodo de validez comprendido total o parcialmente en el periodo de ejecución seleccionado.

La ejecución de métodos se hace de la misma forma en una ejecución con o sin tramos.

Si se hace una llamada mediante esta segunda sintaxis, no se ejecutará la metarregla del elemento. Se ejecutará la regla directamente.

LN4 permite ejecutar reglas cuyo periodo de ejecución no esté comprendido en el periodo de ejecución seleccionado por el usuario al iniciar la aplicación.

Para ello, se utilizará la siguiente sintaxis:

```
NombreElemento [{yyyy-mm-dd}] (arg1, ..., argn)..IdRegla
```

Para ejecutar la regla cuyo periodo de validez comprende la fecha indicada detrás del nombre del elemento, o bien:

```
NombreElemento [{yyyy-mm-dd}] (arg1, ..., argn)
```

Para ejecutar la metarregla cuyo periodo de validez comprende la fecha indicada detrás del nombre del elemento.

Ejecución de elementos en disparadores y eventos

Existe una tercera forma de ejecutar métodos y conceptos: mediante disparadores y eventos.

Los disparadores permiten automatizar la ejecución de elementos de tipo método y concepto. Los eventos sólo se pueden utilizar con elementos de tipo método.

En los dos casos, el funcionamiento es similar:

- Cada disparador o evento contará con un elemento origen y unos elementos destino. Los eventos pueden contar con un número ilimitado de elementos origen.
- Cuando se ejecuta un elemento que se haya declarado origen de un disparador o de un evento, se desencadenará la ejecución de los elementos destino del disparador o del evento.

La ejecución de disparadores también se ve afectada por la selección de una ejecución con tramos o sin tramos.

Si se selecciona la ejecución con tramos y el disparador ejecuta un elemento de tipo concepto, se ejecutará la metarregla del elemento siempre que el periodo de validez de una de sus normas:

- Esté comprendido, total o parcialmente, en el periodo de ejecución indicado por el usuario
- Comprenda el periodo de ejecución

La metarregla solicitará la ejecución de las reglas del elemento. Para cada regla que se ejecute se generará al menos un tramo; es decir, un valor asociado a un periodo de tiempo específico.

Resumen

- Al iniciar la aplicación, es posible indicar fechas de filtro de datos, el periodo de ejecución (fechas de reglas) y la fecha de aplicación.
- El periodo de ejecución y la fecha de aplicación se pueden cambiar desde reglas escritas en lenguaje LN4 con unas funciones especiales.
- Son elementos con ejecución los de tipo método o concepto.
- Un elemento con ejecución puede tener una, dos o más reglas asociadas.
- Cada regla puede tener una o más normas. Norma es el periodo de validez comprendido entre una fecha de inicio y otra de fin. La fecha de fin del periodo de validez de una norma puede ser nula. Sin embargo, la fecha de inicio siempre debe indicarse.
- En el caso de que haya más de una norma para una misma regla, sólo una de ellas puede tener una fecha de fin nula.
- Los periodos de validez de las normas de una misma regla no se pueden solapar.
- Los periodos de validez de las normas de una regla sí pueden solaparse con los periodos de validez de las normas de otras reglas del mismo elemento. De esta forma:
 - Para un mismo elemento puede haber dos o más reglas válidas en el mismo instante del tiempo
 - Para cada una de estas reglas, sólo una de sus normas será válida en un instante del tiempo

- Las metarreglas son un tipo especial de reglas que genera el sistema siempre que se crea la primera regla de un elemento. Las metarreglas se ejecutan siempre que se solicita la ejecución de un método o concepto desde la interfaz de la aplicación, desde disparadores o eventos, y desde reglas LN4, con la sintaxis

`NombreElemento(args)`

- Las metarreglas únicamente contienen llamadas al resto de reglas del elemento.
- Las metarreglas, al igual que las reglas, pueden tener distintas normas o periodos de validez. Los periodos de validez de estas normas no pueden solaparse.
- Siempre que se solicite la ejecución de un método, el intérprete de reglas ejecutará la regla que se haya marcado como metarregla, siempre que tenga al menos una norma con un periodo de validez que comprenda la fecha de aplicación seleccionada al iniciar la sesión de trabajo.
- Cuando se solicita la ejecución de un concepto y se hace una ejecución sin tramos, el intérprete de reglas ejecuta la metarregla cuyo periodo de validez comprenda la fecha de aplicación seleccionada.

La metarregla solicitará la ejecución de todas las reglas del elemento. Sólo se ejecutarán las reglas cuyo periodo de validez comprenda a la fecha de aplicación.

- Cuando se solicita la ejecución de un concepto y se hace una ejecución con tramos, el intérprete de reglas ejecuta la metarregla, si tiene al menos una norma con un periodo de validez que comprenda la fecha de aplicación seleccionada.

La metarregla solicitará la ejecución de todas las reglas del elemento. Sólo se ejecutarán aquellas reglas para las que exista al menos una norma cuyo periodo de validez esté comprendido total o parcialmente en el periodo de ejecución seleccionado al iniciar la sesión de trabajo.

- Desde reglas LN4, es posible ejecutar una regla específica de un método o concepto, sin ejecutar antes su metarregla.

Con la sintaxis:

`NombreElemento( )..IdRegla`

La regla sólo se ejecutará si su periodo de validez:

- Comprende la fecha de aplicación (en el caso de métodos, en una ejecución con o sin tramos; en el caso de los conceptos, en una sin tramos)
- Está comprendido total o parcialmente en el periodo de ejecución (en una ejecución con tramos para los conceptos)

- Desde una regla LN4, es posible ejecutar cualquier regla cuyo periodo de validez no incluya la fecha de aplicación ni esté comprendido en el periodo de ejecución (en el caso de una ejecución con tramos).

Le corresponde la siguiente sintaxis:

```
NombreElemento[{aaaa-mm-dd}]( )..IdRegla
```

Dependencias entre elementos

En la regla LN4 de un elemento de tipo método o concepto se puede hacer referencia a otros elementos de la misma estructura de nodo o de otra diferente.

Las estructuras de nodo que contienen los elementos a los que se hace referencia pueden estar o no incluidas en el mismo Meta4Object que el elemento cuya fórmula contiene la referencia.

Mediante estas referencias, es posible utilizar el valor de otros elementos para calcular el valor de un elemento. Cuando un elemento utiliza otros elementos para calcular su valor, se dice que existe una dependencia entre ellos.

Elemento dependiente es aquél cuya fórmula hace referencia a un segundo elemento. *Elemento referenciado*, el utilizado en la regla LN4 de otro elemento.

Un elemento puede ser utilizado en varias reglas de otros elementos y, a su vez, puede utilizar otros elementos en su propia regla.

EJEMPLO

El concepto TotalPagarHorasExtras tiene la siguiente fórmula:

```
Total=NumeroHorasExtras( )*PrecioHorasExtras  
Return(Total)
```

Para calcular su valor, el concepto utiliza a otros dos conceptos: NumeroHorasExtras y PrecioHorasExtras.

Se dice que existe una dependencia entre ellos, y que TotalPagarHorasExtras depende de NumeroHorasExtras y de PrecioHorasExtras.

Dependencias de valor y de ejecución

Los elementos de tipo concepto recogen el valor que devuelve su fórmula. A estos elementos también se les puede asignar un valor por defecto, que será el que recogerán antes de ser ejecutados.

A partir del momento en que se ejecute, el concepto pasa a recoger el valor obtenido como resultado de la ejecución de su fórmula.

La referencia a un concepto desde la regla de otro elemento se puede hacer de dos formas, escribiendo:

- El nombre del concepto

```
x = IdConcepto
```

- El nombre del concepto seguido de sus argumentos entre paréntesis

```
x = IdConcepto( arg1, ... argn)
```

En el primer caso, únicamente se lee el valor del concepto. Se dice que se trata de una dependencia de valor. Si el concepto se ha ejecutado con anterioridad, se leerá el valor obtenido tras su ejecución.

Si el concepto no se ha ejecutado aún y tiene un valor por defecto, se leerá este valor, pero no se ejecutará su regla.

En el segundo caso, el concepto se ejecutará, independientemente de que se haya ejecutado con anterioridad. Se dice que hay una dependencia de ejecución.

En los dos casos, si el concepto se encuentra en un nodo distinto del `Meta4Object` que contiene el elemento desde cuya regla se hace la llamada, el nombre del concepto deberá ir precedido por el código identificativo del nodo.

Si el concepto se encuentra en un `Meta4Object` distinto del que contiene el elemento desde cuya regla se hace la llamada, el nombre del concepto deberá ir precedido por el alias del `Meta4Object`, así como por el código identificativo del nodo del que forma parte.

Tipos de dependencias

El repositorio de las aplicaciones Meta4 es un conjunto de tablas que contienen la estructura de todos los objetos utilizados por la aplicación: estructuras de nodo, `Meta4Objects`, tablas, eventos, etc.

En el repositorio hay tres tablas que recogen las dependencias entre elementos:

- `M4RCH_INTERNAL_DEP`: las que se producen entre elementos del mismo nodo
- `M4RCH_EXTERNAL_DEP`: las que se producen entre elementos de distintos nodos del mismo `Meta4Object`
- `M4RCH_CHANNEL_DEP`: las que se producen entre elementos de distintos `Meta4Objects`

Ninguna de estas tablas recoge información sobre las dependencias entre elementos que se generan mediante eventos y mediante disparadores.

Se distinguen tres tipos de dependencias entre elementos, según donde se encuentre el elemento al que se hace referencia:

- Si los dos elementos forman parte del mismo nodo, se produce una *dependencia interna*
- Si los elementos se encuentran en un nodo diferente del mismo Meta4Object, se produce una *dependencia externa*
- Si los elementos se encuentran en Meta4Objects diferentes, se produce una *dependencia de nivel 2*

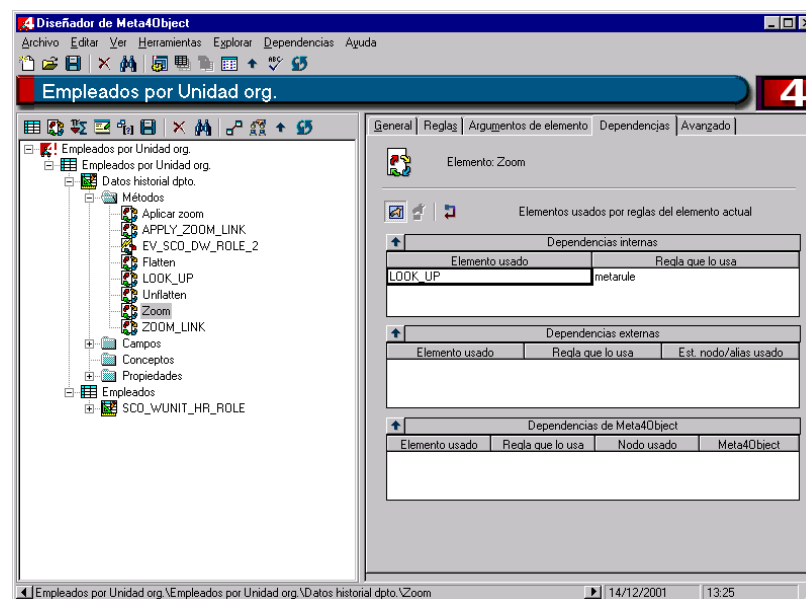
Todas las dependencias entre elementos se registran en el repositorio de la aplicación, que cuenta con una tabla distinta para cada tipo de dependencia.

Cuando se compila una regla, el programa deduce de forma automática todas las dependencias internas, externas y de nivel 2 que existen en la fórmula, siempre que las dependencias hagan referencia a elementos de tipo campo, propiedad o concepto.

Las dependencias obtenidas se guardan en unas tablas del repositorio reservadas para ello: M4RCH_INTERNAL_DEP, M4RCH_EXTERNAL_DEP y M4RCH_CHANNEL_DEP.

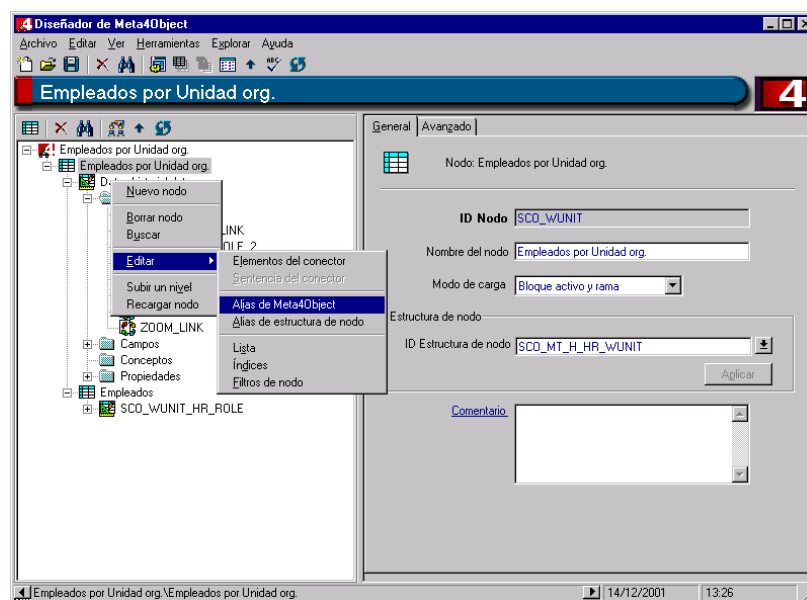
Desde el Diseñador de Meta4Objects es posible consultar las dependencias internas y externas. Esta herramienta muestra las dependencias internas y externas de cada elemento.

Figura 70. Diseñador de Meta4Objects: dependencias



Las dependencias de nivel 2 se pueden ver a través de los formularios de *Definición de alias* y *resolución de alias* de cada Meta4Object. Se tiene acceso a estos formularios desde la herramienta **Diseñador de Meta4Objects**.

Figura 71. Resolución de alias de Meta4Object



Dependencias de nivel 2

Las dependencias entre elementos de Meta4Objects diferentes, o llamadas de nivel 2, requieren un tratamiento especial.

Este tratamiento surge ante la necesidad de distinguir la instancia específica del Meta4Object que contiene el elemento que se quiere utilizar en la regla.

En un mismo instante pueden estar cargadas en memoria distintas instancias de un mismo Meta4Object.

Un mismo usuario puede, por ejemplo, haber cargado en memoria distintas instancias de un mismo Meta4Object. Según el momento en que se haya solicitado la carga del Meta4Object, se habrán cargado en memoria unos datos u otros. En esta situación, es necesario disponer de un mecanismo que permita indicar la instancia del Meta4Object con que se quiere trabajar en el texto de una regla.

EJEMPLO

Un Meta4Object recupera todos los registros de la tabla Histórico de proyectos. El Meta4Object está formado por un único nodo. Un usuario decide cargar este Meta4Object en memoria a las 09:30:00. En ese instante, se cargan de la base de datos los siguientes registros:

HR_ID	PROYECTO	INICIO	FIN
01	OWEN & CIA	01-02-1999	31-07-1999
01	BRADSHEET	05-08-1999	NULO
02	OWEN & CIA	10-03-1999	NULO
03	BRADSHEET	15-04-1999	NULO

Tres minutos más tarde, a las 09:33:00, el usuario solicita la carga en memoria de otra instancia de ese mismo Meta4Object. El sistema creará una nueva instancia que recuperará datos distintos de los que se recuperaron la primera vez, ya que otro usuario ha agregado dos nuevos registros al Meta4Object y ha grabado los cambios en este intervalo.

HR_ID	PROYECTO	INICIO	FIN
02	RAILWAYS BS	11-05-1999	NULO
04	BRADSHEET	11-05-1999	NULO

Por lo tanto, en la segunda operación de lectura, el sistema habrá devuelto los siguientes registros:

HR_ID	PROYECTO	INICIO	FIN
01	OWEN & CIA	01-02-1999	31-07-1999
01	BRADSHEET	05-08-1999	NULO
02	OWEN & CIA	10-03-1999	10-05-1999
03	BRADSHEET	15-04-1999	NULO
02	RAILWAYS BS	11-05-1999	NULO
04	BRADSHEET	11-05-1999	NULO

Como se puede apreciar, en la segunda operación de lectura se han recuperado los dos registros agregados por el otro usuario.

El conjunto de registros recuperados también muestra el cambio de valor producido en el campo FECHA FIN del registro correspondiente al empleado 02 y al proyecto OWEN & CIA: el campo se ha rellenado con la fecha inmediatamente anterior a la propuesta como fecha de inicio del nuevo registro (proyecto RAILWAYS BS).

De esta forma, un mismo usuario trabaja con dos instancias distintas del mismo Meta4Object.

En el ejemplo anterior se parte del supuesto de que, siempre que un usuario solicita la carga en memoria de un Meta4Object, el sistema crea una nueva instancia del Meta4Object. Esto no siempre es así. Por razones de optimización, es posible reutilizar instancias que un mismo usuario ha cargado con anterioridad en memoria.



Sólo los Meta4Objects que se hayan guardado en caché pueden ser reutilizados por más de un usuario de la aplicación.

La reutilización de instancias ya cargadas de Meta4Objects, o la creación de nuevas instancias, se gestiona mediante la utilización de alias en las reglas de LN4 y su posterior resolución.

Uso de alias de Meta4Object en reglas LN4

Siempre que se quiera utilizar un elemento de otro Meta4Object en la regla de un método o concepto, es necesario utilizar un alias para hacer referencia al Meta4Object que contiene el elemento.

El alias consistirá en un código identificativo alfanumérico que se utilizará en lugar del código identificativo del Meta4Object.

EJEMPLO

En la regla de un concepto A del Meta4Object EMPLEADO se utiliza el elemento C de otro Meta4Object: CONDICIONES_EMPLEADO. Este elemento está incluido en el nodo Condiciones.

Se dirá que existe una llamada de nivel 2 entre los dos Meta4Objects.

La sintaxis que establece LN4 para hacer referencia a los elementos de otro Meta4Object es:

```
AliasMeta4Object!IdNodo.IdElemento
```

Donde *AliasMeta4Object* será una cadena de caracteres que sustituye al nombre del Meta4Object. *IdNodo* será el código identificativo del nodo en el que está incluido el elemento, e *IdElemento*, el código identificativo del elemento.

En el ejemplo, se utilizará el alias *EmpCond* para hacer referencia al Meta4Object `CONDICIONES_EMPLEADO`. La llamada de nivel 2 tendrá la forma:

```
EmpCond!Condiciones.C
```

A continuación, es necesario indicar cómo se debe resolver el alias de este Meta4Object.

Para utilizar un alias de Meta4Object en una regla, es necesario:

1. Declarar el alias en el nodo que contiene el elemento para el que se va a escribir la regla.
2. Escribir la regla haciendo referencia al Meta4Object que contiene el elemento, mediante el alias que se ha declarado en el paso anterior.
3. Resolver el alias en el nodo que contiene el elemento para el que se va a escribir la regla.

Todos estos pasos se harán desde el Diseñador de Meta4Objects. Para obtener más información, consulte el capítulo "Administración de memoria" del grupo de manuales de *Arquitectura de componentes*.

Dimensiones: ejecución condicional de reglas

El término *dimensión* hace referencia a una condición o un conjunto de condiciones que deben cumplirse para que se ejecute una regla de un elemento de tipo método o concepto.

Las dimensiones facilitan la ejecución condicional de reglas. Mediante dimensiones se podrá indicar si se debe ejecutar una regla u otra, según el valor que recojan uno o varios elementos determinados.

Por ejemplo, pueden diseñarse varias reglas para un mismo elemento. Una que se ejecute en el caso de que un elemento recoja un valor; otra que se ejecute si el mismo elemento recoge algún otro valor, etc. Para lograr esta funcionalidad, se tendría que definir una dimensión en cada regla.

Una dimensión equivale a una condición o a una serie de condiciones que deben cumplirse para que se ejecute una regla. Las dimensiones siempre están asociadas a una regla. Son aplicables a elementos de tipo método y concepto.



El entorno de desarrollo de la aplicación permite indicar al usuario-configurador las condiciones que deben cumplirse para que el intérprete ejecute una u otra regla al calcular un concepto o ejecutar un método.

Cada condición compara con un valor constante el valor que recoge un elemento de tipo campo, concepto o propiedad del nodo en el que se encuentra el elemento para el que se diseña la regla, de forma que ésta sólo se ejecutará si se cumple la condición. En cualquier otro caso, no se ejecutará.

Una dimensión puede incluir una o más condiciones, cada una de ellas definidas sobre un elemento de tipo campo, concepto o propiedad.

En el caso de que se indique más de una condición en una misma dimensión, el sistema se comporta de la siguiente forma:

- Si las condiciones se han definido sobre elementos diferentes, la regla se ejecutará cuando se cumplan todas las condiciones. Dicho de otra forma, las condiciones se unirán mediante el operador AND.
- Si las condiciones se han definido sobre el mismo elemento, se unirán mediante el operador OR. Es decir, la regla se ejecutará cuando se cumpla al menos una de las condiciones.

Esto último tiene especial importancia en el caso de que se quiera definir una condición que compruebe la inclusión de un valor dentro de un rango.

Para ello, es necesario definir dos dimensiones con el mismo periodo de validez. Una que compruebe que el valor es menor que el valor mayor del rango, y otra que compruebe que es mayor que el valor inferior del rango.

Periodo de validez de una condición

Cada condición que se indica en una dimensión tiene un periodo de validez delimitado por unas fechas de inicio y de fin. Una vez cumplido este periodo, la condición dejará de aplicarse.

El periodo de validez de una condición no tiene por qué estar incluido dentro del periodo de validez de la regla para la que se define. Esta circunstancia da una mayor flexibilidad al diseñador.

EJEMPLO

Una regla puede tener distintos periodos de validez discontinuos en el tiempo. El término *norma* designa a cada uno de estos periodos de validez.

Por ejemplo, una regla R1 puede tener un periodo de validez comprendido entre el 1 de enero de 1990 y el 31 de diciembre de 1995, y otro periodo comprendido del 1 de enero de 1998 en adelante. El periodo comprendido entre el 1 de enero de 1996 y el 31 de diciembre de 1997 no estaría cubierto por la regla.

Si la regla se quiere ejecutar en el único caso de que se cumplan una serie de condiciones, se puede definir una dimensión que abarque desde el 1 de enero de 1990 al infinito. De esta forma, se garantizaría la ejecución de la regla en el caso de que se cumpla la condición.



Los periodos de validez de una regla no tienen por qué incluir a los periodos de validez de las condiciones de la dimensión.

Si la dimensión tiene dos o más condiciones, cada condición tendrá su propio periodo de validez. Las condiciones sólo se evaluarán si la fecha de aplicación está comprendida dentro de su periodo de validez.

Dimensiones y metarreglas

Siempre que se graba la regla de un elemento de tipo método o concepto para la que se haya definido una dimensión, el sistema actualizará las normas de la metarregla del elemento de forma automática.

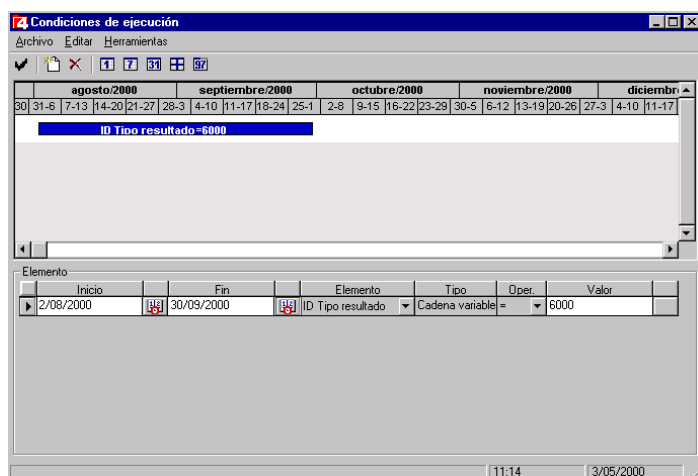
La metarregla contiene una llamada a la regla, previa evaluación del cumplimiento de las condiciones definidas en la dimensión.

EJEMPLO

Se crea una regla de un método y una dimensión que especifica que la regla sólo debe ejecutarse en el caso de que un elemento de tipo campo recoja un valor mayor que 6.000.

Las dimensiones se han definido desde la ventana **Condiciones de ejecución**, a la que se tiene acceso desde la pestaña Reglas.

Figura 72. Condiciones de ejecución



En este caso, al grabar la regla, el sistema generará automáticamente una segunda regla de tipo metarregla, con el siguiente código:

```
If ID Tipo resultado = 6000 Then
  Elemento( )..IdRegla
EndIf
```

El sistema puede generar una norma para la metarregla, o más de una. El número de normas que se generen depende de cómo se solapen los periodos de validez de las condiciones indicadas para la dimensión.

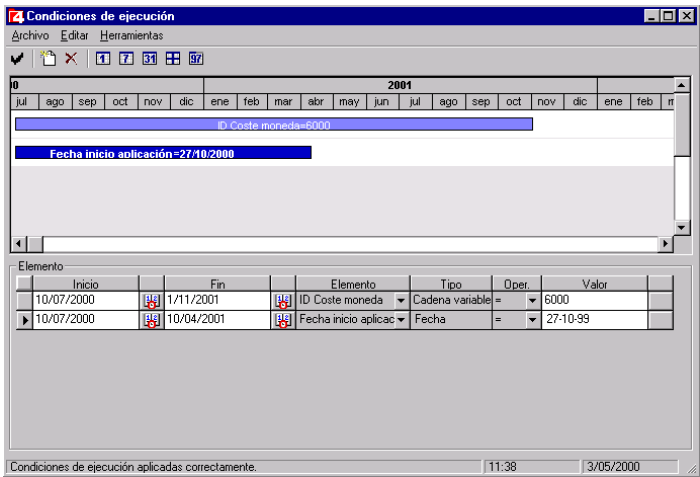
Por lo general, genera distintas normas, según cómo se solapen los periodos de validez de las condiciones incluidos en la dimensión.

EJEMPLO

Se define una regla con un periodo de validez comprendido entre el 1 de Julio de 2000 y el 30 de Noviembre de 2001.

Se agrega a la regla una dimensión que consta de dos condiciones.

Figura 73. Condiciones de ejecución



La primera indica que, durante el periodo comprendido entre el 10 de Julio de 2000 y el 1 de Noviembre de 2001, la regla sólo deberá ejecutarse si el campo SCO_ID_LEVEL del mismo registro recoge el valor 6.000.

La segunda condición indica que, durante el periodo comprendido entre el 10 de Julio de 2000 y el 10 de Marzo de 2001, la regla sólo deberá ejecutarse si el campo SCO_DT_START recoge una fecha posterior al 27 de Octubre de 2000.

Los periodos de validez de las condiciones se solapan.

Cuando se grabe la regla, el sistema generará más de una norma para la metarregla, ya que hay más de una condición. En este caso se generarán cuatro normas de la metarregla, además de la regla normal generada para el concepto y de su correspondiente norma.

La siguiente tabla recoge las características de cada una de las normas de la metarregla:

ID	Inicio validez	Fin validez	Código metarregla
1	01-07-2000	09-07-2000	'Autogenerated System Code. Should not be modified X = Id_Elemento()..R1 return (X)
2	10-07-2000	10-03-2001	'Autogenerated System Code. Should not be modified if (SCO_ID_LEVEL=6000 AND SCO_DT_START>27-10-2000) Then X = Id_Elemento()..R1 end if return (X)
3	11-03-2001	01-11-2001	'Autogenerated System Code. Should not be modified if (SCO_ID_LEVEL=6000) Then X = Id_Elemento()..R1 end if return (X)
4	02-11-2001	30-11-2001	'Autogenerated System Code. Should not be modified X = Id_Elemento()..R1 return (X)

En el ejemplo anterior se puede apreciar que:

- a. Si la fecha de inicio del periodo de validez de la regla no coincide con la fecha de inicio de una de las condiciones de la dimensión, se genera una norma para la metarregla con un periodo de validez comprendido entre:
 - La fecha de inicio de validez de la regla
 - La fecha inmediatamente anterior a la fecha de inicio más antigua de todas las condiciones de la dimensión

Si estas fechas coinciden, no se genera esta norma de la metarregla.

- b. Si la fecha de fin del periodo de validez de la regla no coincide con la fecha de fin de una de las condiciones de la dimensión, se genera una norma para la metarregla con un periodo de validez comprendido entre:
 - La fecha de fin del periodo de validez de la regla
 - La fecha inmediatamente posterior a la fecha de fin más reciente de todas las condiciones de la dimensión

Si estas fechas coinciden, no se genera esta norma de la metarregla

- c. Se genera una norma de la metarregla para cada periodo en el que se solapen condiciones distintas.

EJEMPLO

En el ejemplo anterior, se generan unas normas de la metarregla en el periodo comprendido entre el 10 de Julio de 2000 y el 9 de Marzo de 2001, ya que en este periodo se solapan los periodos de validez de las dos condiciones.

- d. Para los periodos en los que esté activa una condición con un periodo de validez que no se solape con los de otras dimensiones, se generará una norma de la metarregla. En el ejemplo anterior, se genera una norma para el periodo comprendido entre el 10 de Marzo de 2001 y el 01 de noviembre de ese mismo año. En todo este periodo sólo hay una condición activa y su periodo de validez no se solapa con el de ninguna otra condición.

Esta última condición sólo se aplica si:

- La fecha de inicio del periodo de validez de la condición no coincide con la fecha de inicio del periodo de validez de la regla
- La fecha de fin del periodo de validez de la condición no coincide con la fecha de fin del periodo de validez de la regla.

Estos dos casos están cubiertos por la situación descrita en el punto a).

Las metarreglas, al igual que cualquier otra regla, tendrán distintas normas con un periodo de validez determinado entre unas fechas de inicio y de fin. El sistema determinará el periodo de validez de cada norma de la metarregla según las fechas de corte de los periodos de validez de cada una de las condiciones a partir de las cuales se genera.

El sistema nunca creará dos normas para una misma metarregla con periodos de validez solapados.

Cuando se solicite la ejecución de un concepto o método con una fecha de ejecución determinada, el sistema comprobará si existe una norma de la metarregla que comprenda esa fecha.

Si la norma existe, la ejecutará. Según si se cumplen o no las condiciones indicadas en el código de la norma de la metarregla (es decir, las condiciones que se han indicado al definir la dimensión), el sistema ejecutará la regla a la que llama la metarregla.

Si hay dos o más reglas válidas en la fecha de ejecución, o en el periodo de ejecución de la regla, una metarregla puede ejecutarlas todas. El orden en que se ejecutarán las reglas se corresponde con el orden indicado en el campo **Orden ejecución** de cada una de ellas.

Es importante recordar que, cuando una metarregla ejecuta dos o mas reglas seguidas, el sistema únicamente devolverá el valor obtenido por la ejecución de la última. Este factor resulta especialmente importante en el caso de los elementos de tipo concepto, ya que el valor que recogerá el concepto será devuelto por la última regla que se ejecute.

Los valores devueltos por las reglas que se hayan ejecutado con anterioridad no serán recuperables, a menos que se depositen en un elemento que los recoja con carácter temporal.

Creación de metarreglas

El sistema genera de forma automática una metarregla siempre que se crea la primera regla para un elemento.

Sin embargo, es posible definir metarreglas desde las herramientas que proporciona la aplicación. En este caso, el usuario-configurador debe tener en cuenta que:

- Se ejecuta la regla que haya marcado como metarregla.
- La metarregla puede llamar a la ejecución de otras reglas válidas para ese mismo método o concepto. Si en ese instante existen reglas válidas a las que la metarregla no llama, nunca se llegarán a ejecutar, a no ser que se las llame con la sintaxis de llamada directa a reglas desde otras reglas del sistema, es decir:

```
NombreElemento( )..NombreRegla
```

En la fase de diseño, será preciso asegurarse de que los periodos de validez de las reglas incluyen, total o parcialmente, el periodo de validez de las condiciones a partir de las cuales se construye la metarregla.

Es decir, si una condición llama a la ejecución de una regla cuyo periodo de validez no se corresponde con el periodo en el que se ejecuta la llamada a la metarregla, la regla no se ejecutará.

A partir de este momento, cuando se creen nuevas reglas para el elemento, y se agreguen nuevas normas para una regla ya existente, el sistema actualizará de forma automática el conjunto de normas de la metarregla.

A continuación se describe cómo se actualizan las metarreglas. Según el tipo de actualización que se haga, se distinguen los siguientes casos.

Actualización de metarreglas

Las metarreglas creadas de forma automática por el sistema se actualizan también de forma automática, a medida que se agregan nuevas normas o reglas, o se modifican las dimensiones de las reglas a las que se hacen llamadas desde una metarregla.

Se distinguen los siguientes casos:

- Creación de una norma para una regla ya existente
- Borrado de una norma
- Creación de nuevas reglas sin dimensiones para un método o concepto para el que ya existe una regla previa
- Creación de nuevas reglas con dimensiones
- Modificación de dimensiones de una regla existente

Creación de una nueva norma

Siempre que se crea una nueva norma para una regla ya existente, se actualizan las normas de la metarregla del elemento. Se dan los siguientes casos, la norma de la regla se solapa con:

- Una norma de la metarregla
- Más de una norma de la metarregla

Solapamiento con una norma de la metarregla

Existe la posibilidad de que haya un solapamiento de una norma sobre otra existente previamente en una metarregla, por lo que se explican las posibles opciones:

- Si el periodo de validez de la nueva norma está incluido totalmente en el periodo de validez de una norma de la metarregla ya existente, las normas de la metarregla no se modificarán.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 13 y el 30 de Noviembre de 2000.

Cuando se creó esta regla, se generó una nueva metarregla con idéntico periodo de validez.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	30-11-2000	Código de la Regla_1

Se crea una nueva norma para la «Regla 1», con un periodo de validez comprendido entre el 20 y el 25 de Noviembre.

Tras actualizar el conjunto de normas de «Regla 1», se actualizará el conjunto de normas de la metarregla del elemento.

Como, en este caso, el periodo de validez de la norma está incluido totalmente en el periodo de validez de una de las normas de la metarregla ya existentes, no se hará ningún cambio en la norma de la metarregla.



La forma en la que se actualizan las normas de una regla al crear una nueva norma se ha explicado en un apartado anterior.

La situación final sería la que recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	19-11-2000	Código de la Regla_1 - Norma 1
Regla_1	20-11-2000	25-11-2000	Código de la Regla_1 - Norma 2
Regla_1	26-11-2000	30-11-2000	Código de la Regla_1 - Norma 1

- Si el periodo de validez de la nueva norma
 - Está comprendido parcialmente dentro del periodo de validez de una norma de la metarregla ya existente
 - Tiene una fecha de inicio anterior a la fecha de inicio de la norma de la metarregla con la que se solapa

Se ampliará el periodo de validez de la norma de la metarregla. Su fecha de inicio pasará al día de inicio de la nueva norma. Como fecha de fin se mantendrá la fecha de fin de la norma existente.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 13 y el 30 de Noviembre de 2000.

Al crearse esta regla se generó una nueva metarregla con idéntico periodo de validez.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	30-11-2000	Código de la Regla_1

Se crea una nueva norma para la «Regla 1», con un periodo de validez comprendido entre el 1 y el 17 de Noviembre.

Tras actualizar el conjunto de normas de «Regla 1», se actualiza el conjunto de normas de la metarregla del elemento.

En este caso, la fecha de inicio de la metarregla pasará al 1 de Noviembre, para cubrir los periodos de todas las normas de la regla.

La situación final sería la que recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	01-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	01-11-2000	17-11-2000	Código de la Regla_1 - Norma 2
Regla_1	19-11-2000	30-11-2000	Código de la Regla_1 - Norma 1

- Si se cumple que el periodo de validez de la nueva norma:
 - Está comprendido parcialmente dentro del periodo de validez de una norma de la metarregla ya existente
 - Tiene una fecha de fin posterior a la fecha de fin de la norma con la que se solapa

Se ampliará el periodo de validez de la norma de la metarregla. Su fecha de fin pasará al día de fin de la nueva norma de la regla. Como fecha de inicio se mantendrá la fecha de inicio de la norma existente.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 13 y el 30 de noviembre de 2000.

Cuando se creó esta regla se generó una nueva metarregla con idéntico periodo de validez.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	30-11-2000	Código de la Regla_1

Se crea una nueva norma para la «Regla 1», con un periodo de validez comprendido entre el 20 de Noviembre y el 15 de Diciembre.

Tras actualizar el conjunto de normas de «Regla 1», se actualizará el conjunto de normas de la metarregla del elemento.

En este caso, la fecha de fin de la metarregla se ampliará hasta el 15 de diciembre, para cubrir los periodos de todas las normas de la regla.

La situación final sería la que recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	15-12-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	19-11-2000	Código de la Regla_1 - Norma 1
Regla_1	20-11-2000	15-12-2000	Código de la Regla_1 - Norma 2

- Si el periodo de validez de la nueva norma incluye totalmente al periodo de validez de una norma de metarregla ya existente, se ampliará este último periodo de validez. Su fecha de inicio pasará a la fecha de inicio de la nueva norma y su fecha de fin se cambiará por la fecha de fin de la nueva norma.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 13 y el 30 de Noviembre de 2000.

Cuando se creó esta regla se generó una nueva metarregla con idéntico periodo de validez.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	30-11-2000	<i>Código de la Regla_1</i>

Se crea una nueva norma para la «Regla 1», con un periodo de validez comprendido entre el 1 de Noviembre y el 15 de Diciembre.

Tras actualizar el conjunto de normas de «Regla 1», se actualizará el conjunto de normas de la metarregla del elemento.

En este caso, la fecha de fin de la metarregla se ampliará hasta el 15 de Diciembre, para cubrir los periodos de todas las normas de la regla.

La situación final sería la que recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	01-11-2000	15-12-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	01-11-2000	15-12-2000	<i>Código de la Regla_1</i>

Solapamiento con dos o más normas de la metarregla

En el caso de que el periodo de validez de la nueva norma se solape con más de una norma de la metarregla, se distinguen los siguientes casos:

- Si el periodo de validez de la nueva norma está incluido totalmente en dos o varios periodos de validez de normas de la metarregla ya existentes, no se actualizarán las normas de la metarregla.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 13 y el 30 de Noviembre de 2000.

La regla tiene una dimensión con un periodo de validez entre el 13 y el 23 de Noviembre, que indica que la regla sólo se debe ejecutar en ese periodo si el campo ID_LEVEL recoge un valor igual a 8.

Cuando se creó esta regla se generaron dos normas para la metarregla.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified If (Id_Nodo.ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	30-11-2000	Código de la Regla_1

Se crea una nueva norma para la «Regla 1», con un periodo de validez comprendido entre el 20 y el 27 de Noviembre.

Tras actualizar el conjunto de normas de «Regla 1», se actualizará el conjunto de normas de la metarregla del elemento.

En este caso, las normas de la metarregla no se actualizarán, ya que el periodo de validez de la nueva norma de «Regla 1» está comprendido totalmente dentro de los periodos de validez de las normas de la metarregla.

La situación final sería la que recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	13-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified If (Id_Nodo.ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	13-11-2000	19-11-2000	Código de la Regla_1 - Norma 1

ID	Inicio validez	Fin validez	Código metarregla
Regla_1	20-11-2000	27-11-2000	Código de la Regla_1 - Norma 2

- Si el periodo de validez de la nueva norma de la regla:
 - Se solapa totalmente con una o más normas de la metarregla, y parcialmente con una de las normas existentes de la metarregla.
 - Tiene una fecha de fin posterior a la fecha de fin de la norma de la metarregla más reciente:
 - Si esta norma de la metarregla contiene una condición del tipo *If... Then*, generada a partir de una dimensión, se creará una nueva norma de la metarregla con un periodo comprendido entre:
 - El día inmediatamente posterior a la fecha de fin de la norma de la metarregla
 - La fecha de fin de la nueva norma del elemento
 - Si la norma de la metarregla no contiene una condición del tipo *If... Then*, su periodo de validez se ampliará, pasando la fecha de fin hasta el día de fin de la nueva norma de la regla.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 1 y el 30 de Noviembre de 2000.

La regla tiene una dimensión con dos condiciones:

- La primera tiene un periodo de validez entre el 13 y el 23 de Noviembre, e indica que la regla sólo se debe ejecutar en ese periodo si el campo ID_LEVEL recoge un valor igual a 8.
- La segunda tiene un periodo de validez entre el 20 y el 30 de Mayo, e indica que la regla sólo se debe ejecutar si el campo DT_START es posterior al 20-11-1999.

Cuando se creó esta regla se generaron tres normas para la metarregla a partir de esta dimensión. La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	1-11-2000	12-11-2000	'Autogenerated System Code. Should not be modified <pre> X = A()..Regla_1 return (X) </pre>
Metarregla	13-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified <pre> If (ID_LEVEL=8) Then X = A()..Regla_1 End If return (X) </pre>
Metarregla	20-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified <pre> If (ID_LEVEL=8 And DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X) </pre>
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified <pre> If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X) </pre>
Regla_1	01-11-2000	30-11-2000	Código de la Regla_1

Posteriormente se crea una nueva norma para la «Regla 1», con un periodo comprendido entre el 15 de Noviembre y el 15 de Diciembre.

El periodo de la nueva norma se solapará con las normas de la metarregla existentes, con una de ellas (la última) parcialmente.

Tras actualizar las normas de la regla, se actualizarán las de la metarregla. En este caso se tendría que actualizar la norma de la metarregla más reciente que tiene la fecha de fin anterior a la fecha de fin de la nueva norma.

Como esta norma contiene una condición de tipo *If...Then*, el sistema creará una nueva norma para la metarregla, que comprenderá del 1 al 15 de Diciembre.

En el caso de que la norma de la metarregla no contuviese ninguna condición de tipo *If...Then*, únicamente cambiaría su fecha de fin a la fecha de fin de la nueva norma.

La siguiente tabla recoge la situación final:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	1-11-2000	12-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Metarregla	13-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)
Metarregla	20-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8 And DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Metarregla	01-12-2000	15-12-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Regla_1	01-11-2000	14-11-2000	Código de la Regla_1 - Norma 1
Regla_1	15-11-2000	15-12-2000	Código de la Regla_1 - Norma 2

- Si el periodo de validez de la nueva norma de la regla:
 - Se solapa totalmente con una o más normas de la metarregla, y parcialmente con una de las normas existentes de la metarregla.
 - Tiene una fecha de inicio anterior a la fecha de inicio de la norma de la metarregla más antigua:
 - Si esta norma de la metarregla contiene una condición del tipo *If... Then* generada a partir de una dimensión, se creará una nueva norma de la metarregla con un periodo comprendido entre:
 - La fecha de inicio de la nueva norma del elemento
 - El día inmediatamente anterior a la fecha de inicio de la norma de la metarregla

- Si la norma de la metarregla no contiene una condición del tipo *If... Then*, su periodo de validez se ampliará, pasando la fecha de inicio hasta el día de inicio de la nueva norma de la regla.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 1 y el 30 de Noviembre de 2000.

La regla tiene una dimensión con dos condiciones:

- La primera tiene un periodo de validez entre el 13 y el 23 de Noviembre, e indica que la regla sólo se debe ejecutar en ese periodo si el campo ID_LEVEL recoge un valor igual a 8.
- La segunda tiene un periodo de validez entre el 20 y el 30 de Noviembre, e indica que la regla sólo se debe ejecutar si el campo DT_START es posterior al 20-11-1999.

Cuando se creó esta regla se generaron tres normas para la metarregla a partir de esta dimensión.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	1-11-2000	12-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Metarregla	13-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)
Metarregla	20-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8 And DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Regla_1	01-11-2000	30-11-2000	Código de la Regla_1

Posteriormente se crea una nueva norma para la «Regla 1», con un periodo comprendido entre el 12 de Octubre y el 15 de Diciembre.

El periodo de la nueva norma se solapará con las normas de la metarregla existentes.

Tras actualizar las normas de la regla, se actualizarán las normas de la metarregla. En este caso se tendría que actualizar la norma de la metarregla más antigua, que tiene una fecha de inicio posterior a la fecha de inicio de la nueva norma.

Como esta norma no contiene una condición de tipo *If...Then*, se modificará su periodo de validez, cambiando su fecha de inicio por la fecha de inicio de la nueva norma: el 12 de Octubre.

La siguiente tabla recoge la situación final:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	12-10-2000	12-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Metarregla	13-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)
Metarregla	20-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8 And DT_START>'20-05-99') Then X = A()..Regla_1 End If return (X)
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-05-99') Then X = A()..Regla_1 End If return (X)
Regla_1	12-10-2000	30-11-2000	Código de la Regla_1

En el caso de que la norma de la metarregla contuviese una condición de tipo *If...Then*, se crearía una nueva norma para la metarregla con un periodo de validez comprendido entre la fecha de inicio de la nueva norma de la regla, y el día inmediatamente anterior a la fecha de inicio de la norma de la metarregla más antigua.

Los dos últimos casos se pueden dar simultáneamente, siempre que el periodo de validez de la nueva norma de la regla comprenda la totalidad de los periodos de validez de las normas de la metarregla.

Creación de nuevas reglas sin dimensiones

Siempre que se crea la primera regla de un elemento de tipo método o concepto el sistema genera automáticamente una metarregla. El periodo de esta metarregla coincidirá con el que se haya indicado para la primera regla.

Si en un momento posterior se crea una segunda regla para el concepto, con su propio periodo de validez, el sistema regenerará la definición de esta metarregla.

Concretamente, se realizarán dos cambios:

- Se modificará su periodo de validez para que, dentro de él, quede comprendido el periodo de validez de la nueva regla.



Este primer cambio sólo se hará si el periodo de validez de la nueva regla sobrepasa al periodo de validez de la metarregla.

- Se incluirá una llamada a la nueva regla en el código LN4 de la metarregla. El orden en el que se hagan las llamadas a las reglas dependerá de lo que se haya indicado en **Orden ejecución** de las reglas.

EJEMPLO

Se crea un nuevo método, con su correspondiente regla. Se llama a esta «Regla 1». El periodo de validez de la regla comprende desde el 1 de Enero de 2000 al 30 de Abril de 2000.

Cuando se graba la regla, el sistema genera una metarregla con un periodo de validez comprendido entre estas dos mismas fechas. El código LN4 de la metarregla será similar al siguiente:

```
'Autogenerated System Code. Should not be 'modified
X = NombreElemento( )..Regla_1
return (X)
```

Posteriormente, se crea una nueva regla, «Regla 2», para el método, con un periodo de validez comprendido entre el 01 de Junio de 2000 al 31 de Julio de ese mismo año.

Al grabar esta nueva regla, se realizan los siguientes cambios en la metarregla creada inicialmente:

- Se modifica el periodo de validez de la metarregla, ampliándolo hasta el día 31 de Julio de 2000
- Se agrega una nueva condición al código LN4 de la metarregla, que se encargará de solicitar la ejecución de la nueva regla

El código de la metarregla quedaría finalmente de la siguiente forma:

```
'Autogenerated System Code. Should not be 'modified
X = NombreElemento( )..Regla_1
X = NombreElemento( )..Regla_2
return (X)
```



Si el periodo de validez de la segunda regla hubiese estado incluido dentro del de la metarregla, el periodo de validez de ésta no se hubiese modificado.

La ejecución de la metarregla únicamente devolverá el valor que se haya obtenido como resultado de ejecutar la segunda regla.

Cuando se ejecute este método, el sistema ejecutará la regla que esté marcada como metarregla. Esta metarregla se encargará de ejecutar, una tras otra, las reglas «Regla 1» y «Regla 2».

Estas reglas sólo se ejecutarán si la fecha de aplicación está comprendida dentro de sus periodos de validez. De esta forma, si la fecha de aplicación fuese por ejemplo el 20 de Abril de 2000, la «Regla 2» no se ejecutaría. En este caso, como los periodos de validez no se solapan, no es posible que lleguen a ejecutarse las dos reglas en una misma ejecución del elemento.

Es importante señalar lo que sucedería si hubiese más de una norma para la metarregla del método. En este caso, se actualizarán todas las normas de la metarregla cuyo periodo de validez comprenda total o parcialmente el periodo de validez de la nueva regla.

Se realizarán dos cambios:

- Se agregará una llamada a la nueva regla en el código LN4 de todas las normas de la metarreglas que contengan, total o parcialmente, el periodo de validez de la nueva regla.
- Si la fecha de fin del periodo de validez de la nueva regla supera el periodo de validez de la norma de la metarregla con la fecha de fin posterior, se actualizará la fecha de fin de esta norma para igualarla a la fecha de fin de la nueva regla.
- Si la fecha de inicio del periodo de validez de la nueva regla es anterior a la fecha de inicio más baja de todas las normas de la metarregla, se actualizará la fecha de inicio de la norma más antigua para igualarla a la fecha de inicio de la nueva regla.

EJEMPLO

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 1 y el 30 de Noviembre de 2000. La regla tiene una dimensión con dos condiciones:

- La primera tiene un periodo de validez entre el 13 y el 23 de Noviembre, e indica que la regla sólo se debe ejecutar en ese periodo si el campo ID_LEVEL recoge un valor igual a 8
- La segunda tiene un periodo de validez entre el 20 y el 30 de Noviembre, e indica que la regla sólo se debe ejecutar si el campo DT_START es posterior al 20-11-1999

Cuando se creó esta regla, se generaron tres normas para la metarregla a partir de esta dimensión.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	1-11-2000	12-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Metarregla	13-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)
Metarregla	20-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8 And DT_START>'20-05-99') Then X = A()..Regla_1 End If return (X)
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Regla_1	01-11-2000	30-11-2000	Código de la Regla_1

Posteriormente se crea una nueva regla llamada «Regla 2», con un periodo de validez comprendido entre el 15 de Noviembre y el 25 de Diciembre de 2000. Este periodo de validez está incluido parcialmente en los periodos de validez de tres normas de la metarregla.

Como el periodo de validez de la nueva regla sobrepasa los periodos de validez de las normas de la metarregla existentes, y la última norma de la metarregla contiene una condición *If...Then*, se harán los siguientes cambios:

1. Se creará una nueva norma para la metarregla, con un periodo de validez comprendido entre el día inmediatamente posterior a la fecha de fin de la última norma (el 1 de Diciembre de 2000) y la fecha de fin de la norma de la nueva regla (el 25 de Diciembre).
2. Cuando se grabe la regla, el sistema modificará estas tres normas de la metarregla, agregándoles una llamada a la nueva regla, «Regla 2». La llamada a la nueva regla no se verá afectada por las condiciones *If...Then* que contenga la metarregla y que se hayan generado por una dimensión de otra regla del elemento. Es decir, la ejecución de una regla sólo se ve condicionada por sus propias dimensiones.

El estado final queda recogido en la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	1-11-2000	12-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_1 return (X)
Metarregla	13-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8) Then X = A()..Regla_1 End If X=A()..Regla_2 return (X)
Metarregla	20-11-2000	23-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8 And DT_START>'20-11-99') Then X = A()..Regla_1 End If X=A()..Regla_2 return (X)
Metarregla	24-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If X=A()..Regla_2 return (X)
Metarregla	01-12-2000	25-12-2000	'Autogenerated System Code. Should not be modified X=A()..Regla_2 return (X)
Regla_1	01-11-2000	30-11-2000	Código de la Regla_1
Regla_2	15-11-2000	25-12-2000	Código de la Regla_2

Creación de reglas con dimensiones

El siguiente caso que se va a considerar para describir la forma en la que se actualizan las normas de las metarreglas es aquél en que se crea una regla con una dimensión para un elemento ya existente.

Si en la creación de una regla sin dimensiones, el sistema únicamente modificaba la definición de las normas de la metarregla ya existentes cuyos periodos de validez comprendiesen, total o parcialmente, al de la nueva regla, en la creación de reglas con dimensiones resulta un poco más complejo.

En primer lugar, se modifica el código LN4 de las normas de la metarregla existentes, cuyos periodos de validez comprendan, total o parcialmente, el periodo de validez de la nueva regla, agregando una llamada a la ejecución de la nueva regla. En este punto, el sistema se comporta de la misma forma que en el caso anterior.

Sin embargo, cuando se cree una regla con una dimensión, el sistema creará una o más normas para la metarregla. Los periodos de validez de estas normas se obtendrán a partir de los periodos de validez de las condiciones indicadas en la dimensión.

Como los periodos de validez de las normas de la metarregla no se pueden solapar, los periodos de validez de las normas de la metarregla previamente existentes que se solapan con los de las nuevas normas de la metarregla generadas por la nueva regla serán modificados.

Sus fechas de inicio o de fin se cambiarán para evitar su solapamiento con los periodos de validez de las nuevas metarreglas.



Los periodos de validez y el número de normas de la metarregla se crearán de acuerdo a lo indicado en un apartado anterior.

Por lo general, se puede decir que:

- Si la nueva norma se solapa parcialmente con una norma existente, y su fecha de inicio está incluida en el periodo de validez de la norma existente, la fecha de fin de la norma existente se cambiará al día inmediatamente anterior a la fecha de inicio de la nueva metarregla.
- Si la nueva norma se solapa parcialmente con una norma existente, y su fecha de fin está incluida en el periodo de validez de esta norma existente, la fecha de inicio de la norma existente se cambiará al día inmediatamente posterior a la fecha de fin de la nueva norma.
- En el caso de que el periodo de validez de la nueva norma esté incluido totalmente en el periodo de validez de una norma existente:

- La fecha de fin de la norma existente se cambiará por el día inmediatamente anterior a la fecha de inicio de la nueva norma
- Se generará una nueva norma con:
 - Una fecha de inicio igual al día inmediatamente posterior de la fecha de fin de la nueva norma
 - Una fecha de fin igual a la fecha de fin de la norma que ya existía con anterioridad
- Se generará una nueva norma que cubra el periodo comprendido entre las dos normas anteriores
- En todos los casos anteriores, en las normas previamente existentes cuyos periodos de validez incluyan total o parcialmente al periodo de validez de la nueva regla, se agregará una llamada a la ejecución de esta regla.

Si hay dimensiones asociadas a esta regla, las llamadas irán precedidas por la condición indicada por la dimensión válida en el periodo de tiempo cubierto por la norma.

El hecho de que se haga la llamada, no significa que la ejecución de esta regla vaya a tener efecto, ya que su periodo de validez puede ser un subperiodo del periodo de validez de la norma de la metarregla desde la que se le hace la llamada. La ejecución de la regla se hará únicamente si la fecha de aplicación seleccionada por el usuario está incluida en los periodos de validez de sus normas.

EJEMPLO

Para un método existe una metarregla con un periodo de validez comprendido entre el 1 de Julio de 2000 y el 31 de Julio de 2000. Esta metarregla se generó de forma automática al crear la primera regla del método, y únicamente contiene una llamada a esta regla, cuyo código identificativo es «Regla 1».

El código de esta metarregla sería:

```
'Autogenerated System Code.
'Should not be modified
X = ElementoId( )..REGLA_1
return (X)
```

Posteriormente se crea una segunda regla, con código identificativo «Regla 2», con un periodo de validez comprendido entre el 10 y el 20 de Julio de 2000.

Al grabar la regla, se actualiza la metarregla ya existente, agregando en su código LN4 una llamada a «Regla 2». No se modifica el periodo de validez de esta metarregla, ya que comprende en su totalidad al periodo de validez de la nueva regla.

El código actualizado de la metarregla quedaría como sigue:

```
'Autogenerated System Code.  
'Should not be modified  
  
X = ElementoId( )..REGLA_1  
X = ElementoId( )..REGLA_2  
  
return (X)
```

Cuando se solicita la ejecución del método, el sistema comprueba si la fecha de aplicación seleccionada por el usuario está incluido en el periodo de validez de la metarregla. En caso afirmativo, ejecuta ésta.

La metarregla le indica que debe ejecutar «REGLA 1» y «REGLA 2», en este orden. Para cada regla, antes de ejecutarla se comprobará si la fecha de aplicación está incluida en su periodo de validez. Si no está incluida no se ejecutará la regla.

De esta forma, si se seleccionase como fecha de aplicación el 15 de Julio de 2000, se ejecutarían las dos reglas.

Si se hubiese seleccionado como fecha de aplicación el 8 de Julio de 2000, únicamente se ejecutaría «Regla 1».

Apéndice: ejemplos de actualización de metarreglas

A continuación, mediante ejemplos, se describe cómo se comportaría el sistema cuando se crea una nueva regla con dimensiones. Es decir, cuando se tienen que modificar las normas de la metarregla existente por la creación de nuevas normas.

Ejemplo 1

Para un elemento A existe una «Regla 1» con un periodo de validez comprendido entre el 1 y el 30 de Noviembre de 2000.

La regla tiene una dimensión con dos condiciones:

- La primera tiene un periodo de validez entre el 1 y el 15 de Noviembre, e indica que la regla sólo se debe ejecutar en ese periodo si el campo ID_LEVEL recoge un valor igual a 8
- La segunda tiene un periodo de validez entre el 16 y el 30 de Noviembre, e indica que la regla sólo se debe ejecutar si el campo DT_START recoge un valor posterior al 20-11-1999

Cuando se creó esta regla se generaron dos normas para la metarregla a partir de esta dimensión.

La situación inicial la recoge la siguiente tabla:

ID	Inicio validez	Fin validez	Código metarregla
Metarregla	01-11-2000	15-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)
Metarregla	16-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Regla_1	01-11-2000	30-11-2000	Código de la Regla_1

A continuación, se crea una segunda regla para el elemento, «Regla 2», con un periodo de validez comprendido entre el 20 de Noviembre y el 31 de Diciembre de 2000.

Para esta regla se crea una dimensión con un periodo de validez que va del 25 de Noviembre al 15 de Diciembre. La condición comprueba que el campo DT_END recoge un valor anterior al 31 de Diciembre de 1999.

A partir de esta dimensión, se generarían las siguientes normas:

ID	Inicio	Fin	Código metarregla
Metarregla	20-11-2000	24-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_2 return (X)
Metarregla	25-11-2000	15-12-2000	'Autogenerated System Code. Should not be modified If (DT_END<'31-12-99') Then X = A()..Regla_2 End If return (X)
Metarregla	16-12-2000	31-12-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_2 return (X)
Regla_2	20-11-2000	31-12-2000	Código de la Regla_2

Entre estas normas y las ya existentes se producen solapamientos. Concretamente se solapan las tres normas que se indican a continuación.

ID	Inicio	Fin	Código LN4
Metarregla	16-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Metarregla	20-11-2000	24-11-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_2 return (X)
Metarregla	25-11-2000	15-11-2000	'Autogenerated System Code. Should not be modified If (DT_END<'31-12-99') Then X = A()..Regla_2 End If return (X)

Como las normas de una regla no pueden solaparse, el sistema hará una partición para obtener periodos no solapados.

Como resultado del proceso se obtendrán las siguientes normas:

ID	Inicio	Fin	Código LN4
Metarregla	16-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Metarregla	20-11-2000	24-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If X = A()..Regla_2 return (X)
Metarregla	25-11-2000	30-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If If (DT_END<'31-12-99') Then X = A()..Regla_2 End If return (X)
Metarregla	01-12-2000	15-12-2000	'Autogenerated System Code. Should not be modified If (DT_END<'31-06-99') Then X = A()..Regla_2 End If return (X)

Unidas al resto de normas con las que no se producen solapamientos, finalmente se tendría:

ID	Inicio	Fin	Código LN4
Metarregla	01-11-2000	15-11-2000	'Autogenerated System Code. Should not be modified If (ID_LEVEL=8) Then X = A()..Regla_1 End If return (X)

ID	Inicio	Fin	Código LN4
Metarregla	16-11-2000	19-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If return (X)
Metarregla	20-11-2000	24-11-2000	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If X = A()..Regla_2 return (X)
Metarregla	25-11-1999	30-11-1999	'Autogenerated System Code. Should not be modified If (DT_START>'20-11-99') Then X = A()..Regla_1 End If If (DT_END<'31-12-99') Then X = A()..Regla_2 End If return (X)
Metarregla	01-12-2000	15-12-2000	'Autogenerated System Code. Should not be modified If (DT_END<'31-12-99') Then X = A()..Regla_2 End If return (X)
Metarregla	16-12-2000	31-12-2000	'Autogenerated System Code. Should not be modified X = A()..Regla_2 return (X)
Regla_1	01-11-2000	30-11-2000	Código de la Regla_1
Regla_2	20-11-2000	31-12-2000	Código de la Regla_2

Como se puede apreciar, en el código de las normas previamente existentes cuyos periodos de validez se solapan total o parcialmente con los de las nuevas metarreglas, se agrega el código LN4 de las nuevas normas con las que se solapan.

Ejemplo 2

Se crea un método B y una regla con un periodo de validez comprendido entre el 1 de Julio de 2000 y el 31 de Julio de ese mismo año. La regla se llama «Regla 1».

La regla cuenta con una dimensión. La dimensión incluye dos condiciones.

La primera, con un periodo de validez que comprende desde el 1 al 22 de Julio de 2000, indica que la regla sólo se debe ejecutar si el campo SCO_N_LEVEL recoge un valor mayor o igual a 100.

La segunda condición, con un periodo de validez que comprende desde el 22 de Julio al 31 de Julio, indica que la regla sólo se debe ejecutar si el campo DT_START recoge una fecha anterior al 31 de Diciembre de 2000.

A partir de esta dimensión se generarán tres normas para la metarregla del elemento.

ID	Inicio	Fin	Código LN4
Metarregla	01-07-2000	22-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL >=100) Then X = A()..Regla_1 End If return (X)
Metarregla	22-07-2000	22-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL>=100 And DT_START<'31-12-00') Then X = A()..Regla_1 End If return (X)
Metarregla	23-11-2000	31-12-2000	'Autogenerated System Code. Should not be modified If (DT_START<'31-12-00') Then X = A()..Regla_1 End If return (X)
Regla_1	01-07-2000	31-12-2000	Código LN4 de Regla_1

Posteriormente se crea una nueva regla, «Regla 2», con un periodo de validez comprendido entre el 10 y el 20 de Julio de 2000.

Se distinguen dos casos:

- La nueva regla no cuenta con dimensiones (no se generarían nuevas normas para la metarregla)
- La nueva regla cuenta con dimensiones (se generarían nuevas normas para la metarregla)

La regla no tiene dimensiones

Únicamente se modificaría el código LN4 de la primera regla, agregándole una llamada a la «Regla 2»:

```
If SCO_N_LEVEL>=100 Then
    Elemento( )..Regla_1
End If
Regla_2
```

Las otras normas no se verían afectadas. El conjunto de normas resultado para el elemento sería:

ID	Inicio	Fin	Código LN4
Metarregla	01-07-2000	21-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL >=100) Then X = A()..Regla_1 End If X=A()..Regla_2 return (X)
Metarregla	22-07-2000	22-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL>=100 And DT_START<'31-12-00') Then X = A()..Regla_2 End If return (X)
Metarregla	23-11-2000	31-12-2000	'Autogenerated System Code. Should not be modified If (DT_START<'31-12-00') Then X = A()..Regla_2 End If return (X)
Regla_1	01-07-2000	31-07-2000	Código Regla_1
Regla_2	10-07-2000	20-07-2000	Código Regla_2

La nueva regla cuenta con dimensiones

Si se crean dimensiones para la nueva regla, se generarán normas para la metarregla.

Si, por ejemplo, se crea una dimensión para la «Regla 2» con una condición, la condición tiene un periodo de validez que va del 15 al 20 de Julio y comprueba que el campo SCO_TRAINING es igual a 10.

Para esta regla se generarían las siguientes normas de la metarregla:

ID	Inicio	Fin	Código LN4
Metarregla	10-07-2000	14-07-2000	'Autogenerated System Code. Should not be modified X=A()..Regla_2 return (X)
Metarregla	15-07-2000	20-07-2000	'Autogenerated System Code. Should not be modified If (SCO_TRAINING=10) Then X = A()..Regla_2 End If return (X)
Regla_2	10-07-2000	20-07-2000	Código Regla_2

En este caso, el sistema también actualizará las normas de la metarregla cuyos periodos de validez comprendan, total o parcialmente, el periodo de validez de las nuevas normas generadas para «Regla 2». Se aplicaría una partición como la descrita en los apartados anteriores.

Finalmente se obtendrían las siguientes normas:

ID	Inicio	Fin	Código LN4
Metarregla	01-07-2000	09-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL >=100) Then X = A()..Regla_1 End If return (X)
Metarregla	10-07-2000	14-11-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL >=100) Then X = A()..Regla_1 End If X=A()..Regla_2 return (X)

ID	Inicio	Fin	Código LN4
Metarregla	15-07-2000	20-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL >=100) Then X = A()..Regla_1 End If If (SCO_TRAINING=10) Then X = A()..Regla_2 End If return (X)
Metarregla	21-07-2000	21-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL >=100) Then X = A()..Regla_1 End If return (X)
Metarregla	22-07-2000	22-07-2000	'Autogenerated System Code. Should not be modified If (SCO_N_LEVEL>=100 And DT_START<'31-12-99') Then X = A()..Regla_1 End If return (X)
Metarregla	23-07-2000	31-07-2000	'Autogenerated System Code. Should not be modified If (DT_START<'31-12-99') Then X = A()..Regla_1 End If return (X)
Regla_1	01-07-2000	31-07-2000	Código LN4 de Regla_1
Regla_2	10-07-2000	20-07-2000	Código LN4 de Regla_2

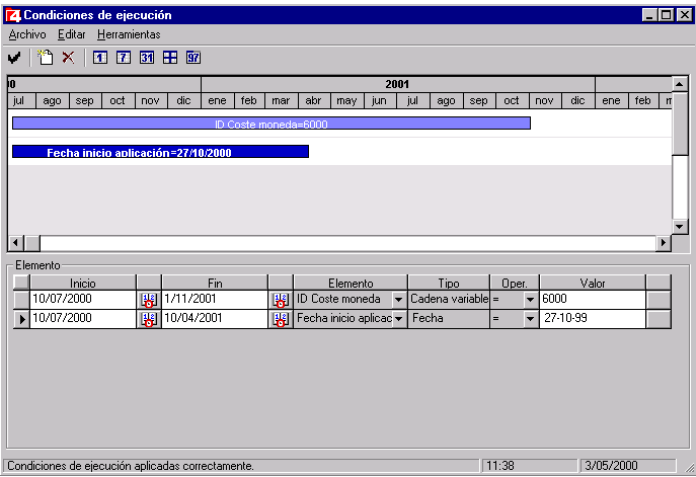
Ventajas de trabajar con etiquetas

Las condiciones que se expresan mediante las condiciones de una dimensión se podrían indicar en el código de la regla asociada al método o al concepto, mediante sentencias condicionales *If... Then*.

Con la separación del código genérico de las reglas y sus condiciones de ejecución, se optimiza el tiempo de ejecución de las reglas, ya que el intérprete no tiene que reconocer código innecesario.

Además, la definición de etiquetas simplifica la creación de reglas, ya que no será necesario editar el código de las fórmulas para poder hacer ejecuciones condicionales.

Figura 74. Diseñador gráfico de dimensiones



Antes de leer el código de una regla, el intérprete del lenguaje LN4 comprobará si se cumplen las condiciones indicadas en la dimensión asociada a la regla.

Si al menos una de las condiciones no se cumple, el intérprete continuará el flujo de ejecución sin necesidad de interpretar reglas innecesarias.

Conclusiones

Entre los puntos clave que se pueden obtener con la lectura de este manual se encuentran:

- Los elementos con ejecución (métodos y conceptos) implementan la funcionalidad de la aplicación.
- Estos elementos contienen código ejecutable escrito en lenguaje LN4, llamadas a métodos de DLL externas o llamadas a recursos C++.
- Las herramientas Meta4[®] facilitan un entorno gráfico para la creación y el mantenimiento de estos elementos.
- LN4 es un lenguaje de programación desarrollado y soportado por Meta4[®], para utilizar en las aplicaciones Meta4[®]. El lenguaje LN4 es independiente de plataformas y sistemas gestores de bases de datos. Su código es transferible entre distintas instalaciones.
- Todas las reglas de un elemento tienen uno o varios periodos de validez, a los que se denominan normas. Cada norma puede tener un código diferente. Los periodos de validez de una misma regla no pueden solaparse.
- Las metarreglas son un tipo especial de reglas que genera el sistema y que se ejecutan siempre que se solicita la ejecución del elemento mediante la sintaxis Elemento(arg) o desde la interfaz de la aplicación.
- Las metarreglas contienen llamadas a otras reglas que no se han marcado como metarreglas. En ocasiones, contendrán llamadas a reglas precedidas por una o varias condiciones, de forma que sólo se llamará a la regla si se cumple la condición.
- Cuando el usuario inicia una sesión de trabajo con la aplicación, puede indicar si quiere hacer una ejecución con tramos o sin tramos.
- Si hace una ejecución sin tramos deberá indicar una fecha de aplicación. Si no se indica esta fecha, el sistema tomará la fecha de hoy.
- Cuando se solicite la ejecución de un elemento de tipo método, el sistema ejecutará únicamente la metarregla cuyo periodo de validez comprenda la fecha de aplicación indicada por el usuario. Esto es siempre así en los métodos (se haga una ejecución con o sin tramos)

- Cuando se solicita la ejecución de un elemento de tipo concepto, se distinguen dos casos:
 - Se hace una ejecución sin tramos
El sistema ejecuta la metarregla cuyo periodo de validez comprende la fecha de aplicación
 - Se hace una ejecución con tramos
El sistema ejecuta todas las metarreglas cuyos periodos de validez estén comprendidos total o parcialmente en el periodo de ejecución seleccionado por el usuario
- Las dimensiones son condiciones de ejecución que se pueden definir para cada una de las reglas. Una dimensión incluye una o más condiciones que deben cumplirse para que se ejecute la regla.
- Las condiciones incluidas en una dimensión tienen un periodo de validez. En cuanto se supere este periodo, la condición dejará de ser evaluada antes de ejecutar la regla.
- Siempre que se diseñan dimensiones para una regla, el sistema generará una o más normas para la metarregla de esa regla.
- El sistema actualiza las normas de la metarregla de un elemento de forma automática, para mantener la integridad en la lógica de la aplicación.

Elementos tramados

9

Introducción

El cálculo con tramos es una de las principales características de las aplicaciones desarrolladas por Meta4.

Los tramos permiten reflejar en los procesos de cálculo los cambios que se producen en los valores de los datos conforme pasa el tiempo.

El motor de cálculo interpreta estas variaciones, de forma que:

- Se evita cualquier pérdida de información en los datos que se toman como base de los cálculos.
- Los resultados de cálculos retrospectivos se pueden interpretar con mayor exactitud y nivel de detalle.
- Se mantiene la fidelidad a la información histórica.

Las aplicaciones desarrolladas por Meta4 permiten trabajar con tramos de forma transparente para el usuario final. Para activar el cálculo con tramos, sólo hay que indicarlo desde la interfaz del programa.

En los siguientes apartados se explica:

- El concepto de tramo y cómo se interpreta en los distintos tipos de elementos (conceptos, campos y propiedades).
- La ejecución con tramos y las fechas de trabajo del sistema.
- Cómo determinar el comportamiento de los elementos en los cálculos con tramos.
- Las funciones que incorpora el lenguaje LN4 para trabajar con tramos.

Introducción a tramos

Existen cuatro tipos de elementos en la arquitectura de Meta4: métodos, propiedades, campos y conceptos. Los métodos son elementos que sólo tienen reglas de ejecución y no pueden almacenar ningún valor. Las propiedades son elementos que pueden almacenar un valor pero no tienen reglas de ejecución ni están asociados a un campo de base de datos. Los campos son elementos que pueden almacenar un valor, no tienen reglas de ejecución y si que están asociados a un campo de base de datos. Los conceptos son elementos que pueden almacenar un valor y tienen reglas de ejecución.

En un caso normal, un elemento que pueda almacenar valores, es decir, una propiedad, un campo o un concepto, sólo almacena un único valor. Si su ámbito es registro un valor por registro, si es de bloque un valor por bloque y si es de nodo un valor por nodo.

En la mayoría de los procesos que se modelizan a través de la arquitectura de Meta4 este comportamiento es suficiente. Esto se debe a que bien el proceso que se modeliza no depende del momento en el que se define ni del momento en el que el usuario lo utiliza, o bien la modelización está preparada para trabajar sólo en un punto determinado del tiempo, que normalmente es el momento en el que el usuario utiliza el sistema.

Sin embargo, hay otros procesos para los cuales no es suficiente con almacenar sólo valores atemporales. Estos son los procesos que dependen del momento en el que se definen y además no se trabaja en un momento determinado del tiempo, sino en un intervalo del tiempo. El caso más importante se tiene en el cálculo de nómina, en el que el proceso abarca normalmente un mes y los valores y reglas que se utilizan dependen del mes en concreto que se esté procesando.

Como resultado, para poder definir correctamente estos procesos, se necesita poder almacenar más de un valor en un mismo elemento. Y además se necesita poder definir en qué periodos del tiempo es válido cada uno de los valores almacenados en el elemento.

La arquitectura Meta4 presenta esta característica y permite almacenar valores temporales en los elementos de tipo propiedad, campo y concepto; pero no en los elementos de tipo método ya que estos no pueden almacenar ningún valor. De los elementos que almacenan valores temporales se dice que son elementos que están tramados. Un elemento tramado almacena una o varias triadas de valores, donde cada triada viene definida:

- por un valor,
- por la fecha de inicio de validez de ese valor,
- y por la fecha de fin de validez de ese valor.

Cada una de estas triadas se denomina tramo.

La arquitectura de Meta4 proporciona las herramientas y las funciones necesarias para parametrizar procesos que requieran el empleo de tramos y para su posterior ejecución.

Parametrizar un proceso para que saque el máximo partido a la característica de tramos es, lógicamente, más complicado que hacerlo sin usar esta característica. Esto es debido a que un proceso que necesite parametrización temporal es intrínsecamente más complejo de entender que no lo necesita y además es necesario seguir una serie de pasos adicionales, que se explicarán más adelante, para definir el proceso correctamente usando las herramientas de Meta4. Sin embargo, si se tiene un buen conocimiento del proceso a modelizar y de las herramientas de Meta4 para trabajar con tramos y sigue un cierto orden y planificación a la hora de definir el proceso con tramos, resultará una tarea asequible en tiempo y esfuerzo.

Si un proceso se parametriza correctamente para que trabaje con tramos se podría hacer, de manera sencilla, que en un momento determinado se ejecutara sin tener en cuenta los tramos. El contrario no es cierto. Un proceso que no esté parametrizado para trabajar con tramos no basta con activar el modo tramos para que el proceso con tramos se realice correctamente.

En cualquiera de los casos, la parametrización de un proceso tramado es (o debería ser) algo transparente para el usuario final del proceso, recayendo toda la responsabilidad de la correcta parametrización en el implantador del proceso.

Parametrización de tramos

Antes de que el usuario final pueda utilizar un proceso tramado, es necesario que un implantador haya definido correctamente el proceso. La definición de un proceso tramado implica la definición de las normas, parametrización del elemento y codificación en LN4.

Esta parametrización permite al implantador determinar varios aspectos:

- qué tramos se crean,
- cómo se realiza la lectura sobre elementos tramados,
- y cómo se compactan los elementos tramados.

Definición de las normas

Cuando se definen las reglas de elementos ejecutables, es decir, de tipo método y concepto, hay que tener en cuenta que estas reglas están formadas por un conjunto de normas.

Son las normas las que realmente contienen el código ejecutable y cada norma tiene asociado un periodo de validez de su código. Es decir, cada norma viene definida, entre otras cosas, por su código, su fecha de inicio de validez y su fecha de fin de validez. Las normas están pensadas para poder reflejar los cambios de comportamiento de las reglas de un elemento a lo largo del tiempo.

Cuando se va a ejecutar un elemento de tipo método o concepto sin estar trabajando con tramos la aplicación de Meta4 elige de forma automática la norma (una y sólo una) que se debe ejecutar basándose en la fecha de trabajo (que se explica más adelante en el apartado Activación de tramos) y que normalmente es la fecha en la que se está ejecutando el proceso.

En los procesos con tramos es importante definir correctamente las normas de una regla. No sólo ya el código de la norma sino también sus fechas de inicio y fin de validez, ya que estas fechas son unos de los criterios que se emplearán para calcular el número de periodos en los que se van a ejecutar los elementos de tipo concepto.

Cuando se va a ejecutar un elemento de tipo método trabajando con tramos, el comportamiento es el mismo que en el caso de no estar trabajando con tramos. Esto es así porque un elemento de tipo método no puede almacenar valores y por lo tanto no puede tener tramos.

Cuando se va a ejecutar un elemento de tipo concepto trabajando con tramos, la aplicación de Meta4 ejecuta el concepto tantas veces como tramos vaya a tener. Existen varios criterios que se emplean para obtener esos periodos. Uno de ellos son las fechas de las normas del concepto que están contenidas total o parcialmente en las fechas de inicio y de fin de trabajo (que se explican más adelante en el apartado Activación de tramos).

El hecho de que las reglas tengan normas es independiente de que se esté parametrizando un proceso con tramos o no. Así, se podría tener un elemento ejecutable cuya regla no depende del tiempo en un proceso con tramos, y por lo tanto sólo tendría una norma cuya validez va desde menos infinito a más infinito. También se podría tener un elemento ejecutable cuya regla depende del tiempo en un proceso sin tramos, y por lo tanto tendría varias normas en los distintos periodos de tiempo. La diferencia es que en un proceso sin tramos sólo se ejecuta una de las posibles normas, que es la que contiene a la fecha de trabajo. Sin embargo, en un proceso con tramos se puede ejecutar una o varias de las normas de las reglas, que son las que están contenidas total o parcialmente en las fechas de inicio y de fin de trabajo.

Parametrización del elemento

Existen una serie de propiedades avanzadas a nivel de elemento que permiten parametrizar el comportamiento del elemento frente a operaciones con tramos.

Estas propiedades son: división en tramos, elemento de tramos y comportamiento en tramos.

División en tramos y elemento de tramos

Estas dos propiedades permiten definir que tramos se van a crear al ejecutar un elemento de tipo concepto.

Los posibles valores de la propiedad División en tramos son: lineal, no lineal y sin tramos.

Para entender el significado de estos valores es necesario tener claro en concepto de dependencias en LN4.

Cuando en el código LN4 de una regla de un elemento de tipo método o concepto se hace referencia a otro elemento diremos que existe una dependencia entre el primer elemento y el segundo. En concreto, se dice que el primer elemento depende del segundo.

Las dependencias pueden ser de tres tipos: de lectura (o uso), de escritura (o asignación) y de ejecución.

Las primeras, o de lectura, se producen cuando en la fórmula del primer elemento se utiliza el valor del segundo elemento. Lógicamente el segundo elemento debe ser de tipo propiedad, campo o concepto. Si x es una variable local y ELEMENTO_SEGUNDO es el segundo elemento:

$$x = \text{ELEMENTO_SEGUNDO}$$

Las segundas, o de escritura, se producen cuando en la fórmula del primer elemento se cambia el valor del segundo elemento. Lógicamente el segundo elemento debe ser de tipo propiedad, campo o concepto. Si x es una variable local y ELEMENTO_SEGUNDO es el segundo elemento:

$$\text{ELEMENTO_SEGUNDO} = x$$

Las terceras, o de ejecución, se producen cuando en la fórmula del primer elemento se invoca la ejecución del segundo elemento. Lógicamente el segundo elemento debe ser de tipo método o concepto. Si x es una variable local y ELEMENTO_SEGUNDO es el segundo elemento:

$$x = \text{ELEMENTO_SEGUNDO}()$$

Cuando se define que un elemento de tipo concepto es lineal significa que cuando se vaya a ejecutar y se determine cuantos tramos va a tener el concepto se tendrán en cuenta los tramos ya creados en todos los demás elementos de los que tenga dependencia de lectura. Las dependencias de lectura se calculan a partir de todas las normas del elemento que se va a ejecutar, independientemente de las fechas de esas normas.

Lo que hace la aplicación de Meta4 es calcular de manera automática la intersección mínima de todos los tramos de los elementos de los que se depende. Esos periodos son uno de los criterios que se emplean para determinar cuantos tramos va a tener el elemento que se va a ejecutar.

Cuando se define que un elemento de tipo concepto es no lineal significa que cuando se vaya a ejecutar y se determine cuantos tramos va a tener el concepto se tendrán en cuenta los tramos ya creados en otro elemento. Este otro elemento debe ser lógicamente de tipo propiedad, campo o concepto.

Para definir cual es este otro elemento hay que rellenar la propiedad avanzada Elemento de tramos. Esta propiedad sólo se utiliza cuando se haya definido que el elemento es de división en tramos no lineal. En cualquier otro caso no hace falta rellenar esta propiedad y si se rellena su valor es ignorado.

Cuando se define que un elemento de tipo concepto es sin tramos significa que cuando se vaya a ejecutar y se determine cuantos tramos va a tener el concepto no se tendrán en cuenta los tramos ya creados de otros elementos. Hay que notar que esto no significa que el elemento no vaya a tener tramos y se ejecute tantas veces como tramos vaya a tener. Sólo significa que los tramos ya creados de otros elementos no constituyen un criterio para la determinación de los tramos que va a tener, pero hay otros criterios (que se verán en el

apartado Creación de tramos) que si que podrían hacer que el elemento acabe ejecutándose de manera tramada.

Comportamiento en tramos

Esta propiedad permite definir cómo se va a comportar un elemento frente a operaciones de lectura y compactación.

Los posibles valores de esta propiedad son: valor base (o precio), valor final (o importe), unidad e incidencia.

El comportamiento detallado de estos distintos valores se puede ver en profundidad en los apartados Lectura de tramos y Compactación de tramos.

Codificación en LN4

Hay veces que la parametrización anterior no es suficiente para conseguir el efecto deseado y la parametrización de tramos requiere de ajustes más específicos. Estos ajustes pueden realizarse en gran medida a través de código LN4, ya que en este lenguaje se dispone de un amplio juego de funciones que permiten el manejo avanzado de los tramos.

Sin explicarlo en detalle, porque ya se irán viendo estas funciones en apartados sucesivos, aquí sólo enumeraremos el conjunto de operaciones que se pueden hacer desde LN4.

- Activar, desactivar y consultar el modo de tramos.
- Establecer y consultar las fechas de trabajo.
- Consultar el número de tramos de un elemento, así como sus fechas de validez y sus valores.
- Crear nuevos tramos en un elemento y asignarles valor.
- Dividir los tramos de un elemento en más tramos.
- Totalizar los tramos de un elemento.
- Invocar funciones que permiten pasar del formato tramos a formato registro y viceversa.

Activación de tramos

Para que la parametrización de los tramos tenga efecto cuando el usuario final ejecute el proceso es necesario que el modo de tramos esté activado y que las fechas de trabajo estén correctamente establecidas.

Modo de tramos

El modo de tramos puede tener dos valores: activado y desactivado. Por defecto el modo de tramos está desactivado.

Se puede activar y desactivar el modo de tramos de un Meta4Object desde LN4, mediante la función `SetSliceMode`, que recibe un parámetro que si es `M4_TRUE` indica que se active el modo de tramos para ese Meta4Object y si es `M4_FALSE` se desactive.

```
SetSliceMode( M4_TRUE )
SetSliceMode( M4_FALSE )
```

Cuando se activa el modo de tramos se hace para un Meta4Object y afecta a todas las operaciones que a partir de ese momento se realicen con ese Meta4Object. Cambiar el modo tramos de un Meta4Object no implica que todos los Meta4Objects que esté utilizando por nivel dos cambien su modo de tramos. Para que cambien su modo tramos hay que ir uno a uno cambiándolo.

El modo de tramos de un Meta4Object se puede consultar desde LN4 mediante la función `GetSliceMode`, que devuelve `M4_TRUE` si el modo tramos está activado y `M4_FALSE` si el modo tramos está desactivado.

```
x = GetSliceMode()
```

Fechas de trabajo

Las fechas de trabajo son un conjunto de fechas que determinan en que momento o momentos del tiempo deben ejecutarse las reglas de un elemento de tipo método o concepto. Esto significa que estas fechas permiten que la aplicación de Meta4 elija cuál o cuales de las normas de una regla deben ejecutarse.

Existen tres fechas de trabajo: fecha de trabajo, fecha de inicio de trabajo y fecha de fin de trabajo.

Cuando el modo de tramos está desactivado sólo la fecha de trabajo tiene efecto sobre las operaciones que se realicen en el Meta4Object. En ese caso, la fecha de inicio de trabajo y la fecha de fin de trabajo no se utilizan y da igual

el valor que puedan tener. Bajo este supuesto, cuando se va a ejecutar un elemento de tipo método o concepto, la aplicación de Meta4 emplea la fecha de trabajo para encontrar la norma (una y sólo una o ninguna) que se debe ejecutar. Esa norma será aquella que contenga a la fecha de trabajo, es decir, cuya fecha de inicio sea menor o igual que la fecha de trabajo y cuya fecha de fin sea mayor o igual que la fecha de trabajo. Si no existe una norma que cumpla tal condición no se ejecutará ninguna norma.

Cuando el modo de tramos está activado las tres fechas tienen efecto sobre las operaciones que se realicen en el Meta4Object.

En este caso, cuando se va a ejecutar un elemento de tipo método, la aplicación de Meta4 emplea la fecha de trabajo para encontrar la norma (una y sólo una o ninguna) que se debe ejecutar. Esa norma será aquella que contenga a la fecha de trabajo, es decir, cuya fecha de inicio sea menor o igual que la fecha de trabajo y cuya fecha de fin sea mayor o igual que la fecha de trabajo. Si no existe una norma que cumpla tal condición no se ejecutará ninguna norma.

Cuando se va a ejecutar un elemento de tipo concepto, la aplicación de Meta4 emplea las fechas de inicio de trabajo y la fecha de fin de trabajo para determinar el conjunto de normas que se deben ejecutar. Esas normas serán aquellas que estén contenidas total o parcialmente en las fecha de inicio de trabajo y la fecha de fin de trabajo, es decir, aquellas cuya fecha de inicio sea menor o igual que la fecha de fin de trabajo y cuya fecha de fin sea mayor o igual que la fecha de inicio de trabajo.

Aparte de servir para que la aplicación determine las normas que deben ejecutarse, las fechas de inicio de trabajo y de fin de trabajo sirven como límites inferior y superior respectivamente para los tramos que se pueden generar en elementos de tipo propiedad, campo o concepto. Es decir, que ningún elemento puede tener valores tramados cuya fecha de validez no esté incluida en las fechas de inicio y de fin de trabajo.

Las fechas de trabajo de un Meta4Object se pueden modificar desde LN4, mediante las funciones SetRunDate, SetStartDate, SetEndDate, que reciben un parámetro que es la fecha que se quiere establecer.

```
SetRunDate( {2003-11-25} )  
SetStartDate( {2003-11-01} )  
SetEndDate( {2003-11-30} )
```

Cuando se cambian las fechas de trabajo desde LN4 se hace para un Meta4Object y afecta a todas las operaciones que a partir de ese momento se realicen con ese Meta4Object. Cambiar las fechas de trabajo de un Meta4Object no implica que todos los Meta4Objects que esté utilizando por nivel dos cambien sus fechas de trabajo. Para que cambien sus fechas de trabajo hay que ir uno a uno cambiándolas.

Las fechas de trabajo se pueden consultar desde LN4 mediante las funciones `GetRunDate`, `GetStartDate`, `GetEndDate`, que devuelven la fecha que se quiere consultar.

```
x = GetRunDate()
y = GetStartDate()
z = GetEndDate()
```

Existen unas funciones similares que también permiten consultar estas fechas.

```
x = AppRunDate()
y = AppStartDate()
z = AppEndDate()
```

A diferencia del modo de tramos, las fechas de trabajo pueden cambiarse a nivel de sesión. Para ello, hay que asignar los valores deseados a los elementos de tipo propiedad `EXECUTION_DATE`, `EXECUTION_START_DATE` y `EXECUTION_END_DATE` del nodo `ROOT_SESSION` del `Meta4Object` sesión. A partir de ese momento, todos los `meta4Object` que se creen tendrán esas fechas de trabajo por defecto. Cambiar las fechas de trabajo a nivel de sesión no tienen efecto sobre los `Meta4Objects` ya creados ni sobre el `Meta4Object` desde el que se haga el cambio de esos elementos (si es que se hace mediante LN4). Por defecto, los valores de esos elementos del `Meta4Object` sesión son: la fecha actual cuando se crea la sesión para la fecha de trabajo, menos infinito para la fecha de inicio de trabajo y más infinito para la fecha de fin de trabajo.

Hay que hacer notar que si la ejecución se realiza en cliente servidor y el cambio de los elementos de fechas del `Meta4Object` sesión se realiza en la parte cliente es necesario invocar al método `COMFIRM` del `Meta4Object` sesión para que esos cambios tengan efecto en la parte servidora.

Fechas de ejecución

A parte de las fechas de trabajo, también es interesante conocer el concepto de fechas de ejecución. Existen tres fechas de ejecución: la fecha de ejecución, la fecha de inicio de ejecución y la fecha de fin de ejecución.

Como ya se ha visto, y se verá con más detalle en el apartado Creación de tramos, cuando se va a ejecutar un elemento de tipo concepto la aplicación de `Meta4` determina de manera automática los tramos que ese concepto va a tener. Para cada uno de esos tramos la aplicación invocará la ejecución de la norma correspondiente al periodo de ese tramo.

La ejecución de esa norma se realizará con fecha inicio de ejecución la fecha de inicio del tramo, con fecha de fin de ejecución la fecha de fin del tramo y con fecha de ejecución la fecha de trabajo.

Estas fechas son las que se emplean cuando desde la norma que se está ejecutando se lee el valor de otro elemento, se asigna el valor de otro elemento o se ejecuta otro elemento. Cuando se lee el valor de otro elemento de tipo propiedad, campo o concepto se utilizan la fecha de inicio de ejecución y la fecha de fin de ejecución para leer el valor en esas fechas (ver apartado Lectura de tramos). Cuando se asigna el valor de otro elemento de tipo propiedad, campo o concepto se utilizan la fecha de inicio de ejecución y la fecha de fin de ejecución para asignar el valor en esas fechas y crear el correspondiente tramo si fuese necesario (ver apartado Creación de tramos). Cuando se ejecuta otro elemento de tipo método o concepto se utilizan la fecha de ejecución, la fecha de inicio de ejecución y la fecha de fin de ejecución para crear la ejecución de ese elemento.

Las fechas de ejecución se pueden consultar desde LN4 mediante las funciones `GetRunningRunDate()`, `GetRunningStartDate()`, `GetRunningEndDate()`, que devuelven la fecha que se quiere consultar.

```
x = GetRunningRunDate()  
y = GetRunningStartDate()  
z = GetRunningEndDate()
```

Creación de tramos

Existen múltiples maneras y circunstancias por las cuales se pueden generar tramos en un elemento de tipo propiedad, campo o concepto. Todas ellas sólo se producen si el modo de tramos está activado. Estas maneras son: al acabar la ejecución de un concepto, al asignar otros elementos, en la ejecución de funciones LN4 de nómina y directamente desde atributos método de LN4.

Ejecución de un concepto

Como ya se ha visto, cuando se va a ejecutar un elemento de tipo concepto, si el modo de tramos está activado, la aplicación de Meta4 determina de manera automática cual es el conjunto de tramos que ese concepto va a tener. Para cada uno de esos tramos la aplicación genera una ejecución con las fechas del tramo correspondiente.

El conjunto de tramos calculado de un elemento depende del valor de la propiedad avanzada División en tramos (ver apartado Parametrización de tramos).

En todos los casos las fechas que se tienen en cuenta para determinar los tramos del elemento son aquellas que está comprendidas entre la fecha de inicio de trabajo y fecha de fin de trabajo, incluyendo estas dos fechas. Es decir, aquellos periodos cuya fecha de inicio sea menor que la fecha de fin de trabajo

y cuya fecha de fin sea mayor o igual que la fecha de inicio de trabajo. Si hay fechas de inicio que deban considerarse y son menores que la fecha de inicio de trabajo, se ignora esa fecha y se toma la fecha de inicio de trabajo. Igualmente si hay fechas de fin que deban considerarse y son mayores que la fecha de fin de trabajo, se ignora esa fecha y se toma la fecha de fin de trabajo.

En todos los casos el algoritmo también es el mismo. Los tramos que se van a obtener vienen dados por la intersección mínima de las fechas de todos los periodos que se consideran en cada caso. Es decir, que lo que se obtiene es un conjunto de tramos que cumplen que son el conjunto de menor número posible de tramos que está totalmente incluido en la unión de todos los periodos que se consideran.

Lineal

Si el concepto es lineal, se determinan sus tramos teniendo en cuenta:

- Las fechas de inicio y de fin de las normas del propio elemento.
- Las fechas de inicio y de fin de los tramos que ya tiene el propio elemento (si los tuviera).
- Las fechas de inicio y de fin de los tramos que ya tienen todos los elementos de los que tiene dependencias de lectura.
- Las fechas de inicio y de fin de las normas de todos los elementos de los que tiene dependencias de ejecución.

No lineal

Si el concepto es no lineal, se determinan sus tramos teniendo en cuenta:

- Las fechas de inicio y de fin de las normas del propio elemento.
- Las fechas de inicio y de fin de los tramos que ya tiene el propio elemento (si los tuviera).
- Las fechas de inicio y de fin de los tramos que ya tiene el elemento que se marca en la propiedad avanzada Elemento de tramos.
- Las fechas de inicio y de fin de las normas de todos los elementos de los que tiene dependencias de ejecución.

Sin tramos

Si el concepto es sin tramos, se determinan sus tramos teniendo en cuenta:

- Las fechas de inicio y de fin de las normas del propio elemento.
- Las fechas de inicio y de fin de los tramos que ya tiene el propio elemento (si los tuviera).
- Las fechas de inicio y de fin de las normas de todos los elementos de los que tiene dependencias de ejecución.

Asignación de elementos

Una manera directa de generar tramos es asignar un valor (estando el modo de tramos activado) a un elemento de tipo propiedad, campo o concepto. La creación del tramo se produce inmediatamente antes de hacer la asignación.

El proceso es el mismo, independientemente si la asignación tiene lugar desde LN4 de manera directa, desde LN4 de manera indirecta, por nivel 2, en algoritmos internos de funciones LN4 o en la asignación que se produce en los elementos de tipo concepto al acabar su ejecución.

Si la fecha de inicio de ejecución y la fecha de fin de ejecución coinciden exactamente con un tramo ya existente en el elemento que se asigna no se crea el tramo y simplemente se asigna.

Si la fecha de inicio de ejecución y la fecha de fin de ejecución no coinciden exactamente con un tramo ya existente en el elemento que se asigna se crea un tramo en esas fechas y se asigna el valor.

En el proceso de creación de un tramo nuevo es posible que haya tramos existentes que desaparezcan o tramos existentes que modifiquen su fecha de inicio o su fecha de fin y su valor. Desaparecerán aquellos tramos que estén completamente comprendidos en el nuevo tramo, es decir, cuya fecha de inicio sea mayor o igual que la fecha de inicio de ejecución y cuya fecha de fin sea menor o igual que la fecha de fin de ejecución. Modificarán sus fechas aquellos tramos que estén comprendidos parcialmente en el nuevo tramo, es decir, aquellos cuya fecha de inicio sea menor que la fecha de inicio de ejecución y cuya fecha de fin sea mayor o igual que la fecha de inicio de ejecución, o bien aquellos cuya fecha de fin sea mayor que la fecha de fin de ejecución y cuya fecha de inicio sea menor o igual que la fecha de fin de ejecución. En estos casos es posible que se tenga que modificar el valor de esos tramos. Esa modificación dependerá del tipo (número, cadena, o fecha) del elemento a asignar y del valor de su propiedad comportamiento en tramos (valor base, valor final, unidad o incidencia) y es llevada a cabo de manera automática por la aplicación de Meta4.

Funciones LN4 de nómina

Hay un conjunto de funciones LN4 que internamente hacen uso intensivo de la característica de tramos. Algunas de ellas pueden, según la parametrización, generar nuevos tramos. Estas funciones son: Unflatten, Flatten, JoinSlices, TransferApplyValue y YTDSearchFiltered. En este apartado se detalla el comportamiento de estas funciones relacionado con la creación de tramos. Para más información de la que aquí se detalla acudir a los manuales específicos de LN4 donde se describen estas funciones.

Unflatten

La función Unflatten está pensada para pasar un bloque con un conjunto de registros sin tramos a un sólo registro con tramos. Es lo que se conoce comúnmente como pasar de 2D a 3D.

La función Unflatten es de ámbito bloque y para que funcione correctamente deben estar definidos en el nodo los elementos con tipo interno fecha de inicio y fecha de fin.

Si se invoca a la función Unflatten sobre un bloque que no tiene registros no se hace nada y se retorna correctamente. Si sólo se tiene un registro y el modo de tramos no está activado no se hace nada pero se produce un warning. Si sólo se tiene un registro y el modo de tramos está activado se creará un registro con tramo.

En cualquier otro caso, cada registro pasa a formar un tramo, donde el valor del tramo es el valor del elemento en el registro, la fecha inicio del tramo es la que venga almacenada en el elemento con tipo interno fecha de inicio y la fecha fin del tramo es la que venga almacenada en el elemento con tipo interno fecha de fin. Es importante tener en cuenta que no puede haber solapamiento de fechas. Al finalizar la asignación de todos los tramos de un elemento se procede a la compactación si ésta fuese necesaria (ver apartado Compactación de tramos). Al acabar el algoritmo, los registros antiguos son eliminados, quedando sólo el registro resultante con tramos.

La función Unflatten admite un número variable de parámetros que indican una serie de elementos del registro que actúan como criterios de agrupación. Sólo se agruparán en un registro con tramos aquellos registros que tengan los mismos valores en los elementos pasados como argumentos. Así, acabará habiendo tantos registros con tramos como conjuntos de valores distintos haya.

Flatten

La función Flatten está pensada para pasar un registro con tramos a un bloque con un conjunto de registros sin tramos. Es lo que se conoce comúnmente como pasar de 3D a 2D. Se podría decir que es la función inversa de Unflatten. Aunque realmente esta función no está relacionada con la creación de tramos se incluye aquí por completitud.

La función Flatten es de ámbito registro y para que funcione correctamente deben estar definidos en el nodo los elementos con tipo interno fecha de inicio y fecha de fin.

Cada tramo pasa a formar un registro, donde el valor del elemento en el registro es el valor del tramo, en el elemento con tipo interno fecha de inicio se almacena la fecha inicio del tramo y en el elemento con tipo interno fecha de fin se almacena la fecha fin del tramo. Al acabar el algoritmo, el registro antiguo con tramos es eliminado, quedando sólo los registros resultantes sin tramos.

La función Flatten admite un parámetro que indica un nodo destino donde se quiere hacer el aplanamiento. En este caso la asignación de elementos se hará por identificador de elemento y no se eliminará el registro antiguo.

La función Flatten es especialmente útil para guardar valores tramados en la base de datos. Dado que las bases de datos no están preparadas para almacenar varios valores en una celda de una tabla, antes de persistir un registro con tramos hay que pasar esos tramos a su representación en formato registro.

JoinSlices

La función JoinSlices realiza un merge entre los elementos tramados de diferentes registros de un mismo bloque. El resultado se deja en un nuevo registro al final del bloque, de forma que en ese registros los elementos quedan tramados con los valores agrupados de los registros existente. Los registros originales son eliminados al final del algoritmo.

El merge se realiza elemento a elemento, de manera que se calcula el conjunto mínimo de tramos correspondiente a la intersección de todos los tramos de un mismo elemento teniendo en cuenta todos los registros del bloque. Es decir, se calcula el conjunto de tramos más pequeño que está completamente incluido en la suma de todos los tramos de todos los registros para ese elemento. Después, para cada tramo calculado se crea ese tramo en el elemento correspondiente del nuevo registro. A continuación se le asigna un valor que se calcula como suma de los valores en esas fechas de ese elemento en todos los registros si es de tipo numérico o como el último valor si es de tipo cadena. Al finalizar la asignación de todos los tramos de un elemento se procede a la compactación si ésta fuese necesaria (ver apartado Compactación de tramos).

TransferApplyValue

Esta función es la segunda fase del aplica valor. En una primera fase, otra función ha generado el conjunto de registros en formato corto (variable valor) con los valores que se deben asignar, habiendo resuelto ya las prioridades en la aplicación. La segunda fase lo único que hace es un pivotado de esos valores para asignarlos a los elementos destino. En ese pivotado, en el caso de que se esté trabajando con el modo de tramos activado, se tendrán en cuenta las fechas de validez de los valores a aplicar y se generarán tramos en esas fechas, asignándose a continuación el valor del tramo. Al finalizar la asignación de todos los tramos de un elemento se procede a la compactación si ésta fuese necesaria (ver apartado Compactación de tramos).

YTDSearchFiltered

Esta función, a grandes rasgos, realiza un copiado de registros de un nodo a otro, teniendo en cuenta una serie de filtros a aplicar, los elementos destino y una serie de criterios de agrupación. En caso de que el argumento a la función KeepValuesPolicy reciba el valor M4_KEEP_SLICES, en vez de asignar el valor en el elemento destino y perder el valor antiguo, lo que se hace es asignar el valor entre fechas, creando para ello un tramo y compactando si esto fuese necesario (ver apartado Compactación de tramos). De esta manera, si esta función es ejecutada varias veces con fechas de ejecución diferentes se puede tener el registro destino con elementos tramados, cada tramo resultante de una de las ejecuciones de la función.

Atributos método de LN4

En se dispone de varios atributos método que permiten la creación directa de un tramo en un elemento de tipo propiedad, campo o concepto. Éstos son SysAddNewSlice, SysCreateSlice y SysSplitSlice.

SysAddNewSlice

Añade un nuevo tramo a un elemento, pero no le asigna un valor. Recibe como parámetros la fecha inicio y la fecha fin del tramo a crear.

```
ELEMENTO..SysAddNewSlice( {2003-11-01}, {2003-11-10} )
```

SysCreateSlice

Añade un nuevo tramo a un elemento y además le asigna un valor. Recibe como parámetros la fecha inicio y la fecha fin del tramo a crear.

```
ELEMENTO..SysCreateSlice( {2003-11-01}, {2003-11-10}, "valor" )
```

SysSplitSlice

Divide un tramo existente en dos. Recibe como argumento la fecha por la que se debe hacer la división. El valor de los dos nuevos tramos se ajusta de acorde al valor antiguo, al tipo (número, cadena o fecha) del elemento a asignar y del valor de su propiedad comportamiento en tramos (valor base, valor final, unidad o incidencia).

```
ELEMENTO..SysSplitSlice( {2003-11-11} )
```

Lectura de tramos

Durante el normal desarrollo de las aplicaciones Meta4 es común hacer uso de la lectura del valor de los elementos de un Meta4Object, ya sea desde el código LN4 de otro elemento o en operaciones internas de la aplicación. Cuando se está trabajando con el modo de tramos desactivado esa lectura es muy sencilla, ya que el elemento que se lee no está tramado y sólo tienen un valor, que es el que se recupera en la lectura. Sin embargo, cuando se está trabajando con el modo de tramos activado el elemento que se lee puede estar tramado. En este caso, ¿qué valor es el que se debe recuperar?

La respuesta a esta pregunta depende de cuatro factores: de las fechas de inicio y de fin de ejecución actuales, del tipo (número, cadena o fecha) del elemento que se lee, del valor de la propiedad comportamiento en tramos (valor base, valor final, unidad o incidencia) del elemento que se lee y de la distribución de los tramos que tenga el elemento a leer.

A la hora de leer un elemento tramado, la lectura no se realiza en un punto del tiempo, por lo que obtener el valor no es tan sencillo como buscar el tramo correspondiente a esa fecha y devolver el valor. La lectura se realiza entre las fechas de inicio de ejecución y fin de ejecución que haya en el momento de la lectura.

Existen dos posibles escenarios en esta lectura:

- Que el periodo determinado por las fechas de inicio y fin de ejecución esté totalmente comprendido en uno de los tramos que tiene el elemento a leer.
- Que el periodo determinado por las fechas de inicio y fin de ejecución abarque más de uno de los tramos que tiene el elemento a leer.

Para explicar como se realiza la lectura en modo tramos se verá de forma independiente para cada uno de los comportamientos en tramos del elemento que se lee. Dentro de cada apartado se tendrán en cuenta cada uno de los dos escenarios descritos anteriormente. Dentro de cada escenario se tendrán en cuenta cada uno de los posibles tipos del elemento que se lee.

Valor base

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución esté totalmente comprendido en uno de los tramos que tiene el elemento a leer.

- Para números: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.
- Para cadenas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.
- Para fechas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución abarque más de uno de los tramos que tiene el elemento a leer.

- Para números: Si todos los tramos contenidos total o parcialmente en las fechas de inicio y fin de ejecución tienen el mismo valor se devuelve ese valor. Si no tienen el mismo valor se produce un error y se devuelve un valor nulo.
- Para cadenas: Si todos los tramos contenidos total o parcialmente en las fechas de inicio y fin de ejecución tienen el mismo valor se devuelve ese valor. Si no tienen el mismo valor se produce un error y se devuelve un valor nulo.
- Para fechas: Si todos los tramos contenidos total o parcialmente en las fechas de inicio y fin de ejecución tienen el mismo valor se devuelve ese valor. Si no tienen el mismo valor se produce un error y se devuelve un valor nulo.

Valor final

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución esté totalmente comprendido en uno de los tramos que tiene el elemento a leer.

- Para números: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución pero prorrateado según el periodo de tramo que abarcan estas fechas. Es decir, que lo que se devuelve es la parte proporcional del valor para las fechas de inicio y fin de ejecución respecto a las fechas de inicio y fin de tramo. Concretamente la operación que se hace es multiplicar el valor del tramo por el número de días determinado por las fechas de inicio y de fin de ejecución (ambas incluidas) y dividirlo por el número de días determinado por las fechas de

inicio y de fin del tramo (ambas incluidas). Consecuentemente el valor que se devuelva será menor o igual al valor del tramo. Por eso se denomina prorrateo .

- Para cadenas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.
- Para fechas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución abarque más de uno de los tramos que tiene el elemento a leer.

- Para números: Se devuelve el valor calculado sumando el valor de todos los tramos contenidos totalmente en las fechas de inicio y fin de ejecución y sumando el valor prorrateado de todos los tramos contenidos parcialmente en las fechas de inicio y fin de ejecución (qué como mucho pueden ser dos, uno al comienzo y uno al final.
- Para cadenas: Si todos los tramos contenidos total o parcialmente en las fechas de inicio y fin de ejecución tienen el mismo valor se devuelve ese valor. Si no tienen el mismo valor se produce un error y se devuelve un valor nulo.
- Para fechas: Si todos los tramos contenidos total o parcialmente en las fechas de inicio y fin de ejecución tienen el mismo valor se devuelve ese valor. Si no tienen el mismo valor se produce un error y se devuelve un valor nulo.

Unidad

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución esté totalmente comprendido en uno de los tramos que tiene el elemento a leer.

- Para números: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución pero prorrateado según el periodo de tramo que abarcan estas fechas. Es decir, que lo que se devuelve es la parte proporcional del valor para las fechas de inicio y fin de ejecución respecto a las fechas de inicio y fin de tramo. Concretamente la operación que se hace es multiplicar el valor del tramo por el número de días determinado por las fechas de inicio y de fin de ejecución (ambas incluidas) y dividirlo por el número de días determinado por las fechas de inicio y de fin del tramo (ambas incluidas). Consecuentemente el valor que se devuelva será menor o igual al valor del tramo.
- Para cadenas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.

- Para fechas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución abarque más de uno de los tramos que tiene el elemento a leer.

- Para números: Se produce un error y se devuelve un valor nulo.
- Para cadenas: Se produce un error y se devuelve un valor nulo.
- Para fechas: Se produce un error y se devuelve un valor nulo.

Incidencia

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución esté totalmente comprendido en uno de los tramos que tiene el elemento a leer.

- Para números: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución pero prorrateado según el periodo de tramo que abarcan estas fechas. Es decir, que lo que se devuelve es la parte proporcional del valor para las fechas de inicio y fin de ejecución respecto a las fechas de inicio y fin de tramo. Concretamente la operación que se hace es multiplicar el valor del tramo por el número de días determinado por las fechas de inicio y de fin de ejecución (ambas incluidas) y dividirlo por el número de días determinado por las fechas de inicio y de fin del tramo (ambas incluidas). Consecuentemente el valor que se devuelva será menor o igual al valor del tramo.
- Para cadenas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.
- Para fechas: Se devuelve el valor que tenga el tramo donde estén contenidas las fechas de inicio y fin de ejecución.

Considérese el caso en el que el periodo determinado por las fechas de inicio y fin de ejecución abarque más de uno de los tramos que tiene el elemento a leer.

- Para números: Se devuelve el valor calculado sumando el valor de todos los tramos contenidos totalmente en las fechas de inicio y fin de ejecución y sumando el valor prorrateado de todos los tramos contenidos parcialmente en las fechas de inicio y fin de ejecución (qué como mucho pueden ser dos, uno al comienzo y uno al final).
- Para cadenas: Si todos los tramos contenidos total o parcialmente en las fechas de inicio y fin de ejecución tienen el mismo valor se devuelve ese valor. Si no tienen el mismo valor se produce un error y se devuelve un valor nulo.

-
- Para fechas: Si todos los tramos contenidos total o parcialmente en las fechas de inicio y fin de ejecución tienen el mismo valor se devuelve ese valor. Si no tienen el mismo valor se produce un error y se devuelve un valor nulo.

Excepciones

Un caso excepcional es aquel en el que las fechas de inicio y fin de ejecución abarquen un periodo del tiempo para el cual no existen tramos creados para el elemento que se lee. Como una vez que se crean tramos para un elemento no existe discontinuidad (no deja de haber tramos) entre el primer y el último tramo, esto significa que para que se de este caso la fecha de inicio de ejecución debe ser menor que la fecha de inicio del primer tramo y/o que la fecha de fin de ejecución debe ser mayor que la fecha de fin del último tramo. En este caso, a efectos de lectura para los periodos que no existe tramo se considerará que son un tramo que va desde la fecha de inicio de trabajo hasta la fecha fin de trabajo y cuyo valor es el valor por defecto del elemento.

A efectos prácticos esto significa que para aquellas operaciones donde se intente leer el valor de ese periodo que no tiene tramo se devolverá el valor por defecto. Y para aquellas operaciones que impliquen un prorrateo se devolverá el valor por defecto prorrateado entre las fechas que se intenta leer sobre las fechas de inicio y fin de trabajo (que son siempre mayores). Esas fechas que se intenta leer serán entre la fecha de inicio de ejecución y la fecha de inicio del primer tramo existente si se lee antes de la existencia de tramos y serán entre la fecha de fin del último tramo existente y la fecha de fin de ejecución si se lee después de la existencia de tramos.

Hay que hacer notar que la característica de utilizar el valor por defecto prorrateado en el supuesto anterior es nuevo a partir de la versión 600.031.008 de la librería m4dm. En versiones anteriores no se realizaba el prorrateo sino que se tomaba directamente el valor por defecto sin prorratear.

Otra excepción es cuando el elemento que se lee no tiene ningún tramo creado. En este caso se devuelve el valor que tenga el elemento en operaciones de lectura normales. En caso de lecturas donde se aplique el prorrateo se devuelve el valor prorrateado entre las fechas pedidas y las fechas de trabajo.

Redondeo

En las operaciones de lectura que se han descrito anteriormente, cuando se necesita realizar el prorrateo del valor se hace una división. Esta división se realiza con la precisión de la máquina, que son al menos 9 decimales. Esta operación puede producir una imprecisión en los valores obtenidos si las divisiones no son exactas.

Para poder evitar este efecto, o al menos paliarlo, la aplicación de Meta4 proporciona una característica que permite al parametrizador del Meta4Object indicar que después de cada operación de prorrateo se redondee el valor que se va a devolver a un número concreto de decimales. Ese redondeo implica además una corrección del redondeo para que los errores no se vayan acumulando y se distribuyan de manera uniforme a lo largo de todos los tramos obtenidos.

La parametrización consiste en indicar en dos propiedades de Meta4Object ROUND_CURRENCY y ROUND_NUMBER si se debe aplicar o no el redondeo, respectivamente para monedas y el resto de números. 1 indica que si se debe aplicar el redondeo y 0 que no. Estas propiedades están siempre creadas en el Meta4Object y se pueden asignar y consultar desde LN4 como cualquier otra propiedad de un Meta4Object.

```
x = MyMeta4Object!GetProperty( "ROUND_CURRENCY" )
MyMeta4Object!SetProperty( "ROUND_CURRENCY", x )
y = MyMeta4Object!GetProperty( "ROUND_NUMBER" )
MyMeta4Object!SetProperty( "ROUND_NUMBER", y )
```

Al crearse un Meta4Object estas propiedades toman el valor que tengan los parámetros de aplicación ROUND_CURRENCY y ROUND_NUMBER. Si no están definidos esos parámetros de aplicación se tomará el valor 0 para esas propiedades, lo que implica que no se efectuará el redondeo.

Si esas propiedades de Meta4Object están activadas (su valor es 1), la aplicación procederá a hacer el redondeo en las operaciones de prorrateo. Para ello utilizará la precisión que venga dada respectivamente en las propiedades de Meta4Object CURRENCY_PRECISION y NUMBER_PRECISION. Estas propiedades están siempre creadas en el Meta4Object y se pueden asignar y consultar desde LN4 como cualquier otra propiedad de un Meta4Object. Por defecto su valor es -1.

```
x = MyMeta4Object!GetProperty( "CURRENCY_PRECISION" )
MyMeta4Object!SetProperty( "CURRENCY_PRECISION", x )
y = MyMeta4Object!GetProperty( "NUMBER_PRECISION" )
MyMeta4Object!SetProperty( "NUMBER_PRECISION", y )
```

Si en el momento de hacer el redondeo esas propiedades tienen un valor negativo o un valor mayor que la escala del elemento que se está leyendo se utilizará la escala del elemento como precisión en la operación.

Hay que hacer notar que la característica de hacer el redondeo utilizando como parametrización propiedades de Meta4Object es nueva a partir de la versión 600.038.000 de las librerías m4dm y m4vm. En versiones anteriores el redondeo se realizaba utilizando una entrada en el registro. Concretamente se tenía en cuenta la entrada:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\xxx\yyy\CVM\FlattenItemPrecision
```

De forma, que sólo si su valor era 1 se realizaba el redondeo. Para realizar el redondeo se tenía en cuenta la escala del elemento como número de decimales, de manera que no podía ser especificada por el parametrizador. Además, el redondeo sólo se realizaba cuando las operaciones de lectura eran hechas desde la función Flatten, pero no en el caso del resto de lecturas. Ahora se hace para todas las lecturas que sucedan.

Ajuste a 30 días

Otra característica importante que hay que tener en cuenta en las operaciones de lectura es el ajuste a 30 días. El ajuste a 30 días permite al parametrizador hacer que la aplicación de Meta4 pueda tratar tramos de una determinada longitud como si tuviesen una longitud distinta en operaciones de lectura. Para ello el parametrizador debe indicar dos fechas. Una fecha, o fecha a tratar, es la fecha que de estar incluida en un tramo que se lee debe tratarse como si fuese una fecha distinta y por lo tanto la aplicación de Meta4 debe considerar una longitud de tramo distinta. Otra fecha, o fecha a considerar, es la fecha que realmente debería ser la fecha a tratar. La diferencia entre la fecha a considerar y la fecha a tratar es el periodo que se debe añadir a la longitud del tramo. Hay que tener muy claro que la longitud del tramo no se ve afectada y después de la operación sigue siendo la misma. Sólo es a efectos de la lectura que cambia su longitud.

Aunque se denomine ajuste a 30 días porque el uso más común es considerar que los meses de 31 días son de 30 días, de modo que lo que hay que hacer es sumar menos 1 día a la longitud del periodo.

Lo que hace la aplicación es verificar si cuando se realiza una lectura en la que hay que prorratear están definidas la fecha a tratar y la fecha a considerar para el elemento del que se va a leer. Si este es el caso y la fecha a tratar se encuentra dentro del tramo que se va a prorratear, se ajusta la longitud del tramo como se ha indicado, es decir, si la fecha de inicio del tramo es menor o igual que la fecha a tratar y si la fecha de fin de tramo es mayor o igual que la fecha a tratar. De igual manera si la fecha a tratar se encuentra dentro del periodo que se va a leer, también se ajusta la longitud del periodo como se ha indicado, es decir, si la fecha de inicio de ejecución es menor o igual que la fecha a tratar y si la fecha de fin de ejecución es mayor o igual que la fecha a tratar.

El ajuste a 30 días sólo tiene efecto cuando se hace un prorrateo, es decir cuando el elemento a leer tiene un valor de la propiedad avanzada comportamiento en tramos valor final, unidad o incidencia y el tipo del elemento es número.

La fecha a tratar debe venir almacenada en otro elemento del mismo registro del que se lee. Este elemento debe estar indicado rellenando la propiedad avanzada Elemento auxiliar de prorrateo. Si esa propiedad avanzada no está

rellena, para el caso de unidades e incidencias se consulta en su lugar la propiedad avanzada Elemento auxiliar. Esto se hace por compatibilidad hacia atrás. La fecha a considerar debe venir almacenada en otro elemento del mismo registro del que se lee. Este elemento debe estar indicado rellenando la propiedad avanzada Elemento auxiliar del elemento donde se almacena la fecha a tratar. Si alguna de estas definiciones no está presente o los elementos auxiliares no están rellenos con unas fechas válidas no se aplica el prorrateo a 30 días.

Hay que hacer notar que la característica de consultar el elemento auxiliar de prorrateo para obtener la fecha a tratar es nueva a partir de la versión 600.001.000 de la librería m4dm. En versiones anteriores se consultaba directamente el elemento auxiliar pero sólo para unidades e incidencias.

Compactación de tramos

La compactación de tramos es un proceso por el cual se pueden unir dos tramos consecutivos en fechas de un elemento para formar un solo tramo.

El tramo resultante tiene como fecha de inicio la fecha de inicio del primer tramo y como fecha de fin la fecha de fin del segundo tramo. El valor con el que queda el tramo resultante depende del tipo de elemento (numero, cadena o fecha) y del valor de la propiedad avanzada comportamiento en tramos (valor base, valor final, unidad o importe).

Los requisitos para que dos tramos consecutivos puedan compactarse dependen igualmente del valor de la propiedad avanzada comportamiento en tramos (valor base, valor final, unidad o importe). Además ambos tramos deben tener la misma prioridad.

El criterio fundamental a la hora de compactar dos tramos es que el proceso sea completamente reversible. Eso significa que si se han compactado dos tramos y posteriormente se intenta leer el valor del elemento en las fechas del primer tramo que se compactó se debe obtener el mismo valor que si no se hubiese compactado. Lo mismo es aplicable a leer el valor en las fechas del segundo tramo. Es decir, que las operaciones de compactación y lectura deben ser coherentes. También significa que si se parte el tramo resultante en dos (por ejemplo con el atributo método SysSplitSlice), los dos tramos obtenidos deben ser iguales que antes de haberse compactado.

El proceso de compactación se realiza de manera automática por la aplicación de Meta4. Esto permite que el número de tramos se mantenga en el mínimo posible, de manera que se eviten cálculos innecesarios y que a la hora de grabar en la base de datos valores tramados el número de registros se mantenga al mínimo posible.

A día de hoy el proceso de compactación se produce al finalizar la ejecución de un concepto, momento en el que se compactan los tramos del propio elemento que se acaba de ejecutar (después de haberlo asignado) y también se compactan los elementos que este concepto ha asignado. Los elementos que se han asignado desde el concepto se calculan utilizando las dependencias de asignación, de modo que los elementos asignados por indirección no se compactarían. La compactación también se produce en los algoritmos internos de funciones LN4 de nómina como: Unflatten, JoinSlices, TransferApplyValue y YTDSearchFiltered.

Hay que hacer notar que existe un requisito para que la compactación se realice en caliente, es decir, siempre que se asigne valor a un tramo nuevo o existente. Eso implicaría que los sitios donde se realiza la compactación a día de hoy no serían ya necesarios, porque se garantizaría sin lugar a dudas que siempre que se pueda se compacta y el conjunto de tramos siempre se mantienen en un mínimo. En el momento de escribir este documento este requisito todavía no se había implementado.

Para explicar como se realiza la compactación de tramos se verá de forma independiente para cada uno de los comportamientos en tramos del elemento que se compacta.

Valor base

- Para números: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.
- Para cadenas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.
- Para fechas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.

Valor final

- Para números: Dos tramos consecutivos se compactan si su valor por unidad de tiempo es el mismo en ambos tramos. Esto significa que los valores de los dos tramos son proporcionales a la longitud del periodo que abarcan. Es decir, que el valor del primer tramo dividido por su número de días es igual al valor del segundo tramo dividido por su número de días. El valor del tramo resultante es la suma de los valores de cada uno de los dos tramos.
- Para cadenas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.
- Para fechas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.

Unidad

- Para números: Dos tramos consecutivos se compactan si su valor por unidad de tiempo es el mismo en ambos tramos. Esto significa que los valores de los dos tramos son proporcionales a la longitud del periodo que abarcan. Es decir, que el valor del primer tramo dividido por su número de días es igual al valor del segundo tramo dividido por su número de días. El valor del tramo resultante es la suma de los valores de cada uno de los dos tramos.
- Para cadenas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.
- Para fechas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.

Incidencia

- Para números: Dos tramos consecutivos se compactan si su valor por unidad de tiempo es el mismo en ambos tramos. Esto significa que los valores de los dos tramos son proporcionales a la longitud del periodo que abarcan. Es decir, que el valor del primer tramo dividido por su número de días es igual al valor del segundo tramo dividido por su número de días. El valor del tramo resultante es la suma de los valores de cada uno de los dos tramos.
- Para cadenas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.
- Para fechas: Dos tramos consecutivos se compactan si su valor es el mismo en ambos tramos, independientemente de la longitud del periodo que abarca cada tramo. El valor del tramo resultante es el mismo que ya tenían cada uno de los tramos.

Redondeo

En el caso en el que la compactación se haga cuando los valores de los dos tramos consecutivos son proporcionales se tendrá en cuenta el redondeo, siempre y cuando el Meta4Object esté parametrizado para trabajar con redondeo y con una determinada precisión.

Tener en cuenta el redondeo es calcular si los dos valores consecutivos son proporcionales o no hasta el número de decimales estipulado.

En ese caso, la forma de determinar si se está trabajando con redondeo y la manera de calcular la precisión a emplear es la misma que en el caso de lectura de elementos tramados.

Funciones LN4 para el manejo de tramos

Existe un amplio abanico de funciones de LN4 para el manejo de los tramos. A Algunas de ellas ya se han visto a lo largo de este capítulo. Aquí se enumeran todas con una breve descripción.

Cada elemento de esta sintaxis se explica en la siguiente tabla:

Elemento	Descripción
NombreElemento	Nombre del elemento para el cual se van a crear o se han creado tramos.
ÍndiceTramo	Identifica el tramo con el que se quiere trabajar. Los tramos se numeran secuencialmente comenzando por 0. En lugar del índice del tramo, puede escribirse una fecha comprendida dentro del periodo de validez del tramo.
Atributo-método	Nombre del atributo-método cuyo valor se quiere recuperar.

La siguiente tabla recoge las funciones y atributo-métodos relacionados. Estos atributo-métodos se aplican sobre:

- Elementos con tramos
- Tramos específicos de un elemento

Función	Descripción
SetSliceMode	Establece el modo de tramos de un Meta4Object. Admite un parámetro que puede ser M4_TRUE o M4_FALSE según se quiere activar o desactivar el modo tramos.
GetSliceMode	Devuelve el modo de tramos de un Meta4Object. Puede devolver M4_TRUE o M4_FALSE según esté activado o desactivado el modo tramos.
SetRunDate	Establece la fecha de trabajo de un Meta4Object. Admite un parámetro que es la nueva fecha a establecer.
SetStartDate	Establece la fecha de inicio de trabajo de un Meta4Object. Admite un parámetro que es la nueva fecha a establecer.
SetEndDate	Establece la fecha de fin de trabajo de un Meta4Object. Admite un parámetro que es la nueva fecha a establecer.
GetRunDate	Devuelve la fecha de trabajo de un Meta4Object.
GetStartDate	Devuelve la fecha de inicio de trabajo de un Meta4Object.
GetEndDate	Devuelve la fecha de fin de trabajo de un Meta4Object.
AppRunDate	Devuelve la fecha de trabajo de un Meta4Object.

Función	Descripción
AppStartDate	Devuelve la fecha de inicio de trabajo de un Meta4Object.
AppEndDate	Devuelve la fecha de fin de trabajo de un Meta4Object.
GetRunningRunDate	Devuelve la fecha de ejecución de un Meta4Object.
GetRunningStartDate	Devuelve la fecha de inicio de ejecución de un Meta4Object.
GetRunningEndDate	Devuelve la fecha de fin de ejecución de un Meta4Object.
SysAddNewSlice	Añade un nuevo tramo a un elemento. Admite dos parámetros que son la fecha de inicio y la fecha de fin del tramo que se quiere crear. Es un método atributo.
SysCreateSlice	Añade un nuevo tramo a un elemento y le asigna un valor. Admite tres parámetros que son la fecha de inicio y la fecha de fin del tramo que se quiere crear y el valor que se le asigna al nuevo tramo. Es un método atributo.
SysSplitSlice	Genera dos tramos a partir de uno. Admite un parámetro que es la fecha por la que se debe partir el tramo existente. Es un método atributo.
SysTotalSlices	Devuelve la totalización de todos los tramos de un elemento. Admite un parámetro que es el tipo de totalización que se quiere. Los valores permitidos son M4_TOTAL_COUNT, M4_TOTAL_SUM, M4_TOTAL_AVG, M4_TOTAL_MAX, M4_TOTAL_MIN, M4_TOTAL_FIRST, M4_TOTAL_LAST. Es un método atributo.
SysSliceNumber	Devuelve el número de tramos de un elemento. Es un atributo.
SysStartDate	Devuelve la fecha de inicio del tramo actual de un elemento. Es un atributo. Se puede indicar entre corchetes el tramo deseado, ya sea por índice o por fecha.
SysEndDate	Devuelve la fecha de fin del tramo actual de un elemento. Es un atributo. Se puede indicar entre corchetes el tramo deseado, ya sea por índice o por fecha.
SysFirstSlice	Devuelve el índice del primer tramo de un elemento. Es un atributo.
SysLastSlice	Devuelve el índice del último tramo de un elemento. Es un atributo.

Trazabilidad de tramos

Hoy en día existe una manera fácil de trazar las operaciones con tramos, consulte el manual Depuración con trazas.

Arquitectura cliente/servidor en las aplicaciones Meta4

10

Acerca de este capítulo

Este capítulo pretende dar una visión tecnológica de la arquitectura cliente/servidor implementada en las aplicaciones Meta4.

Además de ofrecer una visión global sobre la arquitectura cliente/servidor y el modo en el que los componentes interactúan en todos los procesos implementados en la aplicación, se pretende proporcionar a los usuarios e implantadores los mecanismos y procedimientos necesarios para el desarrollo funcionalidades dentro de este marco tecnológico.

Arquitectura cliente/servidor: fundamentos básicos

El sistema de las aplicaciones Meta4 aplica el sistema cliente/servidor en varios niveles. Es altamente modular y se implementa fundamentalmente por medio del software, de forma que se puedan controlar los modos de interacción entre los diversos clientes y servidores.

Es posible enlazar servidores determinados en redes de comunicación y realizar ciertas tareas sin afectar a todos los datos y procesos de la red central del sistema.

Introducción: arquitecturas de 3 capas lógicas y físicas

La aplicación Meta4 está optimizada para trabajar en una arquitectura cliente/servidor de varios niveles.

A diferencia de los modelos informáticos, la arquitectura cliente/servidor de varios niveles distribuye la lógica de la aplicación en tres capas lógicas que, a su vez, pueden implementarse en un número variable de niveles de hardware.

- Nivel de almacenamiento de datos
- Nivel de aplicación
- Nivel de presentación

Generalmente, estos niveles lógicos se corresponden con niveles físicos independientes, tal y como se explicó anteriormente:

- El nivel de datos se corresponde con el servidor en el que está instalado el SGBD relacional.
- El nivel de aplicación se corresponde con un servidor Windows o Unix, en el que se ejecutan la mayoría de los procesos que exigen mayores recursos del sistema.
- El nivel de presentación se corresponde con los equipos cliente.

El esquema anterior responde a la designación de tres niveles físicos. En este modelo, los tres niveles lógicos de la aplicación se implementan en sistemas hardware diferentes.

Se dispone, además, de distintos modelos de implantación y desarrollo. Con este término se designan aquellas implantaciones en las que existe un cliente de entorno de desarrollo que se conecta directamente a la base de datos, sin utilizar los servicios del servidor de aplicaciones. Para que la conexión a la base de datos esté asegurada, es necesario que los drivers ODBC/OCI estén instalados y configurados correctamente.

Existen además implantaciones en las que el SGBD relacional y el servidor de aplicaciones se ejecutan en la misma máquina.

En este modelo se reduce el tráfico de red, ya que las transacciones de datos se producen en la misma máquina, sin necesidad de desplazarse a través de la red corporativa.

Este modelo no compromete la escalabilidad: el hecho de que en la misma máquina se haya instalado el servidor de base de datos y un servidor de aplicaciones no impide que se instalen servidores de aplicaciones adicionales en otros equipos.

Rendimiento y escalabilidad

La arquitectura cliente/servidor de varios niveles garantiza el rendimiento del sistema y su escalabilidad.

Por escalabilidad se entiende la posibilidad de aumentar las solicitudes cursadas a los servidores de aplicaciones por un número creciente de procesos cliente, sin que el número creciente de solicitudes afecte negativamente al rendimiento del sistema.

La arquitectura cliente/servidor multinivel garantiza la escalabilidad de las aplicaciones, ya que en cualquier momento se puede aumentar el número de servidores de bases de datos y de aplicaciones para atender mayor cantidad de servidores de aplicaciones y puestos de trabajo cliente.

Además de garantizar la escalabilidad, las arquitecturas de varios niveles ofrecen las siguientes ventajas respecto a otras arquitecturas basadas en el procesamiento centralizado o en los modelos de dos niveles:

- Reducción en el tráfico de red y mejoras en el rendimiento de la red corporativa.

No todos los datos gestionados por el SGBD relacional tienen que desplazarse hasta los puestos cliente, como sucede en una arquitectura de dos niveles, en la que los puestos cliente implementan el nivel de presentación y el de aplicación.

- Fácil instalación y mantenimiento de los puestos de trabajo cliente.

Como los puestos de trabajo cliente solicitan la ejecución de procesos a los servidores de aplicaciones, que a su vez se encargan de interactuar con el sistema gestor de bases de datos, no es necesario que los puestos cliente dispongan de conexión directa al SGBD relacional. Sólo es necesario que se puedan conectar a los servidores de aplicaciones.

- Integración directa con la tecnología Web.

Las aplicaciones que se ejecutan a través de redes TCP/IP con el protocolo HTTP son arquitecturas de varios niveles, en las que el procesamiento se implementa en un servidor de aplicaciones que, a través de un servidor HTTP, envía los resultados de los procesos a terminales o equipos que interactúan con la aplicación a través de un navegador HTML.

Nivel de almacenamiento de datos

Hace referencia al soporte necesario para gestionar toda la información con la que trabaja la aplicación: las bases de datos y los sistemas que gestionan las operaciones de lectura y escritura en la base de datos. Las aplicaciones Meta4 permiten trabajar con bases de datos distribuidas de distintos fabricantes, así como compartir recursos con otras bases de datos mediante el controlador Meta4ODBC y la adecuación del lenguaje LN4 a la norma ANSI/SQL.

La base de datos central puede ser atendida en un procesador único, en uno múltiple o en un grupo de procesadores. La configuración del complejo de la base de datos se puede adaptar al volumen de datos y al tráfico en el sistema.

Nivel de aplicación

Comprende los componentes software que implementan la funcionalidad de la aplicación.

El nivel de aplicación consiste en:

- Una serie de componentes de software integrados (Base de datos lógica (BDL), Meta4Object Engine, Intérprete LN4, etc.) que gestiona las operaciones y las llamadas que se producen entre distintos Meta4Objects.
- Los Meta4Objects que definen la funcionalidad de la aplicación.
- Una serie de componentes que se encargan de:
 - atender las solicitudes de conexión que llegan al sistema.
 - recibir toda la información que llega procedente de los clientes: tanto de control como de peticiones o datos.
 - analizar y extraer la información de verificación que contiene, verificar que es posible realizarla y planificarla adecuadamente.
 - recoger, distinguir y procesar las respuestas pendientes de enviar al cliente.
 - enviar las respuestas a través de la conexión de transporte establecida con el cliente.

El nivel de aplicación es independiente del sistema gestor de bases de datos

(SGBD). Esta independencia se consigue mediante:

- El acceso a los servicios del sistema gestor a través de la especificación ODBC y controladores directos, por ejemplo interfaz Oracle Call y controladores nativos de Sybase.
- Un componente de software desarrollado por Meta4, la Base de datos lógica, que gestiona la comunicación entre los servidores de aplicaciones y el SGBD relacional.

Entre otras funcionalidades, la base de datos lógica implementa:

- Controles para comprobar la validez de los datos que se intentan escribir en la base de datos: integridad de claves primarias, claves externas, valores nulos, etc.
- Conversión de los tipos de datos gestionados por la aplicación, con los tipos de datos gestionados por los distintos SGBD relacionales: Oracle, Informix, etc (esto se hace de forma transparente para el usuario).
- Traducción de las funciones de totalización (conversión entre tipos de datos, truncamientos de cadenas de caracteres, etc.) utilizadas en la aplicación, a su correspondiente implementación en el SGBD relacional que se esté utilizando.
- Optimización de las conexiones disponibles en el SGBD relacional, lo que permite que varios usuarios reutilicen y compartan una misma conexión de bases de datos.
- Gestión de transacciones lógicas.



La base de datos lógica (BDL) permite al usuario-configurador desarrollar funcionalidades desde el entorno de desarrollo que ofrece la aplicación, que, sin necesidad de hacer ningún tratamiento o adaptación posterior, pueden ejecutarse sobre cualquier SGBD relacional certificado: Oracle, Informix, o SQL-Server.

El nivel de aplicación se ejecuta en los servidores de aplicaciones: plataformas UNIX o Microsoft Windows. En una implementación convencional, es posible que existan varios servidores de aplicaciones que atiendan las solicitudes de usuarios distribuidos en distintas ubicaciones.

La distribución de la ejecución de procesos se explica en el apartado Tipos de ejecución en arquitectura C/S. La descripción detallada de los componentes lógicos de un servidor de aplicaciones se encuentra en un apartado posterior de este mismo capítulo.

Nivel de presentación

Con este término se hace referencia a los sistemas que presentan información al usuario final de la aplicación y permiten la interacción entre el usuario y el sistema.

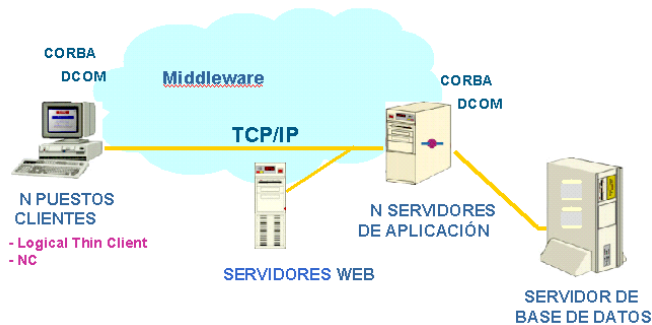
Este nivel gestiona la presentación de los datos en los puestos cliente. La aplicación permite generar distintos modos de presentación de datos para facilitar la visualización dinámica de las relaciones entre datos y simplificar el análisis de informaciones complejas.

En el modelo cliente/servidor de Meta4, el nivel de presentación se implementa en puestos de trabajo PC con MS-Windows, así como en cualquier máquina equipada con un navegador de Internet.

Implementación física de la arquitectura c/s

El modelo cliente/servidor distribuye la lógica de la aplicación en los tres niveles descritos en el apartado anterior. Los niveles se implementan en distintos sistemas y plataformas conectados mediante una red compatible con el protocolo TCP/IP.

Figura 75. Arquitectura física de las aplicaciones Meta4



En la modalidad más simple, se cuenta con un único sistema gestor de bases de datos, un servidor de aplicaciones y varios puestos cliente.

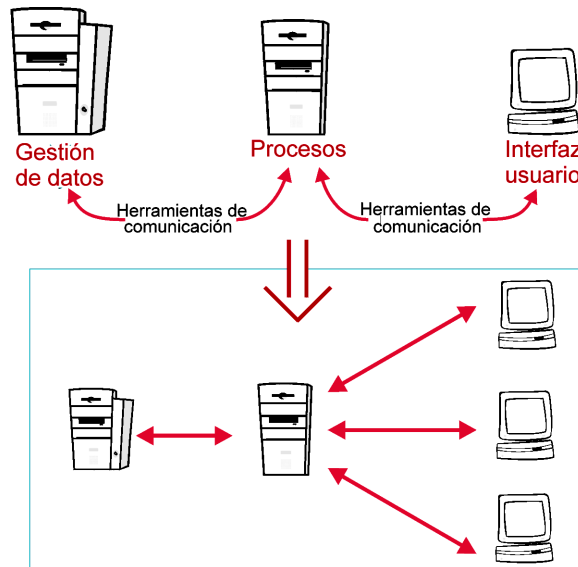
- Los servidores de bases de datos, que se encargan de gestionar los datos con los que trabaja la aplicación: operaciones de lectura y escritura de información en la base de datos física.
- Los servidores de aplicaciones, que se ocupan de ejecutar los procesos que implementan la lógica de la aplicación.
- Los servicios de presentación, que implementan la presentación al usuario de los datos y de los resultados obtenidos por su procesamiento en los servidores de aplicaciones.



En la arquitectura cliente/servidor de las aplicaciones Meta4, los puestos cliente no sólo actúan como servidores de presentación. Además de estos servicios, pueden implementar la lógica de la aplicación y ejecutar código procesable. De esta forma, el nivel de aplicación se reparte entre los servidores de aplicaciones y los puestos cliente.

Estos componentes se comunican a través del *middleware* que permite el intercambio de mensajes. Los servidores de presentación interactúan e intercambian información con los servidores de aplicaciones. Éstos, a su vez, intercambian información con los servidores de bases de datos. El modelo de arquitectura cliente/servidor implementado por las aplicaciones Meta4 es un modelo de tres capas con lógica distribuida. El término 3 capas hace referencia a la posibilidad de que la ejecución de un proceso pueda distribuirse en los servidores de aplicaciones o en los puestos clientes, pudiendo darse un modelo distribuido como el que recoge la siguiente figura:

Figura 76. Modelo distribuido



Además de estos componentes, existe la posibilidad de agregar conectividad al sistema con servidores web que permitan acceso a los procesos de negocio gestionados por la aplicación a través de un explorador capaz de interpretar HTML estándar y applets Java.

Servidores de bases de datos

En los servidores de bases de datos reside la base de datos física, y se almacena el repositorio con el contenido y la parametrización de todos los Meta4Objects que maneja la aplicación.

Meta4 certifica el correcto funcionamiento de la tecnología Meta4 con distintas versiones de los SGBD relacionales como Oracle y Microsoft SQL-Server. En posteriores versiones del producto está previsto que se certifique su funcionamiento con Informix, Sybase y DB2.

Las características más destacables de la arquitectura Meta4 son:

- Independencia del sistema respecto a los fabricantes de SGBD relacionales
- Posibilidad de trabajar con bases de datos distribuidas de distintos fabricantes
- Optimización y accesibilidad en las conexiones al servidor de base de datos

Independencia

En el modelo desarrollado por Meta4, el sistema gestor de bases de datos actúa como un "repositorio" de información. Todas las comprobaciones sobre la validez e integridad de los datos, aunque no exclusivamente, se realizan en los servidores de aplicaciones, a través de la base de datos lógica (BDL).

La aplicación ofrece un entorno de desarrollo integrado, IDE, que permite diseñar nuevas funcionalidades de forma totalmente independiente a las características del sistema gestor.



Con el término IDE (Integrated Development Environment), se hace referencia a un conjunto de herramientas de desarrollo integradas en la aplicación, con las que el usuario puede desarrollar nuevas funcionalidades y personalizar las versiones estándar del producto. El IDE incluye herramientas para diseñar modelos de datos, un lenguaje de programación específico, etc.

En este entorno de desarrollo, los controles de integridad de datos (claves externas, claves primarias, unicidad de valores, control de valores nulos, etc.), los tipos de datos, las funciones de totalización, la conversión de datos, etc., se pueden aplicar sin necesidad de conocer su implementación real en el sistema gestor en el que se va a registrar la información.

El IDE de Meta4 permite diseñar funcionalidades transferibles, de forma automática, a otras implantaciones en las que se ha decidido trabajar con un SGBD relacional diferente; además, el cliente puede hacer cualquier cambio en el sistema que soporta sus datos, sin afectar a las funcionalidades desarrolladas desde las herramientas de diseño de la aplicación.

Otro aspecto importante para la independencia respecto al SGBD relacional se logra implementando la funcionalidad de la aplicación sin recurrir a procedimientos ni disparadores (*triggers*) de base de datos.

El hecho de que todas las validaciones de datos y los procedimientos se ejecuten en servidores de aplicaciones, o en máquinas cliente, no sólo garantiza la independencia respecto al SGBD relacional y la transferibilidad de los desarrollos entre implementaciones diferentes, sino que también:

- Minimiza el tiempo necesario para el desarrollo de nuevas funcionalidades
- Facilita la adaptación de los desarrollos de software a los cambios que se produzcan en los procesos de negocio que soportan
- Minimiza los requerimientos de red y optimiza el rendimiento de la red corporativa, ya que únicamente se transmite a los servidores de datos información ya validada por la BDL de los servidores de aplicaciones

Bases de datos distribuidas

La posibilidad de trabajar de manera transparente con bases de datos distribuidas y gestionadas por distintos servidores es una posibilidad que, desde hace bastante tiempo, vienen ofreciendo los principales fabricantes de SGBD relacionales.

Meta4 ofrece mayores niveles de flexibilidad, pues permite trabajar con bases de datos distribuidas y gestionadas por sistemas gestores de distintos fabricantes, de forma transparente al usuario final y sin tener que realizar ninguna parametrización en los sistemas gestores.

Por ejemplo, un Meta4Object puede extraer datos de una base de datos gestionada por un servidor Oracle, o por una base de datos gestionada por un servidor SQL Server. Sin embargo deberá tener en cuenta que el servidor de aplicaciones exclusivamente puede conectarse contra un gestor de base de datos del mismo tipo (Oracle o SQL Server), aunque puede configurarlo para que el acceso a los datos del SGBD puedan residir físicamente en servidores diferentes.

Conexiones lógicas

En el modelo cliente/servidor convencional, cada usuario conectado al sistema necesita disponer con exclusividad de una conexión de base de datos durante todo el tiempo que dure la sesión de trabajo.

En la arquitectura Meta4, puede determinarse el número de conexiones lógicas a la base de datos mediante opciones de configuración. De este modo, no podrán ejecutarse procesos que impliquen que se supere el número de conexiones lógicas parametrizados en la configuración.

Servidores de aplicaciones

Los servidores de aplicaciones atienden las solicitudes de los puestos cliente e implementan la interfaz entre los procesos de aplicación y los datos gestionados por los servidores de bases de datos. Básicamente se encarga de ejecutar las operaciones de lectura y escritura sobre la base de datos, y enviar los resultados obtenidos a los puestos clientes. Está orientado a la ejecución de Meta4Objects.

Para que se produzca esta comunicación, los servidores de aplicaciones necesitan disponer de conexión a la base de datos; es decir, en los servidores de aplicaciones debe instalarse el *middleware* propietario de cada fabricante de base de datos y los controladores ODBC si son necesarios. El servidor de aplicaciones puede residir en una plataforma con Windows o con UNIX.

En la arquitectura física de la aplicación, el número de servidores de aplicaciones varía en función de los puestos clientes concurrentes y de los procesos de trabajo. La arquitectura cliente/servidor de Meta4 descarga sobre los servidores de aplicaciones la ejecución más pesada de los procesos de trabajo.

Los servidores de aplicaciones están configurados para interactuar con distintos tipos de usuarios y para soportar distintos tipos de ejecución: modo OLTP (*On Line Transaction Processing*) o transaccional, modo Delta o transaccional optimizado, modo Proxy o conversacional, y la ejecución de procesos programada desde el Planificador de tareas (Job Scheduler). Para información más detallada, véase el apartado "Tipos de ejecución en arquitectura C/S", de este mismo capítulo.

Para información más detallada sobre la arquitectura del servidor de aplicaciones y sus componentes, consulte el manual *El Servidor de Aplicaciones de Meta4* y el *Meta4 Server Site*.

Funcionalidad del servidor de aplicaciones

El servidor de aplicaciones está preparado para:

- Recibir solicitudes de los puestos de trabajo cliente

Las solicitudes pueden haberse cursado de distinta forma: ejecución de procesos en lote, transacciones OLTP o proxy.

- Controlar la cola de solicitudes, ejecutando los procesos conforme el momento en que se hayan cursado.
- Ejecutar los procesos pendientes en la cola de solicitudes.

La ejecución de un proceso obliga al servidor de aplicaciones a realizar operaciones de lectura y escritura de datos; para llevar a cabo estas operaciones, se tienen que cursar solicitudes al SGBD relacional. Todas las

operaciones de lectura y escritura en base de datos se tramitan a través de uno de los componentes del servidor de aplicaciones: la base de datos lógica.

- Cargar en la memoria principal los datos que devuelva la base de datos, así como las instrucciones necesarias para su procesamiento.

El servidor puede estar configurado para trabajar con una memoria caché, en la que residirán los datos a los que tienen acceso, con mayor frecuencia, los usuarios de la aplicación.

- Ejecutar el procesamiento de los datos y enviar los resultados al equipo cliente que ha cursado la solicitud, o a la base de datos.
- Liberar, una vez finalizado el proceso, los recursos que se le hayan dedicado.
- Gestionar estadísticas sobre la utilización de recursos.

En este último punto debe hacerse una matización: las aplicaciones Meta4 permiten obtener acceso a datos que se han almacenado temporalmente en una caché de datos. Ésta puede residir en el puesto cliente, en el servidor de aplicaciones o en ambos.

Las cachés dinamizan las operaciones de lectura de datos, ya que evitan la necesidad de realizar una operación de lectura en el SGBD.

En la caché se almacenan datos que no son susceptibles de cambiar con frecuencia: por ejemplo, tablas maestras, de referencia o de validación (tabla de países, provincias, etc.).

Cuando se define un conjunto de datos almacenables en caché, se puede especificar un periodo de actualización de los datos, de modo que, en cuanto se supere este tiempo, el sistema vuelva a leer los datos de la base de datos para actualizar el contenido de la caché.

Si, en ese instante, hay un usuario trabajando con datos de esa caché, el sistema le presentará una notificación, indicándole que se va a actualizar el contenido de la caché y que puede estar trabajando con datos desactualizados. Para información más detallada, véase el apartado **Funcionamiento de las cachés**

Componentes del servidor de aplicaciones

El servidor de aplicaciones esta compuesto por subsistemas que utilizan distintos componentes para la gestión de las transacciones cliente/servidor:

- Base de datos lógica: se encarga de gestionar las operaciones de lectura y escritura sobre la base de datos y controlar la validez de los datos antes de enviarlos al SGBD relacional.
- Meta4Object Engine: se encarga de la carga en memoria de Meta4Objects y de la gestión optimizada de la memoria RAM del servidor de aplicaciones.
- Intérprete/ejecutor LN4: se encarga de ejecutar el código LN4 de las reglas de métodos y conceptos (la funcionalidad y las instrucciones de procesamiento se implementan mediante reglas LN4 de los elementos de tipo método y concepto).



LN4 es el lenguaje de programación desarrollado por Meta4 para implementar la lógica de la aplicación. LN4 ofrece la eficacia y capacidad expresiva de los lenguajes de programación orientados a Meta4Objects.

El ejecutor LN4 interpreta y ejecuta las instrucciones que se hayan indicado en estas reglas. LN4 permite:

- Tratar los registros de Meta4Objects que se hayan cargado en la memoria principal del servidor de aplicaciones: inserciones, actualizaciones, borrados, etc.
- Solicitar a la BDL la grabación en la base de datos física de todos los cambios que se hayan realizado sobre los registros disponibles en la memoria principal.
- Ejecutar sentencias SQL en el servidor de bases de datos, etc.

Servidores de presentación: puestos cliente

Los puestos cliente desde los cuales los usuarios trabajan con la aplicación, implementan el nivel de presentación del sistema. Meta4 distribuye varios tipos de software cliente:

- **Puestos cliente clásico o Visual Basic:** implementan el nivel de presentación, y permiten ejecutar procesos de aplicación que no requieran recursos de máquina elevados. Dentro de los Clientes Visual Basic, cabe diferenciar los clientes distribuidos (DC), que trabajan contra el servidor de aplicaciones, y los clientes de desarrollo (SMOLTP), que emulan la funcionalidad de aquél y se conectan directamente al servidor de base de datos para trabajar en modo local.
- **Puestos clientes en entorno Web:** Los clientes ligeros permiten acceso a determinados módulos de la aplicación a través de un explorador capaz de interpretar HTML estándar. No obstante, para obtener acceso a la aplicación de Meta4 desde un explorador, es necesario haber configurado correctamente la integración del servidor de aplicaciones con un servidor Web.
- **Puestos cliente rico:** son parecidos a los clientes Visual Basic salvo que el interfaz final es mediante un explorador capaz de interpretar HTML estándar. Estos clientes implementan el nivel de presentación en un entorno HTML, y permiten ejecutar procesos de aplicación en local en lugar del servidor en la medida que sea posible. También cabe diferenciar los clientes distribuidos (DC), que trabajan contra el servidor de aplicaciones, y los clientes de desarrollo (SMOLTP), que emulan la funcionalidad de aquél y se conectan directamente al servidor de base de datos para trabajar en modo local.

Interfaz del puesto cliente

Los puestos cliente se basan en una interfaz gráfica de usuario desarrollada en VB, que hace un uso extensivo de las tecnologías basadas en componentes ActiveX. En lo referente a su funcionalidad, implementa los servicios de:

- Inicio de sesiones de trabajo con los servidores de aplicaciones.
Al iniciar una sesión de trabajo, el usuario debe identificarse ante el servidor de aplicaciones mediante un nombre de usuario y una contraseña.
- Solicitud a los servidores de aplicaciones de la ejecución de procesos de trabajo, consultas, informes, etc., empleando uno de los tipos de ejecución gestionados por los servidores de aplicaciones: OLTP, Proxy o Delta desde el Planificador de tareas (los tipos de ejecución se explican en este mismo capítulo)
- Gestión de la presentación en pantalla de los resultados de los procesos ejecutados en los servidores de aplicaciones.
- Posibilidad de que el usuario cambie los modos de presentación y visualización de datos (estas operaciones se realizan en el equipo cliente y no requieren ningún tipo de interacción con los servidores de aplicaciones).
- Cierre de conexiones con los servidores de aplicaciones.

Entre las características más importantes de la interfaz del puesto cliente, destacan:

- Paneles de navegación, que permiten al usuario desplazarse a través de los distintos componentes que configuran un Meta4Object, o Meta4Objects diferentes (la implementación física de éstos, que puede estar distribuida en un número variable de tablas, es totalmente transparente para los usuarios de la aplicación).
- Herramientas de entrada de datos, con controles que asisten al usuario y facilitan la interacción con la aplicación: listas de validación, botones de fechas, grupos de remonte, etc.
- Módulos de entrada masiva de datos, concebidos para aumentar la productividad de los usuarios finales.
- Distintas visualizaciones de los datos.
La misma información puede analizarse desde varios puntos de vista, mediante los modos de presentación disponibles por defecto: formulario, entrada rápida, árboles de datos, etc.
- Representación visual de los datos de tablas de histórico.
En una misma ventana, el usuario puede analizar, simultáneamente, la trayectoria de todos aquellos aspectos susceptibles de evolucionar con el paso del tiempo.
- Entorno de ejecución de informes y módulo de consulta a la base de datos.

- Integración con aplicaciones de terceros; desde la interfaz, el usuario puede:
 - Exportar los datos extraídos desde la herramienta Consulta a hojas de cálculo, bases de datos locales, etc.
 - Ejecutar aplicaciones compatibles con la especificación OLE 2.0.
- Posibilidad de que el usuario personalice el aspecto y los componentes visibles de la interfaz.

Cliente ligero

La tecnología Meta4 además ha desarrollado una arquitectura basada en un cliente 100% HTML, **cliente ligero**. El cliente ligero está compuesto básicamente de páginas HTML que se cargan dinámicamente bajo demanda en el servidor Web, y que se envían a los puestos clientes a través de un navegador capaz de interpretar HTML estándar.

El cliente ligero se compone de un conjunto de módulos (*servlets* y *taglets*) que son asociados al servidor web de cualquier intranet para ofrecer un sistema de comunicación abierto a cualquier navegador capaz de interpretar el lenguaje HTML estándar.

Para acceder a la información contenida en la base de datos, tan sólo es necesario disponer de un navegador web que reconozca el lenguaje HTML. El cliente ligero, situado en el servidor web se encargará de convertir las peticiones recibidas en formato HTML a XML, solicitando al servidor la gestión de las peticiones recibidas.

Tras acceder a la base de datos y realizar las tareas solicitadas, devuelve el resultado al servidor web, en donde el cliente ligero se encarga de transformar esta respuesta en código HTML. Este código es interpretado por el navegador Web, quien muestra al usuario la información solicitada a través de una interfaz gráfica de fácil acceso.

Tipos de ejecución en arquitectura C/S

La arquitectura cliente/servidor de la aplicación permite trabajar con tres tipos de ejecución:

- OLTP (*On Line Transaction Processing*) o transaccional
- Delta o transacción optimizado
- Proxy o conversacional
- Ejecución de procesos programada desde el Planificador de tareas

Los tres primeros tipos de ejecución, OLTP, Delta y Proxy necesitan una conexión en tiempo real entre los puestos cliente y los servidores de aplicaciones. La ejecución programada no necesita una conexión en tiempo real entre el cliente y el servidor.

Un servidor, en función del número de módulos de ejecución que tenga establecidos, estará capacitado para desarrollar transacciones de tipo OLTP, proxy, o de ejecución programada en mayor o menor medida. Cuanto mayor sea el número de módulos de ejecución asignados a un servicio de ejecución, mayor será el número de ejecuciones que se podrán procesar de forma concurrente en ese servicio, siempre teniendo en cuenta los límites físicos o configurados de la máquina.

OLTP o modo transaccional

Este tipo de ejecución requiere intercambios de mensajes en tiempo real entre el cliente y el servidor. El cliente establece una conexión con el servidor y le solicita la ejecución de un proceso. Mediante un módulo de ejecución OLTP, el servidor ejecuta el proceso solicitado, envía los resultados al cliente y cierra la transacción. Tras enviar los resultados al cliente, el servidor no retiene ninguna información del cliente ni del intercambio de datos que acaban de realizar. Puede decirse que el modo OLTP está orientado a conexión.

Delta o modo transacción optimizado

En el modo Delta, el cliente establece una sesión de trabajo con el servidor y le solicita la ejecución de un proceso. Mediante un módulo de ejecución OLTP, el servidor ejecuta el proceso solicitado, envía los resultados al cliente y cierra la transacción.

A diferencia de lo que sucede en el modo OLTP puro, el servidor retendrá información sobre el cliente y sobre el estado de la transacción. Para ello, se guarda los datos del Meta4Object en formato binario en un caché especial de objetos en el servidor en preparación para la siguiente transacción desde el

cliente. Desde ahí se pueden recuperar los datos que no han sido enviados por el cliente. De este modo, se permite reducir los datos enviados desde el cliente al servidor en cada transacción, trabajando sólo con los cambios respecto a la última transacción cliente servidor.

Proxy o modo conversacional

En el modo Proxy, el cliente establece una sesión de trabajo con el servidor y le solicita la ejecución de un proceso. Mediante un módulo de ejecución Proxy, el servidor ejecuta el proceso y envía los resultados al cliente. Sin embargo, a diferencia de lo que sucede en el modo OLTP, el servidor retendrá información sobre el cliente y sobre el estado de la transacción.

Por ejemplo, una ejecución en modo Proxy permite al cliente ejecutar una consulta compleja a la base de datos, y recuperar los resultados de manera progresiva. El servidor mantiene la información sobre los registros que ha enviado al cliente en cada una de las interacciones. Se dice que el modo Proxy está orientado a sesiones.

Ejecución programada: Planificador de tareas

La característica más importante de este tipo de ejecución es que no precisa una conexión en tiempo real entre el cliente y el servidor de aplicaciones. El puesto cliente solicita la ejecución de un proceso a un servidor determinado, o a cualquier servidor de aplicaciones de la red corporativa. En su solicitud, el cliente indica la fecha y hora en la que quiere que comience la ejecución del proceso; también puede indicar que el proceso debe ejecutarse tan pronto como sea posible.

El proceso se ejecuta en la fecha y hora programadas, sin necesidad de que intervenga el usuario.

Las solicitudes se cursan desde una herramienta especial disponible en los puestos cliente de la aplicación.

Al solicitar la ejecución de un proceso desde esta herramienta, se registra la solicitud en la base de datos. Periódicamente, los servidores de aplicaciones leen en la base de datos las solicitudes que se les han dirigido.

Para información más detallada sobre este tipo de ejecución, consulte la ayuda en línea de la aplicación.

Procesamiento distribuido en C/S

Toda la funcionalidad de la aplicación se implementa mediante Meta4Objects. Los Meta4Objects encapsulan datos e instrucciones de procesamiento. Los datos se cargan en elementos de tipo campo y propiedad. El procesamiento se codifica en las reglas de los elementos de tipo método y concepto utilizando el lenguaje LN4. Los elementos de un Meta4Object pueden estar distribuidos en un número variable de nodos conectados que formen una estructura jerárquica.

Procesamiento distribuido de Meta4Objects

En el momento de diseñar un Meta4Object, el usuario-configurador puede distribuir la carga de datos y la ejecución de procesos entre el cliente y el servidor de aplicaciones, pudiendo indicar dónde:

- Deben cargarse los datos correspondientes a los elementos de tipo campo y propiedad: en la memoria principal del servidor de aplicaciones, en la del puesto cliente o en ambas
- Va a ejecutarse cada uno de los elementos de tipo método o concepto: en el puesto de trabajo cliente o en el servidor de aplicaciones.

De esta modo, un mismo Meta4Object puede distribuir sus elementos entre el servidor de aplicaciones y el puesto cliente. La distribución del procesamiento entre las dos máquinas permite optimizar los recursos disponibles, además de sacar el máximo partido de la capacidad de los equipos.

Meta4Object en servidor y en cliente

En el contexto cliente/servidor, en la estructura de un Meta4Object se distinguen dos partes: Meta4Object en servidor y Meta4Object en cliente.

Meta4Object en cliente hace referencia a los elementos que se cargan o ejecutan en la memoria principal del puesto cliente; Meta4Object en servidor designa a los elementos del mismo que se cargan y ejecutan en la memoria principal del servidor de aplicaciones.

Meta4Object en cliente y en servidor forman parte de un mismo Meta4Object, y pueden compartir componentes comunes; algunos elementos formarán parte tanto del Meta4Object en servidor como del Meta4Object en cliente.

En el momento de diseñar el Meta4Object, el usuario-configurador puede indicar si un elemento va a formar parte del Meta4Object en servidor, del Meta4Object en cliente o de los dos. Si forma parte de ambos, se podrá indicar si deben sincronizarse o no los valores recogidos en cada parte.

A continuación se explican los distintos tipos de elementos según su configuración cliente/servidor:

Modos de serialización o transmisión de datos

En la arquitectura cliente/servidor, los puestos cliente intercambian datos e instrucciones de ejecución de métodos con los servidores de aplicaciones. La comunicación entre clientes y servidores se establece a partir de la distribución de los elementos del Meta4Object entre el cliente y el servidor. La aplicación de un modo de comunicación dado en un Meta4Object determina la forma en la que se van a intercambiar datos entre el cliente y el servidor. La comunicación entre clientes y servidores de aplicaciones puede ser de varios tipos: OLTP, Delta o Proxy. Las diferencias entre estos consisten en:

- El volumen de datos que viajan desde el cliente al servidor y vice versa.
- Al finalizar la ejecución en el servidor, si se mantiene el Meta4Object vivo en el servidor y cómo, o si se destruye este Meta4Object.
- Al finalizar la transmisión, si se mantiene la conexión física con la base de datos que puede consumir muchos recursos pero que se necesita bajo ciertas condiciones.

A continuación en la tabla abajo se muestra las diferencias entre cada uno:

Característica	OLTP	DELTA	PROXY
Datos que viajan desde cliente	Meta4Object entero	Meta4Object entero en la primera transacción. Después, Meta4Object con registros cambiados	Meta4Object entero en la primera transacción. Después, Meta4Object con registros cambiados
Datos que viajan desde servidor	Meta4Object con registros cambiados	Meta4Object con registros cambiados	Meta4Object con registros cambiados
Meta4Object en servidor al finalizar la ejecución	Se destruye	Se destruye	Se mantiene vivo en un espacio Proxy
Copia de Meta4Object en caché de objetos	No	Sí	No
Conexión física con la base de datos al finalizar la transmisión	Se libera	Se libera	Se mantiene viva

Meta4Objects en C/S: metodología de desarrollo

La carga en memoria y la ejecución de un Meta4Object pueden distribuirse entre el cliente y el servidor:

- Un Meta4Object en servidor, que incluye una serie de elementos de tipo campo, propiedad, método y concepto, que se cargan y ejecutan en el servidor de aplicaciones.
- Un Meta4Object en cliente, formado por una serie de elementos de tipo campo, propiedad, método y concepto, que se pueden utilizar y ejecutar en los puestos cliente.

Parametrización de Meta4Objects C/S

Para poder distribuir los datos y la capacidad de procesamiento correspondientes a un Meta4Object entre el puesto cliente y el servidor de aplicaciones, el usuario-configurador debe parametrizar el Meta4Object en el momento de su creación.

En la parametrización del Meta4Object, deben seguirse los siguientes pasos:

1. Indicar cuál va a ser el comportamiento del Meta4Object en un entorno de ejecución cliente/servidor: es decir, indicar si es o no **Separable**.
2. Indicar cuál va a ser el tipo de ejecución de un Meta4Object en cliente/servidor: OLTP, Delta o Proxy.
3. Parametrizar la propiedad **ID Tipo C/S** a nivel de nodo, de estructura de nodo o de elemento, del Meta4Object. En esta propiedad se indica si el componente es de tipo Parte servidor, Parte cliente, Parte completa, Ambas partes (sync.). Los elementos heredan los valores indicados a nivel de estructura de nodo y de nodo, a menos que se indique otro valor a nivel de elemento para estas propiedades. En este caso, siempre se da prioridad a lo indicado a nivel de elemento. La propiedad ID Tipo de sincronización se puede parametrizar únicamente a nivel de elemento.
4. Parametrizar la propiedad **ID Tipo C/S** a nivel de los elementos de conector. A diferencia a los otros componentes, las propiedades disponibles son Parte servidor, Parte cliente y Parte completa.
5. Si el Meta4Object es de tipo OLTP, se tendrá que parametrizar la propiedad **ID Tipo de sincronización** en los elementos con ejecución (métodos y conceptos) de tipo Ambas partes (sync.). Conviene recordar que estos elementos siempre se ejecutan en el servidor de aplicaciones.

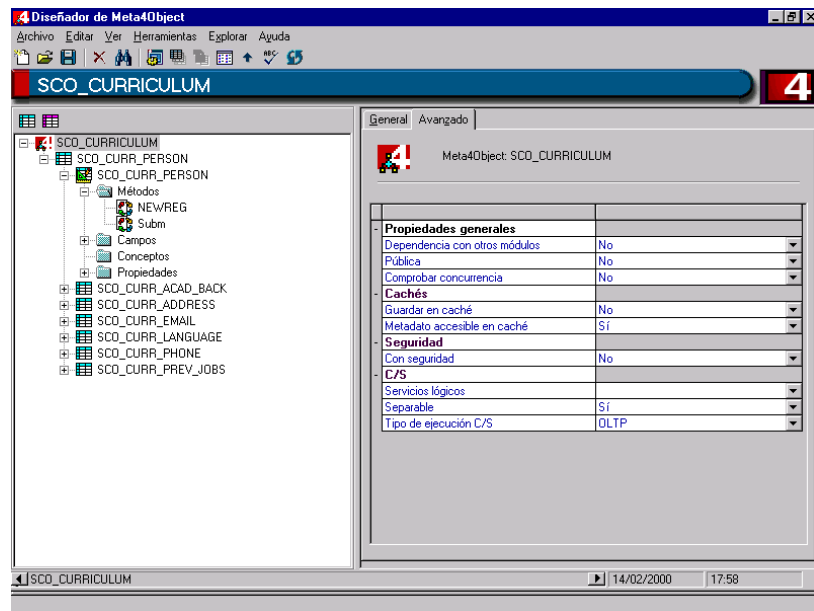
Sin embargo, si el Meta4Object es de tipo Delta, no hará falta parametrizar esta propiedad en los elementos con ejecución de tipo Ambas partes (sync.), porque el sistema se encarga de ello.

- Si el Meta4Object tiene varios nodos, deberá rellenarse la propiedad **ID Tipo C/S** para cada par de elementos conectados.

Parametrización del comportamiento del Meta4Object

El comportamiento de un Meta4Object en un entorno de ejecución distribuido cliente/servidor se parametriza mediante las propiedades **Separable** y **Tipo de ejecución C/S**, disponibles desde la herramienta **Diseñador de Meta4Objects**.

Figura 77. Diseñador de Meta4Objects



La siguiente tabla describe el significado de cada una de estas propiedades:

Propiedad	Descripción
Separable	Se utiliza para indicar si la carga de datos y el procesamiento del Meta4Object puede distribuirse entre el cliente y el servidor de aplicaciones. Si se asigna a esta propiedad el valor: • Sí: el Meta4Object se distribuirá entre el cliente y el servidor. Por defecto, un Meta4Object siempre será separable. • No: el Meta4Object no podrá distribuirse. En este último caso, el Meta4Object se comportará como si todos sus elementos fuesen de tipo Parte completa; es decir, todos los elementos serán visibles tanto para el puesto cliente como para el servidor de aplicaciones, y los métodos y conceptos se ejecutarán en la máquina en la que se solicita su ejecución. Si se indica que el Meta4Object no es separable, no se producirá ninguna interacción entre el cliente y el servidor de aplicaciones y no se obtendrá acceso a la base de datos lógica.
Tipo de ejecución C/S	Se utiliza para indicar el tipo de ejecución que se utilizará en el Meta4Object en cada transacción: • OLTP: En cada transacción viaja el Meta4Object entero de cliente a servidor y sólo vuelven al cliente los deltas o los registros que han cambiado del Meta4Object. Se destruye el Meta4Object en la parte servidor y también se libera la conexión física a la base de datos. • Delta: sólo viajan los deltas del Meta4Object del cliente al servidor y también vuelven al cliente los deltas o los registros que han cambiado del Meta4Object. El viaje del cliente al servidor es en modo OLTP, lo que significa que al acabar la ejecución en servidor se destruye el Meta4Object en la parte servidor y se libera la conexión física a la base de datos para ahorrar el uso de recursos. Además, en la primera transacción siempre viaja el Meta4Object entero del cliente al servidor. Al acabar la transacción de un Meta4Object Delta se guardan los datos del Meta4Object en el servidor en un caché, para que en la siguiente transacción puedan recuperarse los datos que había en el servidor y que no van a ser enviados por el cliente. De esta manera se reduce los datos involucrados en cada viaje. • PROXY: sólo viajan los deltas del Meta4Object del cliente al servidor y también vuelven al cliente los deltas o los registros que han cambiado del Meta4Object. No sólo se mantiene viva la conexión física a la base de datos sino también el Meta4Object en servidor. El último significa que sólo necesitan viajar los cambios desde el cliente. Generalmente, se debe utilizar el tipo de ejecución Delta. La mayoría de los Meta4Objects serán del tipo Delta o OLTP. El tipo de ejecución Proxy sólo se utiliza para aquellos Meta4Objects que implementan funciones complejas que requieren un número elevado de interacciones entre el equipo cliente y el servidor: cálculo de nómina, etc. La diferencia entre estos tipos de ejecución se explicó, en líneas generales, en un apartado anterior: <i>Tipos de ejecución en arquitectura C/S</i>.

Parametrización de los elementos: Propiedad ID Tipo C/S

Una vez que se ha indicado que el Meta4Object se va a ejecutar en un entorno cliente/servidor, el usuario-configurador puede indicar qué elementos del Meta4Object van a formar parte de la ejecución en cliente y cuáles en el servidor.

La pertenencia de un elemento a una de estas partes se indica mediante la propiedad **ID Tipo C/S**. Esta propiedad se puede indicar a nivel de elemento, de nodo y de estructura de nodo.

Si se indica un valor para **ID Tipo C/S** a nivel de nodo, este valor se aplicará a todos los elementos que estén incluidos en el nodo, a no ser que se les haya asignado otro valor para esta propiedad a nivel de elemento. De ser así, siempre tendrá prioridad el valor asignado a nivel de elemento.

ID Tipo C/S puede recoger cuatro valores: *Parte cliente*, *Parte servidor*, *Parte completa* y *Ambas partes (sinc.)*. La siguiente tabla recoge el significado de cada uno de ellos:

Valor	Descripción
Parte cliente	El elemento sólo es visible en el puesto cliente. El servidor de aplicaciones no conoce su existencia, es decir, los elementos de este tipo no forman parte del Meta4Object en servidor. Sólo pueden ser de este tipo los elementos de tipo método, concepto o propiedad. Un elemento de tipo campo no puede ser Parte cliente, ya que todas las operaciones de lectura y escritura sobre la base de datos se gestionan a través del servidor de aplicaciones. Los métodos y conceptos de tipo Parte cliente se procesan en el puesto cliente. Su ejecución no produce, por sí misma, ningún intercambio de datos entre el cliente y el servidor.
Parte servidor	Este tipo de elementos sólo permanece visible para el servidor de aplicaciones. Los puestos cliente no conocen su existencia, es decir, no forman parte del Meta4Object en cliente. Pueden ser de tipo Parte servidor los elementos de tipo campo, método, concepto o propiedad que no tienen que mostrar su valor o el resultado de su ejecución a los puestos cliente. Como norma general, no se debe asignar esta propiedad a ningún elemento que sea necesario mostrar en el cliente. Cualquier elemento invocado por una presentación no podrá tener comportamiento Parte servidor, puesto que las presentaciones se ejecutan en el cliente. Los métodos y conceptos de tipo Parte servidor se ejecutan en el servidor de aplicaciones. Su ejecución no produce, por sí misma, ningún intercambio de datos entre el cliente y el servidor.

Valor	Descripción
Parte completa	Los elementos son visibles para el servidor de aplicaciones y para los puestos cliente, pero ni su valor ni su ejecución están sincronizados. Por ejemplo, si se tiene un elemento de tipo campo en cuya propiedad ID Tipo C/S se ha indicado el valor Parte completa, el sistema mantendrá el elemento tanto en el Meta4Object en cliente (memoria principal del puesto cliente) como en el Meta4Object en servidor (memoria principal del servidor de aplicaciones). Sin embargo, si desde el cliente se actualiza el valor de este elemento, los cambios no se reflejarán en la imagen que, de este mismo elemento, mantiene el servidor. En los elementos de tipo método o concepto, es decir, con ejecución, el elemento se ejecuta en la máquina desde la que se solicita la ejecución. La ejecución de un elemento de tipo Parte completa no desencadena ningún intercambio de datos entre el puesto cliente y el servidor de aplicaciones.
Ambas partes (sinc.)	Los elementos de tipo Ambas partes (sinc.) permanecen visibles tanto para el puesto cliente como para el servidor, y sus valores y su ejecución están sincronizados. Como en los elementos de tipo Parte completa, un elemento Ambas partes (sinc.) forma parte tanto del Meta4Object en cliente como del Meta4Object en servidor. Pueden ser de este tipo los elementos de tipo campo, propiedad, método o concepto. Si desde un puesto cliente se modifica el valor de un elemento de este tipo, el nuevo valor se reflejará en el Meta4Object en servidor en cuanto se solicite la ejecución de cualquier método de tipo Ambas partes (sinc.). En los elementos que requieren ejecución, es decir, los de tipo método o concepto, cuando su ID Tipo C/S es Ambas partes (sinc.), la ejecución se hará en el servidor de aplicaciones, aunque se haya solicitado desde el puesto cliente. El resultado de la ejecución del elemento se hará visible, inmediatamente, en el puesto cliente. Es importante recordar que en la transferencia de datos entre los puestos cliente y los servidores de aplicaciones sólo se serializan los elementos de este tipo.

Como conclusión, se recuerda que:

- Forman parte del Meta4Object en cliente los elementos cuya propiedad **ID Tipo C/S** recoge los valores **Parte cliente, Parte completa y Ambas partes (sinc.)**.
- Forman parte del Meta4Object en servidor los elementos cuya propiedad **ID Tipo C/S** recoge los valores **Parte servidor, Parte completa y Ambas partes (sinc.)**
- Los elementos de tipo campo que van a mostrarse en la interfaz del puesto cliente, o que van a ser requeridos en la ejecución de métodos o conceptos procesables por los equipos cliente, deben ser de tipo **Ambas partes (sinc.)**
- Los elementos de tipo campo que no necesitan mostrarse en la interfaz del puesto cliente, y que no van a ser requeridos en la ejecución de métodos o conceptos, deben ser de tipo **Parte servidor**

- Los métodos que se pueden ejecutar desde la interfaz de usuario deben ser de tipo **Ambas partes (sinc.)**, **Parte cliente** o **Parte completa**
- La ejecución de un método de tipo **Ambas partes (sinc.)** se puede solicitar desde la interfaz del puesto cliente o desde el servidor de aplicaciones, pero su ejecución se procesa en el servidor de aplicaciones
- La ejecución de un método de tipo **Parte cliente** se puede solicitar desde la interfaz del puesto cliente y su ejecución se procesa en el puesto cliente
- Un método o concepto de tipo **Parte cliente** sólo puede utilizar aquellos elementos que forman parte del Meta4Object en cliente
Si en su regla se hace referencia a un elemento de tipo **Parte servidor**, se producirá un error de compilación.
- Un método o concepto de tipo **Parte servidor** sólo puede utilizar aquellos elementos que forman parte de la ejecución del servidor del Meta4Object
Si en su regla se hace referencia a un elemento de tipo **Parte cliente**, se producirá un error.
- La ejecución de un método o concepto de tipo **Ambas partes (sinc.)** se puede solicitar desde el puesto cliente o desde el servidor de aplicaciones (sin embargo, como el elemento se ejecuta en el servidor, su código sólo obtiene acceso a los elementos del Meta4Object en servidor)
- En los intercambios de datos entre los puestos cliente y los servidores de aplicaciones, sólo se serializan los elementos de tipo **Ambas partes (sinc.)**
Los elementos de tipo **Parte servidor**, **Parte cliente** o **Parte completa** no originan ningún intercambio de datos entre el cliente y el servidor.

Posibles usos de los elementos en C/S

La siguiente tabla muestra una lista con la parametrización de los elementos de un Meta4Object en un entorno de desarrollo cliente/servidor.

Como norma general, el usuario deberá tener en cuenta estas consideraciones metodológicas a la hora de parametrizar un Meta4Object, para evitar que se produzcan inconsistencias en un entorno de trabajo C/S.

En el caso de que la propiedad **ID Tipo C/S** no esté parametrizada a nivel de elemento, el comportamiento de este elemento tomará los valores definidos a nivel de nodo. Si la propiedad **ID Tipo C/S** no está parametrizada en ningún caso, los elementos tomarán el comportamiento de tipo Parte completa.

Tipo Elemento	ID Tipo C/S	Usos-Descripción
Campo	Parte cliente	Un elemento de tipo campo no puede ser Parte cliente, ya que todas las operaciones de lectura y escritura sobre la base de datos se gestionan a través del servidor de aplicaciones.
	Parte servidor	Es aconsejable utilizar estos elementos en la parte servidor para campos que se usen de forma auxiliar y que no sea necesario tener en el client, y que por lo tanto el usuario no necesita visualizar. No conviene invocarlos desde las reglas de elementos, ni desde conectores del tipo Parte completa.
	Parte completa	Un elemento de tipo campo extrae los valores de la base de datos, por lo tanto no es conveniente tener un dato de este tipo con valores en cliente y servidor diferentes.
	Ambas partes (sync.)	Esta opción es recomendable para los campos que sea necesario visualizar en el cliente, para los elementos conectados a un elemento de tipo Parte cliente o para los que son invocados por una regla que se ejecuta en el cliente.

Tipo Elemento	ID Tipo C/S	Usos-Descripción
Propiedad	Parte cliente	Los elementos de tipo propiedad en Parte cliente se utilizan para almacenar valores que fueron rellenos desde la regla de un método o concepto, o desde una presentación, pero que no necesita utilizar ninguna regla que se ejecute en el servidor. Los elementos de tipo propiedad en Parte cliente no deberían estar conectados a ningún elemento en parte servidor, ni deberían invocarlos reglas de elementos tipo Parte servidor, Parte completa o Ambas partes (sinc.).
	Parte servidor	Los elementos de tipo propiedad en Parte servidor permiten calcular un valor por defecto o almacenar valores que sirvan para sincronizar la ejecución de reglas de métodos o conceptos que se ejecuten siempre en el servidor. Los elementos de tipo propiedad en Parte servidor no estarán conectados con elementos en Parte cliente, ni podrán invocarlos las reglas de elementos en Parte cliente o Parte completa.
	Parte completa	Los elementos de tipo propiedad en Parte completa sirven para almacenar valores que usan las reglas de métodos o conceptos tanto en el cliente como en el servidor. Los elementos de tipo propiedad en Parte completa almacenan valores que sirven para sincronizar la ejecución de reglas, pero no importa que no coincidan los valores del cliente y del servidor, puesto que sólo se necesita el valor en el entorno en el que se estén ejecutando las reglas.
	Ambas partes (sinc.)	Los elementos de tipo propiedad Ambas partes (sinc.) se utilizan para almacenar valores introducidos por el usuario y necesarios para la ejecución de reglas de métodos o conceptos de tipo Ambas partes (sinc.) o Parte servidor.

Tipo Elemento	ID Tipo C/S	Usos-Descripción
Conceptos	Parte cliente	Los elementos de tipo concepto en Parte cliente sirven para realizar cálculos que devuelvan un valor necesario en la Parte cliente. No se deben utilizar conceptos del tipo Parte cliente cuando estén asociados a métodos o conceptos que sean Parte servidor o del tipo Ambas partes (sinc.), ni se podrá invocar dentro de su regla a elementos de tipo parte servidor.
	Parte servidor	Los elementos de tipo concepto en Parte servidor sirven para realizar cálculos que no precisen ser usados en la parte cliente.
	Parte completa	Los elementos de tipo concepto Parte completa permiten realizar cálculos que sean útiles tanto en el cliente como en el servidor, pero sin que sea necesario que el valor resultante esté sincronizado entre el cliente y el servidor.
	Ambas partes (sinc.)	Los elementos de tipo concepto Ambas partes (sinc.) sirven para realizar cálculos cuyo valor sea necesario tener sincronizado entre el cliente y el servidor. Desde la regla de un concepto Ambas partes (sinc.) no se podrá acceder a datos del tipo Parte cliente.
Métodos	Parte cliente	Los elementos de tipo método Parte cliente permiten realizar ejecuciones sólo en la Parte cliente. Desde las reglas de estos métodos podemos invocar elementos de tipo Parte cliente, Parte completa o Ambas partes (sinc.).
	Parte servidor	Los elementos de tipo método Parte servidor permiten realizar ejecuciones sólo en la Parte servidor. Desde las reglas de estos métodos podemos invocar elementos de tipo Parte servidor, Parte completa y Ambas partes (sinc.).
	Parte completa	Son métodos que podrán ser invocados tanto en el cliente como en el servidor. Como norma general, en las reglas de estos métodos sólo se deberían poder ejecutar llamadas a elementos de tipo Parte completa o Ambas partes (sinc.).
	Ambas partes (sinc.)	Son métodos que se ejecutan en el servidor, pero que pueden invocarse tanto en el cliente como en el servidor. Cuando se invoca uno de estos métodos desde una regla o mediante un conector, la ejecución pasa al servidor. Los métodos del sistema para cargar o guardar datos deberían estar parametrizados con este tipo de comportamiento.

Parametrización de la transmisión de datos para elementos de tipo Ambas partes (sinc.)

Los intercambios de datos sólo se producen cuando se ejecuta un elemento de tipo Ambas partes (sinc.) en cliente. Los datos que se transfieren en cada transacción en elementos de tipo Ambas partes (sinc.) dependen del tipo de ejecución especificado para el Meta4Object.

Básicamente existe dos tipos de volúmenes a transferir que se considera en cada tipo de ejecución que afecta la parametrización:

- **Datos Parciales o Deltas:** contiene únicamente los cambios, también se conoce como Delta. Entre el cliente y el servidor sólo se transfieren la información correspondiente a los elementos de tipo Ambas partes (sinc.) que se hayan modificado desde el Meta4Object en cliente o desde el Meta4Object en servidor. Las modificaciones incluyen los registros que se hayan agregado, eliminado, y modificado desde el último intercambio de datos.
- **Datos Completos o Full:** transfiere todo. Entre el cliente y el servidor se transfieren todos los elementos de tipo Ambas partes (sinc.), se hayan modificado o no.

Los datos que vuelven desde el servidor siempre son datos parciales o deltas, sin embargo los datos que viajan desde el cliente varían entre tipos de ejecución.

A continuación se describen los efectos de cada tipo de ejecución sobre estos datos que vienen desde el cliente y cuando se puede requerir alguna parametrización.

Tipo de ejecución de OLTP

En este modo, en los Meta4Objects por defecto viajan enteros desde el cliente hasta el servidor, sino se establece ningún modo de serialización avanzado.

Sin embargo, los intercambios de datos que parten del servidor hacia el puesto cliente siempre se hacen según el modo Delta.

Limitaciones

Para optimizar los datos que viajan entre el cliente y servidor en el tipo de ejecución OLTP, se debe recurrir a la parametrización manual.

Mediante uno de los modos de transmisión avanzados que se describen a continuación, es posible elegir el modo de transmisión que se va a utilizar entre el puesto cliente y el servidor de aplicaciones.

Modos avanzados de transferencia o serialización

Estos modos avanzados de serialización sólo se pueden aplicar en **Meta4Objects OLTP**, para optimizar el conjunto de datos que se envían desde los puestos cliente al servidor de aplicaciones. En una interacción cliente/servidor realizada con un Meta4Object de tipo OLTP, los datos que envía el servidor al puesto cliente en respuesta a una solicitud siempre se envían en modo de transmisión Delta.

El tipo de transmisión que se quiere utilizar en cada caso se indica en el momento de diseñar el Meta4Object mediante la propiedad **ID Tipo de sincronización**.

El modelo de ejecución seleccionado para el Meta4Object, proxy, determina de manera óptima qué datos se van a transferir entre el puesto cliente y el servidor, y viceversa, cada vez que se ejecute un método Ambas partes (sinc.).

Si al Meta4Object le corresponde un modelo de ejecución proxy, los intercambios entre cliente y servidor (excepto el primero de todos, que parte del cliente y va al servidor) se harán de acuerdo con el modelo Delta; es decir, sólo se transmitirá información sobre los elementos de tipo Ambas partes (sinc.) que se hayan modificado.

Si le corresponde un modelo de ejecución OLTP, los intercambios de datos que vayan del puesto cliente al servidor se harán según el modo Full, y los que vayan del servidor al cliente, de acuerdo con el modo Delta.

Con los Meta4Objects de tipo OLTP, el usuario-configurador puede especificar con mayor precisión qué datos se deben transmitir cada vez que se ejecute un elemento de tipo Ambas partes (sinc.).

En líneas generales, se envían únicamente los datos que se utilizan en la regla del método cuya ejecución se solicita desde el puesto cliente. De esta forma, el cliente no envía al servidor de aplicaciones todos los elementos de tipo Ambas partes (sinc.); sólo envía los necesarios para la ejecución del método.

Esta información se indica en la propiedad **ID Tipo de sincronización** que puede tomar los siguientes valores: Documento, Persist, Bloque, Bloque y sus hijos, Registro, Registro y sus hijos, y Nada. Para información más detallada, véase el apartado Modos de serialización o transmisión de datos.



La propiedad **ID Tipo de sincronización** sólo es aplicable a los elementos de tipo método que sean Ambas partes (sinc). En el resto de los casos, no se tendrá en cuenta el valor indicado para esta propiedad.

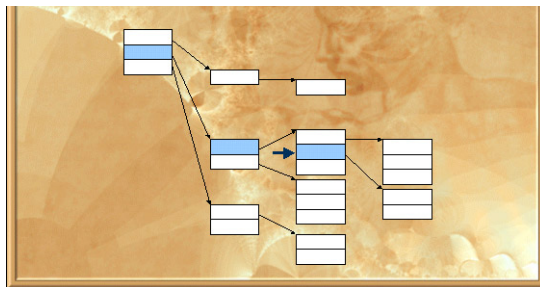
Los modos de transmisión avanzados disponibles para el **ID Tipo de sincronización** se describen a continuación.

Registro

Cuando el puesto cliente solicite la ejecución del método, se transmitirán al servidor de aplicaciones:

- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de registro, con los valores correspondientes al registro que contiene el elemento cuya ejecución ha desencadenado el intercambio de datos
- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de bloque, con el valor correspondiente al bloque de memoria que contiene el elemento cuya ejecución ha desencadenado el intercambio de datos
- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de nodo, del nodo que contiene el elemento cuya ejecución ha desencadenado el intercambio de datos
- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de registro, del registro padre del registro que contiene el elemento cuya ejecución ha desencadenado el intercambio de datos
- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de bloque, del bloque en el que está incluido el registro padre del registro que contiene el elemento cuya ejecución ha desencadenado el intercambio de datos
- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de nodo, del nodo en el que está incluido el registro padre del registro que contiene el elemento cuya ejecución ha desencadenado el intercambio de datos

Figura 78. Modo de transmisión de tipo registro



Registro y sus hijos

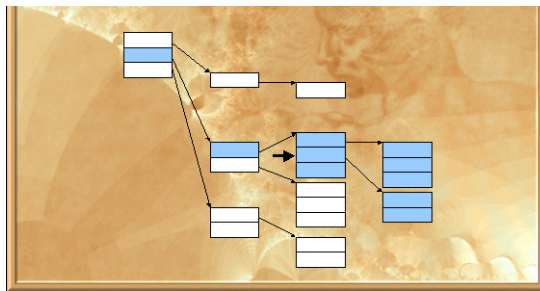
Transmite los mismos datos que el modo Registro, y además:

- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de registro, de todos los registros hijos del registro que contiene el elemento cuya ejecución ha desencadenado la transmisión de datos
- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de bloque, del bloque que contiene los registros hijos del registro que contiene el elemento cuya ejecución ha desencadenado la transmisión de datos

- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de bloque, de todos los bloques hijos del bloque que contiene el elemento cuya ejecución ha desencadenado la transmisión de datos.
- Los elementos de tipo Ambas partes (sinc.) con alcance a nivel de nodo, del nodo hijo del nodo que contiene el elemento cuya ejecución ha desencadenado la transmisión de datos.

Esta propagación por los nodos superiores del Meta4Object continúa hasta alcanzar el nodo raíz.

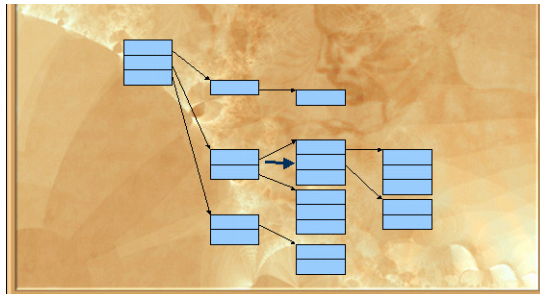
Figura 81. Modo de transmisión bloque y sus hijos



Documento

Cuando el puesto cliente solicite la ejecución del método, se transmitirán al servidor de aplicaciones todos los elementos de tipo Ambas partes (sinc.) del Meta4Object en el que está incluido el método, cualquiera que sea su alcance.

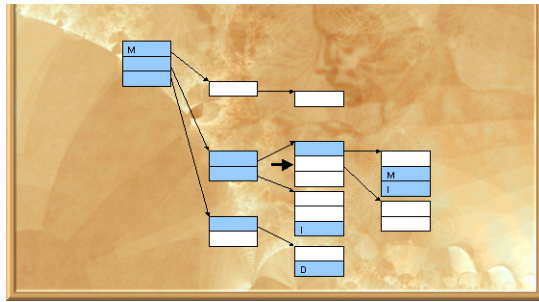
Figura 82. Modo de transmisión Documento



Persist

Se transmite únicamente la información correspondiente a los elementos de tipo Ambas partes (sinc.) que se hayan modificado (cualquiera que sea su alcance).

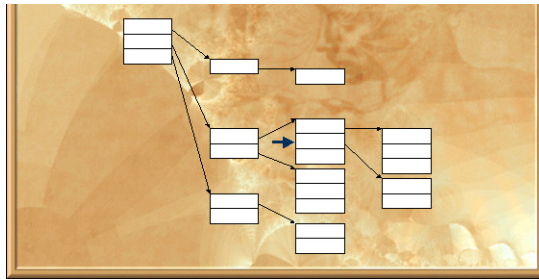
Figura 83. Modo de transmisión Persist



Ninguno

En los modos de transferencia avanzados no se envía nada más aparte de los datos a nivel de nodo de todos los nodos.

Figura 84. Modo de transmisión Ninguno



Tipo de ejecución Delta

Este modo es el que se debería utilizar por defecto en la mayoría de los casos, porque sobre todo ahorra los datos que viajan y no requiere una parametrización adicional o especial debido a sus características de transmisión de datos descritos anteriormente.

Resumiendo, en modo Delta, en el primer intercambio que se produzca entre el cliente y el servidor, se transmitirán todos los elementos de tipo Ambas partes (sync.), modificados o no, ya que el servidor necesita mantener la información relativa a la estructura y contenido del Meta4Object en cliente.

Todos los intercambios posteriores que se produzcan desde el cliente al servidor, serán de tipo Delta.

En el servidor al empezar a ejecutar se recupera el estado de la última transacción de los Meta4Objects que viajan de cliente a servidor (y sólo de esos) y deserializar los deltas sobre esos objetos.

Al finalizar la transacción se guardan los Meta4Objects que vuelven al cliente,

que ya que en la siguiente transacción se recuperan los datos que había y no han sido enviados por el cliente. Estos son solamente los que viajaron de ida. Si se ha creado un nuevo Meta4Object en la parte servidor este no se guarda ni vuelve al cliente.

Tipo de ejecución de Proxy

Este modo, se utiliza en situaciones cuando, por ejemplo, se necesita una carga parcial de datos, como por ejemplo, en los informes.

Resumiendo, en modo Proxy, en el primer intercambio que se produzca entre el cliente y el servidor, se transmitirán todos los elementos de tipo Ambas partes (sync.), modificados o no, ya que el servidor necesita mantener en todo momento la información relativa a la estructura y contenido del Meta4Object en cliente.

Todos los intercambios posteriores que se produzcan desde el cliente al servidor, serán de tipo Delta. En el servidor al empezar a ejecutar se utiliza el espacio proxy reservado del Meta4Object para recuperar su estado y deserializar los deltas sobre ello.

Limitaciones

A diferencia de los tipos de ejecución Delta y OLTP, la conexión lógica no se muere, se mantiene viva, por lo tanto el consumo de recursos es más elevado en este modo, y por eso no se debe utilizar demasiado. Sin embargo, es necesario en situaciones cuando la conexión viva con la base de datos sea imprescindible.

Resumen

- Los modos de transmisión de datos que se aplican en las interacciones entre los puestos cliente y el servidor de aplicaciones dependen del tipo de ejecución seleccionado para el Meta4Object sobre el que se está trabajando.
- El tipo de ejecución del Meta4Object que se recomienda es Delta.
- Si el tipo de ejecución del Meta4Object es proxy o delta, el primer intercambio que se produzca entre el cliente y el servidor se hará según el modo de transmisión Full.

Los demás intercambios se harán de acuerdo con el modo Delta, cualquiera que sea el origen de la transmisión (el puesto cliente o el servidor de aplicaciones).

- Si el tipo de ejecución del Meta4Object es Delta, el sistema

-
- Si el tipo de ejecución del Meta4Object es OLTP, el intercambio se hará según el tipo de ejecución avanzado que se haya seleccionado.
 - El modo de transmisión sólo es parametrizable para los Meta4Objects que tienen un tipo de ejecución OLTP.
El modo de transmisión no se puede parametrizar para los Meta4Objects con tipo de ejecución Proxy ni con Delta.
 - El modo de transmisión se configura a nivel de elemento para aquellos elementos con ejecución que sean de tipo Ambas partes (sync.)
Esto se debe a que sólo los métodos y conceptos de este tipo desencadenan un intercambio de datos entre el cliente y el servidor.
 - Los Meta4Objects amigos de otro no pueden tener un tipo de ejecución distinto.

Conexión de los métodos en entornos C/S

El sistema permite configurar un método para que, antes o después de su ejecución, desencadene la ejecución de otros métodos.

En un entorno de ejecución cliente/servidor, al conectar la ejecución de métodos hay que tener en cuenta:

- Las restricciones que impone la visibilidad de los métodos (es decir, su pertenencia al Meta4Object en cliente o al Meta4Object en servidor)
- La necesidad de racionalizar el flujo de ejecución de métodos para minimizar los intercambios de datos entre el puesto cliente y el servidor de aplicaciones

Visualización de los métodos

La pertenencia de un método al Meta4Object en cliente o al Meta4Object en servidor determina los elementos a los que va a obtener acceso. Los métodos que forman parte del Meta4Object en cliente sólo pueden conectarse con otros métodos que también formen parte del Meta4Object en cliente. Los que forman parte del Meta4Object en servidor sólo pueden conectarse con otros métodos del Meta4Object en servidor.

En los dos casos anteriores hay que tener en cuenta el valor que se haya asignado a la propiedad **ID Tipo C/S** de los métodos que se conectan.

En los elementos de tipo método, el valor que se asigna a esta propiedad determina si el método va a estar accesible en el Meta4Object en cliente o en el Meta4Object en servidor. Si **ID Tipo C/S** recoge el valor Parte cliente, el elemento sólo será visible en el puesto cliente; si recoge el valor Parte servidor el elemento sólo estará disponible para el servidor de aplicaciones.

Cuando **ID Tipo C/S** recoge el valor Parte completa, el método estará visible en el servidor de aplicaciones y en el puesto cliente, y se ejecutará en la máquina desde la que se haga la solicitud de ejecución. Con los elementos de tipo Parte completa, no existe ningún tipo de sincronización ni intercambio de datos entre cliente y servidor, ni antes ni después de la ejecución del método.

Si **ID Tipo C/S** recoge el valor Ambas partes (sinc.), el método estará visible para el servidor de aplicaciones y para el puesto cliente, pero siempre se ejecutará en el servidor de aplicaciones, a pesar de que la solicitud de ejecución se haya hecho desde el puesto cliente.

Como conclusión, se debe recordar que:

- No puede conectarse un método de tipo Parte cliente con un método de tipo Parte servidor, cualquiera que sea el orden de ejecución
- Si se conectan dos métodos de tipo Parte completa, su ejecución se hará en la máquina desde la que se solicite la ejecución del primero de ellos: bien en el cliente, bien en el servidor, pero no se producirá ningún intercambio entre el puesto cliente y el servidor
- Si se conectan dos métodos de tipo Ambas partes (sinc.), los dos se ejecutarán en el servidor de aplicaciones y, para cada uno, se producirán dos intercambios de datos entre el cliente y el servidor
- Si se conecta un método de tipo Ambas partes (sinc.) con un método de tipo Parte cliente, el primer método se ejecutará en el servidor de aplicaciones y generará un intercambio de datos de ida y otro de vuelta

El método Parte cliente se ejecutará en el puesto cliente y no generará ninguna interacción. Para que esto sea posible, es necesario que el conector entre los dos métodos sea de tipo Parte cliente.

Optimización del flujo de ejecución: idas y vueltas

El flujo de ejecución de un Meta4Object en cliente/servidor está determinado por las llamadas que se hacen entre sí los métodos y conceptos, y por los conectores existentes entre estos elementos ejecutables. En general, se produce un flujo de ejecución en el cliente y otro en el servidor.

Siempre, las ejecuciones se inician en la parte cliente ante una acción determinada del usuario. Cuando se ejecuta en la parte cliente la regla de un elemento de tipo Ambas partes (sinc.), la ejecución pasa al servidor y se sincronizan los valores de los elementos. La ejecución continuará en el servidor hasta que no quede ninguna acción por realizar, momento en el que retorna la ejecución al cliente. En este instante se produce una nueva sincronización de los datos entre el cliente y servidor.

A estos intercambios entre el cliente y el servidor se les denomina **idas y vueltas**.

Uno de los objetivos fundamentales del diseño de Meta4Objects es reducir el número de idas y vueltas al mínimo posible. Minimizar el tiempo y el número de idas y vueltas es fundamental para mejorar el rendimiento del sistema y reducir el flujo de carga de la red.

La clave para evitar idas y vueltas innecesarias consiste en analizar cómo se utilizan los elementos del tipo Ambas Partes (sync.) en la parte cliente.

Por ejemplo, siempre que se ejecuta un método de tipo Ambas partes (sync.), se producen dos intercambios de datos entre el cliente y el servidor. El primero, desde el cliente al servidor de aplicaciones, con la solicitud de ejecución y el envío de los datos que requiere la ejecución del método (que se habrá indicado en la propiedad **ID Tipo de sincronización**). El segundo, desde el servidor de aplicaciones al puesto cliente, para indicar que el método se ha ejecutado.

Cuando se conectan dos métodos de tipo Ambas partes (sync.), su ejecución generará cuatro intercambios de datos entre el cliente y el servidor. Sin embargo, al tratarse de métodos Ambas partes (sync.), su ejecución se hará siempre en el servidor de aplicaciones.

Si, por ejemplo, el segundo método únicamente realiza un cálculo y unas operaciones de grabación en la base de datos, el puesto cliente no necesita ver el método. En este caso, con el fin de minimizar el intercambio de datos entre cliente y servidor, el segundo método se podría definir de tipo Parte servidor, al igual que el conector.

Como los métodos Ambas partes (sync.) y Parte servidor forman parte del Meta4Object en servidor, el segundo método estaría visible para el primero, por lo que no habría ningún problema para conectar su ejecución. Como los métodos Parte servidor no generan ningún intercambio de datos entre el puesto cliente y el servidor de aplicaciones, el envío de datos entre el cliente y el servidor quedaría reducido a dos, sin que esto mermase la funcionalidad del conector: el resultado de la ejecución de los dos métodos sería el mismo en ambos casos.

En resumen, un Meta4Object bien diseñado es aquél en el que, o bien todas las acciones que desarrolla un usuario se ejecutan en el cliente, o bien se produce una única ida y vuelta entre el cliente y el servidor.

Instancias de Meta4Objects en entornos C/S

La utilización de un Meta4Object no implica su carga total en memoria. Como se explica en el capítulo *Administración de memoria* de este manual, es posible parametrizar la carga progresiva, bajo demanda, de los registros de los nodos de un Meta4Object. Es decir, es posible estar trabajando con un Meta4Object, sin necesidad de que toda la información recuperable por este Meta4Object esté disponible en la memoria principal del equipo que ejecuta el proceso.

El comportamiento del sistema en la creación de instancias de Meta4Objects y en la reutilización de Meta4Objects con instancia en la memoria se describe en el capítulo *Administración de memoria* de este mismo manual. Como se explica en este capítulo, las políticas de reutilización dependen de la forma en la que:

- Se solicite la carga de datos del Meta4Object: mediante opción de menú o mediante llamadas de nivel 2
- Se haya parametrizado la propiedad que determina la política de reutilización en cada uno de los casos anteriores

En un entorno cliente/servidor, la memoria del servidor se distribuye en varios espacios que se reservan para procesos específicos. Siempre que el servidor cargue en memoria un Meta4Object al que corresponde un modelo de ejecución proxy, los datos se cargarán en un espacio de memoria reservado, denominado espacio proxy. El modelo de ejecución de un Meta4Object se indica, en tiempo de diseño, en la propiedad **Tipo de ejecución C/S**.

El servidor reserva un espacio proxy para cada sesión que lleva asociado este tipo de ejecución.

Si desde un método o concepto del Meta4Object para el que se ha reservado el espacio proxy se solicita la creación de instancia de otro Meta4Object mediante llamadas de nivel 2, el sistema cargará este segundo Meta4Object en este mismo espacio proxy. De esta forma, en el mismo espacio proxy pueden estar disponibles distintas instancias de varios Meta4Objects.

En el caso de que una de las instancias disponibles en un espacio proxy hiciese referencia a un elemento de un Meta4Object para el que anteriormente se ha levantado una instancia en ese mismo espacio proxy, el sistema podría reutilizar la instancia ya disponible o crear una nueva, dependiendo de cómo se haya parametrizado la política de reutilización de Meta4Objects al diseñar la llamada de nivel 2.

Puede encontrar información sobre las políticas de reutilización disponibles en el capítulo *Administración de memoria* de este mismo manual.

Todos los conceptos que se explican en ese capítulo son aplicables al modelo de trabajo cliente/servidor. Únicamente hay que tener en cuenta la siguiente restricción.

Como conclusión, se recuerda que:

- El sistema permite reutilizar instancias de Meta4Objects, siempre que formen parte del mismo espacio proxy
- La posibilidad de reutilizar instancias de Meta4Objects queda restringida al espacio proxy en el que se trabaja
- Una instancia nunca puede reutilizar una instancia que se encuentre en otro espacio proxy

Operatividad en modo C/S

Configuración del cliente Meta4 en C/S

Existen determinados parámetros en el registro de la aplicación que permiten configurar los clientes distribuidos para conectarse al servidor de aplicaciones. Algunos de estos parámetros, visibles desde la herramienta de configuración del cliente o desde el archivo de registro, toman los valores que el instalador de la aplicación asignó durante el proceso de instalación.

El típico cliente estándar con comportamiento C/S es el cliente distribuido que se conecta únicamente al servidor de aplicaciones y delega en éste la conexión y la ejecución de las operaciones de lectura y escritura sobre la base de datos. Durante el proceso de instalación del cliente distribuido es necesario instalar el servidor de aplicaciones. La instalación del servidor de aplicaciones obliga a tener instalados los componentes necesarios para asegurar la conectividad con la base de datos, y a configurar los servidores de aplicaciones.

Desde el editor de registro, podrá comprobar y modificar las configuraciones de los servidores de aplicaciones para obtener acceso a la base de datos.

Timeouts

El concepto de **timeout** hace referencia al efecto causado cuando una operación que implica transacciones en cliente/servidor no finaliza correctamente en el tiempo prefijado.

En las aplicaciones Meta4 se emplean dos términos con similar significado, pero con distinto planteamiento.

De este modo, existe una limitación en el tiempo máximo que el cliente esperará una respuesta por parte del servidor ante la solicitud de ejecución de una transacción. Una vez transcurrido el tiempo máximo de espera, si no se ha recibido una respuesta apropiada, se muestra al usuario un mensaje de alerta en el que se ofrecen tres posibilidades:

- Reintentar: se solicita de nuevo la realización de la transacción
- Anular: se anula la solicitud de ejecución que está pendiente.
- Modificar Timeout: establece un nuevo tiempo de espera.

Los tiempos de **timeout** pueden parametrizarse a través del archivo de registro en la parte cliente mediante las propiedades **CsDefaultTimeOut**, que define el tiempo por defecto de **timeout** para una ejecución OLTP/PROXY contra el servidor de aplicaciones, y **CsGenericTimeOut**, que hace referencia a otro tipo

de transacciones (conexión, metadatos, control etc.). Estos *timeouts* se miden en segundos.

Sobre los distintos componentes del servidor de aplicaciones existen parámetros asociados a *timeouts* que determinan cómo interactúan estos servicios. Durante la ejecución, el administrador puede modificar los valores sobre los tiempos de ejecución de los componentes. El Meta4Object Engine se encarga internamente de cancelar la ejecución cuando se sobrepasan estos tiempos.

Se han implementado además un tipo de Timeouts que se caracterizan porque si no se establece ninguna transacción se interrumpe la conexión. Son lo denominados **Timeouts de inactividad**.

Está previsto que en el futuro se limiten los tiempos de ejecución desde el nivel de la aplicación a través de fórmulas del LN4.

Funcionamiento de las cachés

Tipos de caché

Las cachés son espacios de memoria temporales y/o persistentes reservados para almacenar la estructura o el contenido de los objetos de la aplicación. En el *core* de la aplicación de Meta4 existen tres tipos de cachés:

- Caché de metadatos
- Caché de diccionario
- Caché de datos

Caché de metadatos

Estas cachés almacenan definiciones y estructuras de los metadatos de la aplicación.

Hay cuatro cachés de metadatos:

- Estructura de un Meta4Object
- Seguridad de un Meta4Object
- Estructura de una presentación
- Mapeos de Meta4Object y de presentaciones

Este tipo de cachés son persistentes en disco.

Caché del diccionario

Esta caché almacena los metadatos de las tablas de base de datos y de su definición de seguridad para su uso en la base de datos lógica.

Este tipo de cachés no son persistentes en disco, de modo que en posteriores inicializaciones de los ejecutables de PeopleNet comenzarán desde cero.

En entornos tipo producción como los que nos ocupan, las fuentes de cambios siempre son trasposos de tablas hechas con RAM-DL.

Caché de datos

La caché de datos contiene los datos de los Meta4Objects marcados como cacheables en datos, es decir, que la unidad de almacenamiento en este tipo de cachés es el contenido de un Meta4Object.

Las tablas de base de datos no son en sí mismas un dato cacheable, pero tienen un atributo IS_CACHEABLE para permitir el refresco automático de este tipo de cachés.

Se considera que un canal debe ser cacheable en datos si los datos que está manejando son solo de lectura, es decir, son tablas que se usan principalmente para leer pero casi nunca para escribir (el típico ejemplo es la tabla de sexos). Para optimizar el rendimiento de la plataforma, algunos Meta4Objects tecnológicos, como los que consultan las opciones de menú, tareas, opciones de programa, idiomas, sociedades, árbol de sociedades, roles para opciones de menú o monedas también se han hecho recientemente cacheables en datos para optimizar su rendimiento.

Este tipo de cachés son persistentes en disco.

Caché de datos: comportamiento de un Meta4Object cacheable

El Meta4Object cacheable se almacena en la memoria caché, que en realidad es un espacio de memoria RAM, de tal modo que cuando el sistema necesita leer datos sobre ese Meta4Object, lo hace directamente desde la caché y no desde la base de datos. Las cachés dinamizan las operaciones de lectura de datos, ya que evitan la necesidad de realizar una operación de lectura en el SGBD.

Las aplicaciones Meta4 permiten obtener acceso a datos que se almacenaron temporalmente en una caché de datos. Ésta puede residir en el puesto cliente, en el servidor de aplicaciones o en ambos.

Para que entre en funcionamiento el mecanismo de caché de datos es necesario hacer una carga completa del Meta4Object. Los Meta4Objects toman los parámetros de configuración de cachés desde la Herramienta de

Configuración cuando se cargan en los puestos de trabajo clientes, y del archivo de arranque e inicialización del servidor **startup.obl** cuando se cargan en el servidor de aplicaciones.

La política de planificación que se utiliza para la gestión de cachés tiene la peculiaridad de que se gestiona de forma paralela al almacenamiento en disco y memoria, y se planifica en base a la memoria asignada para cada subsistema de cachés y al número de objetos de la misma.

Refresco de las cachés

Los tres tipos de cachés disponen de un modo automático de refresco. El efecto de este refresco automático difiere según la plataforma en la que se esté trabajando. Y según esto se necesitará o no realizar acciones adicionales (o por lo menos se debe ser consciente de ello para entender los datos que se están manejando en cada momento).

También existe una gestión automática por parte de RAM-DL cuando se traspasa algún objeto que es susceptible de afectar a los objetos cacheados, de manera que los objetos afectados se marcan como obsoletos en la tabla de versionado de cachés.

En el servidor de aplicaciones

Las cachés se actualizan cada tres minutos (por defecto) y al arrancar el servidor.

Existe un elemento del servidor, llamado supervisor de cachés (con una frecuencia DELAY_TIME por defecto de 180 segundos), que recorre la tabla de versionado de cachés de todos los objetos cacheados buscando si hay versiones más recientes.

Adicionalmente, un administrador puede efectuar una gestión más detallada desde el monitor del servidor de aplicaciones o mediante un comando SNMP, para borrar cualquier caché, refrescar la caché de metadatos o cambiar propiedades de las mismas.

En el cliente

Las cachés se actualizan solo al conectarse al servidor.

Cuando el cliente se conecta, le manda al servidor la fecha de su última actualización de las cachés (tanto de metadatos, como de datos). Si el número de objetos con fecha posterior es menor que un límite (que está por defecto en 250 objetos) se realiza un borrado selectivo. Si es un número mayor, se borran todas las cachés del cliente al realizarse la conexión.

Adicionalmente, cuando el cliente hace una petición de ejecución al servidor, si

se detecta que el metadato de un Meta4Object es más moderno en el servidor que en el cliente, se borra ese objeto de la caché cliente, para que en posteriores peticiones al servidor se pueda comunicar correctamente.

En la estación de desarrollo

Las cachés no se actualizan automáticamente.

El usuario puede, mediante las opciones avanzadas, actualizar y limpiar las cachés.

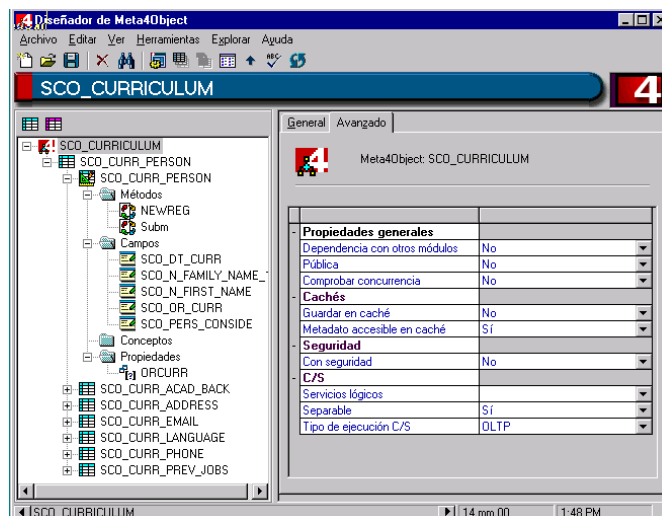
Parámetros de configuración de la caché de datos y metadatos

Los parámetros de configuración de la caché se definen a través de la **Herramienta de Configuración**, del archivo **startup.obl** (en el caso del servidor) y de las propiedades del Meta4Object.

Parametrización de la caché

Podrá parametrizar las cachés de los Meta4Objects desde la pestaña de propiedades avanzadas que aparece en el diseñador de Meta4Objects, tal y como muestra la imagen siguiente:

Figura 85. Parametrización de la caché



Las propiedades parametrizables a nivel de Meta4Object son: **Guardar en caché** y **Metadato accesible en caché**.

La propiedad **Guardar en caché** hace referencia a la caché de datos. Esta propiedad puede recoger los siguientes valores:

- En el caso de que recoja el valor **No**, los datos del Meta4Object no se almacenan en la caché de datos; lo que significa que cada vez que el Meta4Object necesite cargar esos datos, se conectará directamente sobre el sistema gestor de base de datos para recuperarlos. Esto implica un tráfico y un coste innecesario de recursos de red, puesto que obliga a establecer tantas conexiones como peticiones demande el Meta4Object.
- En el caso de que recoja el valor **Sí**, los datos del Meta4Object se almacenan en memoria caché, de tal modo que cuando se demanda la carga de datos del Meta4Object, éstos se extraen de la caché, evitando así una conexión con la base de datos, y por tanto, minimizando recursos e idas y vueltas innecesarios. Para información más detallada, véase el apartado Comportamiento de cachés en C/S

La propiedad **Metadato accesible en caché** indica si se va a cachear o no la estructura (los metadatos) del Meta4Object.

Esta propiedad puede recoger los siguientes valores:

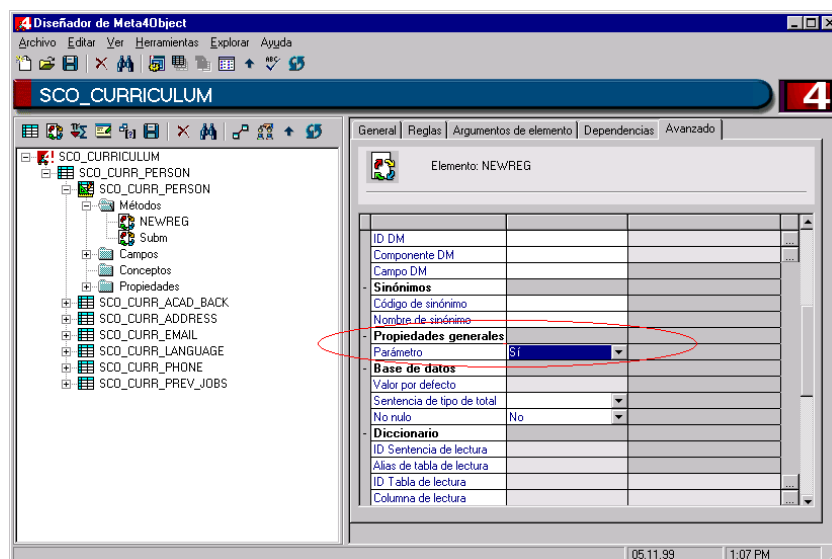
- En el caso de que recoja el valor **Sí**, los metadatos se almacenan en el subsistema de cachés, con la particularidad, con respecto a los datos, de que se almacenan tanto en la caché del cliente como en la caché del servidor. Todos los Meta4Objects toman por defecto esta propiedad, sobre todo los modificados por los usuarios en la implantación. Para información más detallada, véase el apartado Comportamiento de cachés en C/S
- En el caso de que recoja el valor **No**, los metadatos no se almacenan en el subsistema de cachés. S



Si los datos de un Meta4Object son cacheables, interesaría que también lo fueran sus metadatos. Por eso, si un Meta4Object se marca como no cacheable de metadatos, automáticamente el sistema lo considera también como no cacheable de datos.

Con el fin de optimizar los recursos de memoria RAM del subsistema de cachés, es posible parametrizar la propiedad **Parámetro** a nivel de elemento para que en la caché de ese dato el valor del elemento conste como parámetro de entrada. Esto permite tener distintos juegos de datos cacheables dependiendo del valor de ese elemento.

Figura 86. Parámetros de elementos con cachés



EJEMPLO

Suponga que necesita almacenar en la caché los departamentos que tiene cada centro de trabajo.

Si cachea cada departamento en un Meta4Object distinto, cuando necesite consultar la información de un departamento determinado no tendrá que cargar la información de todos los departamentos, sino sólo la del que le interese. Si utiliza el centro de trabajo como parámetro de la caché, conseguirá un Meta4Object diferente para cada centro de trabajo.

Configuración de caché en parte cliente y servidor a través de registro

En la **Herramienta de Configuración** están almacenados todos los parámetros de configuración del comportamiento con cachés de datos y metadatos. Los valores de estas propiedades son editables.

CacheDir: Indica el nombre del directorio donde se almacenan los datos guardados de la caché. En la ruta indicada en este parámetro, se generará una estructura de directorios Emul cuando la aplicación se esté ejecutando en un entorno de desarrollo; y se generará una estructura Client cuando se esté ejecutando en la parte cliente de un entorno cliente/servidor, y Server en la parte servidor. En cada directorio se encontrarán los siguientes subdirectorios:

- **Cache:** para cada subsistema se almacena un archivo .cac llamado CMCRCache.cac, donde se almacenan los índices de los contenidos del subsistema al que se refiere.

- **Channel:** hace referencia al contenido de los Meta4Objects (datos) que son cacheables.
- **Scr:** se almacena la caché de seguridad de los Meta4Objects. Las cachés de seguridad se generan desde la herramienta de seguridad desarrollada en la aplicación.
- **Prs:** se refiere a la caché de presentaciones.
- **Mcr:** almacena los metadatos, es decir la estructura de los Meta4Objects.



Las cachés de seguridad, presentaciones y mapeo también se consideran metadatos.

DataCacheDefaultMaxPeriod: indica el número de días máximo que un Meta4Object puede permanecer en la caché por defecto. Admite valores decimales para las horas. Cada Meta4Object podrá especificar su propio tiempo de expiración en la base de datos.

Este parámetro se comprobará cuando se cumpla el número de accesos marcado en la propiedad DataCacheRefreshratio.

DataCacheExpiryRatio: Factor reductor del tiempo de caducidad de los Meta4Objects de datos. Cada Meta4Object de datos puede tener una política propia de caducidad. Sobre este tiempo se aplica el factor reductor en el servidor.

Se calcula dividiendo el valor que aparece en el parámetro **DataCacheDefaultMaxPeriod** entre el valor que aparece en este parámetro.

DataCacheMaxMemory: Tamaño máximo en bytes de RAM que se puede usar en la caché de datos antes de realizar el intercambio con el disco.

DataCacheMaxNumObjects: Número máximo de Meta4Objects en la caché. Esta propiedad permite refrescar la memoria caché.

DataCacheMode: Puede recoger las siguientes propiedades:

- **NONE:** No se trabaja con cachés.
- **READ:** Sólo se trabaja con los datos que están en disco, pero no se graban, a pesar de que se guardan en archivos temporales.
- **WRITE:** Sólo se graba en disco; se sobrescribirán los datos que se encuentren almacenados en disco.
- **READ_WRITE:** se graba y lee los valores de disco.
- **MEM_ONLY:** Sólo se trabaja con cachés en memoria, no se graba ni se utiliza lo que había en disco.

DataCacheRefreshRatio: Número de accesos a la caché de datos para grabar la caché y revisar los Meta4Objects. Indica cada cuántos accesos se graba en

disco, y marca cada cuánto se revisa la caducidad de los Meta4Objects de la caché.

MDCacheDefaultMaxPeriod: Caducidad para los Meta4Objects de la caché de metadatos.

MDCacheExpiryRatio: Se aplica dividiendo la caducidad de la caché de metadatos entre este ratio.

MDCacheMaxMemory: Tamaño máximo de RAM (en bytes) que se puede usar en caché de metadatos antes de realizar el intercambio con el disco.

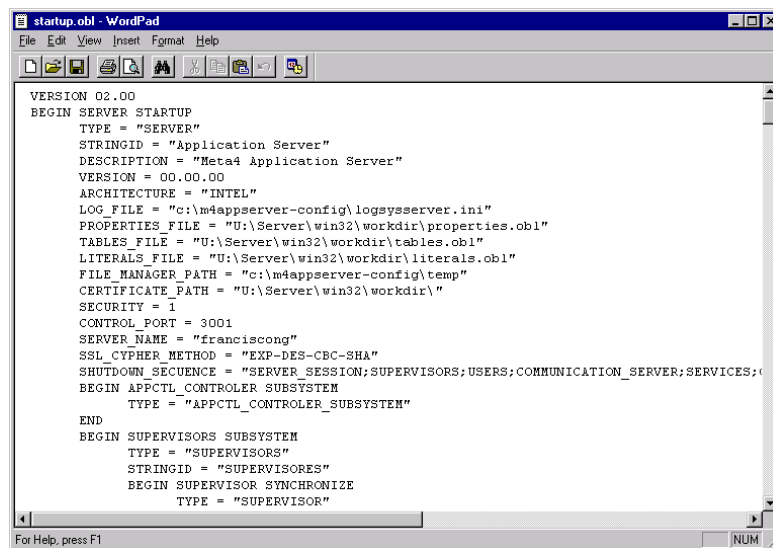
MDCacheMaxNumObjects: Número máximo de Meta4Objects de metadatos en la caché.

MDCacheRefreshRatio: Valor numérico que indica cada cuántos accesos se graba en el disco y cada cuánto se revisa la validez de los metadatos de la caché.

Configuración de caché en parte servidor (startup.obl)

Los parámetros de configuración de las cachés en el servidor de aplicaciones se gestionan desde el archivo de arranque e inicialización del servidor **startup.obl**.

Figura 87. Archivo startup.obl



De este modo, los Meta4Objects que tengan parametrizado el modo de carga en el servidor de aplicaciones tomarán los parámetros de caché desde este archivo.

Este archivo establece la política de cachés en varios subsistemas de cachés: caché de metadatos, caché de presentaciones, caché de seguridad, caché de

mapeos y caché de Meta4Objects. Para cada uno de estos subsistemas existen propiedades parametrizables:

- **MAX_CACHE_NUM_ITEMS**: indica el número de elementos que se admiten en la caché para ese subsistema, tanto para memoria como para disco.
- **MAX_CACHE_SIZE_MEMORY**: indica el tamaño máximo que ocupará la caché para ese subsistema en memoria.

Es recomendable que el administrador tenga en cuenta que para evitar inconsistencias entre los metadatos de la base de datos y los de la caché, existe un parámetro editable en el **startup.obl** que permite sincronizar ambos valores, **METADATA_MOD_DATE = "0000-00-00 00:00:00"**. Por lo tanto, siempre que se produzcan modificaciones en metadatos durante las fases de desarrollo e implantación, será necesario borrar las cachés del servidor, editar el archivo **startup.obl** y modificar el parámetro **METADATA_MOD_DATE** para que recoja la fecha de modificación de los metadatos. Esta nueva fecha del servidor se envía de vuelta a los puestos de trabajo cliente en el proceso de conexión del servidor, para que éstos desechen los metadatos obsoletos, es decir aquéllos que son anteriores a la fecha indicada en el parámetro, y se sincronicen entre sí.

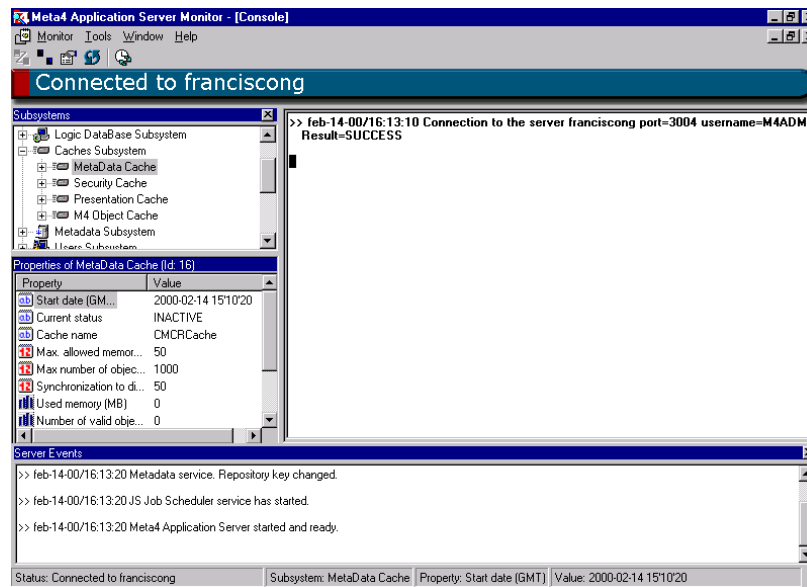
Supervisión de cachés desde la herramienta de administración del servidor

La herramienta de administración del servidor de aplicaciones muestra información sobre las cachés de datos y metadatos que se hayan definido en el momento de instalar el servidor de aplicaciones, o posteriormente, al editar el archivo de configuración **startup.obl**. Esta herramienta permite supervisar y gestionar todas las operaciones referentes al borrado de Meta4Objects de la caché del servidor.

El subsistema de cachés del servidor de aplicaciones se compone de varios elementos: Metadata cache, Security cache, Presentation cache, Meta4Object cache y Map cache.

Cada uno de los elementos de los distintos subsistemas tienen propiedades parametrizables distintas, y los procesos de ejecución, como el borrado de cachés, son independientes unos de otros.

Figura 88. Propiedades de la caché



Consulte el manual sobre Procedimientos de administración del servidor de aplicaciones para mayores detalles.

Borrado de cachés del servidor

En los subsistemas de cachés, cuando se alcanza al máximo de memoria permitido o al máximo número de objetos permitidos para esa caché, se descartan los Meta4Objects cacheados que llevan más tiempo sin ser usados.

Los comandos que permiten ejecutar operaciones de borrado de cachés desde la herramienta de administración del servidor de aplicaciones son: **Empty Out**, **Refresh** y **Refresh caché**. Para ejecutar estos comandos, haga clic con el botón derecho del ratón desde el componente sobre el que esté colocado.

- **Empty Out:** borra todos los Meta4Objects que se encuentran en la caché. Si están siendo usados en ese momento se borrarán cuando dejen de usarse.
- **Refresh cache:** elimina de la caché los objetos que están obsoletos. Sirve para liberar memoria en el servidor.
- **Synchronize to Disk:** guarda en disco la información actualizada de la caché.

Existen otras propiedades de parametrización que inciden directamente sobre el borrado de cachés: **Máximo número de objetos** (Set new max. number of objects allowed) y **Máximo tamaño de memoria** (Set new max. allowed memory).

En el caso de que se sobrepase el valor asignado en la propiedad **Máximo número de objetos**, el sistema intenta eliminar de la caché los objetos que no estén referenciados. Si encuentra objetos en la caché que no están referenciados, los elimina, y si detecta que no puede liberarse espacio de la caché, aumenta el tamaño de la misma a través de una asignación dinámica de memoria.

En el caso de que se sobrepase el valor asignado a la propiedad **Máximo tamaño de memoria**, el sistema intenta buscar en la caché objetos no referenciados. Si encuentra objetos en la caché que no están referenciados, los almacena en disco y los elimina del espacio reservado en la caché. Si no existen objetos referenciados, el sistema desencadena una asignación dinámica de memoria para aumentar su tamaño.

Comportamiento de cachés en C/S

Las cachés de metadatos funcionan de la siguiente manera: cuando un puesto cliente necesita usar un objeto cacheable lo busca primero en la caché local. Si no lo encuentra en la caché, se lo pide al servidor de aplicaciones. Este, a su vez, lo busca en su caché. Si no lo encuentra, lo construye y guarda en la caché. Finalmente lo devuelve al cliente, que lo almacena en su caché local.

Las cachés de datos funcionan de modo similar, salvo que no piden el objeto cacheable al servidor, sino que lo construyen directamente de los datos que el servidor de aplicaciones les proporciona de la BD.

Este modo de trabajo permite optimizar el tráfico entre los clientes y el servidor, y entre el servidor y la base de datos.

Sincronía/asincronía

El modelo de diseño con espacios de memoria caché optimiza la velocidad de acceso a los datos, pero además debe garantizar la integridad de los mismos, asegurando la sincronización de los datos y metadatos de la caché con los de la base de datos.

La aplicación cuenta con un sistema de sincronización que en la mayoría de los casos es automático para el usuario. El objetivo es conseguir que las cachés solo descarten los metadatos obsoletos y que se sincronicen entre sí.

Las formas de sincronización se detallan en el apartado **Refresco de las cachés**.

Consideraciones especiales en el diseño con cachés

La política en el diseño con cachés está encaminada a definir qué

Meta4Objects son o no cacheables, siempre dependiendo del entorno de trabajo con la aplicación.

Los Meta4Objects cacheables:

- No deben tener seguridad de datos, ya que si un usuario cambia de rol de aplicación será imposible tener acceso a datos que hasta ahora eran visibles.
- No deben tener filtros SQL, puesto que estos filtros tienen acceso a la base de datos, cuando realmente deberían tenerlo a la RAM, donde están todos los datos.
- No deberían ser Meta4Objects demasiado grandes. Si es necesario cachear algún Meta4Object demasiado grande, se debe dividir en varios Meta4Objects cacheados.

Trazas cliente/servidor

Los usuarios del lenguaje LN4 y en general diseñadores de Meta4Objects con información cliente/servidor se encuentran con dificultades cuando quieren mejorar el rendimiento de ciertos procesos. Uno de los puntos a tomar en cuenta es evidentemente la cantidad y tipo de transacciones que implica dicho proceso.

Por ello, se ofrece una herramienta que permite a estos usuarios, el obtener la información necesaria para poder depurar el comportamiento cliente/servidor.

Es de vital importancia que esta información se pueda obtener tanto en versiones de depuración como de uso estándar del cliente, pero sólo al realizar la conexión con un servidor (tanto en cliente estándar como en cliente de desarrollo).

La información relativa al seguimiento de las comunicaciones realizadas entre el cliente y el servidor se guardan en un archivo denominado *CSTrace.txt*, contenido en un directorio asignado por el usuario. Por defecto, este archivo se genera en el directorio temporal del propio usuario.

Esta metodología no pretende sustituir a las estadísticas de Meta4Objects (archivos de traza detallados), ya que la información mostrada aquí será la mínima necesaria para la depuración de parámetros en el funcionamiento cliente/servidor.

Requerimientos de alto nivel

Para realizar esta optimización, el usuario debe ser un desarrollador de Meta4Objects (LN4) y con un conocimiento medio de la tecnología relacionada con cliente/servidor y un alto conocimiento a nivel funcional. Los casos de uso típico son los siguientes:

- Después de detectar un caso obvio de mal rendimiento (tiempos malos), el usuario intenta detectar las causas y sospecha del comportamiento cliente servidor del Meta4Object y/o presentación (en caso de que esta exista).

En este caso se debería poder aislar el conjunto de trazas relacionadas con esta operación para su análisis posterior más detallado. Con una simple visualización de las trazas el usuario podrá detectar muchas situaciones de error (no será necesaria el uso de herramientas avanzadas).

- En fase de optimización, se activan estas trazas para detectar algún punto que quede en el desarrollo. Ya que la cantidad de información que se puede generar es grande, los datos podrían ser post-procesados y convertidos en advertencias mas manejables por el usuario.

Esto implica que el usuario debe definir su política de detección de posibles errores y facilitar esta tarea mediante el empleo de herramientas convencionales disponibles en el mercado.

Se propone usar una filosofía semejante a la usada actualmente en estadísticas para la acumulación de eventos relacionados con transacciones cliente servidor y facilitar su posterior análisis. En un principio se mostrará información de:

- Arranque y parada del Meta4Object Engine
- Apertura y cierre de sesiones
- Ejecución de métodos remotos
- Peticiones de metainformación (metadatos, presentaciones, seguridad)
- Peticiones que son interceptadas por el caché local y evitan transacciones contra el servidor (peticiones de metainformación o carga de datos en M4Objetos cacheables).

En todos los casos, se hará hincapié en detallar la información que puede tener que ver con la influencia de la transacción en el rendimiento general (por ejemplo: tamaños de mensajes enviados al/recibidos desde el servidor).

Otras funcionalidades de interés son:

- Como ayuda adicional al usuario, que permite fijar etiquetas para identificar desde un nivel mas alto cuales son las operaciones que se están realizando (inicio/fin de etiqueta)
- Poder habilitar/deshabilitar estas trazas
- Poder habilitar/deshabilitar el mostrar trazas de transacciones inexistentes debido a su resolución con ayuda de la caché local
- Inicialización de las trazas (eliminación de toda información previa)



Consulte el manual de Depuración para mayores detalles.

Apéndice: Introducción al APISQL



¿Que es APISQL?

APISQL es el lenguaje empleado para el acceso a los datos de la base de datos lógica.

Es un lenguaje con base en el SQL estándar con particularidades propias de la tecnología Meta4.

Uso del APISQL:

Tipos de acceso a datos a través de API SQL

Empleando un concepto invocado desde LN4

Para ello es necesario crear un elemento concepto de tipo long y ámbito nodo en el nodo donde se va a emplear la sentencia. El concepto debe tener la propiedad avanzada "Argumentos variables" con valor "si". A este elemento se le asociará una de las siguientes funciones de CPP (Estas funciones ejecutan la sentencia de diferentes maneras): ExecuteSQL, ExecuteDirectSQL ó ExecuteRealSQL.

- **ExecuteSQL:** Es la más restrictiva. Cuando se asocia el elemento a esta función se lanza la sentencia lógica con todos los controles internos de seguridad establecidos en la base de datos lógica. No tiene ningún argumento fijo.
- **ExecuteDirectSQL:** Tiene dos argumentos fijos:

- *IdConn*: Es el identificador de la conexión lógica. La BASE DE DATOS LÓGICA permite tener datos repartidos entre distintos usuarios ó bases de datos. De tal manera que si tenemos cinco usuarios habrá cinco conexiones reales distintas (todas sobre una misma conexión lógica). La BASE DE DATOS LÓGICA utilizará la conexión contra el usuario de base de datos que tenga la tabla a la que hace referencia la sentencia. Si su valor es -1, se tomará la conexión por defecto de la tabla. Para más información consulte la ayuda en línea sobre conexiones.
- *Mask*: Es la máscara que contiene la información necesaria para deshabilitar validaciones internas de la base de datos lógica. Los valores admitidos pueden ser 0 ó 1, es decir se realizan todos los controles ó ninguno.
- *ExecuteRealSQL*: Se diferencia de las funciones anteriores en que permite lanzar sentencias reales, es decir la sentencia está escrita en SQL nativo y no con la sintaxis propia de Meta4 como en los casos anteriores. En este caso la BASE DE DATOS LÓGICA no realiza ninguna operación intermedia, y carece de control interno. Sólo tiene el *IdConn* como argumento fijo.

Empleando el elemento SysSentence

SysSentence es una propiedad de tipo long y de ámbito nodo donde se almacena una sentencia APISQL que se ejecutará en la carga de la Estructura de Nodo correspondiente. Este elemento tiene un tipo interno especial: el 20.

El elemento *SysSentence* puede contener:

- Una sentencia SELECT completa.
- Una sentencia de filtro añadido.

Cuando la Estructura de Nodo se carga se comprueba si existe un elemento con tipo interno 20, si no existe se lanza la sentencia asociada a la estructura de nodo. Si se detecta este elemento y lo que contiene es una sentencia SELECT completa, se sustituye la sentencia generada para la Estructura de Nodo por la que contiene el elemento *SysSentence*. Si existen filtros de seguridad y filtros automáticos se añadirán a la sentencia anterior. Si por el contrario, el elemento contiene solo la parte FROM se mezcla esta parte con la sentencia SELECT de la Estructura de Nodo junto con los filtros de seguridad y los filtros automáticos. El elemento *SysSentence* puede contener N partes WHERE (N filtros) separados por ",".

Estos filtros y los comentados anteriormente son acumulativos de tal manera que podemos lanzar una sentencia formada por la sentencia SELECT de la Estructura de Nodo + filtros API SQL + filtros *SysSentence* + filtros de seguridad + filtros automáticos.

La secuencia de ejecución sería la siguiente:

LN4	F_{LN4}	F_{LN4}	S_{LN4}
SYSENTENCE	F_{SYS}	S_{SYS}	-----
SENTENCIA TI	S	S	-----
FILTROS APLICADOS	$S + F_{SYS} + F_{LN4}$	$S_{SYS} + F_{LN4}$	S_{LN4}



Para pasar parámetros al elemento SysSentence existe un elemento SysParam. Este elemento es de tipo long y de ámbito nodo y tiene un tipo especial 33. La sintaxis para el paso de parámetros es: **Parámetro|[Parámetro|]**

Sintáxis de APISQL

Los elementos que componen una sentencia APISQL son los siguientes:

- **CONST** (Constantes): las constantes de LN4: ejm: PLUS_INF y MINUS_INF
- **ITEM** (Elemento): @ID_ITEM (elemento de la estructura de nodo que contendrá un valor asociado a la sentencia)
- **FUNCTION** (Función): #ID_FUNCTION
- **FIELD** (Campo): ALIAS.ID_FIELD
- **PARAM** (Parámetro): ? (m4Type, Scale, Prec,) ver Valores posibles de m4type
- **OBJECT** (Tabla): ID_OBJECT ALIAS
- **BASE** (Tabla base): &ID_OBJECT ALIAS



Alias obligatorio

Valores posibles de m4type

- 0 = Nulo
- 1 = Cadena fija
- 2 = Cadena variable
- 3 = Long
- 4 = Fecha
- 5 = Fecha y Hora
- 6 = Número
- 7 = Variant
- 8 = Moneda
- 9 = Variant Numérico
- 10 = BLOB
- 11 = Cadena binaria
- 12 = Hora

Tipos de sentencias:

Hay cuatro tipos de sentencias soportados que son:

SELECT

Para seleccionar los valores de una tabla.

La sintaxis de la sentencia tendría esta apariencia.

```
SELECT [DISTINCT] @ID_ITEM=ALIAS.ID_FIELD,  
[@ID_ITEM=ALIAS.ID_FIELD] FROM &ID_OBJECT ALIAS,[ID_OBJECT  
ALIAS2] WHERE [ALIAS.ID_FIELD=ALIAS2.ID_FIELD] [AND] [OR]  
[ALIAS.ID_FIELD=? (m4Type,Scale, Prec,)] [GROUP BY  
ALIAS.ID_FIELD,[ALIAS.ID_FIELD]] [ORDER BY  
ALIAS.ID_FIELD,[ALIAS.ID_FIELD]]
```

INSERT

Para insertar los valores de una tabla.

La sintaxis de la sentencia tendría esta apariencia.

```
INSERT INTO &ID_OBJECT ALIAS(Alias.ID_FIELD,[Alias.ID_FIELD]) VALUES  
([@ID_ITEM], [?(m4Type,Scale, Prec, )])
```

Limitaciones por el uso de nombres lógicos:

- No permite la definición de constantes ni operaciones en la cláusula de las columnas (dentro de la parte INTO)
- El alias es obligatorio.

UPDATE

Para actualizar los valores de una tabla.

La sintaxis de la sentencia tendría esta apariencia.

```
UPDATE &ID_OBJECT ALIAS SET [Alias ID_FIELD = @ID_ITEM], [Alias  
ID_FIELD = ? (m4Type,Scale, Prec, )] WHERE  
[ALIAS.ID_FIELD=ALIAS2.ID_FIELD] [AND] [OR]  
[ALIAS.ID_FIELD=? (m4Type,Scale, Prec,)]
```

Limitaciones por el uso de nombres lógicos:

- Sólo son validas las subconsultas en el filtro (LA PARTE WHERE).
- No se permiten ni funciones ni constantes en la parte del SET

DELETE

Para el borrado de datos en tablas.

La sintaxis de la sentencia tendría esta apariencia.

```
DELETE &ID_OBJECT ALIAS WHERE [ALIAS.ID_FIELD=ALIAS2.ID_FIELD]
[AND] [OR] [ALIAS.ID_FIELD=? (m4Type, Scale, Prec,)]
```

Limitaciones:

- Alias obligatorio

Palabras clave de APISQL

ASC	■ Para ordenaciones ascendentes.
DESC	■ Para ordenaciones descendentes.
NOT	■ Negación de una condición
HAVING	■ Condición para el valor de un campo; ejm HAVING ALIAS.ITEM=2
IS	■ Para ciertas condiciones; ejm ALIAS.ITEM IS NULL
NULL	■ Nulos en general; ejm SELECT NULL
EXISTS	■ Subselects; ejm WHERE EXISTS (SELECT)
NOT IN	■ Subselect; ejm WHERE NOT ALI.DOS NOT IN (SELECT)
SELECT DISTINCT	■ Selecciona solo los valores diferentes
FOR UPDATE	■ Indica que la Select es para actualizar ciertos campos

Funciones APISQL

Si quiere consultar sobre las funciones disponibles acceda al editor de sentencia en la opción Funciones podrá acceder al listado completo de las mismas.

